Library Management System (LMS)

NEA Project A level

Version:	v0.7
Author:	Saarah Bedekar
Date:	12/05/2025

Distribution list for Review and Sign off

#	Recipient	Title	Role (Review/Sign off)
1	Miss Ahmed	Head of Computer Science & ICT	Review
2	Mr R.B	Owner of Bedekar's library	Sign off

Document Amendment History

Version #	Date	Author	Changes
v0.1	07/09/2024	Saarah Bedekar	Initial draft. Issued for Review/Sign off
v0.2	21/09/2024	Saarah Bedekar	Changes based on review comments by Miss Ahmed
v0.3	22/10/2024	Saarah Bedekar	Changes based on self review. Added Design section.
v0.4	20/11/2024	Saarah Bedekar	Added to Class Design
v0.5	03/01/2025	Saarah Bedekar	Added Test approaches
v0.6	13/03/2025	Saarah Bedekar	Added all code
v0.7	12/05/2025	Saarah Bedekar	Updated based on feedback from Miss Ahmed

Table of Contents

1 Introduction

1.1. Document Purpose

As part of a computer science A level, students are expected to create a NEA project for public examination submission. I will be implementing a semi-automated **Library Management System (LMS)**. The timeline given is until April 2025 to complete my project.

This document covers the scope of my project and provides all necessary details as required by the NEA checklist.

1.2. Main Technology References

Language: https://devdocs.io/openjdk~17/, https://www.w3schools.com/java/

REST API Framework: https://spring.io/projects/spring-boot, https://www.w3schools.blog/spring-boot-tutorial

Database: https://www.mysql.com/, https://www.w3schools.com/MySQL/default.asp

1.3 Miscellaneous Information

1.3.1 Project Timeline

- 2 -3 months: Researching in the right technology and architecture to implement the project.
- 3 months: System development and initial testing.
- 1 month: Documentation and presentation.

1.3.2 Assumptions

- Wi-fi internet availability throughout the library building.
- No physical space constraint for shelfing of books.
- At least some member users are aware of using IT systems (for the usage of self-service system).

2 Analysis

2.1 Background & Problem Definition

A library serves as a key repository of information, providing access to books for a wide range of users, including students, teachers, researchers, and the (general) public. With the advancement in technology, many libraries have automated their operations to use computerised systems. However, many smaller libraries or those in schools may still rely on manual systems that use physical registers, card catalogs, or spreadsheets to keep track of their book stock, books, users, and transactions.

The problem arises from these manual methods, which are prone to errors and inefficiencies. The library staff spend a significant amount of time maintaining and updating the library database, issuing books, and checking returns, which leads to delays in service. Manual data entry can also result in loss or misplacement of information, and there is no real-time tracking of books or members activity. These inefficiencies hinder the library's primary purpose of providing easy and quick access to information required for the library to function. To address this challenge, Mr R.B has approached us to initiate the semi-automation development of his library system.

Bedekar's library is a manual library since 1990. They purchase multiple copies of various books. They have IT staff, Library staff and registered users. They lend books for 2 weeks and track timelines. They have various manual reports to help their daily tasks and possible future projections.

2.1.1 Proposed idea

This project aims to develop a computerised/semi-automated **LMS** that will streamline the day-to-day activities of the library, automate repetitive tasks, and allow for more efficient data management.

Functionalities will be extracted from the consolidated interviews conducted and the observations done at the library site.

2.2 Description of the Current System

The current system in Bedekar's library is largely manual, relying heavily on paper-based records and simple spreadsheets. When a user borrows a book, the library staff logs the details into a register, noting down the borrower's details and the date it was issued. Similarly, when the book is returned, it is crossed off the register, and a fine is imposed if the book is overdue. The stock of books is maintained in a spreadsheet where details such as book title, author, category, availability etc are listed.

There is no digital database that consolidates all the records, making it difficult to track book circulation and user activity over time. This manual system is highly time-consuming and inefficient, especially during Easter time, Summer Holidays and Christmas times, when users borrow and return books in large numbers. Moreover, book reservations are done in person, and there is no online access to check the availability of books.

2.3 Interview

A few interviews were conducted to understand their perspective on the current system and identify the key pain points.

- With the Library owner
- With some Library staff
- With some frequent Member Users

2.3.1 Responses to interviews

Consolidated answers from Interviews with Mr R.B [Sole Owner of Bedekar's Library], Some Staff members and some Member users

Date of interview - 28/08/24 to 21/09/24
Place of interview - Owner's home (London E1), Library premises

QI - Can you give me some background information about your business?

- Manual library since 1990
- We buy books (multiple copies)
- We have IT staff, library staff and registered users
- We lend books for 2 weeks and track timelines
- We generate weekly manual reports
- We would like to have on demand automatic reports
- We would like to have an IT system managing our library

Q2 - What happens when you receive a request for a new user?

- Customer walks in with the required proof of address
- We ask for personal information document
- We verify the document to be the same person
- We issue a card (which has a unique card number) assigning it to the person in the system
- We activate the card and give a default password to the user. The card number acts as the username
- We update our user information system

Q3 - What happens when you receive a request for a book that is not in the library?

- We contact our suppliers
- We take the pricing quote
- We order a title and multiple copies of it depending on its demand to be delivered to the library
- We update our book inventory system

Q4 - What happens when you receive a request for an existing book, but the copies are already borrowed by other customers?

We apologise to the customer and inform them the next expected return date of a copy of that book

Q5 – How/When do you delete books from the system?

- Worn out
- New version available and the current version is obsolete
- If the customer is not contactable anymore and it's past 6 months

Q6 – What are the functions of the staff?

IT admin team

- Create staff logins
- Give staff permissions to staff logins
- Update and patch the IT system
- Provide and maintain self-service system within the library

Library staff

- receive the book copy and update the system (return)
- give the book copy and update the system (Issue)
- view the system for user queries
- take and process request for new customers
- generate reports
- Add, update, and remove books from the inventory
- Track inventory levels and suggest restocking

Q7 - How big is your library shelfing space? Is there a space concerns when buying more books?

Library is big enough and there are no physical shelf space concerns

Q8- What are your library times?

Monday-Saturday 9-5pm. Sunday 9-12pm.

Q9 – How many branches do you have?

Only one, but we are planning to increase the business in the next 2 years

2.3.2 Response summary of the consolidated interviews

Library Owner's comments: The owner expressed the need to expand the business but making it easier to handle operations across all staff functions.

Library Staff's (and Admin) comments: The library staff expressed frustration over the time it takes to manage day-to-day operations, especially during busy periods. Due to the manual nature of the system, tracking which member borrowed which book and when it's due is challenging. Additionally, the lack of an automated fine system means that fines are sometimes forgotten or miscalculated.

Member User's comments: Member users expressed a desire for an interface where they could check book availability. They also wanted convenience in terms of checking their borrowing history and fines.

Both, the library staff and member users highlighted the need for faster and more reliable management of library resources, especially as the library business grows and the number of users increases.

2.4 Current System (ISPO, DFD, Data Dictionary)

The main entities in the current system are the Users [Admin, Library staff, Members], Books and **LMS** Process.

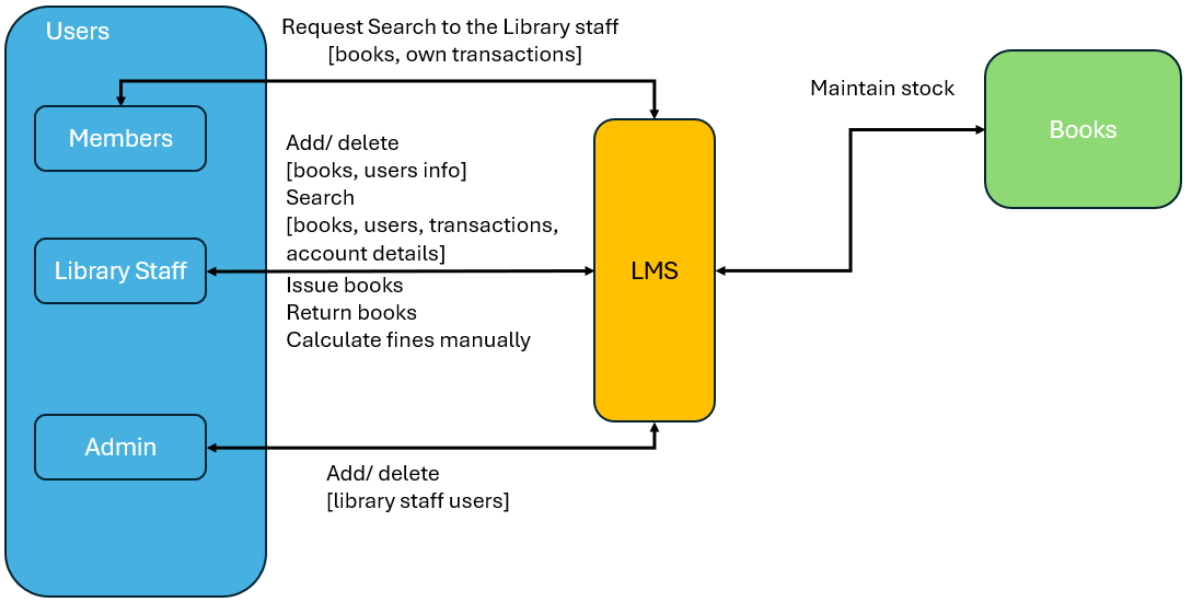
2.4.1 IPSO

IPSO	**Information**	**Evidence**
Input	Book information • Title • Author • Genre • Category • ISBN • Publisher • Price • Quantity • Shelf number • Library location • Edition	Interview, observation
Input	User information • Username • Password • First name • Middle name • Last name • Email • Mobile number • DOB • User type • Last login date • Address	Interview, observation
Input	Address information • Door number • Line 1 • Line 2 • City • Postcode	Interview, observation, government assigned addresses
Process	Generate report for issued books.	Interview
Process	Generate report for returned books.	Interview
Process	Calculate late fee. £2 for every 1-week delay	Interview, observation

Process	Calculate number of available copies of a book. Total – issued.	Interview, observation
Process	CRUD of users and books	Interview, observation
Store	Book information User information Late fee information Book issue information Book return information Library information	Interview
Output	Book information User information Late fee information Book issue information Book return information Issued books report Returned books report	Interview

2.4.2 Level 0 DFD (Context Diagram)

The Level 0 DFD will detail the key entities involved in the current manual/spreadsheet-based Library Management System



2.4.2.1 Entities

Library staff:

Provides book and member information to the system.
Receives book availability, overdue fine details, and borrower information from the system.

Inputs:
Member Details (for CRUD Users)
Book Details (for CRUD Books)
Issue/Return Details (when issuing or returning books)

Outputs:
User Information stored
Transaction Information stored (list of issues/borrowed books, due dates)
Overdue Fines (manual calculation for late returns)
Availability Status (checks book status)

Member:

Requests book issue or return.
Receives information on book availability and can see the confirmation of issue/return in the register.

Inputs:
Request book availability
Request to Borrow a Book
Request to Return a Book

Outputs:
Confirmation of transaction, due dates
Book Catalog and Availability Information

Admin staff:

Provides admin and Library staff information to the system.

Inputs:
Admin, Library staff Details (for CRUD Users)

Outputs:
Admin, Library staff Information

Books:

Updates book stock (availability) based on input from the library staff.

Inputs:
New Books are added to the system via spreadsheets or manually in the register
Book availability is updated by the library staff after issues and returns

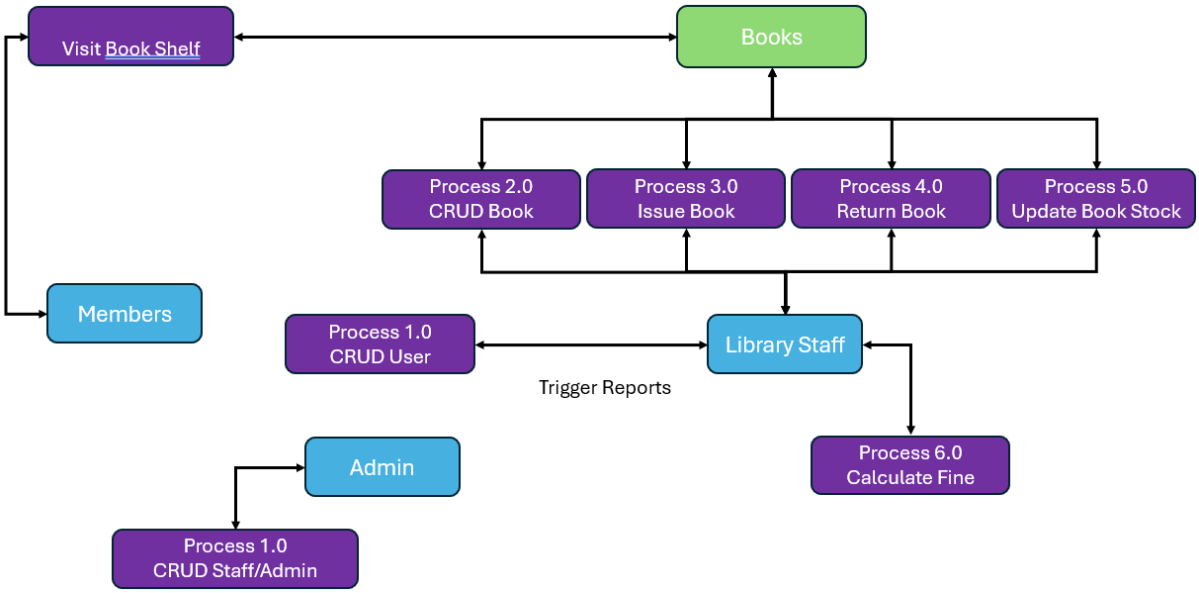
Outputs:
Status of the book (available/issued)

Process (Central System): The Library System (Manual/Spreadsheet):

Interacts with external entities (Users and Books).
Performs functions like maintaining all books information, maintaining all user information, issuing and returning books, maintaining book stock & availability, and recording fines.

2.4.3 Level 1 DFD: (Processes)

The Level 1 DFD will detail the key processes involved in the current manual/spreadsheet-based Library Management System



2.4.3.1 Processes

Below are the documented processes of the system.

Process 1.0: Create Read Update Delete [CRUD] Users

The library staff / admin [as per their roles] collects and record member/staff details (name, contact details etc) manually or in a spreadsheet.

Data Flow: Users provides details → Library staff/Admin records the information.
Inputs: Member User's details (ID, name, etc.).
Outputs: The user's information is CRUD and recorded by the library staff.

Process 2.0: Create Read Update Delete [CRUD] Books

The library staff manually CRUDs the book to a log (physical registers). Details include the book's title, author, genre, availability status etc.

Data Flow: Library staff inputs associated book information → Book information is updated accordingly to the registers.
Inputs: Book's details (ID, title, name, etc.).
Outputs: The book's information is CRUD and recorded by the library staff.

Process 3.0: Issue Book

When a user requests to borrow a book, the Library staff manually checks the availability by searching through the physical inventory or spreadsheets.
The Library staff records the transaction, updating the book's availability, and notes the issue date.

Data Flow: Users requests book → Library staff checks availability → Book issued.
Inputs: New book details (title, author, genre, etc.).
Outputs: The book is added to the library's inventory manually.

Process 4.0: Return Book

The user returns the book to the library. The library staff checks if the book is overdue and updates the transaction log.
If the book is overdue, the library staff calculates the fine manually.
Data Flow: Users returns the book → Library staff checks return date → Fine (if applicable) calculated manually.
Inputs: User requests book.
Outputs: The book is issued to the member, and the library staff updates the system to reflect that the book is available/unavailable as per remaining numbers.

Process 5.0: Update Book Stock/Inventory

After each transaction (issue or return), the library staff updates the stock/inventory to reflect the availability of books. This process ensures that the status of each book (available/issued) is up-to-date in the system.
Data Flow: Issue/Return information → Stock/Inventory updated.
Inputs: Issue/Return transaction.
Outputs: The inventory is updated to reflect the current status of the book (available/unavailable).

Process 6.0: Calculate Fine

When a book is returned late, the library staff manually calculates the fine based on the number of overdue days and the rate per day. This information is recorded manually or in a spreadsheet.
Data Flow: Return date exceeds due date → Fine calculated → User notified.
Inputs: Book returned late.
Outputs: A fine is calculated manually based on the number of overdue days, and the member is informed of the amount.

2.4.4 Data Dictionary

Data Dictionary for the Level 1 DFD

Process	Input	Output	Description
CRUD User	User details (ID, name, etc.)	User CRUD'ed	CRUD operations on Users saved to register
CRUD Book	Book details (title, author, etc.)	Book CRUD'ed	CRUD operations on Books saved to register
Issue Book	Borrow request, user ID, book ID	Book issued, due date set	Issues a book to the member and updates availability
Return Book	Return request, book ID, user ID	Book returned, fine calculated (if overdue)	Processes book returns and calculates overdue fines
Update Inventory	Issue/return details	Stock/inventory updated	Updates the stock/inventory after book issue/return
Calculate Fine	Due date, return date	Fine amount	Manually calculates the fine for overdue returns

2.5 Problem with the Current System

The current system suffers from multiple inefficiencies:

1. **Time-consuming processes**: Each book transaction is logged manually, resulting in delays and errors, especially during high-traffic periods.
2. **Human error**: Handwritten logs or spreadsheet entries can be easily miswritten, leading to confusion regarding book availability, overdue fines, or lost inventory.
3. **Lack of real-time tracking**: There is no easy way to track which books are available or who has borrowed them without manually combing through the records.
4. **No online access**: Members must visit the library physically to check for book availability, leading to inconvenience.
5. **Difficulty in tracking fines**: Fines must be calculated manually, which often leads to disputes or delays in fine collection.
6. **Inventory management**: As the collection of books grows, it becomes increasingly difficult to maintain an accurate record of the library's holdings using a basic spreadsheet.

2.6 Similar Systems

Many libraries, especially larger ones, use automated **Library Management Systems** to handle their daily operations. I have researched some existing similar popular library product systems like **IdeaStore (**from Idea library store at Whitechapel), **Koha**, **Evergreen**, and **LibSys**. These systems allow for seamless book management, user registration, and online catalogs. Features include automatic fine calculation, reservation systems, online access to catalogues, and automated notifications when books are due. However, these systems are often costly and require a higher level of IT infrastructure, which may not be feasible for smaller or budget-limited libraries like Bedekar's Library for which I will be helping to automate.

2.7 Proposed Solution

The proposed solution is to design and implement a new **LMS** that semi-automates processes such as book issue, return, and fine calculation. The system will use a relational database to store information on books, users, and transactions. It will provide an easy-to-use interface for both library staff and general users (via in house library PCs), where the library staff can manage the inventory and user transactions, and the users can independently check book availability, and view their account info.

The system will also incorporate an automated fine calculation system.

Key features include:

1. **Book management** to maintain books and bookInfos
2. **User management** to maintain users
3. **Transaction management** to maintain transaction history & fines in real-time.
4. **Automatic fine calculation** based on overdue days, as of the day it is checked and/or returned.
5. **Report Generation** for tracking business and predicting future trends.

To decide the programming language to use for an API solution, I researched two main languages that support microservices architecture for API development.

Python	Java
+ Already have experience and knowledge in the core language. - No knowledge of FastAPI, Flask, Django frameworks used for API development.	+ Easy to use + Relatively easy to learn + Can run on the operating system of **LMS** + Easy integration with third party Spring Boot used for API development. + Spring Boot provides features to reduce lines of code, making it concise and efficient programming. - Little knowledge of Java. - No knowledge of Spring Boot.

I have chosen to use Java with Spring Boot so I can learn a new technology, while implementing **LMS**. I do not necessarily need to create a **Graphical User Interface (GUI)** because the API calls with input and output can be verified using the Postman tool, which acts as a **GUI** equivalent. If time permits, I will try to create basic UI screens, but for now, that is optional for my project scope.

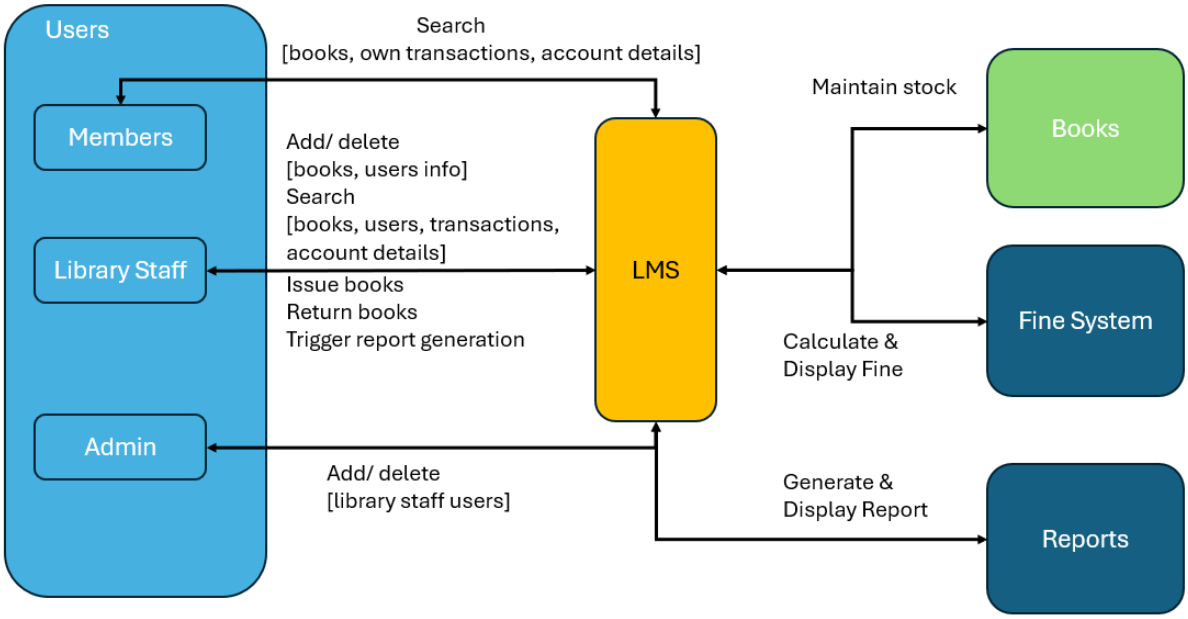
I will have to use a database platform to maintain the data required for **LMS**. There are multiple platforms available that all appear similar however, some have their major draw backs.

MySQL	Oracle
+ Most popular open-source database. + Can be installed on the operating system of **LMS.** + Is supported by Java and Spring Boot + Provides libraries for integrating the data with Java objects. + Uses standard SQL statements which I am already aware of. + Easy to use. - Little knowledge of MySQL DB.	+ Can be installed on the operating system of **LMS.** + Is supported by Java and Spring Boot + Provides libraries for integrating the data with Java objects. + Uses standard SQL statements which I am already aware of. - No knowledge of Oracle. - Not free but needs licence.

I will be using MySQL DB as my database. I have chosen this for easy of development of a smaller project like **LMS**.

2.8 Proposed System (DFD, Data Dictionary)

2.8.1 Level 0 DFD (Context Diagram)



All the entities and associated information from current system apply here in context of automation rather than manual processing. To avoid duplication, I have not re written it. In addition, the below will be done by the new system

Automatically Calculate fine:

Updates the fine for overdue books based on the book return date. **Inputs**: Book return date **Outputs**: Fine calculated as of the view day

Generate reports:

Generate scheduled and manual reports to show various stats across books, Users and Transactions. **Inputs**: Book information User information Transaction information **Outputs**: Online reports generated and shown on screen and scheduled reports stored on disk

Additionally, Member users can do the following (from the PC/self-serve screen in the library)

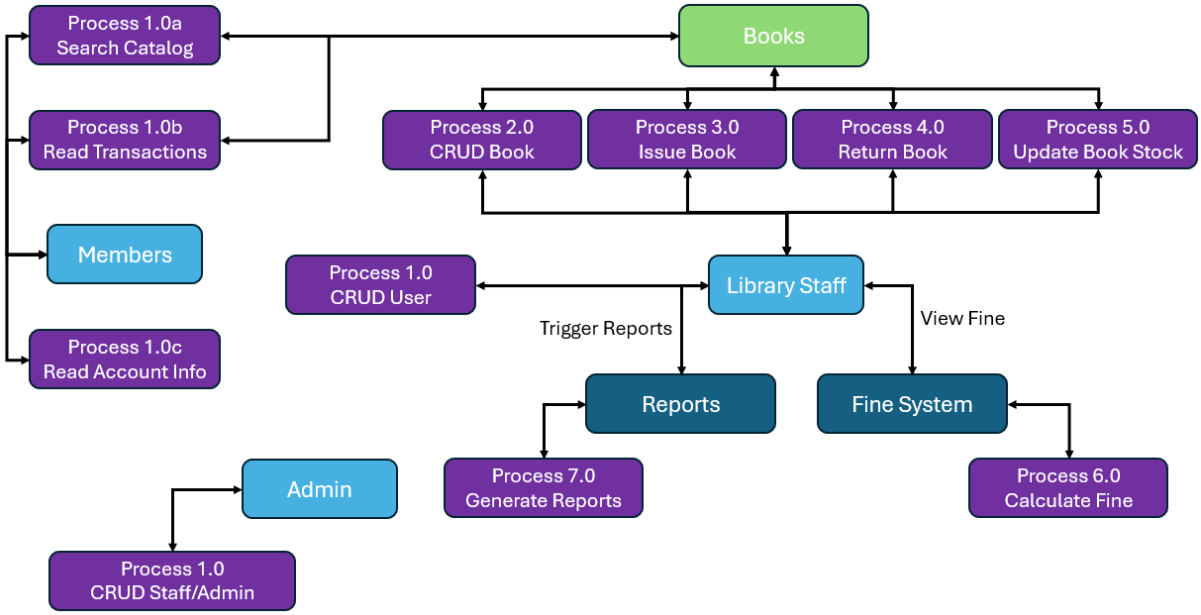
Member Users:

Receives information on book availability and can see their confirmation of issue/return in the system.

Inputs:
Request to see (self) account details
Request to see (self) transaction details
Request to see book catalog and availability

Outputs:
Account Details
Issuing/Borrowing Information (confirmation of transaction, due dates)
Book Catalog and Availability Information

2.8.2 Level 1 DFD: (Processes)



All the processes and associated information from current system apply here in context of automation rather than manual processing. To avoid duplication, I have not re written it. In addition, the below will be done by the new system

Process 7.0: Generate Reports

The library staff can request the system to automatically generate online report in real time.
Data Flow: Library staff triggers report menu and provides report type and input details→ Online report generated → Report displayed on screen (and can be manually saved in pdf format and stored/emailed).

Inputs:

Report type and input details.

Outputs:
Online report is displayed.

2.8.3 Data Dictionary

Data Dictionary for the Level 1 DFD

All the processes and associated information from current MANUAL system apply here in context of automation rather than manual processing. To avoid duplication, I have not re written it. In addition, the below will be done by the new system

Process	Input	Output	Description
Generate report	Report type, details	Online report displayed. Location link shared of the scheduled report.	Online report displayed on screen. Location link shared of the scheduled report.

2.9 Acceptable Limitations

Despite the benefits, there are certain limitations to the system. It may not be feasible to include advanced features like integration with external databases or mobile app access in the initial stages due to time and resource constraints. The system is also not intended for Day1 to handle multiple branches of a library, meaning it is most suitable for single-location library. In future the solution can be expanded to multiple locations. Data loss due to hardware failures, cyber-attacks while mitigated by backup mechanisms, could still be a potential risk.

2.10 Prospective Users

The prospective users for this project are those who will be using it:

- **Admin Staff**: They will use the system having access to full system.
- **Library Staff**: CRUD users, CRUD books, CRUD transactions (i.e issue/return books) etc.
- **Member User**: View book availability, , update their own details, view their own details/transactions/fines (if any), check the availability of books etc.

2.11 Set of Objectives (Traceability matrix)

The needs of the project are for creating an IT system managing our library to automate the core functions as listed below:

- Automate the library's book issue and return process.
- Provide real-time tracking of book availability.
- Enable access to the library's catalog and user accounts.
- Automate fine calculations and manually notify users (by email) of long overdue books.
- Simplify the management of user records and borrowing history.
- Minimize human error and reduce manual labour for the library staff.
- Provide a scalable system that can handle a growing library inventory.
- Generate on demand automatic reports for various library functions (e.g view users, view books, view issued books, view returned books, view books due for return etc.)

Below is a matrix to identify each functional requirement. These are broken down into measurable success criteria.

2.11.1Success Criteria

	#	Main Objective	Explanation	Success Criteria	Priority
1	LMS_U_001	Add User	The user of LMS will be added to the system with all the associated information required for the user to perform their functions.	1. User should not be created with a userId that was previously allocated to another user. 2. All mandatory fields should be validated. 3. A success message should be shown when the User is Successfully added 4. An error message should be shown when the User is not Successfully added 5. This functionality should not be available to Member user	HIGH
2	LMS_U_002	View a user by Id OR via Name pattern match condition	The user of LMS can be searched in the system to get all information about the user.	1. User should be searched by searching the userId 2. User should be searched by searching the user's name pattern which will search across the user's First name, Middle Name and Last Name. 3. A list of users should be displayed for all users matching the search criteria. 4. The user should be able to select any user from the list of	HIGH

				users to see the full details of that selected user. 5. When there is only user searched, the system should be able to auto select the user to see the full details of that user. 6. The selected user's address details should also be displayed 7. An appropriate error message should be shown when the User is not Successfully searched 8. This functionality should not be available to Member user to see details of other users 9. This functionality should be available to Member user to see only personal details and amend it at the same time	
3	LMS_U_003	Update a user	An existing user of LMS can be updated in the system to reflect a change of associated user's data.	1. User should first be searched by searching the userId to verify it is an existing user. 2. An appropriate error message should be shown when the User is not Successfully searched. 3. When the search is Successful, the user details should be retrieved and displayed to the user, giving an option to update only limited fields allowed for	HIGH

				update (email, mobile) via the LMS app. 4. A success message should be shown when the User is Successfully updated 5. An appropriate error message should be shown when the User is not Successfully updated 6. This functionality should not be available to Member user to see details of other details 7. This functionality should be available to Member user to see only personal details and amend it at the same time	
4	LMS_U_004	Delete a user	An existing user of LMS can be deleted to remove a user from the system.	1. User should first be searched by searching the userId to verify it is an existing user. 2. An appropriate error message should be shown when the User is not Successfully searched. 3. When the search is Successful, the user details should be retrieved and displayed to the user, who can then confirm the deletion of that user. 4. A User cannot be deleted if the user has one or more issued books and not yet returned.	HIGH

				5. A User cannot be deleted if the user has one or more past transactions done as this will require admin to first remove the data integrity dependency. 6. An appropriate error message should be shown when the User is not Successfully deleted 7. This functionality should not be available to Member user	
5	LMS_BI_001	Add bookInfo	The BookInfo of LMS will be added to the system with all the associated information required.	1. BookInfo should not be created with a bookInfoId that was previously allocated to previous book. 2. All mandatory fields should be validated. 3. A success message should be shown when a new BookInfo is Successfully added 4. An error message should be shown when the BookInfo is not Successfully added 5. This functionality should not be available to Member user	HIGH
6	LMS_BI_002	View a bookInfo by Id OR via Title pattern match condition	The BookInfo of LMS can be searched in the system to get all information about the BookInfo.	1. BookInfo should be searched by searching the bookInfoId. 2. BookInfo should be searched by searching the BookInfo's Title pattern which will search across the Title.	HIGH

				3. A list of BookInfos should be displayed for all BookInfos matching the search criteria. 4. The user should be able to select any BookInfo from the list of BookInfos to see the full details of that selected BookInfo. 5. When there is only BookInfo searched, the system should be able to auto select the BookInfo to see the full details of that BookInfo. 6. An appropriate error message should be shown when the BookInfo is not Successfully searched	
7	LMS_BI_003	Update a bookInfo	An existing BookInfo of LMS can be updated in the system to reflect a change of associated BookInfo's data.	1. BookInfo should first be searched by searching the bookInfoId, to verify it is an existing BookInfo. 2. An appropriate error message should be shown when the BookInfo is not Successfully searched. 3. When the search is Successful, the BookInfo details should be retrieved and displayed to the user, giving an option to update the fields.	HIGH

				4. A success message should be shown when the BookInfo is Successfully updated 5. An appropriate error message should be shown when the BookInfo is not Successfully updated 6. This functionality should not be available to Member user	
8	LMS_BI_004	Delete a bookInfo	An existing BookInfo of LMS can be deleted to remove the BookInfo from the system.	1. BookInfo should first be searched by searching the bookInfoId to verify it is an existing BookInfo. 2. An appropriate error message should be shown when the BookInfo is not Successfully searched. 3. When the search is Successful, the BookInfo details should be retrieved and displayed to the user, who can then confirm the deletion of that BookInfo. 4. A BookInfo cannot be deleted if there are Book(s) against it in the database. 5. An appropriate error message should be shown when the User is not Successfully deleted 6. This functionality should not be available to Member user	HIGH

9	LMS_B_001	Add a book	The Book of LMS will be added to the system with all the associated information required.	1. Book should not be created with a bookId that was previously allocated to another Book. 2. All mandatory fields should be validated. 3. A success message should be shown when the Book is Successfully added 4. When a Book is added, the number of books in its associated BookInfo should increment by 1. 5. An error message should be shown when the Book is not Successfully added 6. This functionality should not be available to Member user	HIGH
10	LMS_B_002	View a book by Id	The Book of LMS can be searched in the system to get all information about the Book.	1. Book should be searched by searching the bookId. 2. Book should be searched by searching the associated bookInfoId. 3. A list of Books should be displayed for all BookInfos matching the search criteria. 4. The user should be able to select any book from the list of books to see the full details of that selected book. 5. When there is only book searched, the system should be	HIGH

				able to auto select the book to see the full details of that book. 6. The selected book's BookInfo details should also be displayed 7. An appropriate error message should be shown when the Book is not Successfully searched	
11	LMS_B_003	Update a book	An existing Book of LMS can be updated in the system to reflect a change of associated Book's data.	1. Book should first be searched by searching the bookId to verify it is an existing book. 2. An appropriate error message should be shown when the Book is not Successfully searched. 3. When the search is Successful, the Book details should be displayed to the user, giving an option to update only limited fields (Shelf Reference, Location, Edition) allowed for update via the LMS app. 4. A success message should be shown when the Book is Successfully updated 5. An appropriate error message should be shown when the Book is not Successfully updated 6. This functionality should not be available to Member user	HIGH

12	LMS_B_004	Delete a book	An existing Book of LMS can be deleted to remove the Book from the system.	1. Book should first be searched by searching the bookId to verify it is an existing Book. 2. An appropriate error message should be shown when the Book is not Successfully searched. 3. When the search is Successful, the Book details should be retrieved and displayed to the user, who can then confirm the deletion of that Book. 4. A Book cannot be deleted if it is currently borrowed by a user and not yet returned. 5. A Book cannot be deleted if it has historical transactions (to maintain referential integrity). 6. An appropriate error message should be shown when the User is not Successfully deleted 7. When a Book is deleted, the number of books in its associated BookInfo should decrement by 1. 6. This functionality should not be available to Member user	HIGH
13	LMS_T_001	Issue book (Add Transaction)	The Transaction (i.e Book Issue event) of LMS will be added to the system with all the associated information required.	1. Transaction should not be created with a transactionId that was previously allocated to another Transaction.	HIGH

				2. All mandatory fields should be validated. 3. Verify User Exists and if not, then show error message 4. Verify Book Exists and if not, then show error message 5. A transaction cannot be added if the associated user already has 3 non-returned, borrowed books 6. A transaction cannot be added if the associated book is already borrowed and not yet marked as returned 7. Issue date should be auto assigned as of current date. 8. Return date should be auto calculated as of 2 weeks. 9. Mark the associated book to be unavailable for borrowing 10. A success message should be shown when the Transaction is Successfully added (i.e a Book successfully issued to a User) 11. An error message should be shown when the Transaction is not Successfully added 12. An error message should be shown when the Book is not Successfully updated	

				13. This functionality should not be available to Member user	
14	LMS_T_002	Return book (Update Transaction)	An existing Transaction of LMS can be updated in the system to reflect the return of an issued Book.	1. Transaction should first be searched by searching the transactionId to verify it is an existing Transaction or by its associated userId or by its associated bookId. 2. An appropriate error message should be shown when the Transaction is not Successfully searched. 3. When the search is Successful, the Transaction details (i.e Book Issue details) should be displayed to the user, giving an option to update the Transaction (i.e to mark the Book as returned). 4. Fine should be calculated if the return has gone beyond the Return Date. 5. A success message should be shown when the Transaction is Successfully updated (i.e The Book is marked as returned). 6. Actual Return date should be auto updated for the current date. 7. Fine should be auto updated with the calculated Fine.	HIGH

				8. Mark the associated book to be now available for borrowing 9. An appropriate error message should be shown when the Book is not Successfully updated 10. This functionality should not be available to Member user	
15	LMS_T_003	View a transaction by Id OR by associated User or by associated Book match condition	The Transaction of LMS can be searched in the system to get all information about the Transaction.	1. Transaction should be searched by searching the transactionId 2. Transaction should be searched by searching userId associated to the Transaction, either for all associated Transactions or when the Transaction is still open (i.e the issued book is not yet returned). 3. Transaction should be searched by searching bookId associated to the Transaction, either for all associated Transactions or when the Transaction is still open (i.e the issued book is not yet returned). 4. A list of Transactions should be displayed for all Transactions matching the search criteria. 5. The user should be able to select any Transaction from the	HIGH

				list of Transactions to see the full details of that selected book.	
				6. When there is only Transaction searched, the system should be able to auto select the Transaction to see the full details of that Transaction.	
				7. The fine should be auto calculated for each transaction details displayed based on the rule that it will be £2 for every 1-week delay. The delay is calculated for days between the current date and the scheduled returnDate of that transaction	
				8. The selected Transaction's User details should also be displayed.	
				9. That user's address details should also be displayed.	
				10. The selected Transaction's Book details should also be displayed.	
				11. That book's BookInfo details should also be displayed.	
				12. An appropriate error message should be shown when the Transaction is not Successfully searched	
				13. This functionality should be available to Member user to	

				search only their transaction details	
16	LMS_RI_001	Add reportInfo	The ReportInfo of LMS will be added to the system with all the associated information required.	1. ReportInfo should not be created with a reportInfoId that was previously allocated to previous ReportInfo. 2. All mandatory fields should be validated. 3. A success message should be shown when a new ReportInfo is Successfully added 4. An error message should be shown when the ReportInfo is not Successfully added 5. This functionality should not be available to Staff and Member user	HIGH
17	LMS_RI_002	View a reportInfo by Id OR via name pattern match condition	The ReportInfo of LMS can be searched in the system to get all information about the ReportInfo.	1. ReportInfo should be searched by searching the reportInfoId. 2. ReportInfo should be searched by searching the ReportInfo's Name pattern which will search across the Name. 3. A list of ReportInfos should be displayed for all ReportInfos matching the search criteria. 4. The user should be able to select any ReportInfo from the list of ReportInfos to see the full	HIGH

				details of that selected ReportInfo. 5. When there is only ReportInfo searched, the system should be able to auto select the ReportInfo to see the full details of that ReportInfo. 6. An appropriate error message should be shown when the ReportInfo is not Successfully searched 7. This functionality should not be available to Staff and Member user	
18	LMS_RI_003	Update a reportInfo	An existing ReportInfo of LMS can be updated in the system to reflect a change of associated ReportInfo's data.	1. ReportInfo should first be searched by searching the reportInfoId to verify it is an existing ReportInfo. 2. An appropriate error message should be shown when the ReportInfo is not Successfully searched. 3. When the search is Successful, the ReportInfo details should be retrieved and displayed to the user, giving an option to update the field (only the generation time field).	HIGH

				4. A success message should be shown when the ReportInfo is Successfully updated	
				5. An appropriate error message should be shown when the ReportInfo is not Successfully updated	
				6. This functionality should not be available to Staff and Member user	
9	LMS_RI_004	Delete a reportInfo	An existing ReportInfo of LMS can be deleted to remove the ReportInfo from the system.	1. ReportInfo should first be searched by searching the reportInfoId to verify it is an existing ReportInfo.	HIGH
				2. An appropriate error message should be shown when the ReportInfo is not Successfully searched.	
				3. When the search is Successful, the ReportInfo details should be retrieved and displayed to the user, who can then confirm the deletion of that ReportInfo.	
				4. A ReportInfo cannot be deleted if there are Report(s) against it in the database.	
				5. An appropriate error message should be shown when the User is not Successfully deleted	

				6. This functionality should not be available to Staff and Member user	
20	LMS_R_001	Add a report	The Report of LMS will be added to the system with all the associated information required.	1. Report should not be created with a reportId that was previously allocated to another Report. 2. All mandatory fields should be validated. 3. A success message should be shown when the Report is Successfully (manually) added 4. An error message should be shown when the Report is not Successfully (manually) added 5. This functionality should not be available to Staff and Member user	HIGH
21	LMS_R_002	View a report by Id	The Report of LMS can be searched in the system to get all information about the Report.	1. Report should be searched by searching the reportId. 2. Report should be searched by searching the associated reportInfoId. 3. A list of Reports should be displayed for all ReportInfos matching the search criteria. 4. The user should be able to select any report from the list of reports to see the full details of that selected report.	HIGH

				5. When there is only report searched, the system should be able to auto select the report to see the full details of that report. 6. The selected report's ReportInfo details should also be displayed 7. An appropriate error message should be shown when the Report is not Successfully searched 8. This functionality should not be available to Staff and Member user	
22	LMS_S_001	Authenticate user for login	An existing User of LMS will be checked in the system to verify if they can use the system.	1. User should be presented with an option to input user Id and password. 2. Password should be encrypted before transmitting it over the network. 3. User authentication details should be checked against the User's data stored in the DB. 4. Users Firstname and Surname Initials, User's ID and User's Type should be returned when the User is Successfully searched 5. An appropriate error message should be shown when the User is not Successfully searched	HIGH

				6. An appropriate error message should be shown when the User provides an incorrect password	
23	LMS_S_002	Authorise user for functions	An existing User's type will be checked in the system to provide access to specific menu items in the system.	1. Once authenticated, User should be presented with only those menu options for which the User is authorised to access. 2. Admin User will have access to all menu items in the system. 3. Staff User will have access to all menu items in the system except the ReportInfo and Report items. 4. Member User will have access to very limited menu items in the system. They can only read their own details, update some of their details (Password, mobile number and email address), search and read a BookInfo, search and read a Book and search and read their own transaction history.	HIGH
24	LMS_H_001	Housekeeping logs	Log Rotation	Rotate logs and zip	HIGH
25	LMS_H_002	Housekeeping data			Optional
26	LMS_UUI_001	GUI Screens	User Screens	1. Add a User 2. Search and read a User 3. Search and update a User	HIGH

				4. Search and delete a User 5. Update self's User details	
27	LMS_BIUI_001	GUI Screens	BookInfo Screens	1. Add a BookInfo 2. Search and read a BookInfo 3. Search and update a BookInfo 4. Search and delete a BookInfo	HIGH
28	LMS_BUI_001	GUI Screens	Book Screens	1. Add a Book 2. Search and read a Book 3. Search and update a Book 4. Search and delete a Book	HIGH
29	LMS_TUI_001	GUI Screens	Transaction Screens (Issue and Return Book)	1. Add a Transaction 2. Search and read a Transaction 3. Search and update a Transaction 4. Search self's Transactions	HIGH
30	LMS_RIUI_001	GUI Screens	ReportInfo Screens	1. Add a ReportInfo 2. Search and read a ReportInfo 3. Search and update a ReportInfo 4. Search and delete a ReportInfo	HIGH
31	LMS_RUI_001	GUI Screens	Report Screens	1. Add a Report 2. Search and read a Report	HIGH

2.11.1.1 NFR (Non Functional Requirements)

Non-functional requirements (NFRs) are a set of specifications that describe a system's qualities and characteristics, rather than its specific functions. They are used to guide the design of a solution and can act as constraints on the system.

Logging

In programming, logging refers to the process of recording or storing information or messages that help in understanding the behaviour and execution of a program. It is a way to track the flow of the program and identify any issues or bugs.

I will be logging the application logs that can help track and help debug the application.

Application Logs

On the server, it will be kept for 4 weeks and then will be transferred to backup.

The library retains the backup for last 5 years.

User Data Storage

User data must be kept on systems according to the local GDPR rules.

The **LMS** system will backup user data (as part of UK GDPR) for the last 2 years after the last system usage of that member. This is an optional feature for this A Level project.

Housekeeping

The Log housekeeping is essential to free the space of old and unused log files. This can be achieved in the following ways: Log rotation settings. Setting log rotation for the individual log files that are collected from different components of **LMS** system.

Housekeeping of logs will be done in accordance with the library's retention guidelines.

The Housekeeping of logs is an optional feature for this A Level project.

3 Design

3.1 Overall system design

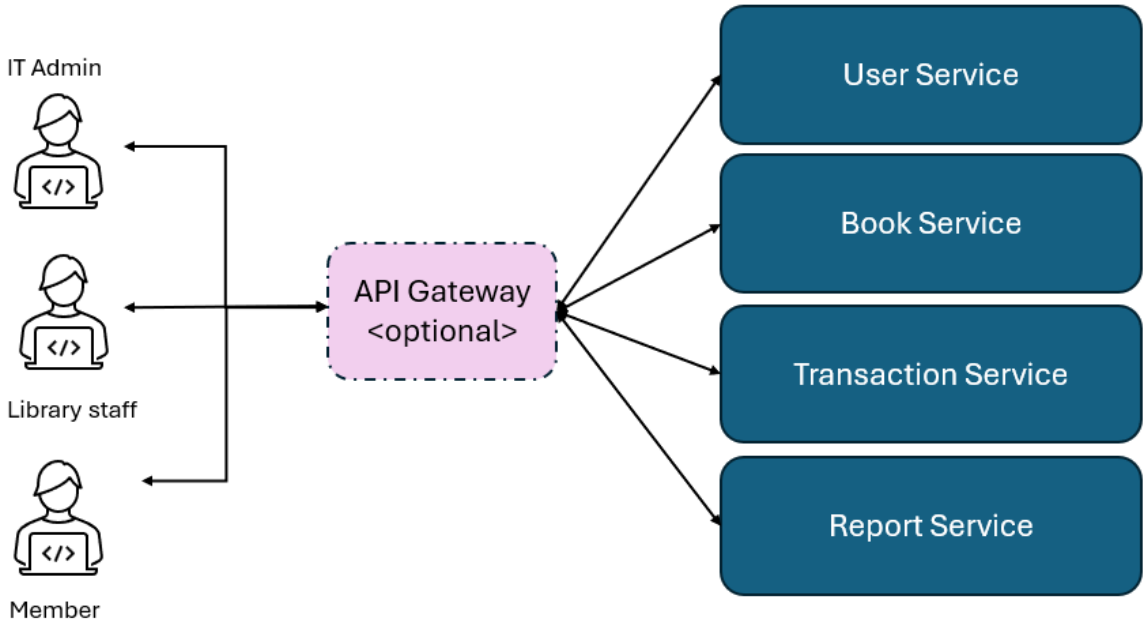
The overall design of the system will include the logical and the physical design. This will cover all aspects of designing required for **LMS** design.

3.1.1 Logical System Diagram

The below information gives a logical view around how the new system will be designed.

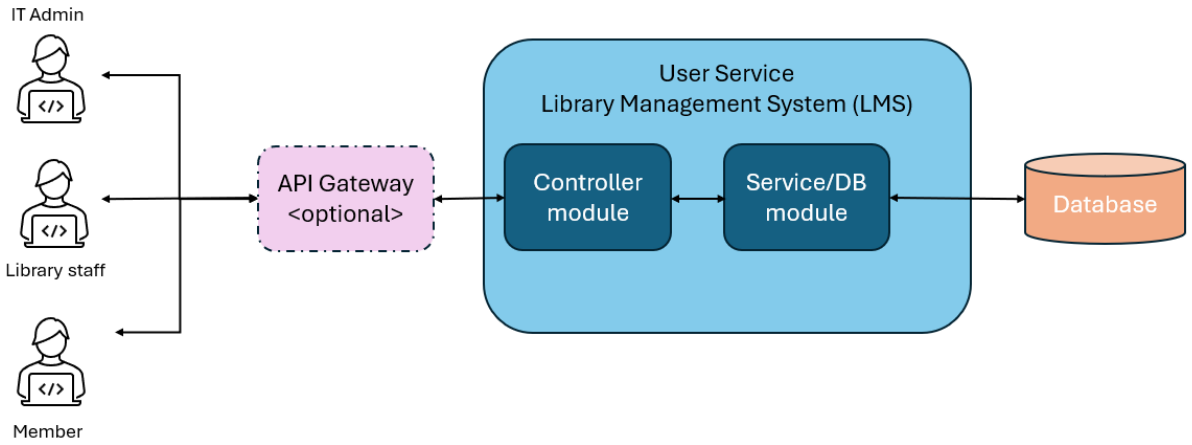
- **Users** of the system with roles across Admin, library staff and Member users
- **Controller module** will help interact with human/API interaction and the process (business) module.
- **Service module** will help to establish connection to the Database and perform CRUD and other DB operations on data from the database.
- **Database** on which data will be saved.
- **Gateway** to sit between the Users and actual services for users to reach only one service from which all other sub services will be reached out to. **This is optional** and will be developed only if time permits.

3.1.1.1 Overall System view



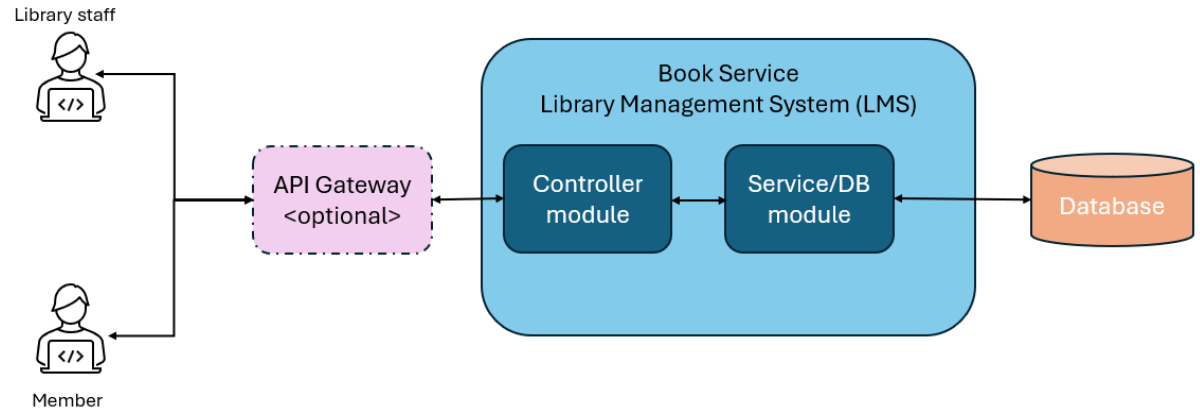
This represents the overall view of the system connections. The actual access/authorisation will be covered in more details in its relevant sections.

3.1.1.2 User Service



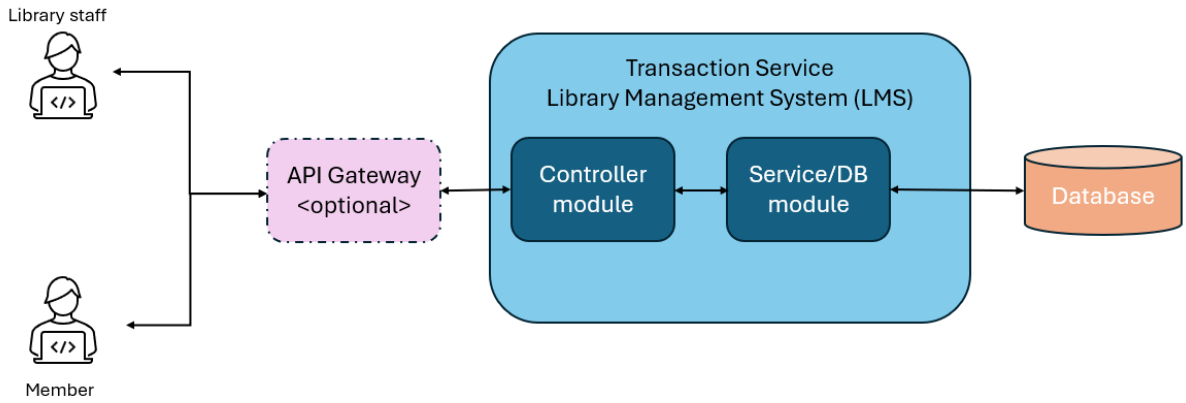
This service will be used for all CRUD operations on maintenance of Users

3.1.1.3 Book Service



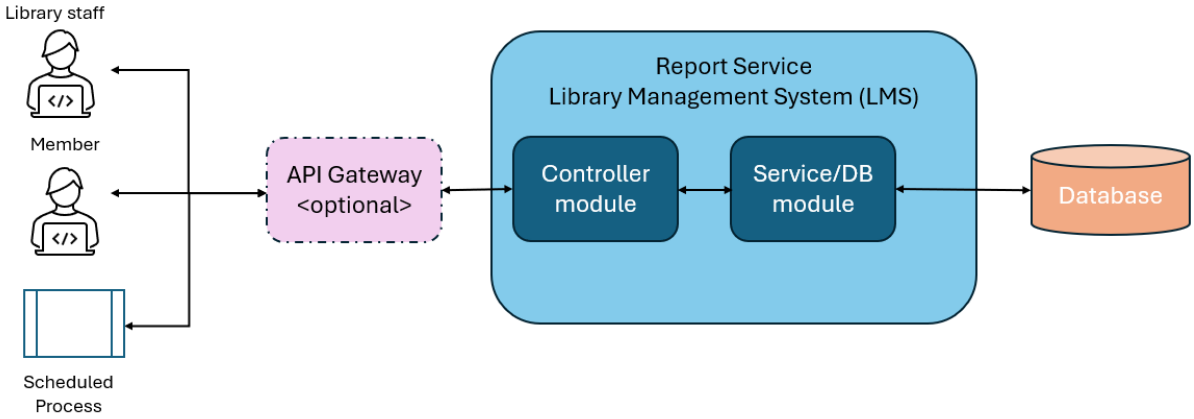
This service will be used for all CRUD operations on maintenance of BookInfos and Books.

3.1.1.4 Transaction Service



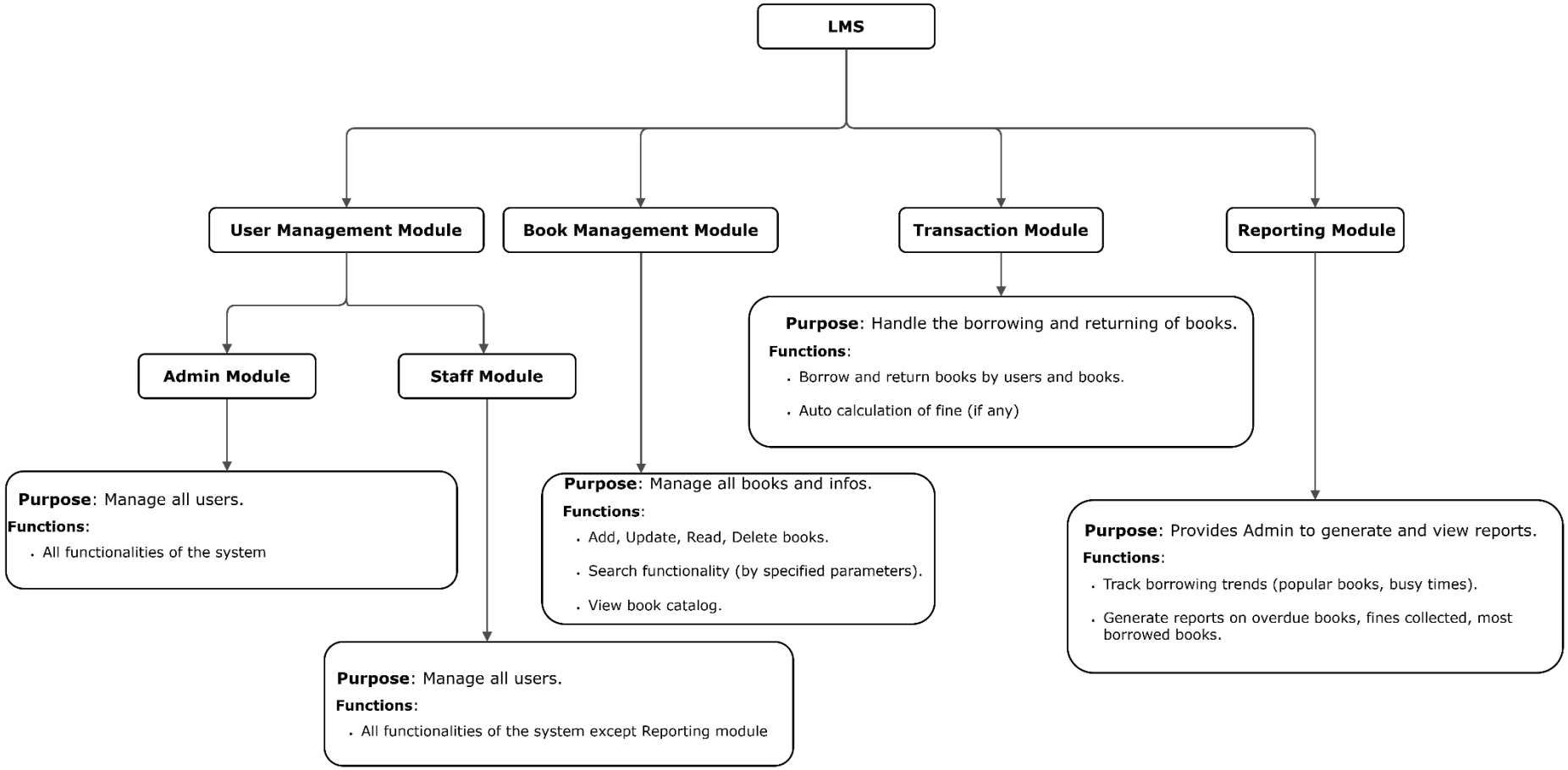
This service will be used for all CRU operations on maintenance of transactions and fine calculations.

3.1.2 Report Module



This service will be used for all report definitions and generations.

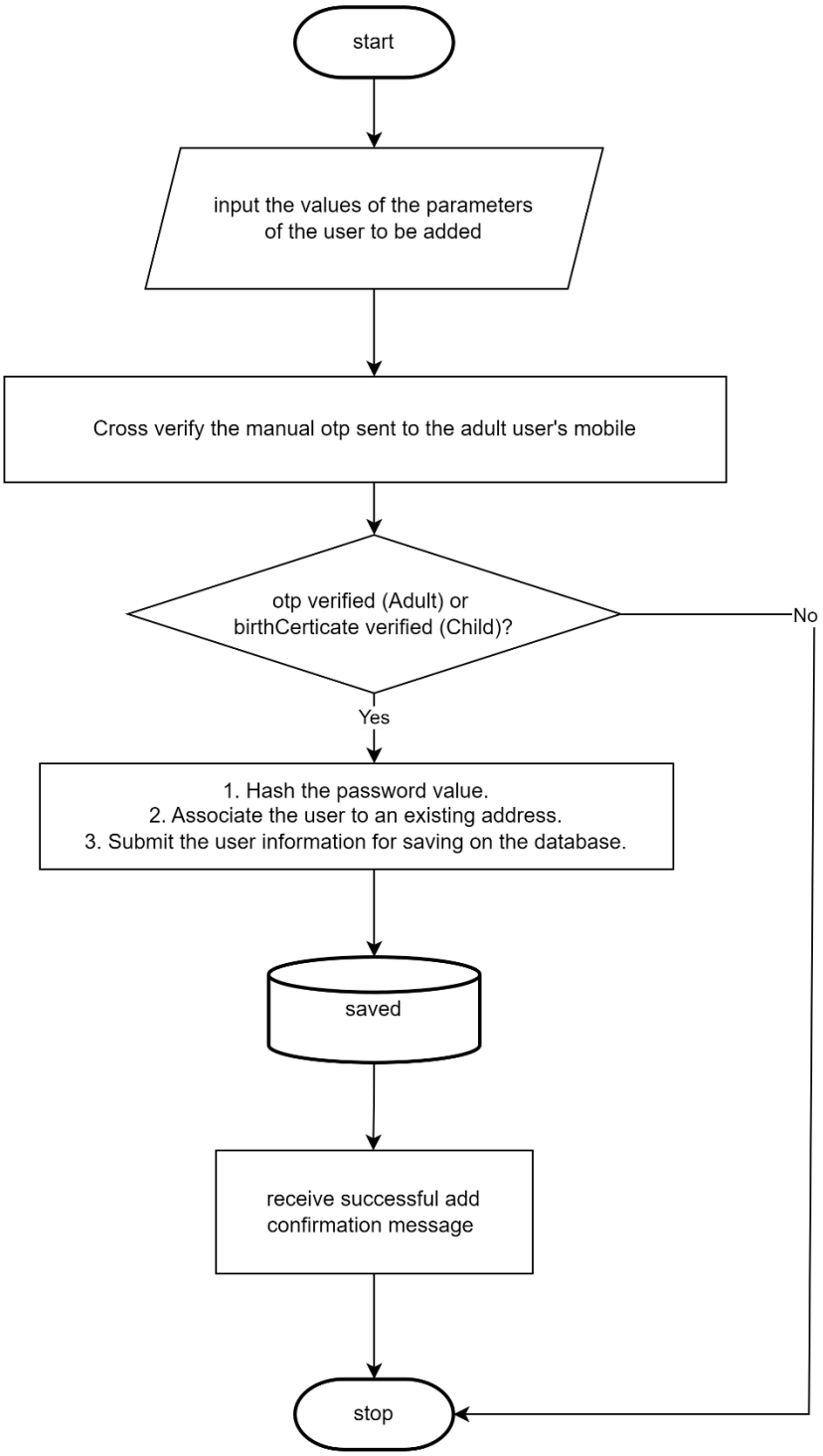
3.2 Modular structure



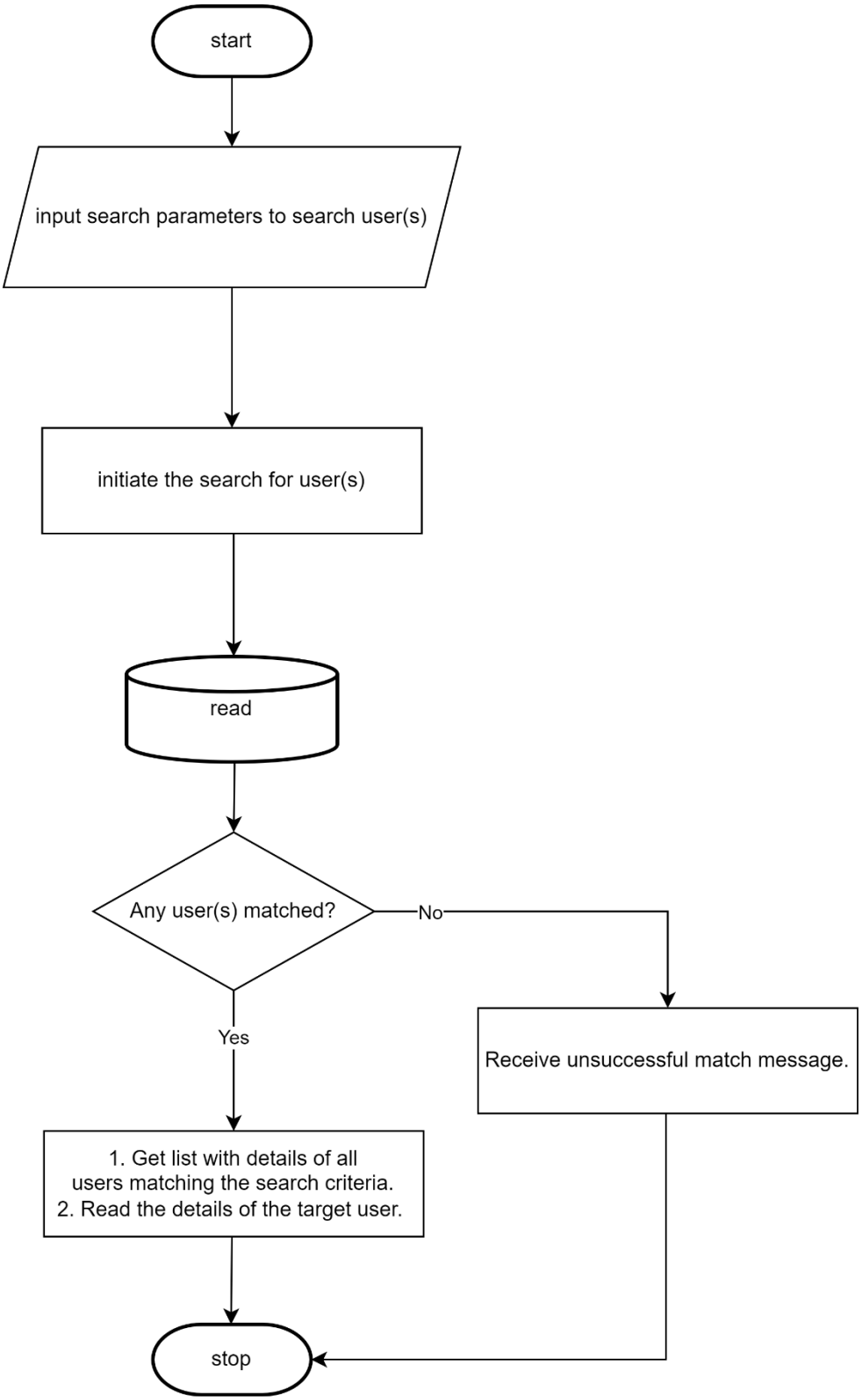
This depicts the modular structure of the whole application. I have kept it to only the diagram and not the explanation as it will be a repetition from other sections.

3.3 System flowchart

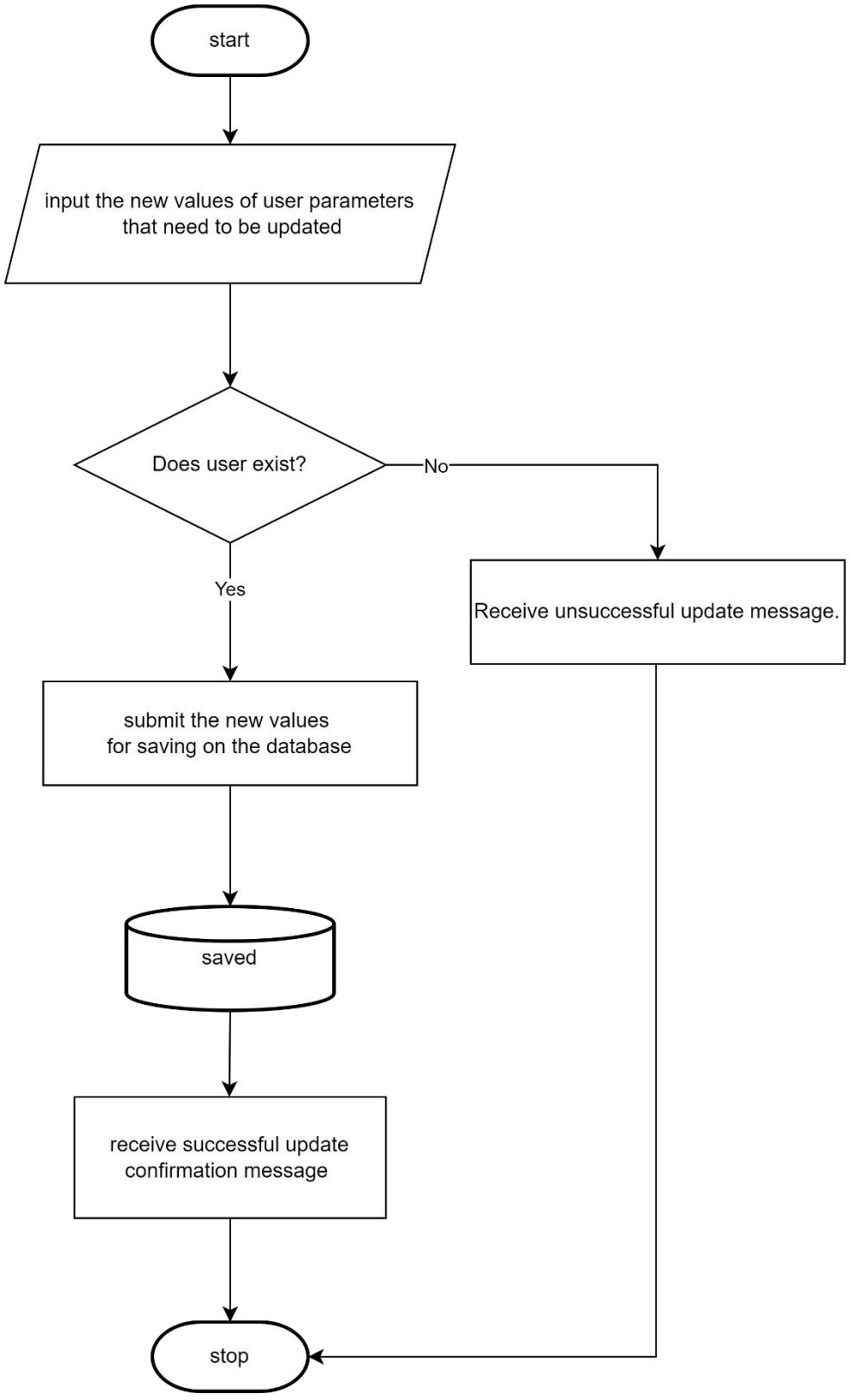
3.3.1 Create/Add a user



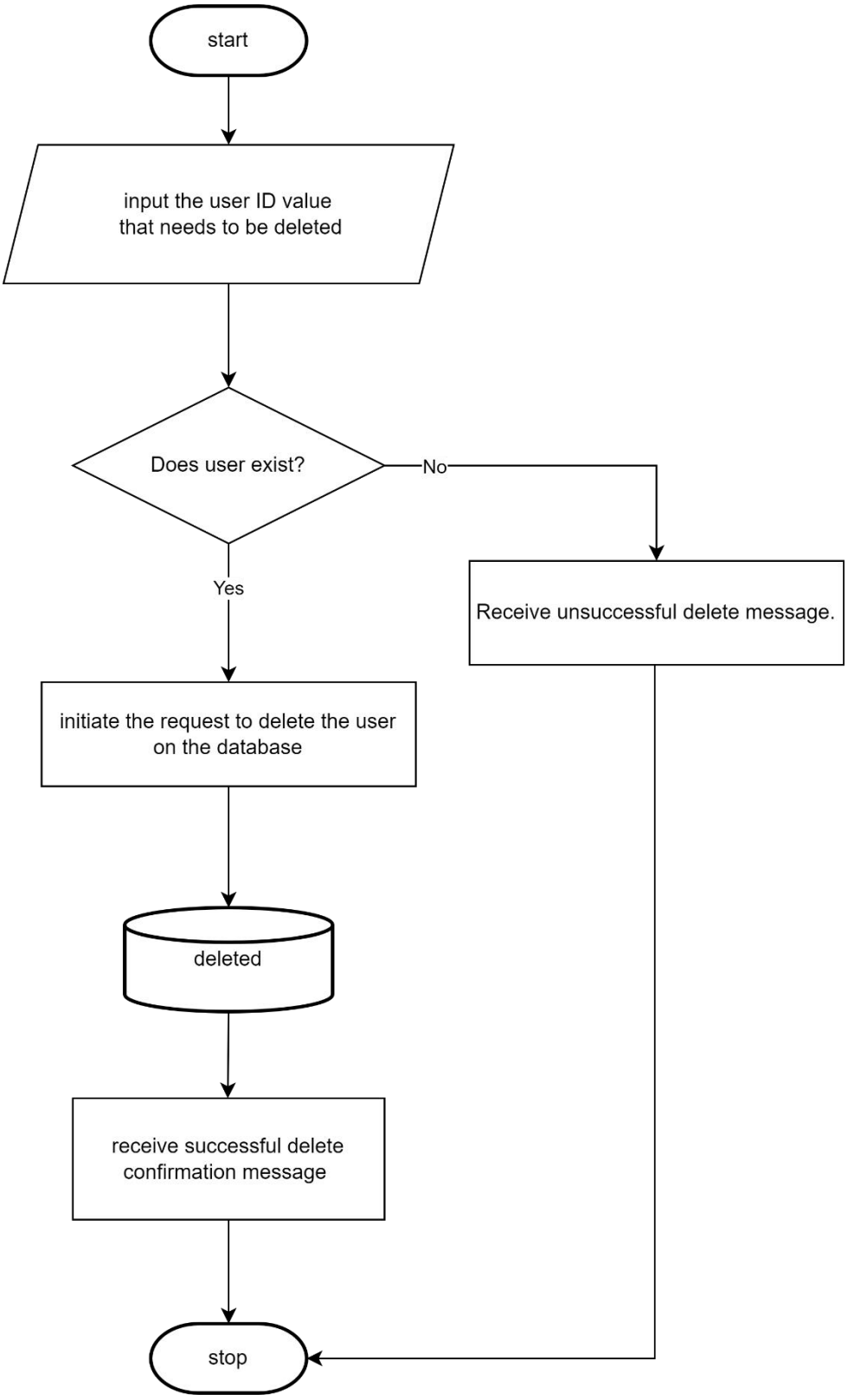
3.3.2 Search list of users and Read/Get a user



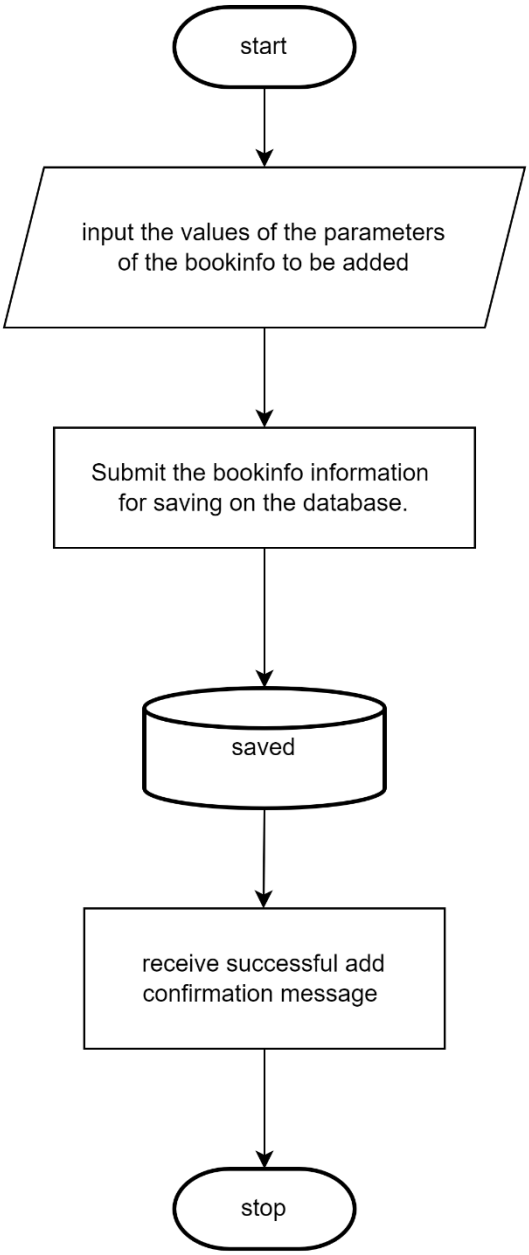
3.3.3 Update a user



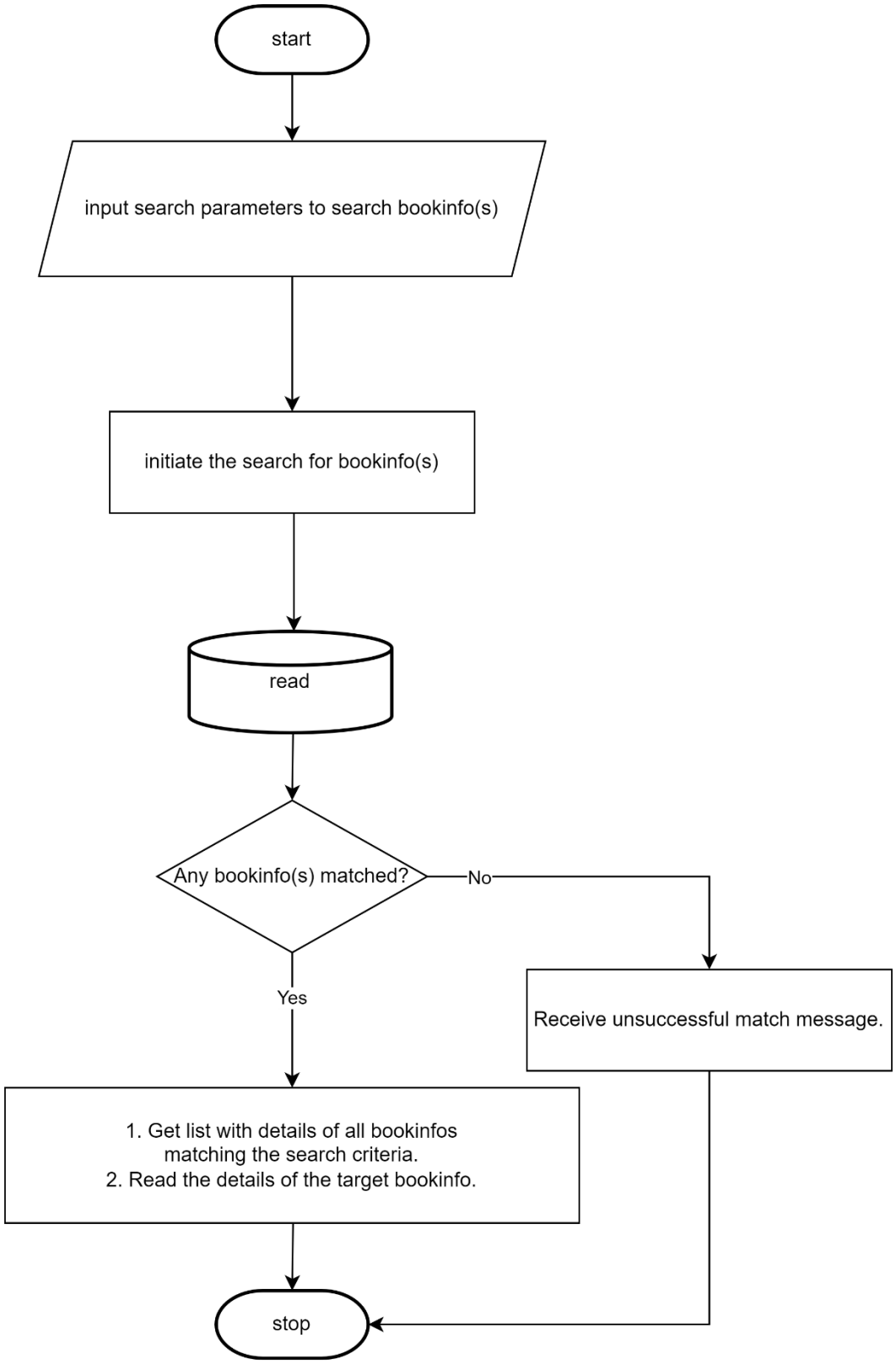
3.3.4 Delete a user



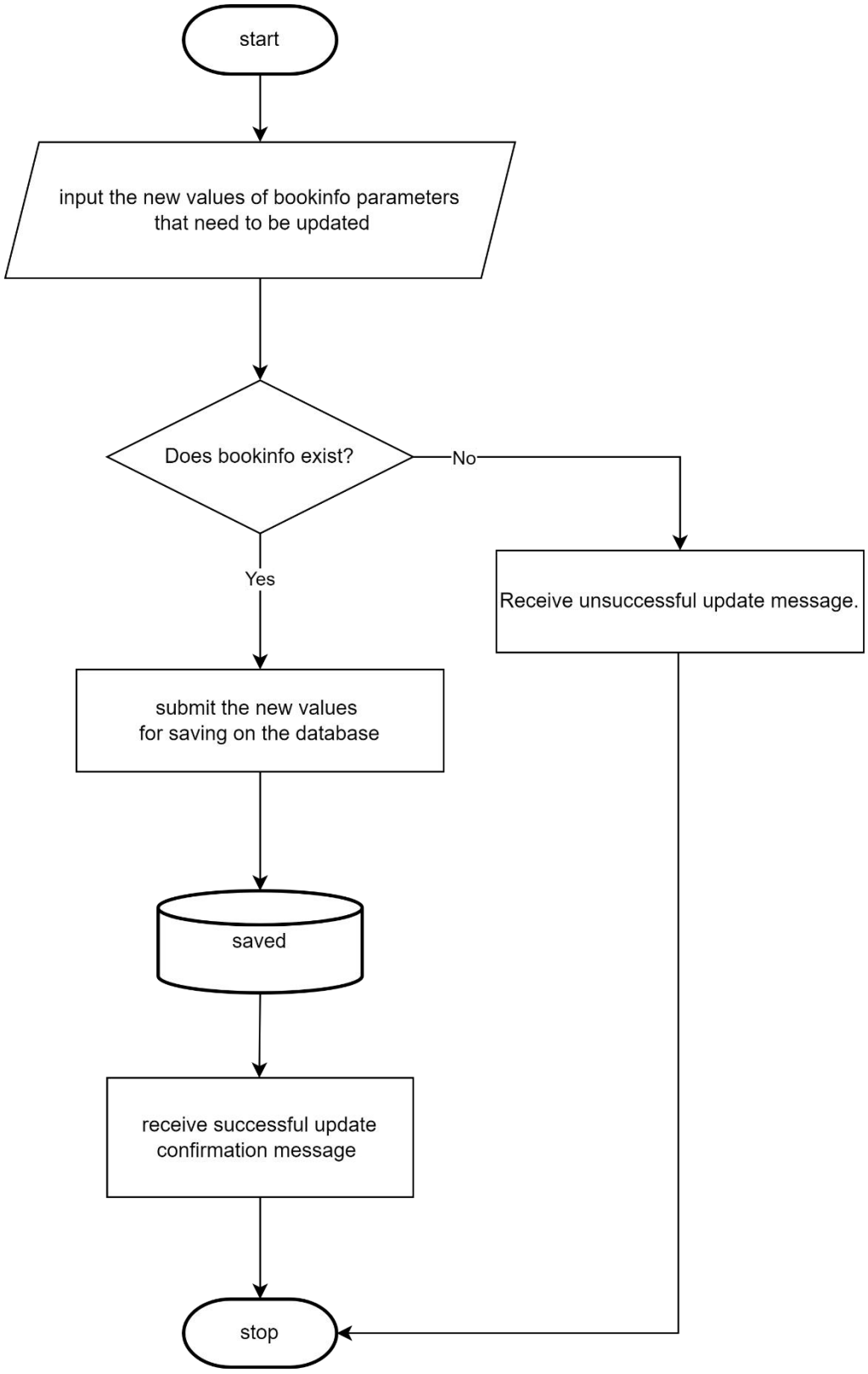
3.3.5 Create/Add a bookInfo



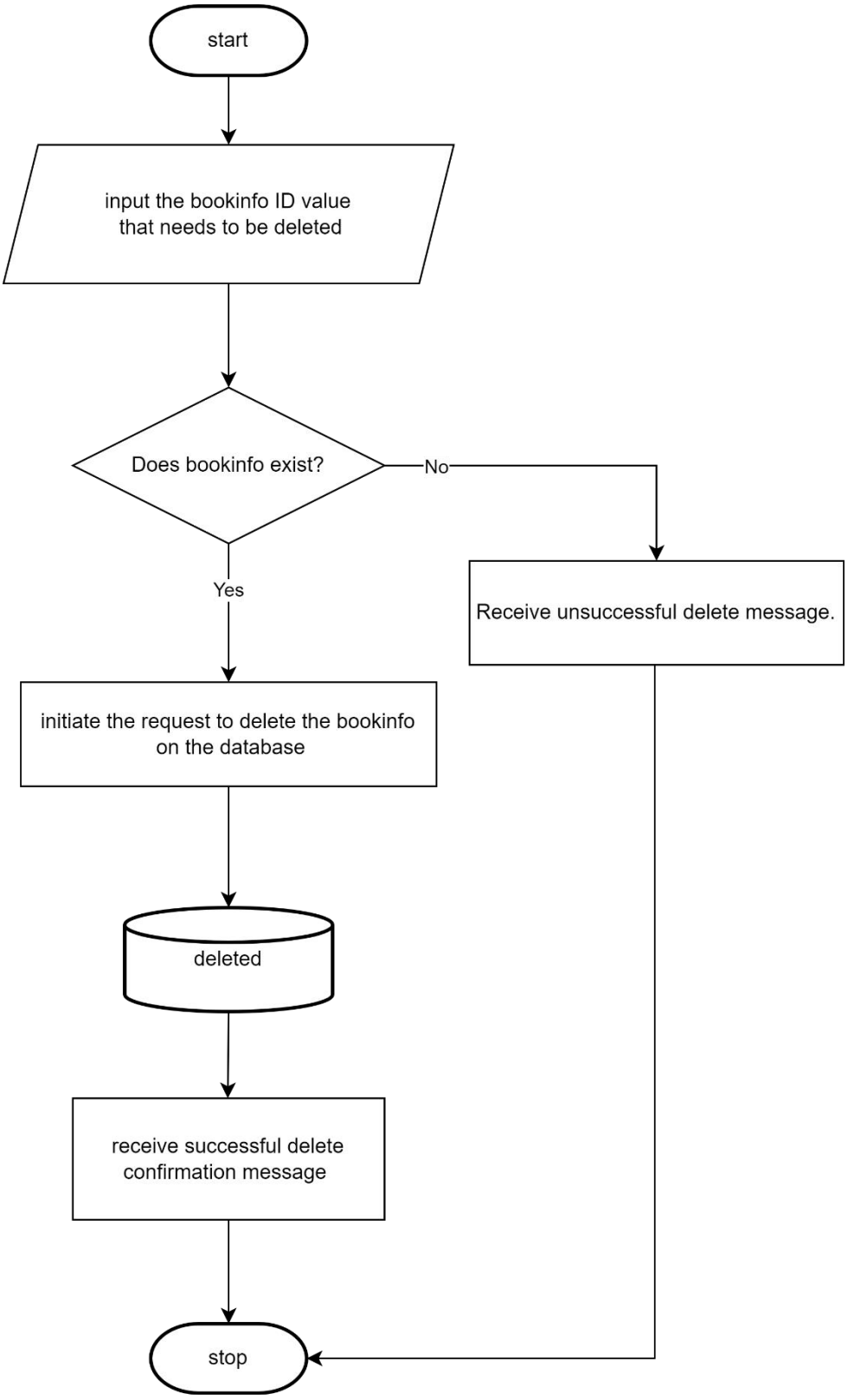
3.3.6 Search list of bookInfos and Read/Get a bookInfo



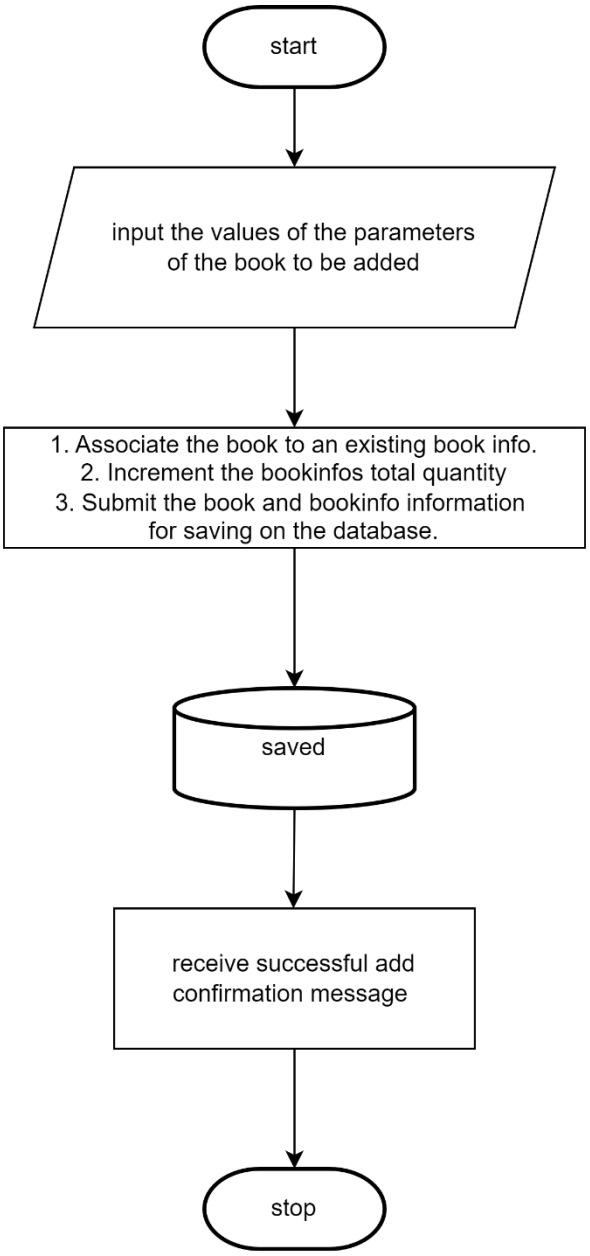
3.3.7 Update a bookInfo



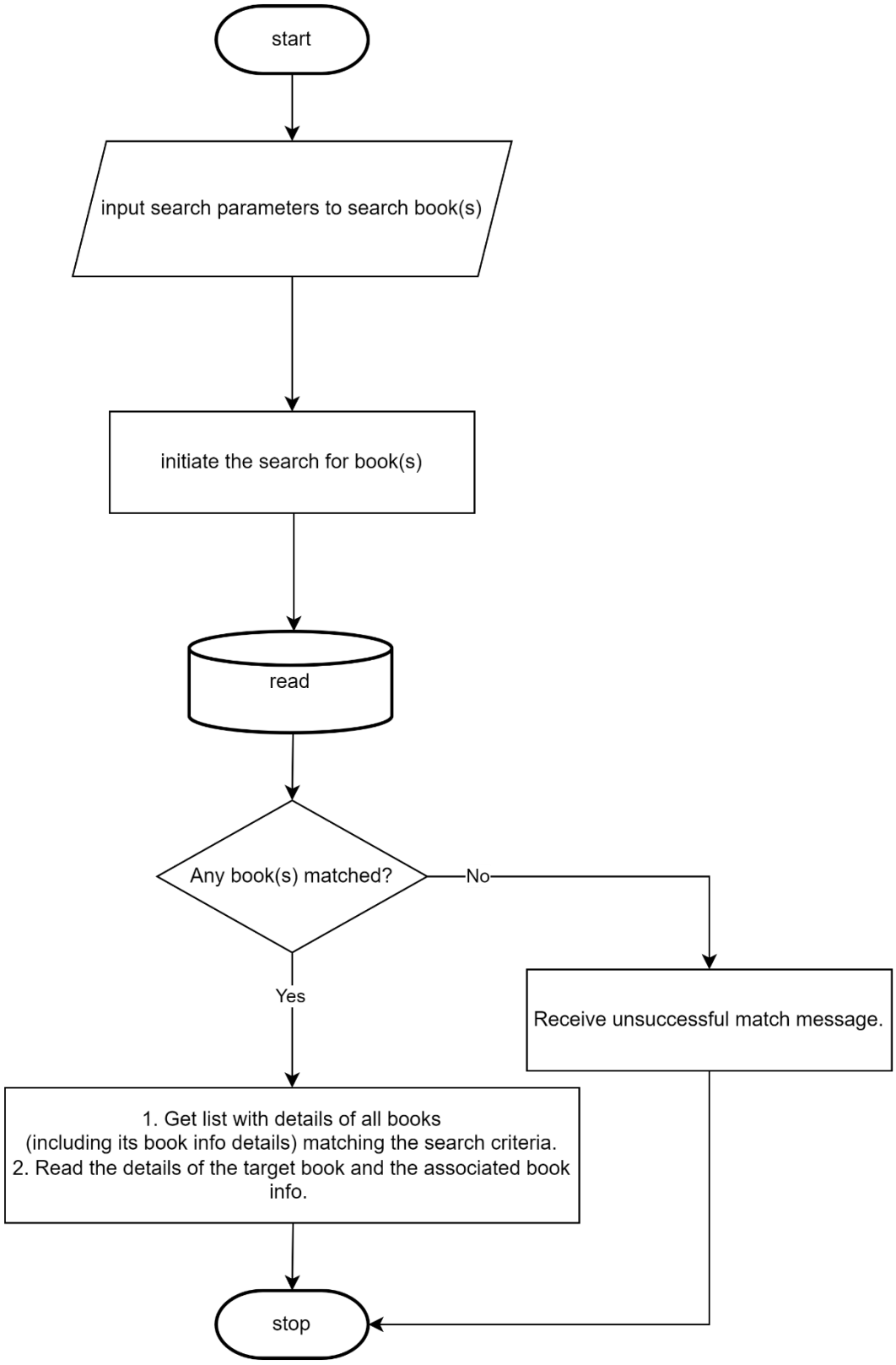
3.3.8 Delete a bookInfo



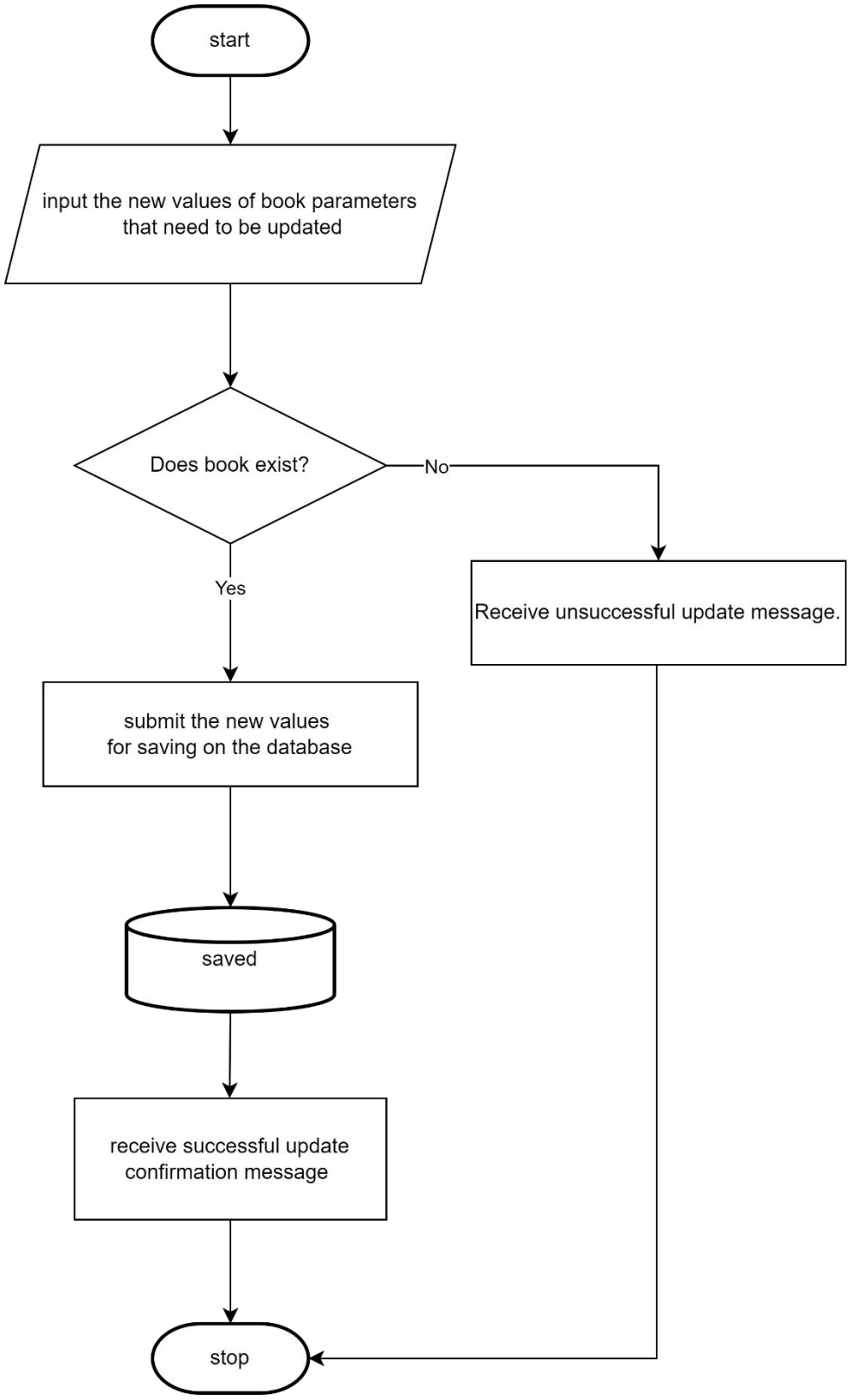
3.3.9 Create/Add a book



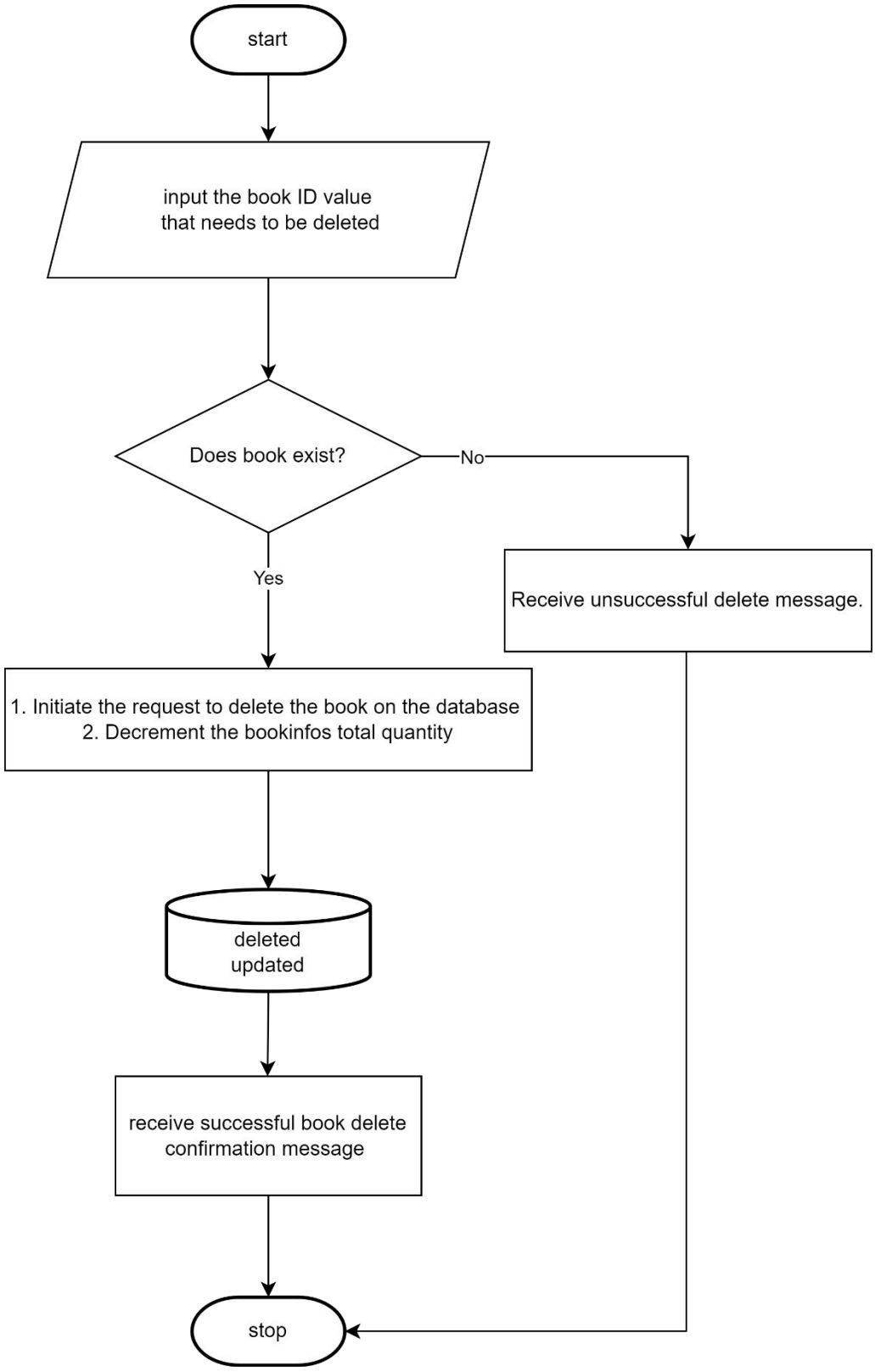
3.3.10 Search list of books and Read/Get a book



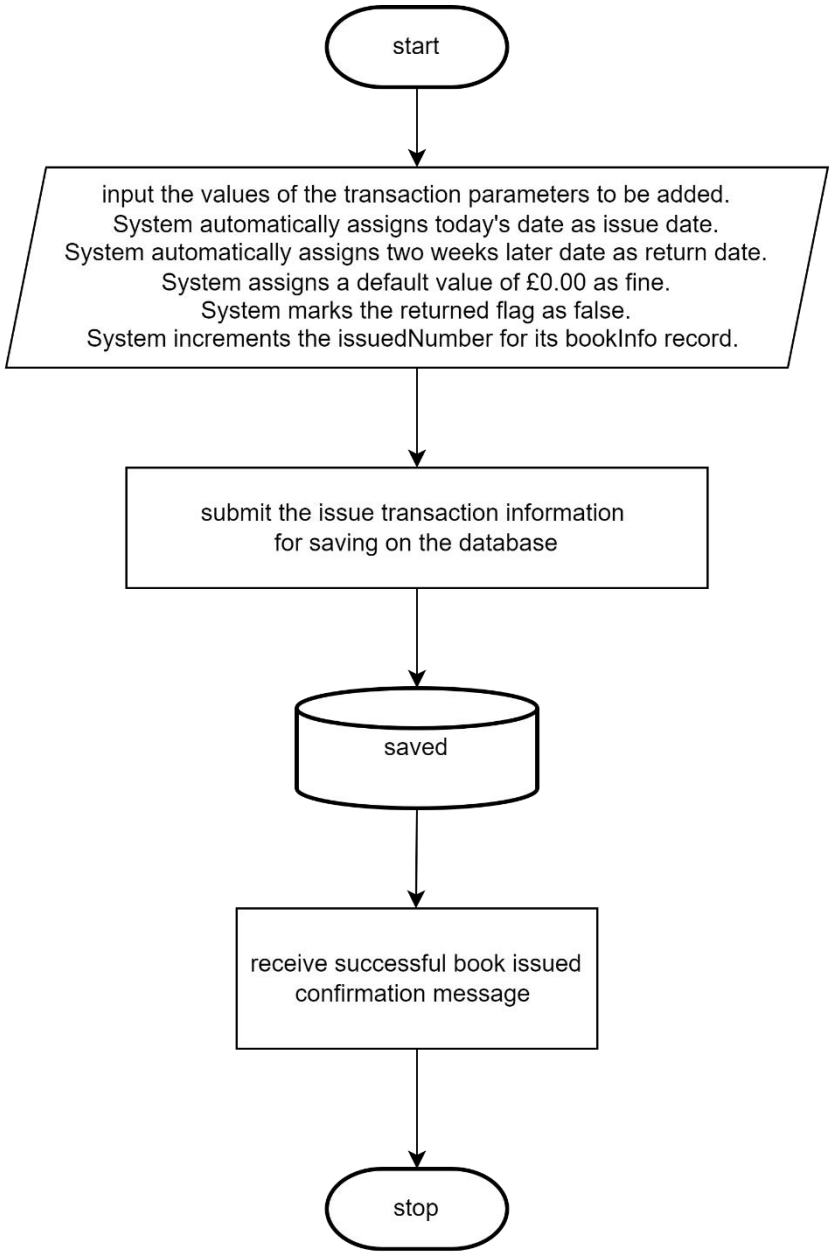
3.3.11 Update a book



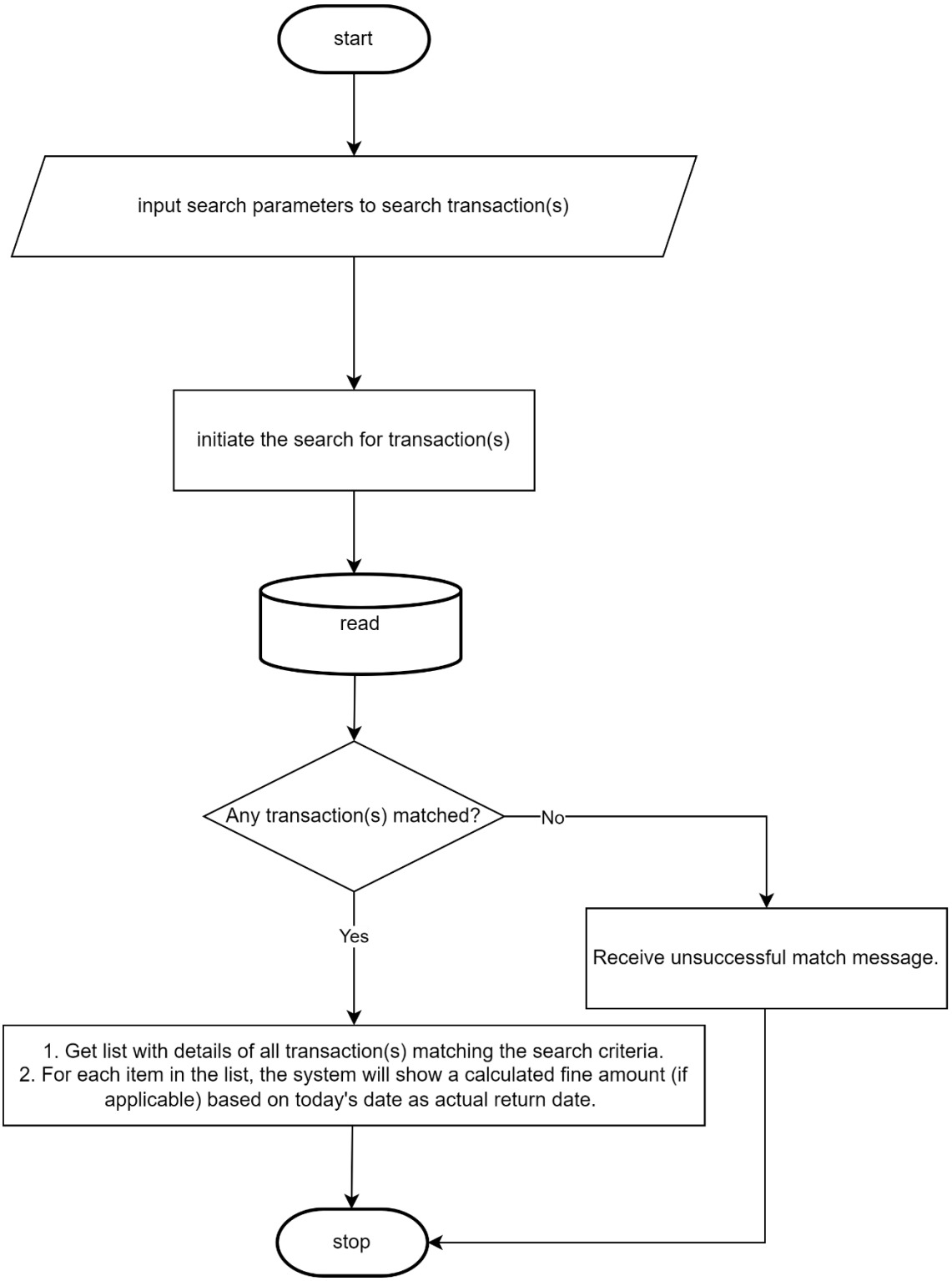
3.3.12 Delete a book



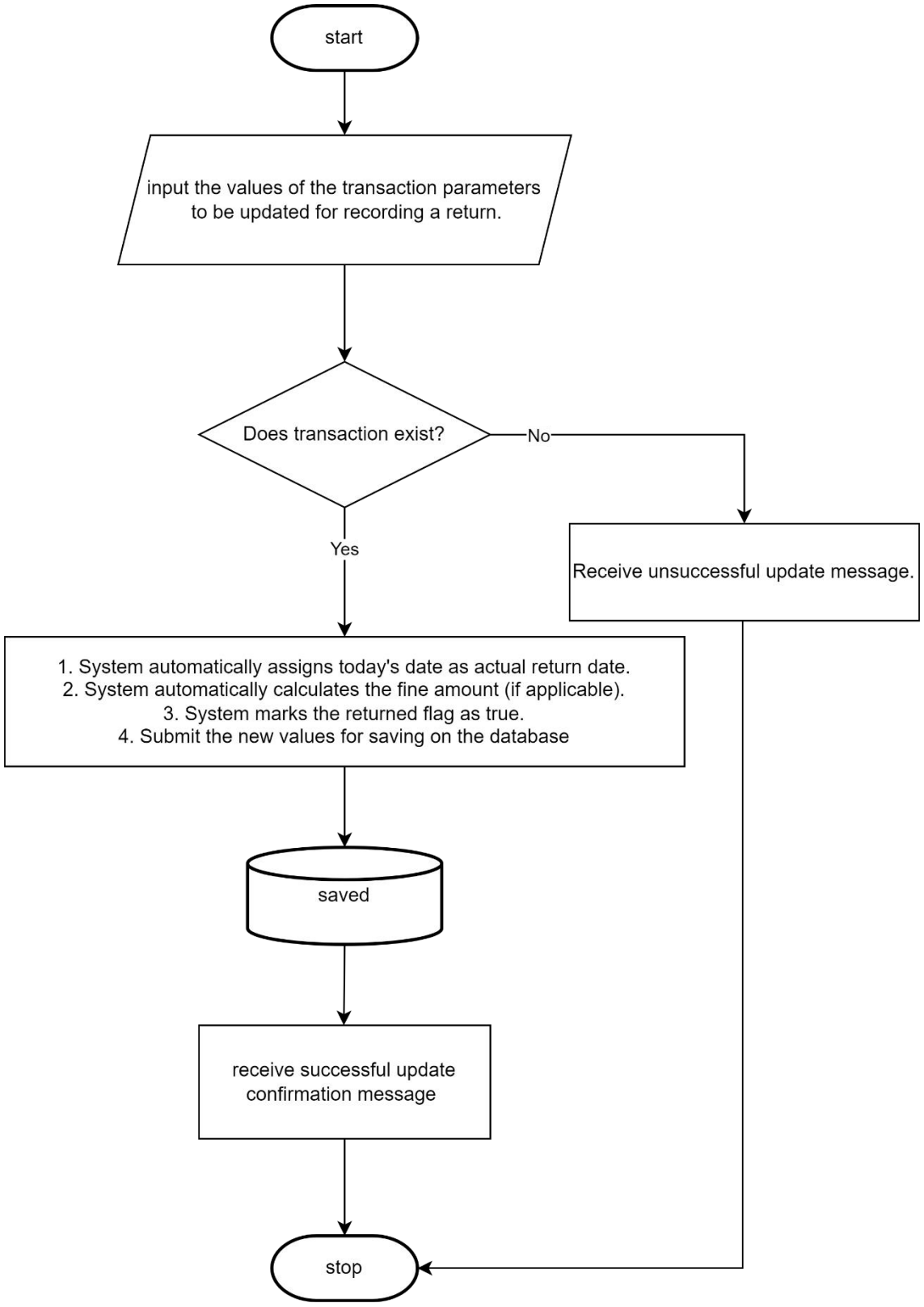
3.3.13 Create/Add a Transaction – Issue Book



3.3.14 Search list of transactions and Read/Get a transaction search/read



3.3.15 Update a Transaction – Return Book



3.3.16 Delete a transaction

I will not have Delete transaction functionality because Book Issue and Return record is used for audit and trend determinations. The transactions will be run by a batch job once the transactions have completed their audit cycle which is generally 2 years. So this LMS functionality cannot be demonstrated as part of demo.

3.3.17 Address

I will not have CRUD functionality for storing address because they should ideally come from government store of addresses, which will be available via PAID APIs. For demo purposes, I will manually update the Address table with maximum 10 addresses so the LMS functionality can be demonstrated.

The associated SQL to add an Address to the Address table is

INSERT INTO Address (door_number, line1, line2, city, postcode)

VALUES ("< door_number >", "< line1 >", "< line2 >", "< city >", "< postcode >");

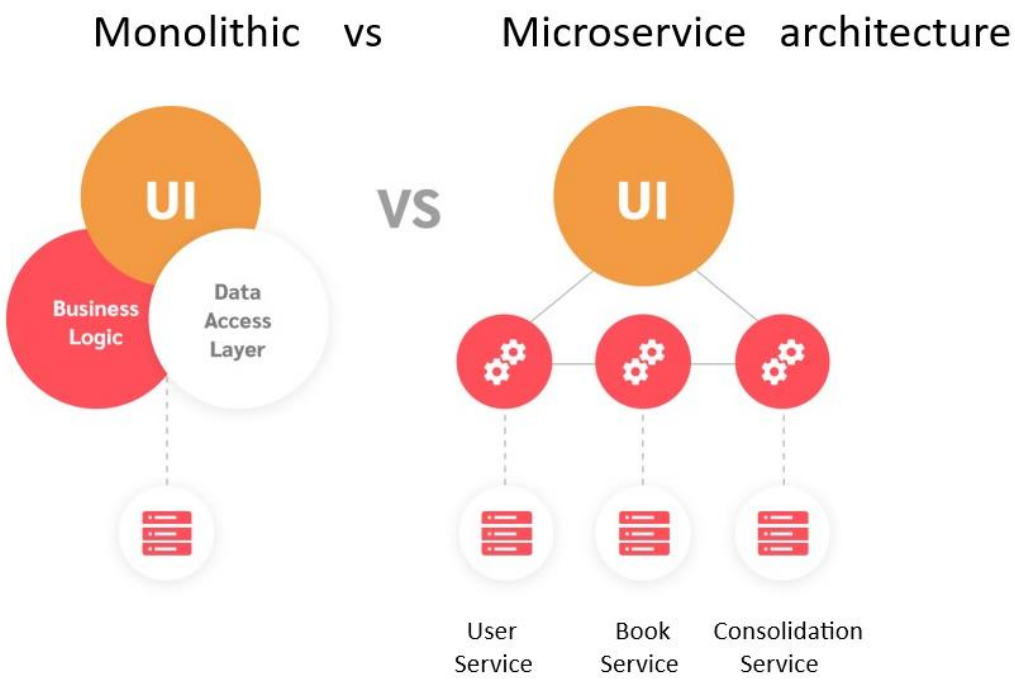
The associated SQL to read an Address from the Address table is

Select * from Address where address_id = < address_id >;

https://www.api.gov.uk/os/os-places-api/#os-places-api

3.4 System Architecture

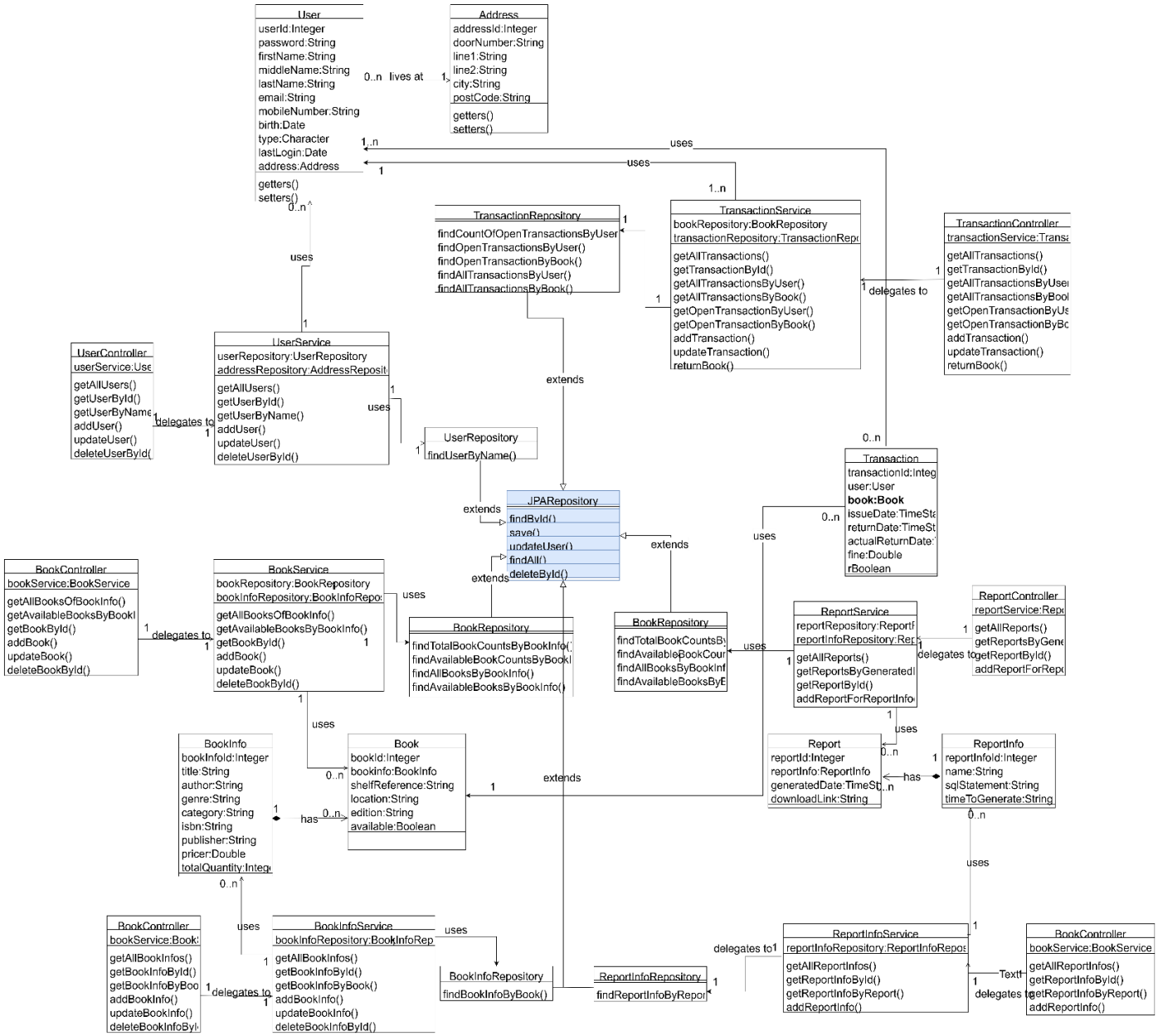
I will divide my overall solution into Service/View/Controller layer, Business layer and Data Service layer. If feasible, the implementation of this solution will be done using micro services architecture as opposed to monolithic architecture.



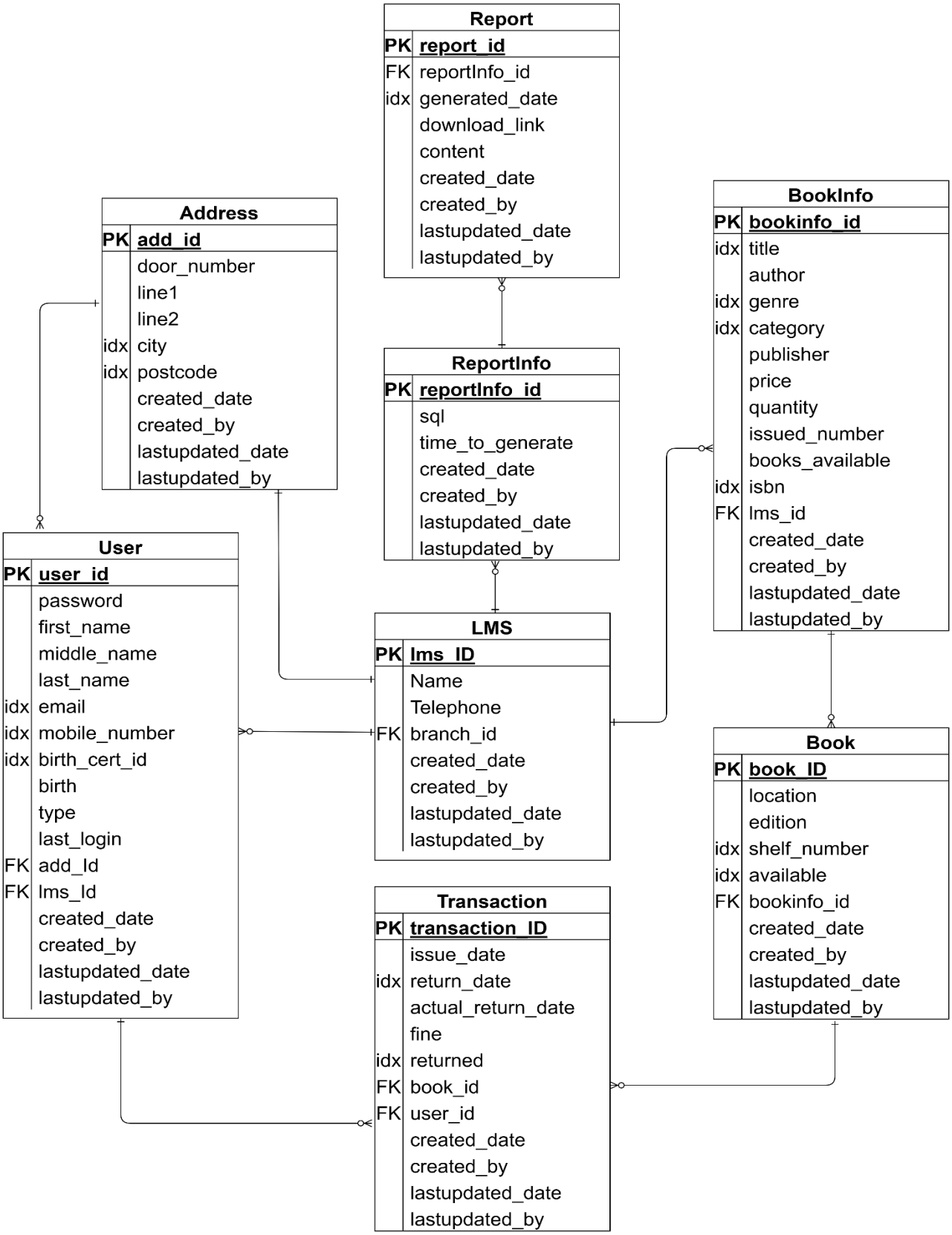
3.5 Class Diagram

I will create classes to program the **LMS** system. As this is a Microservice architecture based, so the interaction between classes is not as tightly coupled as in monolithic architecture. So you will see clear distinct pattern for each micro service with lightly tied up to model classes.

- Controller class: to interact with human/API interaction and the Service module.
- Data Service class: to CRUD data from the database and perform any business intelligence logic on the data, if required.
- Model class: to hold each line/record of data as a Record.


User
userId:Integer
password:String
firstName:String
middleName:String
lastName:String
email:String
mobileNumber:String
birth:Date
type:Character
lastLogin:Date
address:Address
getters()
setters()
0..n lives at 1
Address
addressId:Integer
doorNumber:String
line1:String
line2:String
city:String
postCode:String
getters()
setters()
uses
TransactionRepository
findCountOfOpenTransactionsByUser
findOpenTransactionsByUser()
findOpenTransactionByBook()
findAllTransactionsByUser()
findAllTransactionsByBook()
TransactionService
bookRepository:BookRepository
transactionRepository:TransactionRepo
getAllTransactions()
getTransactionById()
getAllTransactionsByUser()
getAllTransactionsByBook()
getOpenTransactionByUser()
getOpenTransactionByBook()
addTransaction()
updateTransaction()
returnBook()
TransactionController
transactionService:Transa
getAllTransactions()
getTransactionById()
getAllTransactionsByUse
getAllTransactionsByBool
getOpenTransactionByUs
getOpenTransactionByBo
addTransaction()
updateTransaction()
returnBook()
delegates to
UserController
userService:Use
getAllUsers()
getUserById()
getUserByName
addUser()
updateUser()
deleteUserById(
delegates to
UserService
userRepository:UserRepository
addressRepository:AddressReposit
getAllUsers()
getUserById()
getUserByName()
addUser()
updateUser()
deleteUserById()
UserRepository
findUserByName()
extends
JPARepository
findById()
save()
updateUser()
findAll()
deleteById()
Transaction
transactionId:Integ
user:User
book:Book
issueDate:TimeSta
returnDate:TimeSt
actualReturnDate:
fine:Double
rBoolean
BookController
bookService:BookService
getAllBooksOfBookInfo()
getAvailableBooksByBookI
getBookById()
addBook()
updateBook()
deleteBookById()
BookService
bookRepository:BookRepository
bookInfoRepository:BookInfoRepo
getAllBooksOfBookInfo()
getAvailableBooksByBookInfo()
getBookById()
addBook()
updateBook()
deleteBookById()
BookRepository
findTotalBookCountsByBookInfo(
findAvailableBookCountsByBookI
findAllBooksByBookInfo()
findAvailableBooksByBookInfo()
BookRepository
findTotalBookCountsBy
findAvailableBookCour
findAllBooksByBookInf
findAvailableBooksByE
ReportService
reportRepository:ReportI
reportInfoRepository:Rep
getAllReports()
getReportsByGeneratedI
getReportById()
addReportForReportInfo
ReportController
reportService:Repo
getAllReports()
getReportsByGene
getReportById()
addReportForRepo
BookInfo
bookInfoId:Integer
title:String
author:String
genre:String
category:String
isbn:String
publisher:String
pricer:Double
totalQuantity:Integ
Book
bookId:Integer
bookInfo:BookInfo
shelfReference:String
location:String
edition:String
available:Boolean
has
Report
reportId:Integer
reportInfo:ReportInfo
generatedDate:TimeSt
downloadLink:String
ReportInfo
reportInfoId:Integer
name:String
sqlStatement:String
timeToGenerate:String
BookController
bookService:Book
getAllBookInfos()
getBookInfoById()
getBookInfoByBoo
addBookInfo()
updateBookInfo()
deleteBookInfoByI
BookInfoService
bookInfoRepository:BookInfoRep
getAllBookInfos()
getBookInfoById()
getBookInfoByBook()
addBookInfo()
updateBookInfo()
deleteBookInfoById()
BookInfoRepository
findBookInfoByBook()
ReportInfoRepository
findReportInfoByRepor
ReportInfoService
reportInfoRepository:ReportInfoRepos
getAllReportInfos()
getReportInfoById()
getReportInfoByReport()
addReportInfo()
BookController
bookService:BookService
getAllReportInfos()
getReportInfoById()
getReportInfoByReport()
addReportInfo()
delegates to
Text

3.6 Entity Relationship – Data model



I will be creating approx. 8 tables that will form a relational database. This allows data to be organised into one or more tables with a unique key identifying each row. The above diagram shows the layout of Data model as how it will be had it been a monolithic application. In the context of Microservice architecture, the tables will still be the same but will be loosely related.

I will have the scenario of many-to-many relationships (between book and user tables for recording book issues and book returns). These occur between book and user tables (for recording book issues and book returns) when they contain records that are related to more than one other record. Many-to-many relationships can be resolved by creating a link/join table (in my case the Transaction table) which includes the primary keys from the two related tables; these are then known as foreign keys. For the records in the new Transaction table to be unique, a new primary key is needed. Now you have three tables with two one-to-many relationships.

I will have the scenario of referential integrity. This ensures that all the references, from the foreign keys, are valid. This column that is declared a foreign key can only contain non null values (or null values when a record was deleted) from a parent's table's primary key. Referential integrity can be caused by invalid referencing or the deletion of a record that contains a value referred to by a foreign key.

NOTE:
Each table will have 5 common attributes associated to metadata of each record i.e created_date, created_by, lastupdated_date and lastupdated_by. I will implement this functionality only if time permits. Otherwise at least, you will know that I have already thought about it as a Design item.

3.7 Data Dictionary

Some Abbrevations used are

<PK> = Primary Key

<FK> = Foreign Key

<idx> = Indexed column for quick searches

3.7.1 User model

Data item	Data type	Validation	Sample data
user_id <PK>	Integer	Not Null	101
Password	String	Not Null, VLD Contains 8 to 12 characters. Contains a capital letter. Contains a lowercase. Contains a number. Contains an special character.	(Default to) qwerty12
First_name	String	Not Null, No special characters, Space allowed	Saarah
Middle_name	String	No special characters	<Optional>

Last_name	String	Not Null, No special characters	Bedekar
Email <idx>	String	Not Null, VLD Contains prefix and domain, separated by @	a.b@c.com
Mobile_number <idx>	String	Not Null, 11 digits Only digits no space First 2 digits = 07 or 00 Default for infants is 11 0s	07771234555 OR 00000000000
Birth	Date	Not Null , >= Current Date	12-12-2010
Type (Admin/Staff/Member)	Character	Not Null	M
Last_login	Date	Not Null, >= Current Date	12-12-2012
address_id <FK>	Integer	Not Null	12

3.7.2 Address model

Data item	Data type	Validation	Sample data
address_id <PK>	Integer	Not Null	12
Door_number	String	Not Null	352A
Line 1	String	Not Null	Commercial road
Line 2	String		<Optional>
City <idx>	String	Not Null	London
Postcode <idx>	String	Not Null, Is the string between 5 to 7 characters? Is the inward code first character numeric? Are the last 2 characters non-numeric?	E1 0LB

3.7.3 BookInfo model

Data item	Data type	Validation	Sample data
bookinfo_id <PK>	Integer	Not Null	51
Title	String	Not Null	The Elegant Universe

Author	String	Not Null	Brian Greene
Genre	String	Not Null	Science
Category	String	Not Null	Theoretical physics
Publisher	String	Not Null	W. Norton & Company
Price	Float	Not Null	38.99
Quantity	Integer	Not Null	10
ISBN	String	Not Null	ISBN123-4-284-14253-0

3.7.4 Book model

Data item	Data type	Validation	Sample data
book_id <PK>	Integer	Not Null	87
Bookinfo_id <FK>	Integer	Not Null	51
Location	String	Not Null	Whitechapel
Edition	Integer	Not Null	3
Shelf_number <idx>	String	Not Null	W_3F_4Sec_2Sub
Available <idx>	Boolean	Not Null	True

3.7.5 Transaction model (Link Table)

Data item	Data type	Validation	Sample data
transaction_id <PK>	Integer	Not Null	33
Book_id <FK>	Integer	Not Null	87
User_id <FK>	Integer	Not Null	101
Issue_date	Date	Not Null	1-1-2024
Return_date	Date	Not Null	14-1-2024
Actual_return_date	Date	Null	14-2-2024

Fine	Decimal	Not Null	0.0
Returned <idx>	Boolean	Not Null	TRUE

3.7.6 ReportInfo model

Data item	Data type	Validation	Sample data
Reportinfo_id <PK>	Integer	Not Null	2
name	String	Not Null	SCH_TODAYs_RETURN
sql	String	Not Null	select * from transaction where DATE(return_date) = CURDATE()
Time_to_generate	String	Not Null	23:59

3.7.7 Report model

Data item	Data type	Validation	Sample data
report_id <PK>	Integer	Not Null	133
reportinfo_id <FK>	Integer	Not Null	2
generated_date	Date	Not Null	1-1-2024
Download_link	String	Not Null	“http://.....”
content	String	Not Null	<report content>

3.8 User Interface Rationale (Input)

I will be listing my design choices for how the user will provide data to the system.

- Inputs are needed by the system
    - The system needs ID of the entity to be searched or the name, title etc patterns, date etc to be searched for either Id match or pattern match search.
- User method of entering the inputs
    - I have used clear labels for Input fields and placed them logically for a smooth user experience.

  - The inputs are mainly text boxes for user to enter the value.
  - The password field is a masked field to hide the value
  - I have used drop down options only in case of Transaction search, where the user can specify if the transaction search is for open or all transactions.
  - Buttons are used to trigger an action for either submitting the input or selection of menu or resetting the screen values.
  - Hyperlink is used to display generated report from its stored location on server
  - Radio buttons are used for selection of a single row from the multiple rows searched by the system

- Reasoning of my choice of these methods
  - At a high level, these methods provide usability and accessibility
  - I have used drop down as this reduces user error and limits options to valid entries only.
  - I have used radio buttons as that is the only option to select 1 from many records
  - I have used buttons as that is a user convenience to trigger an action

- Use of user-friendly features
  - I have used Error sections to specify error details when an error occurs
  - I have triggered input validation just before submission of user input date to the API server
  - I have included colour contrast features to support accessibility and usability

- Validations applied
  - Validation rules include presence check, email format check, UK mobile number format check, positive Integer/Decimal number check, valid date check, valid report time check, Future date check to ensure mandatory data condition, data is accurate and within limits etc.

3.9 User Interface Rationale (Output)

I will be listing my decisions for how the system presents results or information to the user that eventually helps the user understand or take an action.

- Information details shown
  - Search parameters are shown for specifying data to be retrieved
  - Retrieved search results are displayed
  - Add, Update and Delete confirmation messages are displayed

    - Reports stored in HTML format are displayed when location link clicked

- How I have chosen to show the information
    - Retrieved search results are displayed in tables
    - Error/Exception details are displayed in tables
    - Delete confirmation messages are shown as modal screens
    - Status of successful Add, Update and Delete is shown as coloured Label / Text / Table
    - Reports (HTML format) are displayed in web browser when location link clicked
- My opinion on why this approach helps the user
    - The use of tables neatly formats the data and presents it to the user with nice readability and clarity. It also helps users compare data, and may help in understanding patterns
- Functional requirements met
    - This meets the requirement for showing details of Users, BookInfos, Books, Transactions, ReportInfos and Reports. Transaction shows scheduled Returned date and fine calculated. BookInfo shows total books against it. Book shows its availability etc.

3.10 Algorithms

To prepare for implementation I will structure and create the main algorithms that will allow my **LMS** project to work. Already having thought about the detailed design, I am aware of the limits of my project and the main algorithms that will take place.

Algorithm to Authenticate User

- Get id and Password form user
- Encrypt the password and send it to the server
- Check for password match
- Report successful match
- Report unsuccessful match
- Report User not found

Algorithm CRUD User

User creation

- Obtain the **User** details to input in the system
- Verify the mandatory details
- Verify the correct format for email

- Verify the correct format for mobile number
- Verify the correct valid past or present date of birth
- Associate the User with a valid Address
- Store the fields as part of generic add to the DB.
- Display appropriate error messages wherever applicable

Read/Search a User

- Verify the mandatory search details
- Search if the User exists
- Display the (selected) User (if multiple exists)
- Display the associated Address details
- Display appropriate error messages wherever applicable

Update User

- Verify the mandatory search details
- Search if the specific User exists
- Display the User details with only specific editable fields
- Verify the mandatory details
- Verify the correct format for the editable fields
- Store the updated fields as part of generic updates to the DB.
- Display appropriate error messages wherever applicable

Delete User

- Verify the mandatory search details
- Search if the specific User exists
- User cannot be deleted if they have borrowed a book and not yet returned
- User cannot be deleted if they had historical transactions. [To maintain referential integrity on DB, the transaction will have to be first deleted from the link table. This will be done manually by the support staff based on approvals only]
- Warn of a permanent deletion of that record
- Delete the User only if deletion is confirmed
- Display appropriate error messages wherever applicable

Algorithm CRUD BookInfo

BookInfo creation

- Obtain the **BookInfo** details to input in the system
- Verify the mandatory details
- Verify the correct format for price
- Store the fields as part of generic add to the DB.
- Display appropriate error messages wherever applicable

Read/Search a BookInfo

- Verify the mandatory search details
- Search if the BookInfo exists
- Display the (selected) BookInfo (if multiple exists)
- Display appropriate error messages wherever applicable

Update BookInfo

- Verify the mandatory search details
- Search if the specific BookInfo exists
- Display the BookInfo details with only specific editable fields
- Verify the mandatory details
- Verify the correct format for the editable fields
- Store the updated fields as part of generic updates to the DB.
- Display appropriate error messages wherever applicable

Delete BookInfo

- Verify the mandatory search details
- Search if the specific BookInfo exists
- Bookin BookInfo fo cannot be deleted if it has associated Book(s)
- Warn of a permanent deletion of that record
- Delete the BookInfo only if deletion is confirmed
- Display appropriate error messages wherever applicable

Algorithm CRUD Book

Book creation

- Obtain the **Book** details to input in the system
- Verify the mandatory details
- Associate the Book with a valid BookInfo
- Store the fields as part of generic add to the DB.
- Increment the total copies of the associated BookInfo and save it to the DB.
- Display appropriate error messages wherever applicable

Read/Search a Book

- Verify the mandatory search details
- Search if the Book exists
- Display the (selected) Book (if multiple exists)
- Display the associated BookInfo details
- Display appropriate error messages wherever applicable

Update Book

- Verify the mandatory search details
- Search if the specific Book exists
- Display the Book details with only specific editable fields
- Verify the mandatory details
- Store the updated fields as part of generic updates to the DB.
- Display appropriate error messages wherever applicable

Delete Book

- Verify the mandatory search details
- Search if the specific Book exists
- Book cannot be deleted if it is currently borrowed and not yet returned
- Book cannot be deleted if it has historical transactions. [To maintain referential integrity on DB, the transaction will have to be first deleted from the link table. This will be done manually by the support staff based on approvals only]
- Warn of a permanent deletion of that record
- Delete the Book only if deletion is confirmed
- Decrement the total copies of the associated BookInfo and save it to the DB
- Display appropriate error messages wherever applicable

Algorithm CRUD Transaction

Transaction creation

- Obtain the **Transaction** details to input in the system
- Verify the mandatory details
- Verify the provided User and Book exists
- Verify the associated User is within the borrowing limit
- Verify the associated Book is available and not currently borrowed
- Issue date will be set to current date
- Return date will be set to 15th day from the current date (i.e 2 weeks)
- Mark associated Book to be now unavailable for further borrowing until returned
- Save the updated Book to the DB
- Store the Transaction fields as part of generic add to the DB.
- Display appropriate error messages wherever applicable

Read/Search a Transaction

- Verify the mandatory search details
- Search if the User exists (is searched by User)
- Search if the Book exists (is searched by Book)
- Display the (selected) Transaction (if multiple exists)
- Display the associated User details
- Display the associated User's Address details
- Display the associated Book details

- Display the associated BookInfo details
- Display appropriate error messages wherever applicable

Update Transaction

- Verify the mandatory search details
- Search if the specific Book with Open Transaction exists
- Display the Transaction details with noneditable fields
- Mark associated Book to be now available for borrowing
- Save the updated Book to the DB
- Actual Return date will be set to current date
- Fine will be auto calculated
- Money will be manually taken from the user
- Transaction will be auto marked as (Book) returned
- Store the auto updated fields as part of generic updates to the DB.
- Display appropriate error messages wherever applicable

Delete Transaction

- A Transaction once created cannot be deleted for audit reasons. But if there is a need to delete it for DB referential integrity reasons, then this will be done manually by the support staff based on approvals only

Algorithm Calculate fine

Fine will be auto calculated with respect to the difference between the Actual Return date and the Scheduled returned date. For every week delay there will be a charge as configured in the system. Currently the charge is £2 per week.

Algorithm CRUD ReportInfo

ReportInfo creation

- Obtain the **ReportInfo** details to input in the system
- Verify the mandatory details
- Verify the correct format for generation time
- Store the fields as part of generic add to the DB.
- Display appropriate error messages wherever applicable

Read/Search a ReportInfo

- Verify the mandatory search details
- Search if the ReportInfo exists
- Display the (selected) ReportInfo (if multiple exists)
- Display appropriate error messages wherever applicable

Update ReportInfo

- Verify the mandatory search details
- Search if the specific ReportInfo exists
- Display the ReportInfo details with only specific editable fields
- Verify the mandatory details
- Verify the correct format for the editable fields
- Store the updated fields as part of generic updates to the DB.
- Display appropriate error messages wherever applicable

Delete ReportInfo

- Verify the mandatory search details
- Search if the specific ReportInfo exists
- ReportInfo cannot be deleted if it has associated Report(s)
- Warn of a permanent deletion of that record
- Delete the ReportInfo only if deletion is confirmed
- Display appropriate error messages wherever applicable

Algorithm CRUD Report

Report creation

- Obtain the **Report** details to input in the system
- Verify the mandatory details
- Associate the Report with a valid ReportInfo
- Store the fields as part of generic add to the DB.
- Display appropriate error messages wherever applicable

Read/Search a Report

- Verify the mandatory search details
- Search if the Report exists
- Display the (selected) Report (if multiple exists)
- Display the associated ReportInfo details
- Display appropriate error messages wherever applicable

Update Report

- A Report once created cannot be deleted for audit reasons

Delete Report

- A Report once created cannot be deleted for audit reasons. It can only be purged once the retention period (currently 10 yrs) is over

3.11 System Security and Data Integrity

Authentication

Authentication is the process of determining whether someone or something is who or what they say they are. Authentication provides access control for systems by checking to see if a user's credentials match the credentials in a database of authorised users. In doing this, authentication ensures that systems, processes and information are secure.

There are several authentication types. For user identity, users are typically identified with a user ID; authentication occurs when the user provides credentials, such as a password, that match their user ID.

The application will be accessible only to authenticated users. Users will be authenticated at the point of logging into the system via the login page.

Authorisation

Authorisation is the process of granting or denying access to resources based on the identity and permission level of a user. The authorisation process implements access controls that determine the exact activities a person/system is allowed to execute on a specific resource. Authorisation is a crucial aspect of software, ensuring that only authorised users may access and perform operations on them. It protects sensitive information from unauthorised access, whether due to internal mistakes or external attacks.

Once a user is successfully authenticated, the **LMS** application will allow the user to access functionalities that are authorised for that user. This will be enforced by defined user types (IT admin, library staff, member users).

System security is the practice of protecting information systems from unauthorised access, modification, or destruction. System security measures help organisations protect sensitive data and prevent cyber threats.

The **LMS** will need to ensure that it's system functionality can only be accessed by **authenticated** users. This will be done using system login.

The **LMS** will need to ensure that its resources/information is selective accessible to users based on their **authorised** roles. This requires a granular security model to apply for user access. This will be achieved by User's HttpSession.

Backups when done are kept secured and only admin backup staff can access it to restore if need be via manual procedures.

Data integrity refers to the accuracy, consistency, and completeness of data throughout its lifecycle. It's a critically important aspect of systems which process or store data because it protects against data loss and data leaks. Maintaining the integrity of your data over time and across formats is a continual process involving various processes, rules, and standards.

I will be maintaining proper Foreign Keys to ensure the data integrity is maintained.

3.12 Development Environment/Language

I will be using Java 17 as an Object-Oriented Programming language for developing the code of the **LMS** system.

```
C:\ws\lms>java -version
java version "17.0.12" 2024-07-16 LTS
Java(TM) SE Runtime Environment (build 17.0.12+8-LTS-286)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.12+8-LTS-286, mixed mode, sharing)
```

Since I am targeting to use Web API, so I will be using Spring Boot framework to ease development effort for creating microservices.

I am using Maven to build the code

```
C:\ws\lms>mvn -version
Apache Maven 3.9.9 (8e8579a9e76f7d015ee5ec7bfcdc97d260186937)
Maven home: C:\apache-maven-3.9.9
Java version: 17.0.12, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-17
Default locale: en_GB, platform encoding: Cp1252
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

3.13 Database setup

These instructions are for the DB admin to setup the database from sctracth and populate some tables which are required for the application to start.

Install My SQL Server

Once you have installed MySQL Server, you will need to configure it for Database creation and usage with the below parameter values

```
username=root
password=Root1234
port= 3306
database=lms
```

Before you start the SpringBoot application execute the below on the MySQL WorkBench Client

```
DROP DATABASE IF EXISTS LMS;
CREATE DATABASE IF NOT EXISTS LMS;
USE LMS;
```

Since we are using SpringBoot integrated with JPA, Hibernate will automatically create the tables based on the Model/Data (Java) classes

When you start the Springboot application, the below will be automatically created and executed

```
create table address (address_id integer not null auto_increment, city varchar(255), door_number varchar(255), line1 varchar(255), line2 varchar(255), postcode varchar(255), primary key (address_id))

create table book (book_id integer not null auto_increment, available bit, edition varchar(255), location varchar(255), shelf_reference varchar(255), bookinfo_id integer, primary key (book_id))

create table bookinfo (bookinfo_id integer not null auto_increment, author varchar(255), category varchar(255), genre varchar(255), isbn varchar(255), price float(53), publisher varchar(255), title varchar(255), total_quantity integer, primary key (bookinfo_id))

create table report (report_id integer not null auto_increment, download_link varchar(255), generated_date datetime(6), reportinfo_id integer, primary key (report_id))

create table reportinfo (reportinfo_id integer not null auto_increment, name varchar(255), sql_statement varchar(255), time_to_generate varchar(255), primary key (reportinfo_id))

create table transaction (transaction_id integer not null auto_increment, actual_return_date_date datetime(6), fine float(53), issue_date datetime(6), return_date datetime(6), returned bit, book_id integer, user_id integer, primary key (transaction_id))

create table user (user_id integer not null auto_increment, birth datetime(6), email varchar(255), first_name varchar(255), last_login datetime(6), last_name varchar(255), middle_name varchar(255), mobile_number varchar(255), password varchar(255), type char(1), address_id integer, primary key (user_id))

alter table book add constraint FKcodng1etg40u11cmkrasdk3sx foreign key (bookinfo_id) references bookinfo (bookinfo_id)

alter table report add constraint FKa3ugeeaex0ymf1ykk7ccgu14e foreign key (reportinfo_id) references reportinfo (reportinfo_id)

alter table transaction add constraint FK8hddvclv2iqa3sg1dm8295pqw foreign key (book_id) references book (book_id)

alter table transaction add constraint FKsg7jp0aj6qipr50856wf6vbw1 foreign key (user_id) references user (user_id)

alter table user add constraint FKddefmvbrws3hvl5t0hnnsv8ox foreign key (address_id) references address (address_id)
```

The application is now ready to process. So I will give it the minimum data for the application to be used.

Since the application needs an Admin user to start off with, execute the below first

```
INSERT INTO Address (door_number, line1, line2, city, postcode)
VALUES
```

("100", "Commercial Road", "Shadwell", "London", "E1 1NL"),
("102", "Commercial Road", "Shadwell", "London", "E1 1NL"),
("104", "Commercial Road", "Shadwell", "London", "E1 1NL"),
("106", "Commercial Road", "Shadwell", "London", "E1 4PL"),
("108", "Commercial Road", "Shadwell", "London", "E1 4PL"),
("110", "Commercial Road", "Shadwell", "London", "E1 4PL");

INSERT INTO USER (password, first_name, middle_name, last_name, email, mobile_number, birth, type, last_login, address_id)
VALUES
("pass", "Saarah", "R", "Bedekar", "s.b@gmail.com", "07900000002", "2000-02-02", "A", "2025-03-09", 2);

Now you have a valid user to login to your web application's Login page
From there, the Admin user can perform all operations required for the LMS application.

The Admin user can also use SQL to directly load data into the Database via the MySql client application.

To show all tables you can use below sql
SHOW TABLES;

To see the details of any table you can use below sql example
Describe Book;

To get a record by its Id you can use below sql example
select * from user where user_id = 1

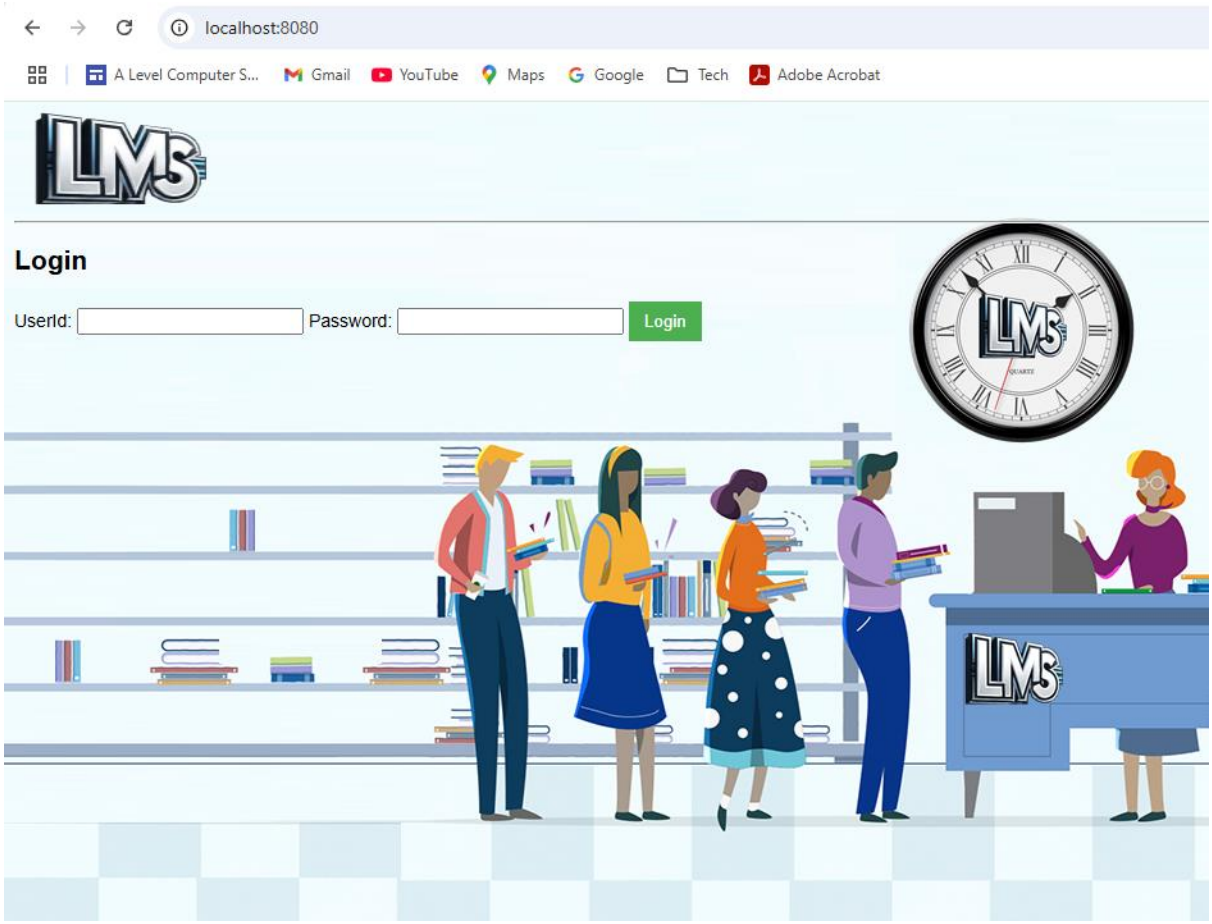
4 Implementation

All the Code is located at **=> Git Hub Link**.

4.1 Contents Page created for code

4.2 All Code shown

Login



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Login Page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../login/js/login.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>
<body style="background-image: url('../img/librarybg.png'); background-repeat: no-repeat;">
    <SCRIPT src="../js/header1.js"></SCRIPT>
    <h2>Login</h2>
    <form action="/login" method="post" onsubmit="return validateLogin(event);">
        <label for="userid">UserId:</label>
        <input type="text" id="userid" name="userid" th:value="${userId}" required />
        <label for="password">Password:</label>
```

```
        <input type="password" id="password" name="password" required />
        <button type="submit">Login</button>
        <br><br>
        <div th:if="!${#strings.isEmpty(error)}">
            <!-- <img style="vertical-align: middle" src="../img/errorIcon.png" /> -->
            <span id="hdnLoginError" style='color:red' th:text="${error}"></span>
        </div>
        <div id="divError" style='color:red'></div>
    </form>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function validates the input information given on the UI
*/
function validateLogin(event) {
    let userId = document.getElementById("userid").value.trim();
    let password = document.getElementById("password").value;
    let divError = document.getElementById("divError");
    let errorSpan = document.getElementById("hdnLoginError");
    let error = "";

    // Validation: Check if the value is an integer
    if (userId && !isPositiveNumber(userId)) {
        error = "User Id value can only be a positive number";
           divError.innerHTML = error;
        if (errorSpan) errorSpan.textContent = "";
           event.preventDefault(); // Stop form submission
        return false;
    }
    divError.innerHTML = ""; // Clear error if valid
    return true;
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles GET requests to map url path to the login page to show login options
     * @return the next mapped url path
     */
    @GetMapping("/")
    public String showLoginPage() {
        return "login";  // No .html needed, Thymeleaf will find login.html in templates
    }

    /**
     * Handles GET requests to map url path to Login page taking user's login credentials
     * @return the next mapped url path
     */
    @PostMapping("/login")
    public String login(@RequestParam Integer userid,
                        @RequestParam String password,
                        HttpSession session,
```

```
                                Map<String, Object> model) {
        log.info("Starting LoginController::login. userid = " + userid);
        String userState = userService.authenticate(userid, password);

        String result = userState.split("-")[0];

        if ("success".equals(result)) {
            String userInfo = userState.split("-")[1];
            String userId = userState.split("-")[2];
            String userType = userState.split("-")[3];
            session.setAttribute("userInfo", userInfo + "-" + userId + "-" + userType);
            session.setAttribute("userId", userId);
            session.setAttribute("userType", userType);

            log.info("Returning LoginController::login. success - userid = " + userid);
            return "redirect:/menu";
        } else if ("invalid_password".equals(result)) {
            model.put("userId",userid);
            model.put("error", "Invalid password.");
            log.info("Returning LoginController::login. invalid password");
            return "login";  // Return to login.html on error
        } else {
            model.put("error", "User not found.");
            model.put("userId",userid);
            log.info("Returning LoginController::login. User not found");
            return "login";  // Return to login.html on error
        }
    }

    /**
     * Handles GET requests to map url path to the Menu page
     * @return the next mapped url path
     */
    @GetMapping("/menu")
    public String showMenu(HttpSession session, Map<String, Object> model) {
        return Utils.handleSession(session, model, "menu");
    }

    /**
     * Handles GET requests to map path to Login page having logged out the existing user
     * @return the next mapped url path
     */
    @GetMapping("/logout")
    public String logout(HttpSession session) {
        session.invalidate();
        return "login";  // Return to login.html on error
    }
```

Repository Class [Java Code]

NA

Service Class [Java Code] => Git Hub Link

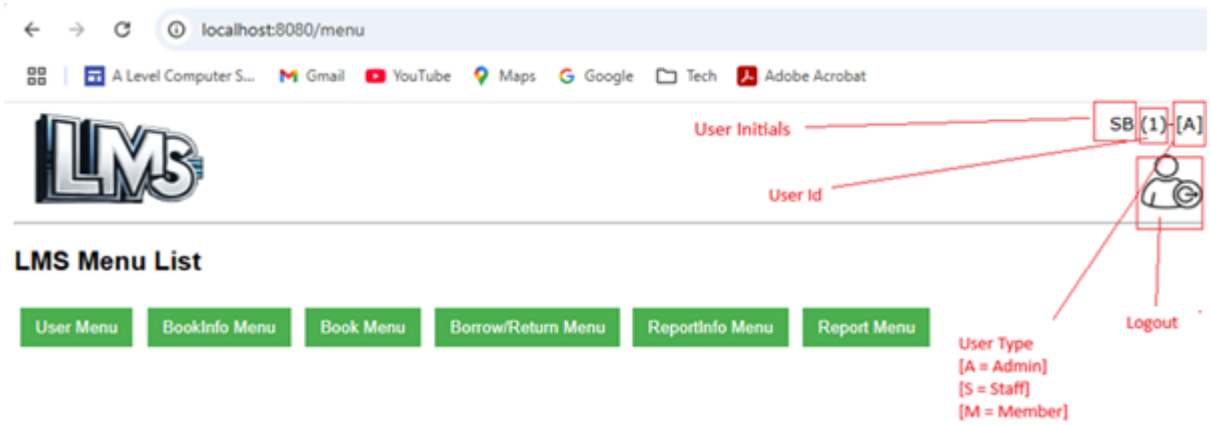
```
    /**
     * Handles the one authentication of the user to the LMS UI application
     * @param userId the userId of the User to be autheticated
```

```
     * @param rawPassword the password of the User to be autheticated
     * @return the state of successful/failure authentication of the User
     */
    public String authenticate(Integer userId, String rawPassword) {
        log.info("Started UserService::authenticate. userId = " + userId);
        Optional<User> userOptional = userRepository.findById(userId);
        if (userOptional.isPresent()) {
            User userDB = userOptional.get();

            // Build the display profile of the User
            String userInfo = userDB.getFirstName().charAt(0) + "" +
             userDB.getLastName().charAt(0) + "-" + userDB.getUserId() + "-" +
              userDB.getType();

            // Hash the raw password
            String hashedPassword = BCryptPasswordHasher.hashPassword(rawPassword);
            //log.info("BCrypt Hashed Password: " + hashedPassword);

            // Verify password match
            boolean isMatch = BCryptPasswordHasher.verifyPassword(
                                userDB.getPassword(), hashedPassword);

            // return the state of authetication whether success or failure
            if (isMatch) {
                log.info("Returning UserService::authenticate. success" + userId);
                return "success-" + userInfo;
            }
            log.info("Returning UserService::authenticate. invalidPwd");
            return "invalid_password-";
        }
        log.info("Returning UserService::authenticate. UserNotFound - userId = " + userId);
        return "user_not_found-";
    }
```

BCryptPasswordHasher [Java Code] => Git Hub Link

```
    /** Handles the hashing of a provided password
     * @param password the password for which the hash encoding has to be done for
     * @return a String of the hashedPassword
     */
    public static String hashPassword(String password) {
        return passwordEncoder.encode(password);
    }

    /** Handles the match verification of a raw password with its hashed password
     * @param rawPassword the raw password provided by the user
     * @param hashedPassword the hashed password generated by the BCryptPasswordHasher
     * @return a boolean indicator to indicate the result of the match verification
     */
    public static boolean verifyPassword(String rawPassword, String hashedPassword) {
        return passwordEncoder.matches(rawPassword, hashedPassword);
    }
```

LMS Menu Screen



UI Form/Screen [HTML Code] => Git Hub Link

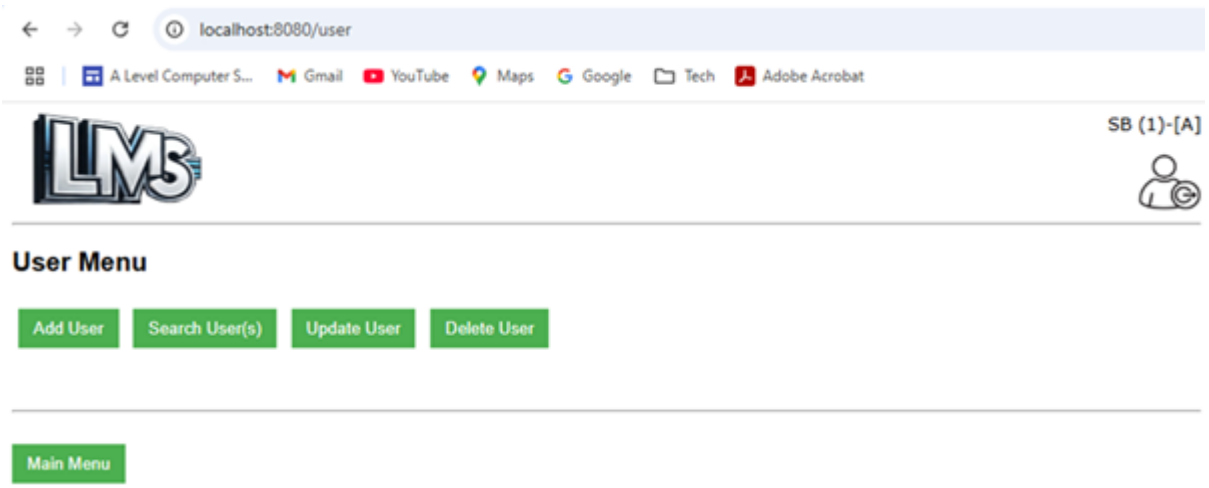
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>LMS Menu Page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <link rel="stylesheet" type="text/css" href="css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>
<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div>
        <h2>LMS Menu List</h2>

        <div class="menubox">
        <!-- Link to user.html -->
        <a th:href="@{/user}"><button>User Menu</button></a>  
            <!-- Link to bookinfo.html -->
            <a th:href="@{/bookinfo}"><button>BookInfo Menu</button></a>  
            <!-- Link to book.html -->
            <a th:href="@{/book}"><button>Book Menu</button></a>  
            <!-- Link to transaction.html -->
            <a th:href="@{/transaction}"><button>Borrow/Return Menu</button></a>
        </div>
        <div class="menubox" th:if="${userType == 'A'}">
            <!-- Link to reportinfo.html -->
            <a th:href="@{/reportinfo}"><button>ReportInfo Menu</button></a>  
            <!-- Link to report.html -->
            <a th:href="@{/report}"><button>Report Menu</button></a>
        </div>
    </div><br>
</body>
</html>
```

Client Side scripting [Javascript Code]

NA

CRUD User

User Menu Screen



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <title>LMS User Menu Page</title>
  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
  <link rel="stylesheet" type="text/css" href="../css/main.css">
  <link rel="icon" href="../img/lms.png">
</head>

<body>
  <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
  <SCRIPT src="../js/header.js"></SCRIPT>
  <div >
    <h2>User Menu</h2>
    <div class="menubox" th:if="${userType != 'M'}">
      <a th:href="@{/usera}"><button>Add User</button></a>  
      <a th:href="@{/userr}"><button>Search User(s)</button></a>  
      <a th:href="@{/useru}"><button>Update User</button></a>  
      <a th:href="@{/userd}"><button>Delete User</button></a>
    </div>
    <div class="menubox" th:unless="${userType != 'M'}">
      <a th:href="@{/userum}"><button>Update Self Details</button></a>
    </div>
    <br>
  </div>

  <br><br><br><br><hr><br>
  <div>
    <a th:href="@{/menu}"><button>Main Menu</button></a>
  </div>
```

```
</body>
</html>
```

Client Side scripting [Javascript Code]

NA

Server Code [Java]

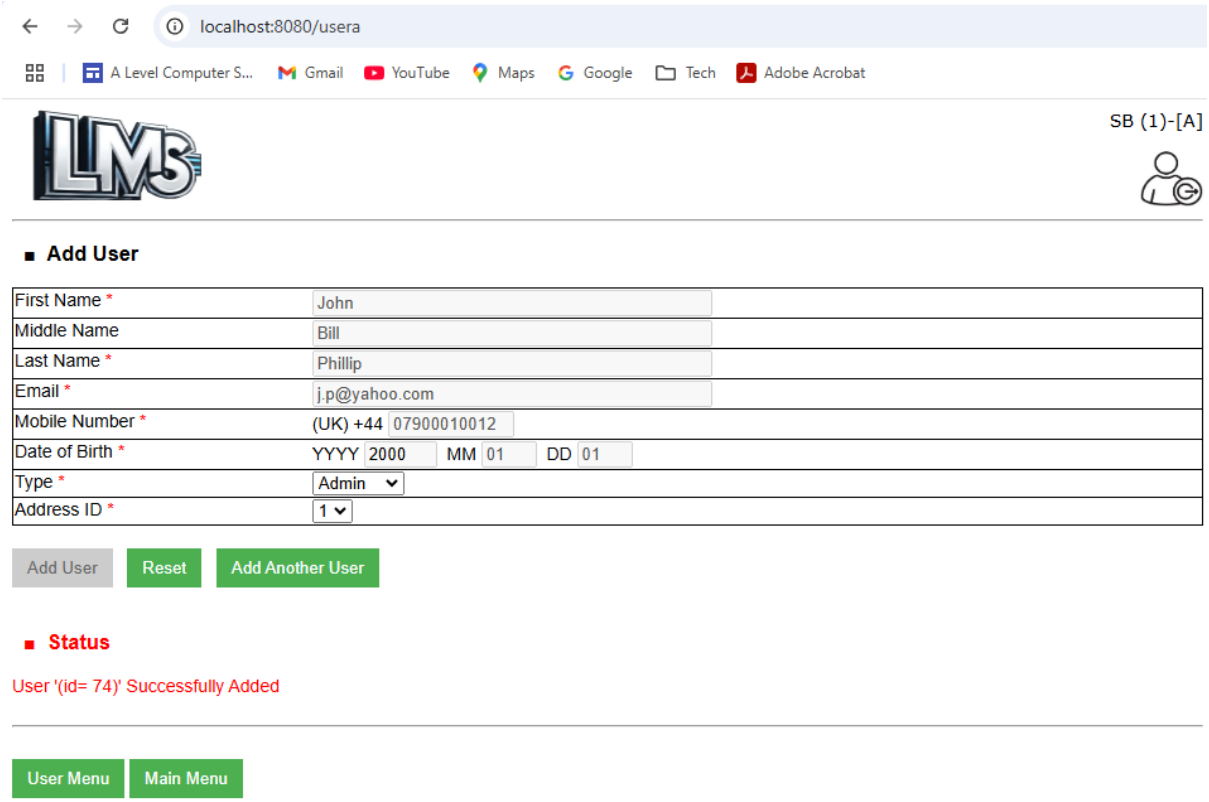
Model/Data Object Class [Java Code] => Git Hub Link

```java
public class User {
    @Id // Specifies its an ID column
    @Column(name = "user_id") // Name of the column in DB
    @GeneratedValue(strategy = GenerationType.IDENTITY) // DB will auto generate this ID
    private Integer userId;

    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)  // insert/update but no reads
    @Column(name = "password")
    @ColumnDefault("qwerty12") // Default value which can be updated later
    private String password;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "middle_name")
    private String middleName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "email")
    private String email;

    @Column(name = "mobile_number")
    private String mobileNumber;

    @Column(name = "birth")
    private Date birth;

    @Column(name = "type")
    private Character type;

    @Column(name = "last_login")
    private Date lastLogin;

    @ManyToOne
    @JoinColumn(name = "address_id")
    private Address address;

    @Transient
    private LmsFault fault;
}
```

Address => Git Hub Link

```
public class Address {
    @Id
    @Column(name = "address_id")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer addressId;

    @Column(name = "door_number")
    private String doorNumber;

    @Column(name = "line1")
    private String line1;

    @Column(name = "line2")
    private String line2;

    @Column(name = "city")
    private String city;

    @Column(name = "postcode")
    private String postcode;

    @Transient
    private String error;
}
```

Create User



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Add  User</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <script src="../user/js/usera.js"></script>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}">
        <h3>  &#x25A0  Add User</h3>
        <table>
            <tr><td>First Name <label style='color:red'>*</label>
                    <td><input type="text" id="txtFirstName" size="40"></input></tr>
            <tr><td>Middle Name
                    <td><input type="text" id="txtMiddleName" size="40"></input></tr>
            <tr><td>Last Name <label style='color:red'>*</label>
                    <td><input type="text" id="txtLastName" size="40"></input></tr>
            <tr><td>Email <label style='color:red'>*</label>
                    <td><input type="text" id="txtEmail" size="40"></input></tr>
```

```
            <tr><td>Mobile Number <label style='color:red'>*</label>
                <td>(UK) +44 <input type="text" id="txtMobileNumber"
                     maxlength="11" size="9"></input></tr>
            <tr><td>Date of Birth <label style='color:red'>*</label>
                <td>YYYY <input type="text" id="txtYear" maxlength="4" size="3"></input>
                 MM <input type="text" id="txtMonth" maxlength="2" size="1"></input>
                 DD <input type="text" id="txtDate" maxlength="2" size="1"></input>
            <tr><td>Type <label style='color:red'>*</label><td>
                <select id='selType'>
                    <option value='A'>Admin</option>
                    <option value='S'>Staff</option>
                    <option value='M'>Member</option>
                </select></tr>
            <tr><td>Address ID <label style='color:red'>*</label><td>
                <select id='selAddr'>
                    <option value='1'>1</option>
                    <option value='2'>2</option>
                    <option value='3'>3</option>
                </select></tr>
        </table>
        <br><button id="btnAddUser" onclick="fnAddUser();">Add User</button>   <button
             onclick="fnReset();">Reset</button>
          <button id="btnAddAnotherUser" onclick="fnAddAnotherUser();"
             class='dbtn'>Add Another User</button>
    </div>
    <div th:unless="${userType != 'M'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
    <div id="divStatus" style="color:red"></div>
    <br><hr><br>
    <div>
        <a th:href="@{/user}"><button>User Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the User information provided on the page and submits the information
to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnAddUser() {
    resultDivStatus = document.getElementById('divStatus');
    let apiUrl = server + apiContext;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateAdd()) {
        return;
    }
    let thisYear = document.getElementById('txtYear').value;
    let thisMonth = document.getElementById('txtMonth').value;
    let thisDate = document.getElementById('txtDate').value;
    // information to be submitted for saving
```

```
    let payload = {
        firstName: document.getElementById('txtFirstName').value,
        middleName: document.getElementById('txtMiddleName').value,
        lastName: document.getElementById('txtLastName').value,
        email: document.getElementById('txtEmail').value,
        mobileNumber: document.getElementById('txtMobileNumber').value,
        birth: thisYear + "-" + thisMonth + "-" + thisDate,
        type: document.getElementById('selType').value,
        lastLogin: Date.now(),
        address: {
            addressId: document.getElementById('selAddr').value
        }
    }

    // create json body for submitting the Add User request
    let options = {method: "POST", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};

    fetch(apiUrl, options).then(response => {if (!response.ok) {
            //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
            displayError(response.status, resultDivStatus, resultDiv);
            return;
        }
        return response.json();
        })
        .then(dataList => {
           if (dataList.fault) {
            let strFault = "<br>  &#x25A0  Error Details";
            strFault += "<table><tr><th>Http</td><td style='color:red'>" +
             dataList.fault.http + "</td></tr>";
            strFault += "<tr><th>Code</td><td style='color:red'>" + dataList.fault.code +
             "</td></tr>";
            strFault += "<tr><th>Message</td><td style='color:red'>" +
             dataList.fault.message + "</td></tr>";
            strFault += "<tr><th>Path</td><td style='color:red'>" + dataList.fault.path +
             "</td></tr></table>";

            resultDivStatus.innerHTML += strFault;
        } else { // If server processed the request successfully
            let strStatus = "<br><h3>  &#x25A0  Status</h3>";
            strStatus += "User '(id= " + dataList.userId + ")' Successfully Added";
            resultDivStatus.innerHTML = strStatus;

           document.getElementById("btnAddUser").className = "dbtn";
           document.getElementById("btnAddAnotherUser").className = "";
      }});
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles POST requests to save a new User.
     * @param user the User entity to be saved
     * @return the saved User entity
     */
```

```
@PostMapping
public ResponseEntity<User> addUser(@RequestBody User user) {
    log.info("Started UserController::addUser. user name = " + user.getFirstName() + "
   " + user.getLastName());
    return ResponseEntity.status(HttpStatus.CREATED).body(userService.addUser(user));
}
```

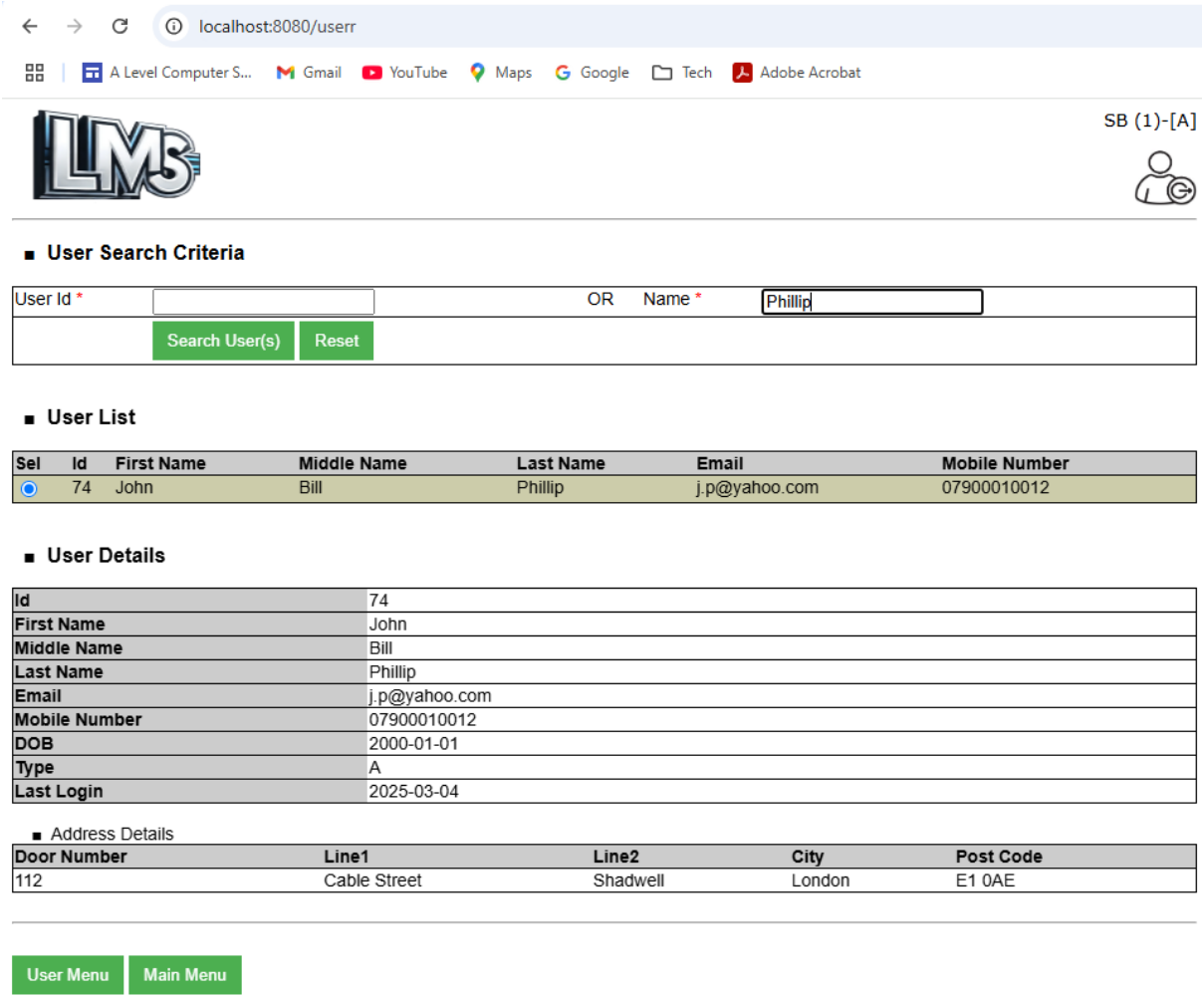
Repository Class [Java Code]

NA

Service Class [Java Code] => Git Hub Link

```
/**
 * Handles requests to save a new User.
 * @param user the User entity to be saved
 * @return the saved User entity
 */
public User addUser(User user) {
    log.info("Started UserService::addUser. user name = " + user.getFirstName() + " " +
         user.getLastName());

    // Gets the Address entity by its associated ID.
    // select * from address where address_id=?
    Address addressDB = addressRepository.findById(
               user.getAddress().getAddressId()).orElse(null);

    user.setAddress(addressDB);

    // System assigns this default password
    user.setPassword(DEFAULT_PASSWORD);

    log.info("Returning UserService::addUser userId = " + user.getUserId());
    //insert into user (address_id,birth,email,first_name,last_login,last_name,
    //     middle_name,mobile_number,password,type) values (?,?,?,?,?,?,?,?,?,?)
    return userRepository.save(user);
}
```

Read User



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Search User</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../user/js/userr.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}" >
        <h3>  &#x25A0  User Search Criteria</h3>
        <table>
            <tr><td>User Id <label style='color:red'>*</label><td><input type="text"
            id="txtUserId"></input>
            <td colspan=6>OR<td>Name <label style='color:red'>*</label><td><input
```

```
            type="text" id="txtUserName"></input>
            <tr><td><td><button onclick = "fnSearchUser();">Search User(s)</button>
            <button onclick = "fnReset();">Reset</button>
        </table>
    </div>
    <div th:unless="${userType != 'M'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
    <div id="divListUser"></div>
    <div id="divDtlsUser"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/user}"><button>User Menu</button></a>
        <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>

</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the search information provided on the page and submits the
information to the server as an API request.
It calls the display user function to show the data on the page
*/
async function fnSearchUser() {
    resultDivList = document.getElementById('divListUser');
    resultDivDtls = document.getElementById('divDtlsUser');
    let apiUrl = server + apiContext;

    if (document.getElementById('txtUserId').value != "") {
        apiUrl += document.getElementById('txtUserId').value;
    } else if (document.getElementById('txtUserName').value != "") {
        apiUrl += 'name/' + document.getElementById('txtUserName').value;
    }
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateSrch()) {
        fnResetSrch();
        return;
    }
    resultDivList.innerHTML = "";

    try {
       const response = await fetch(apiUrl); // Make the API call
       if (!response.ok) {
           fnReset();
           // Handle HTTP errors
           displayError(response.status, resultDivDtls, resultDivList);
           return;
       }

       const data = await response.json(); // Parse JSON response
       globalData = data;
```

```
            //console.log("data = " + data); // csv data items in object
            fnDisplayUserList(data); // Display data on the page
        } catch (error) {
            displayError(error, resultDivDtls, resultDivList);
        }
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles GET requests to get/retrieve an existing User.
     * @param userId the ID of the User to be retrieved
     * @return the requested User entity
     */
    @GetMapping("/{userId}")
    public ResponseEntity<User> getUserById(@PathVariable Integer userId) {
        log.info("Started UserController::getUserById. userId = " + userId);
        return ResponseEntity.status(HttpStatus.OK).body(userService.getUserById(userId));
    }

    /**
     * Handles GET requests to get/retrieve existing Users.
     * @param name the name pattern of the Users to be retrieved
     * @return a list of all requested Users
     */
    @GetMapping("/name/{name}")
    public ResponseEntity<List<User>> getUserByName(@PathVariable String name) {
        log.info("Started UserController::getUserByName. name = " + name);
        return ResponseEntity.status(HttpStatus.OK).body(userService.getUserByName(name));
    }
```

Repository Class [Java Code] => Git Hub Link

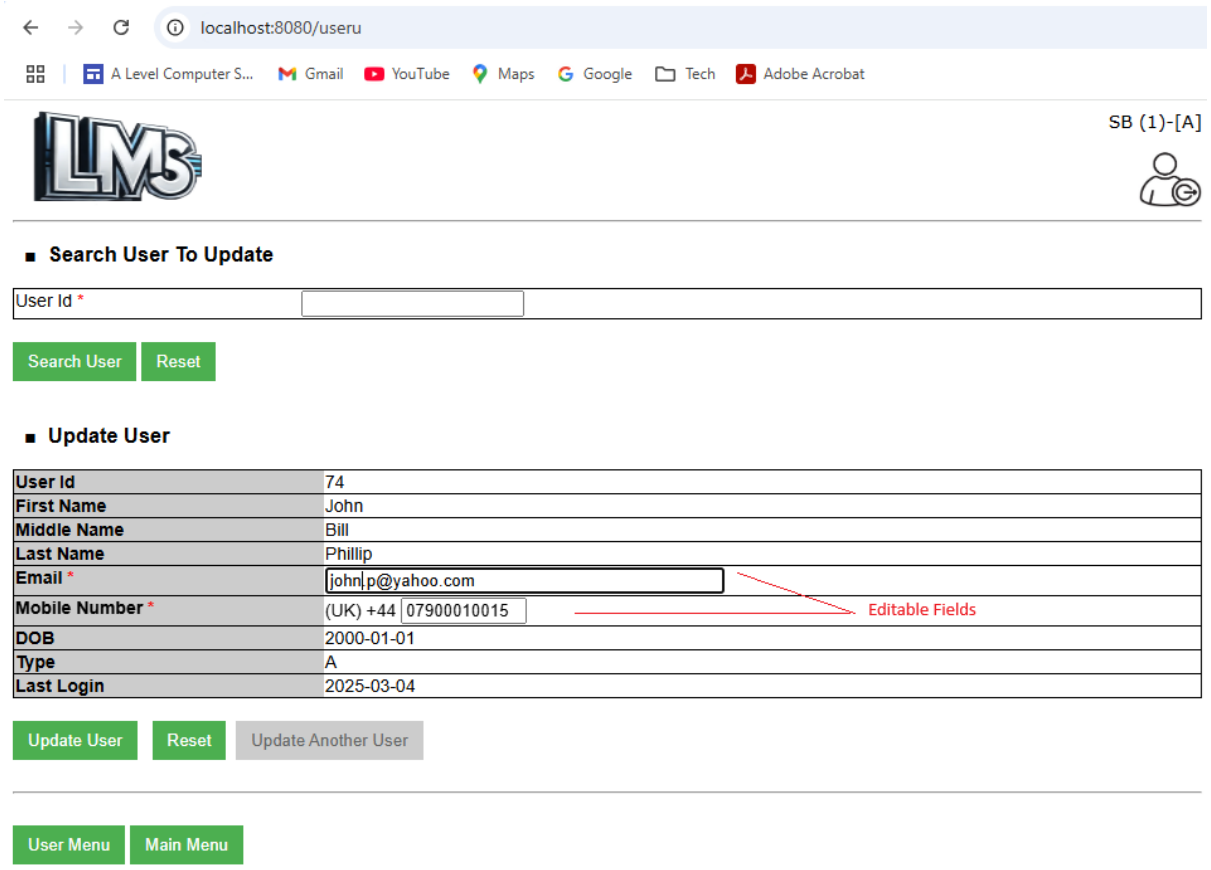
```
    /**
     * Handles the DB call to get/retrieve existing Users by a name pattern.
     * @param name the name pattern of the Users to be retrieved
     * @return a list of the requested User entities
     */
    @Query(
            value = "SELECT * FROM User WHERE first_name LIKE %?1% or last_name LIKE
                    %?1%", nativeQuery = true)
    List<User> findUserByName(String name);
```

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to get/retrieve an existing User.
     * @param userId the ID of the User to be retrieved
     * @return the requested User entity
     */
    public User getUserById(Integer userId) { // SELECT * FROM USER WHERE USER_ID=?;
        log.info("Started UserService::getUserById. userId = " + userId);

        // Gets the user entity by its ID.
```

```
        // select * from user u left join address a on a.address_id=u.address_id where
        // user_id=?
        User userDB = userRepository.findById(userId).orElse(null);

        log.info("Returning UserService::getUserById");
        return userDB;
    }

    /**
     * Handles requests to get/retrieve existing Users by their name pattern.
     * @param name the name pattern of the Users to be retrieved
     * @return a list of all requested Users
     */
    public List<User> getUserByName(String name) {
        log.info("Started UserService::getUserByName. name = " + name);

        // Gets the user entity by its name pattern.
        // select * from user where first_name like %firstName% or first_name like
        // %lastName%
        List<User> userDBList = userRepository.findUserByName(name);

        log.info("Returning UserService::getUserByName");
        return userDBList;
    }
```

Update User



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Update User</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../user/js/useru.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}" >
        <h3>  &#x25A0  Search User To Update</h3>
        <table>
            <tr><td>User Id <label style='color:red'>*</label>
             <td><input type="text" id="txtUserId"></input>
        </table>
        <br><button onclick = "fnSearchUser();">Search User</button>
             <button onclick = "fnResetSrch();">Reset</button>
    </div>
    <div th:unless="${userType != 'M'}" style="color:red">
```

```
        <script>document.write(accessDeniedMsg);</script>
    </div>

    <div id="divUpdateUser"></div>
    <div id="divStatus" style="color:red"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/user}"><button>User Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the User information provided on the page and submits the information
to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnUpdateUser() {
    let apiUrl = server + apiContext + document.getElementById('tdUserId').innerText;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateUpdate()) {
        return;
    }
    // information to be submitted for saving
    let payload = {
        email: document.getElementById('txtEmail').value,
        mobileNumber: document.getElementById('txtMobileNumber').value
    };

    // create json body for submitting the Update User request
    let options = {method: "PUT", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};
    const response = fetch(apiUrl, options);

    let strStatus = "<br><h3>  &#x25A0  Status</h3>";
    strStatus += "User '(id= " + document.getElementById('tdUserId').innerText
              + ")' Successfully Updated";
    resultDivStatus.innerHTML= strStatus;

    toggleFields(true);
    fnResetSrch();

    document.getElementById("btnUpdateUser").className = "dbtn";
    document.getElementById("btnUpdateAnotherUser").className = "";
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles PUT requests to update an existing User.
     * @param userId the ID of the User to be updated
     * @param user the User entity with updated information
     * @return the updated User entity
     */
    @PutMapping("/{userId}")
    public ResponseEntity<User> updateUser(@PathVariable Integer userId, @RequestBody User
            user) {
        log.info("Started UserController::updateUser. userId = " + userId);
        return ResponseEntity.status(HttpStatus.OK).body(
            userService.updateUser(userId, user));
    }
```

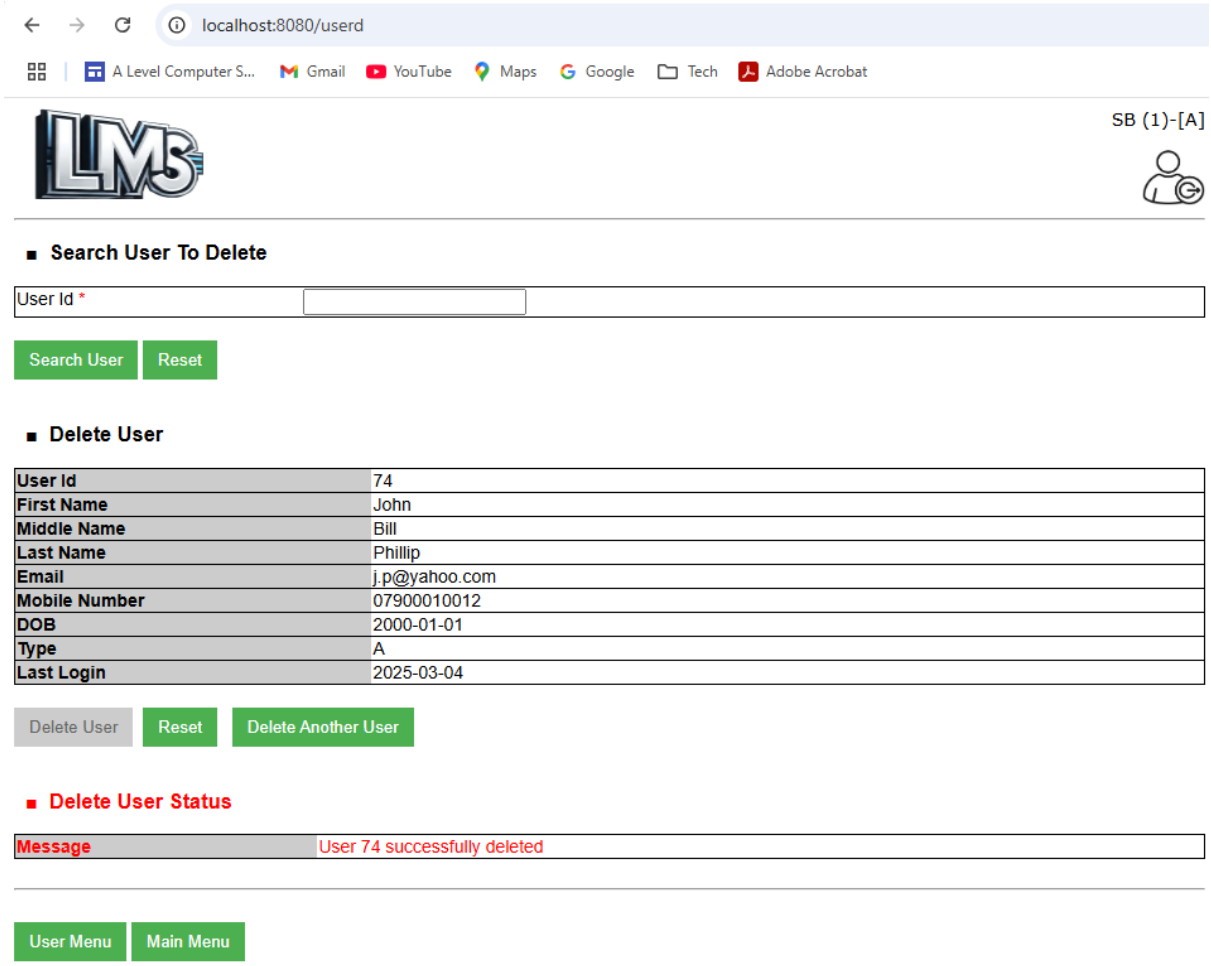
Repository Class [Java Code]

NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save an existing User.
     * @param userId the ID of the User to be updated
     * @param userFromClient the User Object with updated values
     * @return the updated User entity
     */
    public User updateUser(Integer userId, User userFromClient) {
        log.info("Started UserService::updateUser. userId = " + userId);

        // Finds the existing User by ID.
        User userDB = this.getUserById(userId);

        // Map all non null values
        User userUpdated = UserUtils.nonNullMapper(userFromClient, userDB);

        // Saves and returns the updated entity.
        log.info("Returning UserService::updateUser userId = " + userId);

        // update user set email=?,mobile_number=?,password=? where user_id=?
        return userRepository.save(userUpdated);
    }
```

Deleted After confirmation



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Delete User</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../user/js/userd.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div id="divMainWindow">
        <div th:if="${userType != 'M'}">
            <h3>  &#x25A0  Search User To Delete</h3>
            <table>
                <tr><td>User Id <label style='color:red'>*</label>
```

```
                    <td><input type="text" id="txtUserId"></input>
                </table>
                <br><button onclick = "fnSearchUser();">Search User</button>
                 <button onclick = "fnResetSrch();">Reset</button>
            </div>
            <div th:unless="${userType != 'M'}" style="color:red">
                <script>document.write(accessDeniedMsg);</script>
            </div>

            <div id="divDeleteUser"></div>
            <div id="divStatus" style="color:red"></div>
            <br><hr><br>
            <div>
                 <a th:href="@{/user}"><button>User Menu</button></a>
                <a th:href="@{/menu}"><button>Main Menu</button></a>
            </div>
        </div>
        <div id="divModal" style="background-color:#cff5d0;display:none;height:380px;"></div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gets the user ID on the page and submits the information to the server as an
API request.
It also clears any previous output text from any areas on the page
*/
//async function fnDeleteUser() {
async function fnDoAction() {
    let apiUrl = server + apiContext + glbDelUserId;
    //document.getElementById('tdUserId').innerText;
    //console.log("apiUrl = " + apiUrl);

    // create json body for submitting the Delete User request
    let options = {method: "DELETE"};
    const response = await fetch(apiUrl, options);

    if (!response.ok) {
        //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
        displayError(response.status, resultDivStatus, resultDiv);
        return;
    }

    const data = await response.text(); // Text response
    fnDisplayDeleteResponse(data); // Display data on the page
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles DELETE requests to remove a User by ID.
     * @param userId the ID of the User to be deleted
     * @return a success/failure message
     */
    @DeleteMapping("/{userId}")
    public ResponseEntity<String> deleteUserById(@PathVariable Integer userId) {
        log.info("Started UserController::deleteUserById. userId = " + userId);
        return ResponseEntity.status(HttpStatus.OK).body(
             userService.deleteUserById(userId));
    }
```

Repository Class [Java Code]

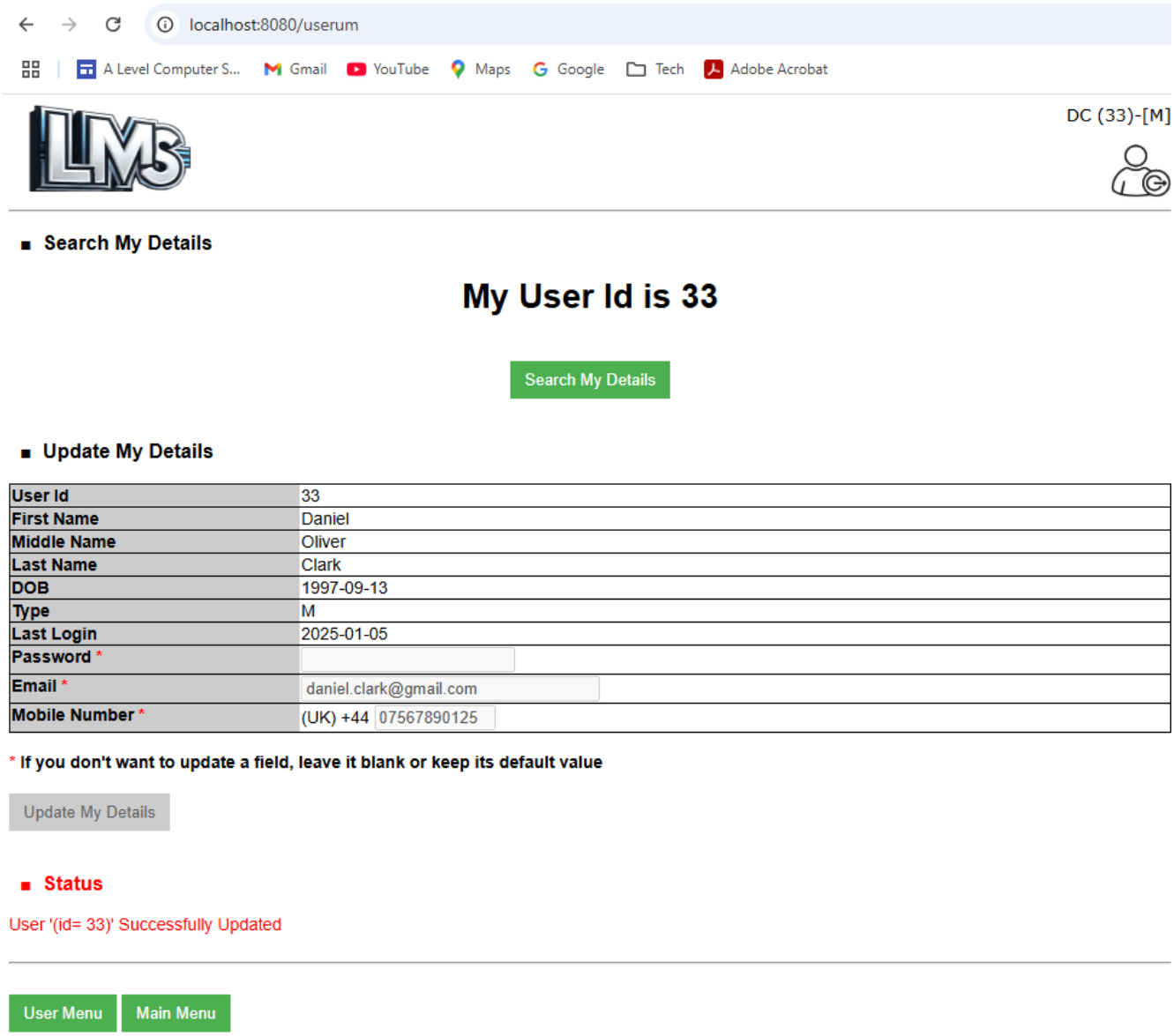
NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to delete a User by ID.
     * @param userId the ID of the User to be deleted
     * @return a success/failure message of deletion
     */
    public String deleteUserById(Integer userId) {
        log.info("Started UserService::deleteUserById. userId = " + userId);

        // Finds the existing User by ID.
        User userDB = this.getUserById(userId);

        // Check if the User has any book(s) issued
        // SELECT COUNT(*) FROM TRANSACTION WHERE user_id =? and returned=false
        int userBorrows = transactionRepository.findCountOfOpenTransactionsByUser(userId);

        // User cannot be deleted if they have borrowed books and not yet returned
        boolean userHasBorrowedBooks = userBorrows > 0;
        if (userHasBorrowedBooks) {
            log.info("User " + userId + " cannot be deleted. User has "
                    + userBorrows + " book(s) issued");
            log.info("Returning UserService::deleteUserById. userId = " + userId);
            return "error:User " + userId + " cannot be deleted. User has "
                    + userBorrows + " book(s) issued";
        }

        // Check if the User has any past transactions
        // SELECT COUNT(*) FROM TRANSACTION WHERE user_id =?
        int userHistoryEntries =
                    transactionRepository.findCountOfAllTransactionsByUser(userId);

        // User cannot be deleted if they have a record of any past transactions because
        // the transaction will first have to be deleted from the system to protect
        // referential integrity of the Database
        boolean userHasHistoricalTransactions = userHistoryEntries > 0;
        if (userHasHistoricalTransactions) {
            log.info("User " + userId + " cannot be deleted. User has " +
```

```
                userHistoryEntries + " historical transaction(s)");
        String strError = "error:User " + userId + " cannot be deleted. " +
                "User has " + userHistoryEntries + " historical transaction(s)";
        strError += "<br>Details: SQL Error: 1451, SQLState: 23000 Cannot delete or "
                    + "update a parent row: a foreign key constraint fails";
        strError += "<br>Please escalate to technical support";

        log.info("Returning UserService::deleteUserById. userId = " + userId);
        return strError;
    }

    // Deletes the user entity by its ID.
    // delete from user where user_id=?
    userRepository.deleteById(userId);
    log.info("Returning UserService::deleteUserById. userId = " + userId);
    return "success:User " + userId + " successfully deleted";
}
```

Update Member User (Self)



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Update Member User</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../user/js/userum.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
```

```
    <h3>  &#x25A0  Search My Details</h3>
    <div id="divSrchUser" style="text-align: center;">
        <h1>My User Id is <span id="txtUserId" th:text="${userId}"></span></h1>
        <br><button id='btnSearchUser' onclick = "fnSearchUser();">
                    Search My Details</button>
    </div>

    <div id="divUpdateUser"></div>
    <div id="divStatus" style="color:red"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/user}"><button>User Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

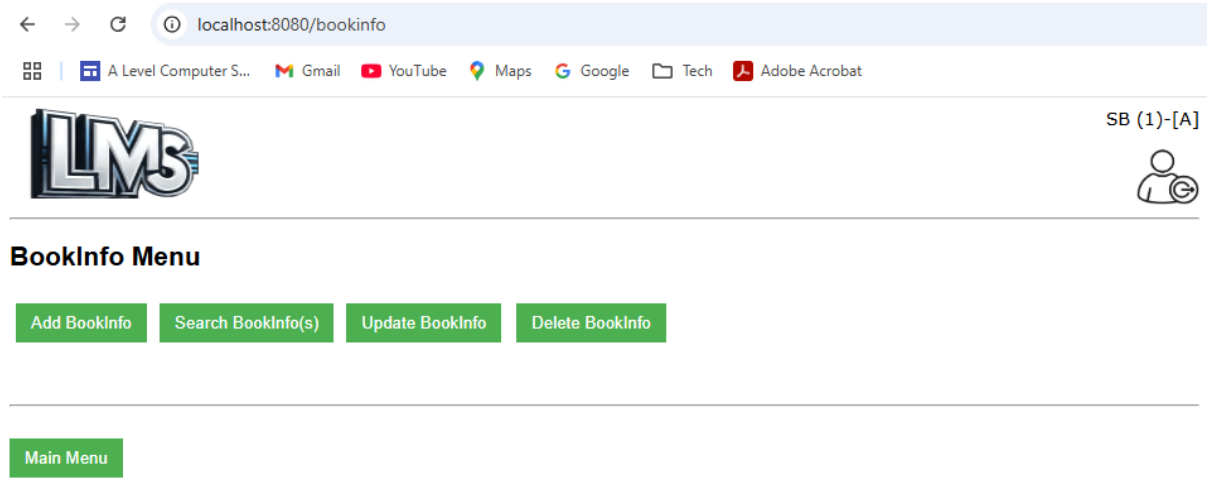
```
/*
This function gathers the User information provided on the page and submits the information
to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnUpdateUser() {
    let apiUrl = server + apiContext + document.getElementById('txtUserId').innerText;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateUpdate()) {
        return;
    }

    let payload = {
        password: document.getElementById('txtPassword').value,
        email: document.getElementById('txtEmail').value,
        mobileNumber: document.getElementById('txtMobileNumber').value
    };

    // create json body for submitting the Update User request
    let options = {method: "PUT", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};
    const response = fetch(apiUrl, options);

    let strStatus = "<br><h3>  &#x25A0  Status</h3>";
    strStatus += "User '(id= " + document.getElementById('tdUserId').innerText
             + ")' Successfully Updated";
    resultDivStatus.innerHTML= strStatus;

    toggleFields(true);
    document.getElementById("btnUpdateUser").className = "dbtn";
}
```

Server Code [Java]

Same as Update User Server Code details

CRUD BookInfo

BookInfo Menu



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>LMS BookInfo Menu Page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div>
        <h2>BookInfo Menu</h2>
        <div class="menubox" th:if="${userType != 'M'}">
            <a th:href="@{/bookinfoa}"><button>Add BookInfo</button></a>
        </div>
        <div class="menubox" >
            <a th:href="@{/bookinfor}"><button>Search BookInfo(s)</button></a>
        </div>
        <div class="menubox" th:if="${userType != 'M'}">
            <a th:href="@{/bookinfou}"><button>Update BookInfo</button></a>  
            <a th:href="@{/bookinfod}"><button>Delete BookInfo</button></a>
        </div>
        <br>
    </div>
    <br><br><br><br><hr><br>
    <div>
        <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

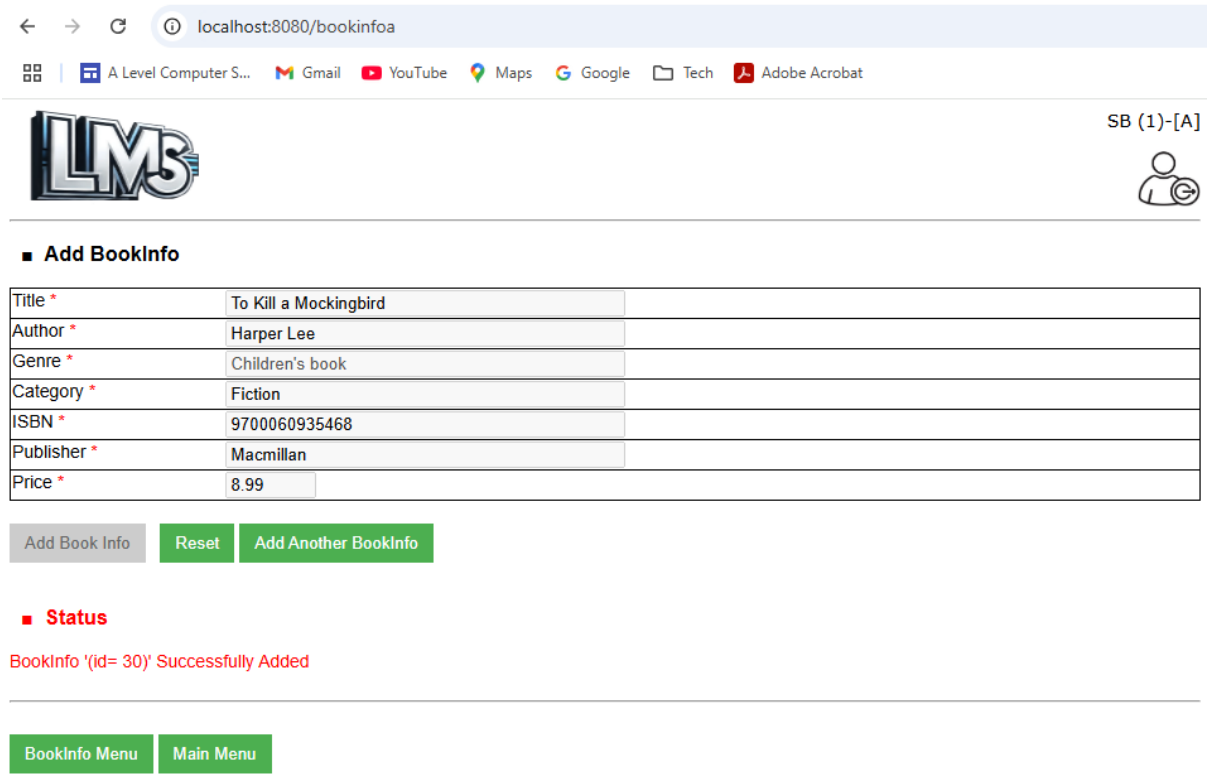
Client Side scripting [Javascript Code] => Git Hub Link

Server Code [Java]

Model/Data Object Class [Java Code] => Git Hub Link

```
public class BookInfo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "bookinfo_id")
    private Integer bookInfoId;

    @Column(name = "title")
    protected String title;

    @Column(name = "author")
    private String author;

    @Column(name = "genre")
    private String genre;

    @Column(name = "category")
    private String category;

    @Column(name = "isbn")
    private String isbn;

    @Column(name = "publisher")
    private String publisher;

    @Column(name = "price")
    private Double price;

    @Column(name = "total_quantity")
    private Integer totalQuantity = 0;

    @Transient
    private LmsFault fault;

}
```

Create BookInfo



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Add BookInfo</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../bookinfo/js/bookinfoa.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}">
        <h3>  &#x25A0  Add BookInfo</h3>
        <table>
            <tr><td>Title <label style='color:red'>*</label><td><input type="text"
                    id="txtTitle" size="40"></input></tr>
            <tr><td>Author <label style='color:red'>*</label>
                    <td><input type="text" id="txtAuthor" size="40"></input></tr>
            <tr><td>Genre <label style='color:red'>*</label>
                    <td><input type="text" id="txtGenre" size="40"></input></tr>
            <tr><td>Category <label style='color:red'>*</label>
                    <td><input type="text" id="txtCategory" size="40"></input></tr>
            <tr><td>ISBN <label style='color:red'>*</label>
```

```
                    <td><input type="text" id="txtIsbn" size="40"></input></tr>
                <tr><td>Publisher <label style='color:red'>*</label>
                    <td><input type="text" id="txtPublisher" size="40"></input>
                <tr><td>Price <label style='color:red'>*</label>
                    <td><input type="text" id="txtPrice" maxlength="6"
                 size="5"></input></tr>
            </table><br>
            <button id="btnAddBookInfo" onclick="fnAddBookInfo();">Add Book Info</button>
                   <button onclick="fnReset();">Reset</button>
            <button id="btnAddAnotherBookInfo" onclick="fnAddAnotherBookInfo();"
                    class="dbtn">Add Another BookInfo</button>
        </div>
        <div th:unless="${userType != 'M'}" style="color:red">
            <script>document.write(accessDeniedMsg);</script>
        </div>
        <div id="divStatus" style="color:red"></div>

        <br><hr><br>
        <div>
            <a th:href="@{/bookinfo}"><button>BookInfo Menu</button></a>
            <a th:href="@{/menu}"><button>Main Menu</button></a>
        </div>
    </body>
    </html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the BookInfo information provided on the page and submits the
information to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnAddBookInfo() {
    resultDivStatus = document.getElementById('divStatus');
    let apiUrl = server + apiContext;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateAdd()) {
        return;
    }
    // information to be submitted for saving
    let payload = {
        title: document.getElementById('txtTitle').value,
        author: document.getElementById('txtAuthor').value,
        genre: document.getElementById('txtGenre').value,
        category: document.getElementById('txtCategory').value,
        isbn: document.getElementById('txtIsbn').value,
        publisher: document.getElementById('txtPublisher').value,
        price: document.getElementById('txtPrice').value
    }

    // create json body for submitting the Add BookInfo request
    let options = {method: "POST", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};

    fetch(apiUrl, options).then(response => {
```

```
        if (!response.ok) { // process if bad response
            //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
            displayError(response.status, resultDivStatus, resultDiv);
            return;
        }
        return response.json(); // process if good response
        })
        .then(dataList => {
        if (dataList.fault) { // If server encountered an error
            let strFault = "<br><h3>  &#x25A0  Error Details</h3>";
            strFault += "<table><tr><th width=30%>Http</td><td style='color:red'>"
                    + dataList.fault.http + "</td></tr>";
            strFault += "<tr><th>Code</td><td style='color:red'>" + dataList.fault.code
                    + "</td></tr>";
            strFault += "<tr><th>Message</td><td style='color:red'>"
                    + dataList.fault.message + "</td></tr>";
            strFault += "<tr><th>Path</td><td style='color:red'>" + dataList.fault.path
                    + "</td></tr></table>";

            resultDivStatus.innerHTML += strFault;
        } else { // If server processed the request successfully
            let strStatus = "<br><h3>  &#x25A0  Status</h3>";
            strStatus += "BookInfo '(id= " + dataList.bookInfoId + ")' Successfully Added";
            resultDivStatus.innerHTML = strStatus;

            // Disable the fields once the request is submitted so no further changes can
            // be done on the same form
            toggleFields(true);
            document.getElementById("btnAddBookInfo").className = "dbtn";
            document.getElementById("btnAddAnotherBookInfo").className = "";

        }});
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles POST requests to save a new BookInfo.
     * @param bookInfo the BookInfo entity to be saved
     * @return the saved BookInfo entity
     */
    @PostMapping
    public ResponseEntity<BookInfo> addBookInfo(@RequestBody BookInfo bookInfo) {
        log.info("Started BookInfoController::addBookInfo. bookInfoId = " +
                                        bookInfo.getBookInfoId());
        return ResponseEntity.status(HttpStatus.CREATED).body(
                        bookInfoService.addBookInfo(bookInfo));
    }
```

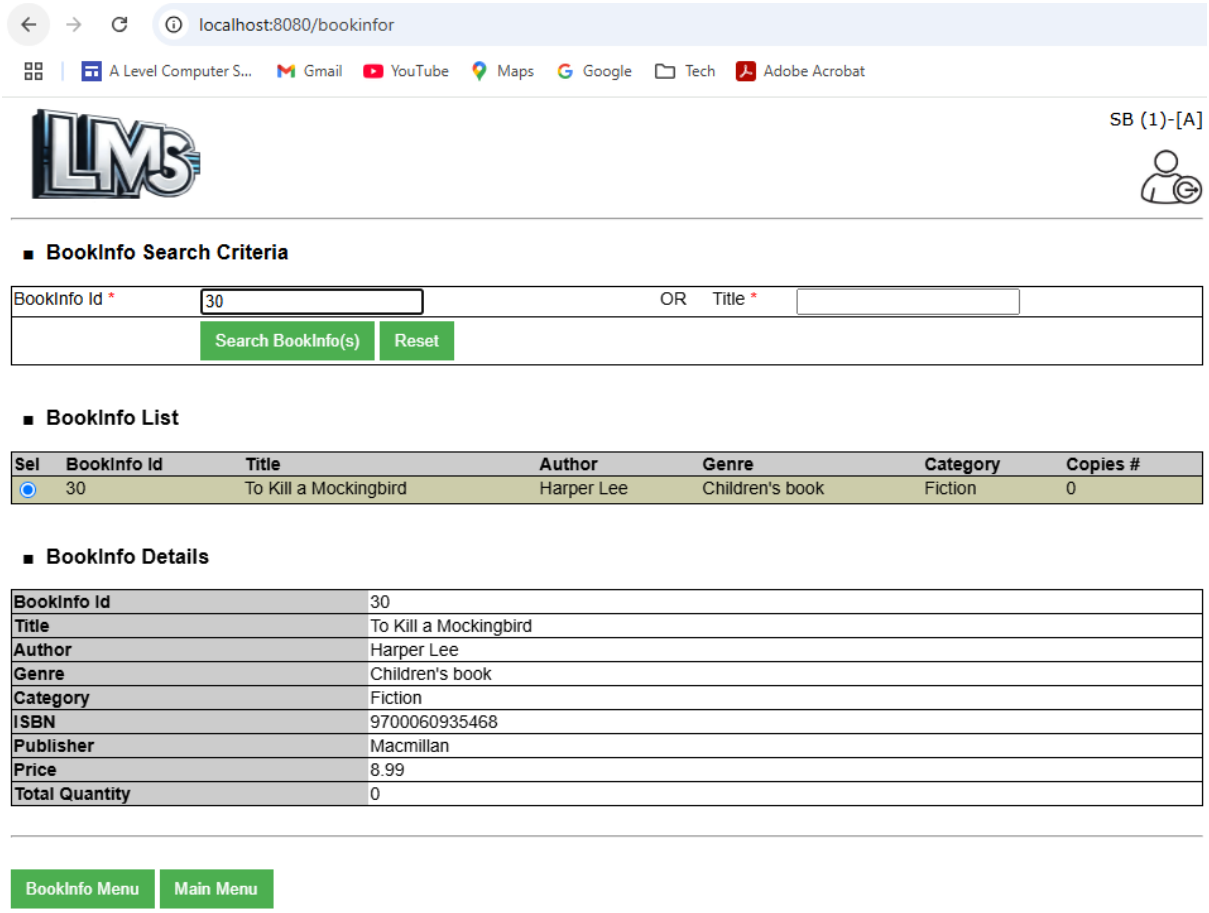
Repository Class [Java Code]

NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save a new BookInfo.
     * @param bookInfo the BookInfo entity to be saved
     * @return the saved BookInfo entity
     */
    public BookInfo addBookInfo(BookInfo bookInfo) {
        log.info("Started BookInfoService::addBookInfo. BookInfo Title = " +
                          bookInfo.getTitle());

        // insert into bookinfo (author,category,genre,isbn,price,publisher,
        //      title,total_quantity) values (?,?,?,?,?,?,?,?,?)
        return bookInfoRepository.save(bookInfo);
    }
```

Read BookInfo



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Search BookInfo</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../bookinfo/js/bookinfor.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
       <div>
        <h3>  &#x25A0  BookInfo Search Criteria</h3>
        <table>
            <tr><td>BookInfo Id <label style='color:red'>*</label>
                <td><input type="text" id="txtBookInfoId"></input>
                <td colspan=6>OR<td>Title <label style='color:red'>*</label>
                <td><input type="text" id="txtBookInfoTitle"></input>
            <tr><td><td><button onclick = "fnSearchBookInfo();">Search BookInfo(s)</button>
            <button onclick = "fnReset();">Reset</button>
        </table>
    </div>
    <div id="divListBookInfo"></div>
    <div id="divDtlsBookInfo"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/bookinfo}"><button>BookInfo Menu</button></a>  <a
th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the search information provided on the page and submits the
information to the server as an API request.
It calls the display BookInfo function to show the data on the page
*/
async function fnSearchBookInfo() {
    resultDivList = document.getElementById('divListBookInfo');
    resultDivDtls = document.getElementById('divDtlsBookInfo');
    let apiUrl = server + apiContext;
    if (document.getElementById('txtBookInfoId').value != "") {
        apiUrl += document.getElementById('txtBookInfoId').value;
    } else if (document.getElementById('txtBookInfoTitle').value != "") {
        apiUrl += "title/" + document.getElementById('txtBookInfoTitle').value;
    }
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateSrch()) {
        fnResetSrch();
```

```
        return;
    }

    resultDivList.innerHTML = "";

    try {
       const response = await fetch(apiUrl); // Make the API call
       if (!response.ok) {
           fnReset();
           displayError(response.status, resultDivDtls, resultDivList);
           return;
       }

       const data = await response.json(); // Parse JSON response
       globalData = data;
       //console.log("data = " + data); // csv data items in object
             fnDisplayBookInfoList(data); // Display data on the page
       } catch (error) {
             displayError(error, resultDivDtls, resultDivList);
       }
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles GET requests to get/retrieve an existing BookInfo.
     * @param bookInfoId the ID of the BookInfo to be retrieved
     * @return the requested BookInfo entity
     */
    @GetMapping("/{bookInfoId}")
    public ResponseEntity<BookInfo> getBookInfoById(@PathVariable Integer bookInfoId) {
        log.info("Started BookInfoController::getBookInfoById. bookInfoId = "
             + bookInfoId);
        return ResponseEntity.status(HttpStatus.OK).body(
                    bookInfoService.getBookInfoById(bookInfoId));
    }

    /**
     * Handles GET requests to get/retrieve existing BookInfos.
     * @param title the title pattern of the BookInfos to be retrieved
     * @return a list of all requested BookInfos
     */
    @GetMapping("/title/{title}")
    public ResponseEntity<List<BookInfo>> getBookInfoByTitle(@PathVariable String title) {
        log.info("Started BookInfoController::getBookInfoByTitle. title = " + title);
        return ResponseEntity.status(HttpStatus.OK).body(
                    bookInfoService.getBookInfoByTitle(title));
    }
```

Repository Class [Java Code] => Git Hub Link

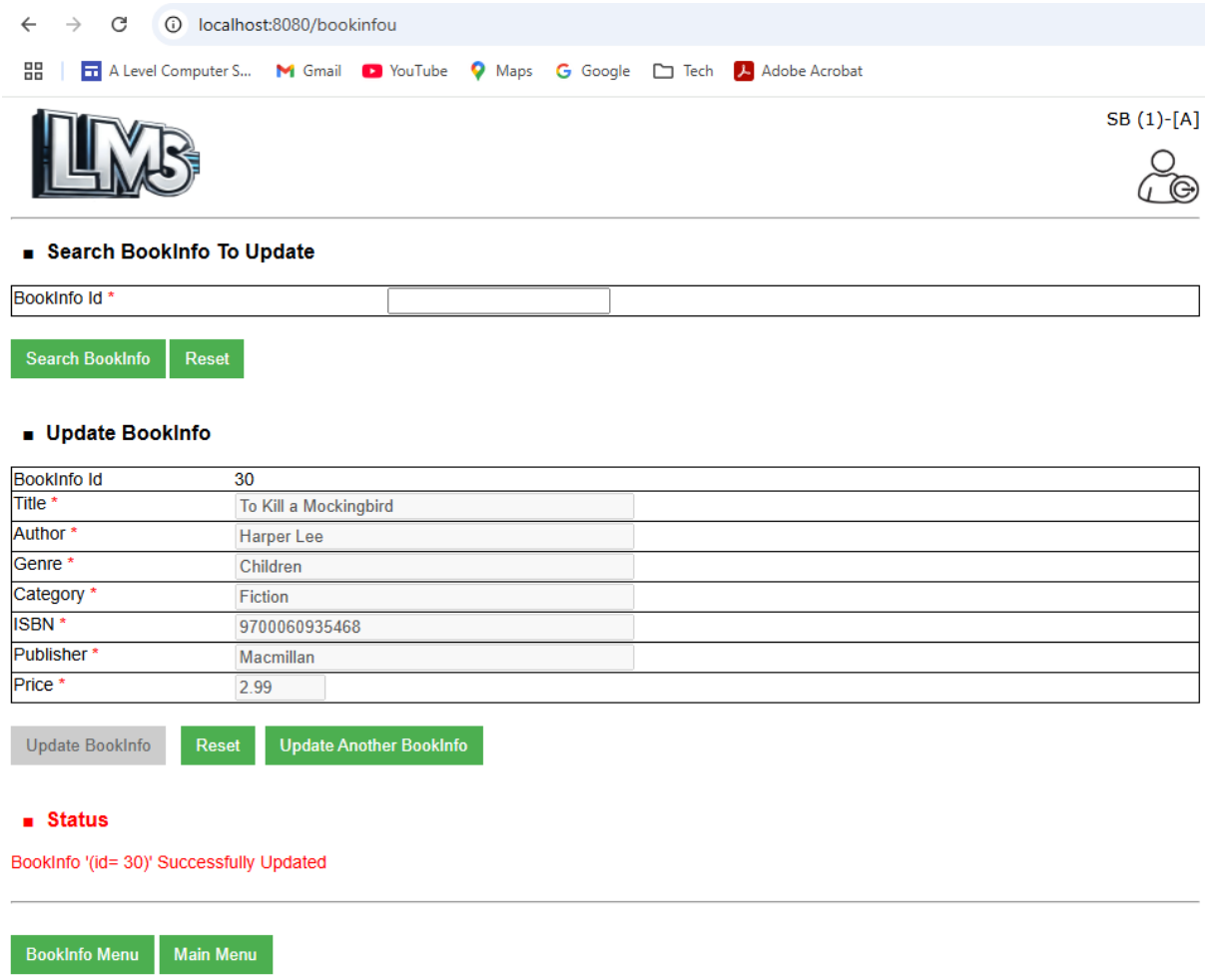
```
    /**
     * Handles the DB call to get/retrieve all BookInfos by a title pattern.
```

```
     * @param title the title pattern of the BookInfos to be retrieved
     * @return a list of the requested BookInfo entities
     */
    @Query(
            value = "SELECT * FROM BOOKINFO WHERE title LIKE %?1%",
            nativeQuery = true)
    List<BookInfo> findBookInfoByTitle(String title);
```

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to get/retrieve an existing BookInfo.
     * @param bookInfoId the Id of BookInfo to be retrieved
     * @return the requested BookInfo entity
     */
    public BookInfo getBookInfoById(Integer bookInfoId) {
        log.info("Started BookInfoService::getBookInfoById. bookInfoId = " + bookInfoId);

        // Gets the bookinfo entity by its ID.
        // select * from bookinfo where bookinfo_id=?
        BookInfo bookInfoDB = bookInfoRepository.findById(bookInfoId).orElse(null);


        log.info("Returning BookInfoService::getBookInfoById");
        return bookInfoDB;
    }

    /**
     * Handles requests to get/retrieve existing BookInfos by their name pattern.
     * @param title the title pattern of the BookInfos to be retrieved
     * @return a list of all requested BookInfos
     */
    public List<BookInfo> getBookInfoByTitle(String title) {
        log.info("Started BookInfoService::getBookInfoByTitle. title = " + title);

        // Gets the BookInfo entity list by its title pattern.
        // SELECT * FROM BOOKINFO WHERE title LIKE %?%
        List<BookInfo> bookInfoDBList = bookInfoRepository.findBookInfoByTitle(title);

        log.info("Returning BookInfoService::getBookInfoByTitle");
        return bookInfoDBList;
    }
```

Update BookInfo



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Update BookInfo</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../bookinfo/js/bookinfou.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}">
        <h3>  &#x25A0  Search BookInfo To Update</h3>
        <table>
            <tr><td>BookInfo Id <label style='color:red'>*</label>
             <td><input type="text" id="txtBookInfoId"></input>
```

```
        </table>
        <br><button onclick = "fnSearchBookInfo();">Search BookInfo</button>
        <button onclick = "fnResetSrch();">Reset</button>
    </div>
    <div th:unless="${userType != 'M'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>

  <div id="divUpdateBookInfo"></div>
  <div id="divStatus" style="color:red"></div>
  <br><hr><br>
  <div>
      <a th:href="@{/bookinfo}"><button>BookInfo Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
  </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the BookInfo information provided on the page and submits the
information to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnUpdateBookInfo() {
    let apiUrl = server + apiContext + document.getElementById('tdBookInfoId').innerText;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateUpdate()) {
        return;
    }
    // information to be submitted for saving
    let payload = {
        title: document.getElementById('txtTitle').value,
        author: document.getElementById('txtAuthor').value,
        genre: document.getElementById('txtGenre').value,
        category: document.getElementById('txtCategory').value,
        isbn: document.getElementById('txtIsbn').value,
        publisher: document.getElementById('txtPublisher').value,
        price: document.getElementById('txtPrice').value
    }

    // create json body for submitting the Update BookInfo request
    let options = {method: "PUT", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};

    const response = fetch(apiUrl, options);

    let strStatus = "<br><h3>  &#x25A0  Status</h3>";
    strStatus += "BookInfo '(id= " + document.getElementById('tdBookInfoId').innerText
              + ")' Successfully Updated";
    document.getElementById('divStatus').innerHTML= strStatus;
```

```
    toggleFields(true);
    fnResetSrch();

    document.getElementById("btnUpdateBookInfo").className="dbtn";
    document.getElementById("btnUpdateAnotherBookInfo").className="";
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /** Handles PUT requests to update an existing BookInfo.
     * @param bookInfoId the ID of the BookInfo to be updated
     * @param bookInfo the BookInfo entity with updated information
     * @return the updated BookInfo entity
     */
    @PutMapping("/{bookInfoId}")
    public ResponseEntity<BookInfo> updateBookInfo(@PathVariable Integer bookInfoId,
                    @RequestBody BookInfo bookInfo) {
        log.info("Started BookInfoController::updateBookInfo. bookInfoId = " + bookInfoId);
        return ResponseEntity.status(HttpStatus.OK).body(
             bookInfoService.updateBookInfo(bookInfoId, bookInfo));
    }
```

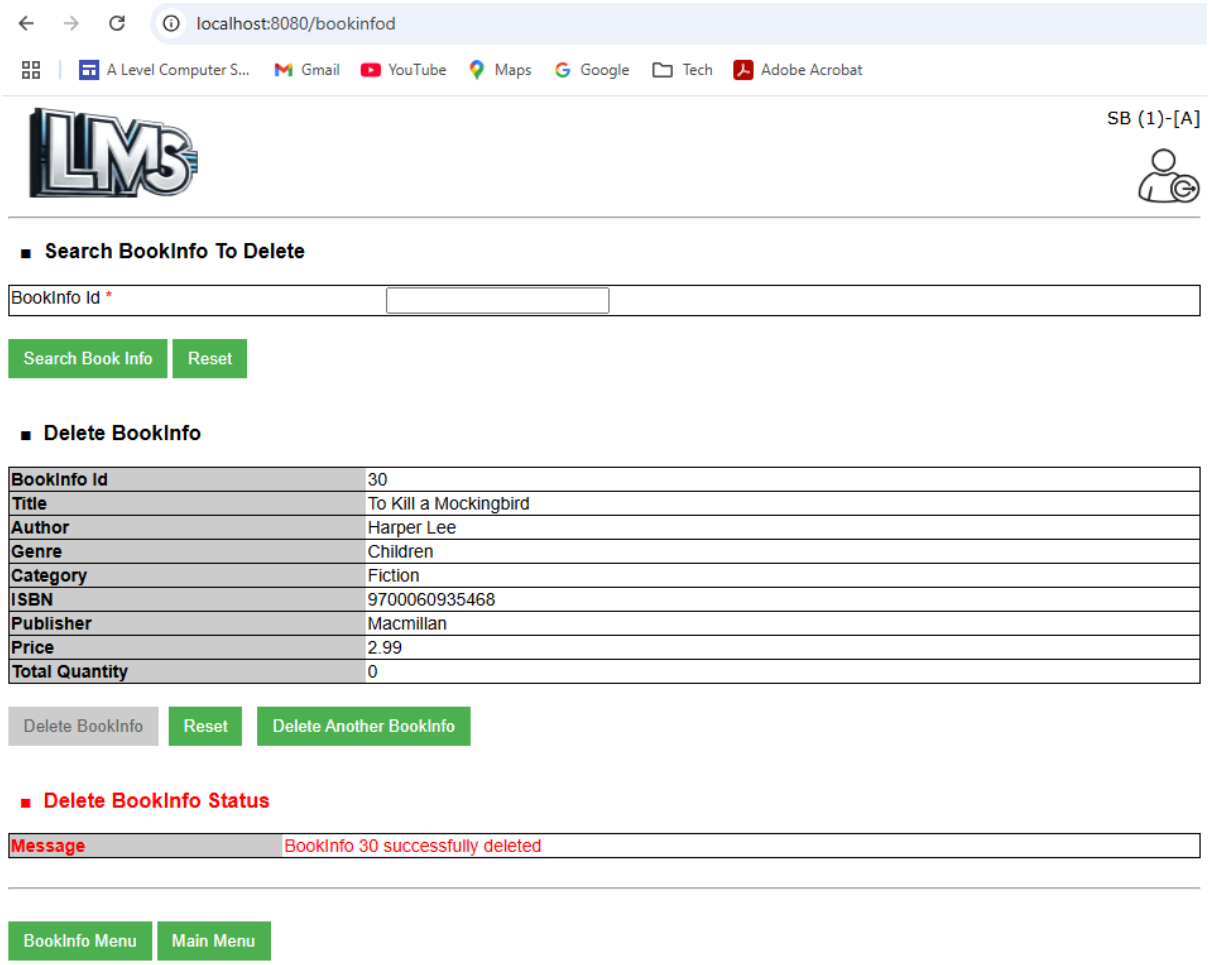
Repository Class [Java Code]

NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save an existing BookInfo.
     * @param bookInfoId the ID of the BookInfo to be updated
     * @param bookInfoFromClient the BookInfo Object with updated values
     * @return the updated BookInfo entity
     */
    public BookInfo updateBookInfo(Integer bookInfoId, BookInfo bookInfoFromClient) {
        log.info("Started BookInfoService::updateBookInfo. bookInfoId = " + bookInfoId);

        // Finds the existing User by ID.
        BookInfo bookInfoDB = this.getBookInfoById(bookInfoId);

        // Map all non null values
        BookInfo bookInfoUpdate = BookInfoUtils.nonNullMapper(bookInfoFromClient,
                                  bookInfoDB);

        // Saves and returns the updated entity.
        log.info("Returning BookInfoService::updateBookInfo");

        // update bookinfo set author=?,category=?,genre=?,isbn=?,price=?,
        // publisher=?,title=? where bookinfo_id=?
        return bookInfoRepository.save(bookInfoUpdate);
    }
```

Deleted BookInfo After confirmation



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Delete BookInfo</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../bookinfo/js/bookinfod.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div id="divMainWindow">
        <div th:if="${userType != 'M'}">
            <h3>  &#x25A0  Search BookInfo To Delete</h3>
            <table>
                <tr><td>BookInfo Id <label style='color:red'>*</label>
                    <td><input type="text" id="txtBookInfoId"></input>
            </table>
```

```
            <br><button onclick = "fnSearchBookInfo();">Search Book Info</button>
            <button onclick = "fnResetSrch();">Reset</button>
        </div>
        <div th:unless="${userType != 'M'}" style="color:red">
            <script>document.write(accessDeniedMsg);</script>
        </div>

        <div id="divDeleteBookInfo"></div>
        <div id="divStatus" style="color:red"></div>
        <br><hr><br>
        <div>
            <a th:href="@{/bookinfo}"><button>BookInfo Menu</button></a>
            <a th:href="@{/menu}"><button>Main Menu</button></a>
        </div>
    </div>
    <div id="divModal" style="background-color:#cff5d0;display:none;height:380px;"></div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gets the BookInfo ID on the page and submits the information to the server as
an API request.
It also clears any previous output text from any areas on the page
*/
//async function fnDeleteBookInfo() {
async function fnDoAction() {
    let apiUrl = server + apiContext + glbDelBookInfoId;
    //console.log("apiUrl = " + apiUrl);

    // create json body for submitting the Delete BookInfo request
    let options = {method: "DELETE"};
    const response = await fetch(apiUrl, options);

    if (!response.ok) {
        //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
        displayError(response.status, resultDivStatus, resultDiv);
        return;
    }

    const data = await response.text(); // Text response
    fnDisplayDeleteResponse(data); // Display data on the page
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles DELETE requests to remove a BookInfo by ID.
     * @param bookInfoId the ID of the BookInfo to be deleted
     * @return a success/failure message
```

```
     */
    @DeleteMapping("/{bookInfoId}")
    public ResponseEntity<String> deleteBookInfo(@PathVariable Integer bookInfoId) {
        log.info("Started BookInfoController::deleteBookInfo. bookInfoId = " + bookInfoId);
        return ResponseEntity.status(HttpStatus.OK).body(
                    bookInfoService.deleteBookInfoById(bookInfoId));
    }
```

Repository Class [Java Code]

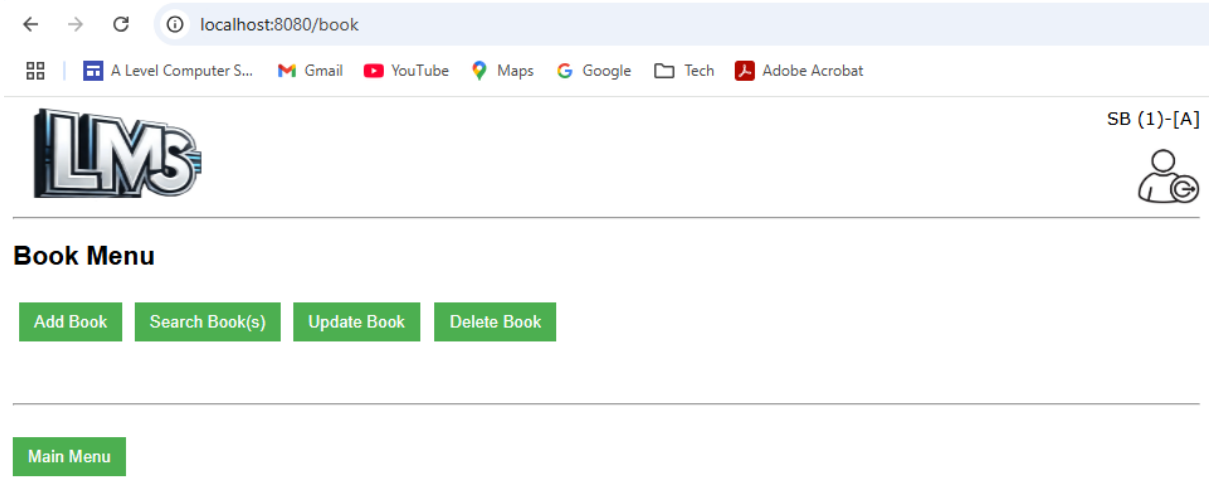
NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to delete a BookInfo by ID.
     * @param bookInfoId the ID of the BookInfo to be deleted
     * @return a success/failure message of deletion
     */
    public String deleteBookInfoById(Integer bookInfoId) {
        log.info("Started BookInfoService::deleteBookInfoById. bookInfoId = "
                    + bookInfoId);

        // Finds the existing BookInfo by ID.
        BookInfo bookInfo = this.getBookInfoById(bookInfoId);

        // Check if the BookInfo has any associated books
        boolean bookInfoHasAssociatedBooks = (bookInfo.getTotalQuantity() > 0);
        if (bookInfoHasAssociatedBooks) {
            log.info("error:BookInfo " + bookInfoId + " cannot be deleted. It has " +
                    bookInfo.getTotalQuantity() + " associated book(s).");
            log.info("Returning BookInfoService::deleteBookInfoById");
            return "error:BookInfo " + bookInfoId + " cannot be deleted. It has " +
                    bookInfo.getTotalQuantity() + " associated book(s).";
        }

        // Deletes the BookInfo entity by its ID.
        // delete from bookinfo where bookinfo_id=?
        bookInfoRepository.deleteById(bookInfoId);
        log.info("Returning BookInfoService::deleteBookInfoById");
        return "success:BookInfo " + bookInfoId + " successfully deleted";
    }
```

CRUD Book

Book Menu



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>LMS Book Menu Page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div>
        <h2>Book Menu</h2>
        <div class="menubox" th:if="${userType != 'M'}">
            <a th:href="@{/booka}"><button>Add Book</button></a>
        </div>
        <div class="menubox">
            <a th:href="@{/bookr}"><button>Search Book(s)</button></a>
        </div>
        <div class="menubox" th:if="${userType != 'M'}">
            <a th:href="@{/booku}"><button>Update Book</button></a>  
            <a th:href="@{/bookd}"><button>Delete Book</button></a>
        </div>
        <br>
    </div>
    <br><br><br><br><hr><br>
    <div>
        <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
 </body>
 </html>
```

Client Side scripting [Javascript Code]

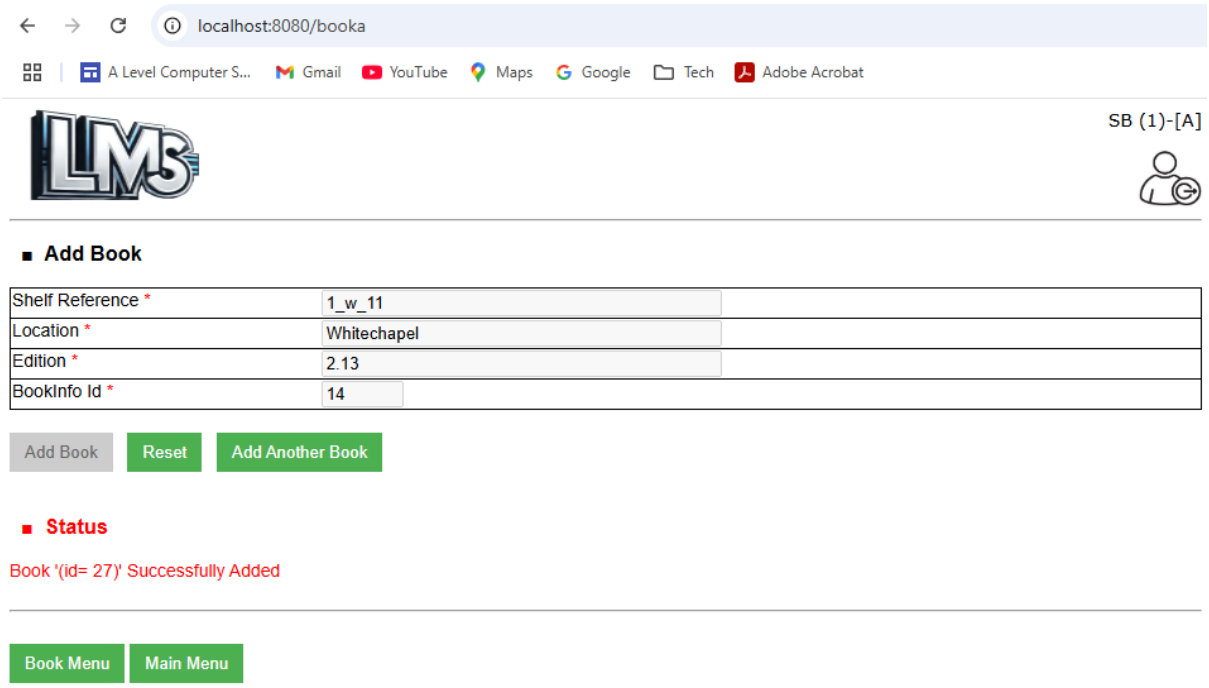
NA

Server Code [Java]

Model/Data Object Class [Java Code] => Git Hub Link

```
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "book_id")
    private Integer bookId;

    @ManyToOne
    @JoinColumn(name = "bookinfo_id")
    private BookInfo bookInfo;

    @Column(name = "shelf_reference")
    private String shelfReference;

    @Column(name = "location")
    private String location;

    @Column(name = "edition")
    private String edition;

    @Column(name = "available")
    private Boolean available = true;

    @Transient
    private LmsFault fault;
}
```

Create Book



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Add Book</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../book/js/booka.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}">
        <h3>  &#x25A0  Add Book</h3>
        <table>
            <tr><td>Shelf Reference <label style='color:red'>*</label>
              <td><input type="text" id="txtShelfReference" size="40"></input></tr>
            <tr><td>Location <label style='color:red'>*</label>
              <td><input type="text" id="txtLocation" size="40"></input></tr>
            <tr><td>Edition <label style='color:red'>*</label>
              <td><input type="text" id="txtEdition" size="40"></input></tr>
            <tr><td>BookInfo Id <label style='color:red'>*</label>
             <td><input type="text" id="txtBookInfoId" maxlength="4" size="4"></input></tr>
        </table>
        <br><button id="btnAddBook" onclick="fnAddBook();">Add Book</button>  
            <button onclick="fnReset();">Reset</button>  
             <button id="btnAddAnotherBook" onclick="fnAddAnotherBook();"
                    class="dbtn">Add Another Book</button>
```

```
    </div>
    <div th:unless="${userType != 'M'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
    <div id="divStatus" style="color:red"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/book}"><button>Book Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the Book information provided on the page and submits the information
to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnAddBook() {
    resultDivStatus = document.getElementById('divStatus');
    let apiUrl = server + apiContext;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateAdd()) {
        return;
    }
    // information to be submitted for saving
    let payload = {
        shelfReference: document.getElementById('txtShelfReference').value,
        location: document.getElementById('txtLocation').value,
        edition: document.getElementById('txtEdition').value,
        bookInfo: {
            bookInfoId: document.getElementById('txtBookInfoId').value
        }
    }

    // create json body for submitting the Add Book request
    let options = {method: "POST", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};

    fetch(apiUrl, options).then(response => {if (!response.ok) {
            //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
            displayError(response.status, resultDivStatus, resultDiv);
            return;
        }
        return response.json();
        })
        .then(dataList => {
        if (dataList.fault) { // If server encountered an error
            let strFault = "<br><h3>  &#x25A0  Error Details</h3>";
            strFault += "<table><tr><th width=30%>Http</td><td style='color:red'>"
                    + dataList.fault.http + "</td></tr>";
            strFault += "<tr><th>Code</td><td style='color:red'>" + dataList.fault.code
```

```
                + "</td></tr>";
            strFault += "<tr><th>Message</td><td style='color:red'>"
                + dataList.fault.message + "</td></tr>";
            strFault += "<tr><th>Path</td><td style='color:red'>" + dataList.fault.path
                + "</td></tr></table>";

            resultDivStatus.innerHTML = strFault;
        } else { // If server processed the request successfully
            let strStatus = "<br><h3>  &#x25A0  Status</h3>";
            strStatus += "Book '(id= " + dataList.bookId + ")' Successfully Added";
            resultDivStatus.innerHTML = strStatus;

            // Disable the fields once the request is submitted so no further changes
            // can be done on the same form
            toggleFields(true);
            document.getElementById("btnAddBook").className = "dbtn";
            document.getElementById("btnAddAnotherBook").className = "";
        }});
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles POST requests to save a new Book.
     * @param book the Book entity to be saved
     * @return the saved Book entity
     */
    @PostMapping
    public ResponseEntity<Book> addBook(@RequestBody Book book) {
        log.info("Started BookController::addBook. bookId = " + book.getBookId());
        return ResponseEntity.status(HttpStatus.CREATED).body(bookService.addBook(book));
    }
```

Repository Class [Java Code]

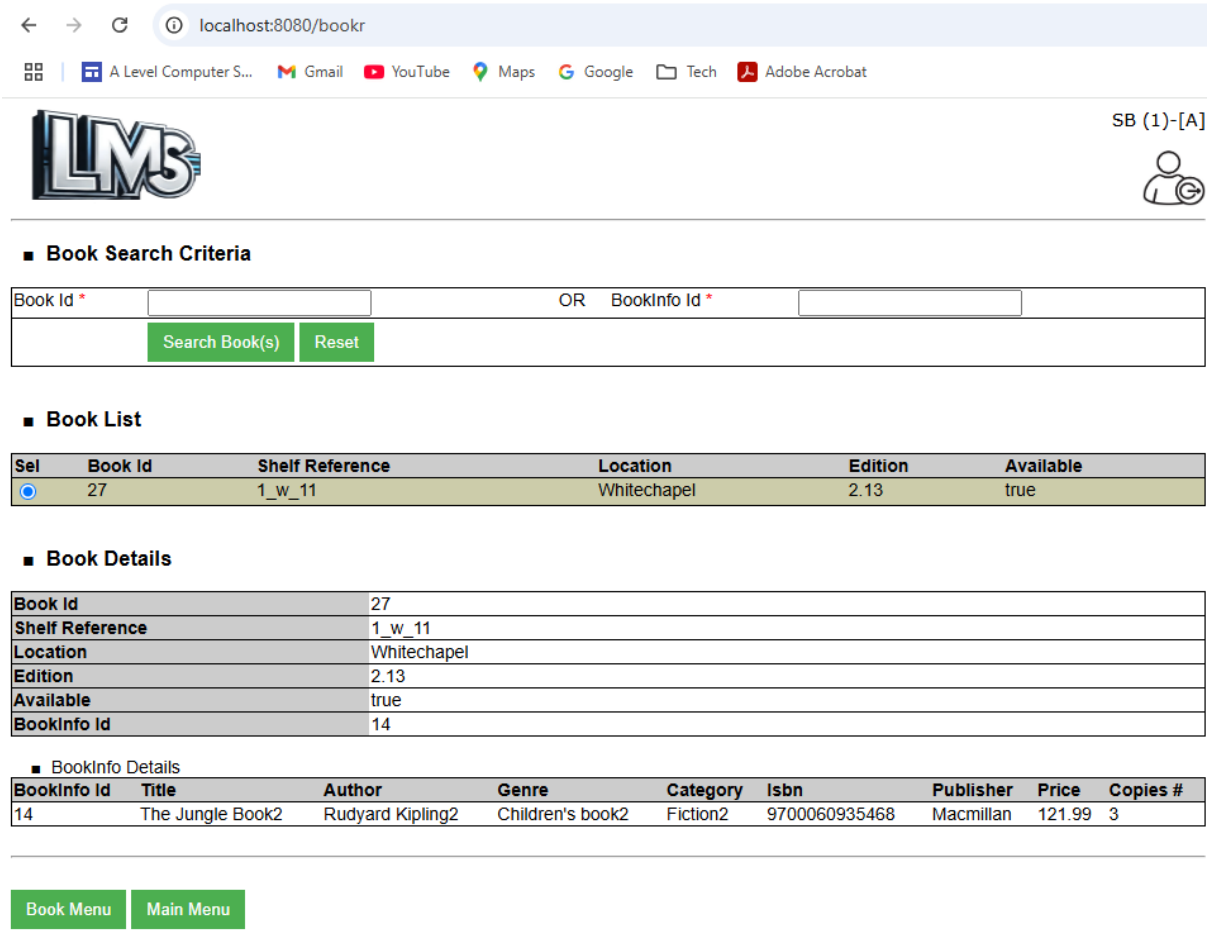
NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save a new Book.
     * @param book the Book entity to be saved
     * @return the saved Book entity
     */
    public Book addBook(Book book) {
        log.info("Started BookService::addBook. book info id = "
                + book.getBookInfo().getBookInfoId());

        // Finds the existing bookInfo by ID from another rest service.
        // Equivalent of BookInfo bookInfoDB = bookInfoRepository.findById(
```

```
        //                  book.getBookInfo().getBookInfoId()).orElse(null);
        // select * from bookinfo where bookinfo_id=?
        String url = BOOKINFO_BASE_URL + book.getBookInfo().getBookInfoId();
        BookInfo bookInfoDB =  restTemplate.getForObject(url, BookInfo.class);


        // Increment the count of total Books in the associated BookInfo entity
        bookInfoDB.incrementTotalQuantity();

        // save the updated BookInfo entity
        // update bookinfo set total_quantity=? where bookinfo_id=?
        bookInfoDB = bookInfoRepository.save(bookInfoDB);

        //book.setAvailable(true); // already set as default in Model class
        // set the updated BookInfo to the current Book
        book.setBookInfo(bookInfoDB);

        log.info("Returning BookService::addBook");

        // insert into book (available,bookinfo_id,edition,location,shelf_reference)
         // values (?,?,?,?,?)
        return bookRepository.save(book);
    }
```

Read Book



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Search Book</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../book/js/bookr.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div id="divSrchUser">
        <h3>  &#x25A0  Book Search Criteria</h3>
        <table>
            <tr><td>Book Id <label style='color:red'>*</label>
                <td><input type="text" id="txtBookId"></input>
                <td colspan=6>OR<td>BookInfo Id <label style='color:red'>*</label>
```

```
                <td><input type="text" id="txtBookInfoId"></input>
            <tr><td><td><button onclick = "fnSearchBook();">Search Book(s)</button>
            <button onclick = "fnReset();">Reset</button>
        </table>
    </div>
    <div id="divListBook"></div>
    <div id="divDtlsBook"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/book}"><button>Book Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/* This function gathers the search information provided on the page and submits the
information to the server as an API request.
It calls the display Book function to show the data on the page
*/
async function fnSearchBook() {
    resultDivList = document.getElementById('divListBook');
    resultDivDtls = document.getElementById('divDtlsBook');
    let apiUrl = server + apiContext;
        if (document.getElementById('txtBookId').value != "") {
        apiUrl += document.getElementById('txtBookId').value;
    } else if (document.getElementById('txtBookInfoId').value != "") {
        apiUrl += 'all/' + document.getElementById('txtBookInfoId').value;
    }
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateSrch()) {
        fnResetSrch();
        return;
    }
    resultDivList.innerHTML = "";

      try {
            const response = await fetch(apiUrl); // Make the API call
            if (!response.ok) {
                fnReset();
           //throw new Error(`Database Error: ${response.status}`); // Handle HTTP errors
           displayError(response.status, resultDivDtls, resultDivList);
           return;
            }

            const data = await response.json(); // Parse JSON response
            globalData = data;
            //console.log("data = " + data); // csv data items in object
            fnDisplayBookList(data); // Display data on the page
      } catch (error) {
            displayError(error, resultDivDtls, resultDivList);
      }
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles GET requests to get/retrieve an existing Book.
     * @param bookId the ID of the Book to be retrieved
     * @return the requested Book entity
     */
    @GetMapping("/{bookId}")
    public ResponseEntity<Book> getBookById(@PathVariable Integer bookId) {
        log.info("Started BookController::getBookById. bookId = " + bookId);
        return ResponseEntity.status(HttpStatus.OK).body(bookService.getBookById(bookId));
    }


    /**
     * Handles GET requests to get/retrieve all Books of a particular BookInfo
     * @param bookInfoId the ID of the BookInfo associated to Books to be retrieved
     * @return a list of all associated Books
     */
    @GetMapping("/all/{bookInfoId}")
    public ResponseEntity<List<Book>> getAllBooksOfBookInfo(
                                @PathVariable Integer bookInfoId) {
        log.info("Started BookController::getAllBooksOfBookInfo. bookInfoId = "
                + bookInfoId);
        return ResponseEntity.status(HttpStatus.OK).body(
                bookService.getAllBooksOfBookInfo(bookInfoId));
    }
```

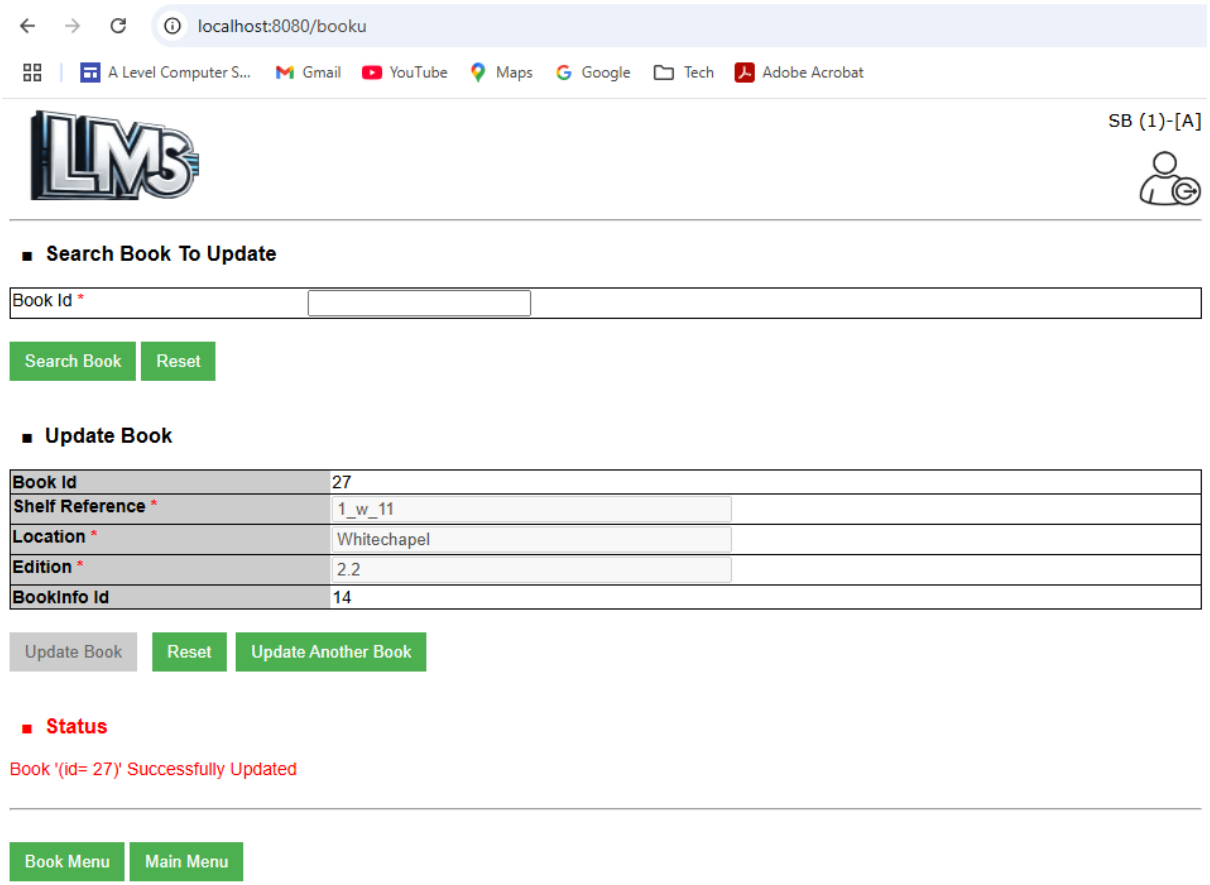
Repository Class [Java Code] => Git Hub Link

```
    /**
     * Handles the DB call to get/retrieve all Books by a bookinfoId.
     * @param bookinfoId the bookinfoId of all the Books to be retrieved
     * @return a list of the requested Book entities
     */
    @Query(
            value = "SELECT * FROM BOOK WHERE bookinfo_id =?1",
            nativeQuery = true)
    List<Book> findAllBooksByBookInfo(Integer bookinfoId);

    /**
     * Handles the DB call to get/retrieve all available Books by a bookinfoId.
     * @param bookinfoId the bookinfoId of all the available Books to be retrieved
     * @return a list of the requested Book entities
     */
    @Query(
            value = "SELECT * FROM BOOK WHERE bookinfo_id =?1 and available=true",
            nativeQuery = true)
    List<Book> findAvailableBooksByBookInfo(Integer bookinfoId);
```

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to get/retrieve an existing Book.
     * @param bookId the Id of Book to be retrieved
     * @return the requested Book entity
     */
    public Book getBookById(Integer bookId) {
        log.info("Started BookService::getBookById. bookId = " + bookId);

        // Gets the bookinfo entity by its ID.
        // select * from book b left join bookinfo bi on bi.bookinfo_id=b.bookinfo_id
        // where b.book_id=?
        Book bookDB = bookRepository.findById(bookId).orElse(null);

        log.info("Returning BookService::getBookById");
        return bookDB;
    }
```

Update Book



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Update Book</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../book/js/booku.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}">
        <h3>  &#x25A0  Search Book To Update</h3>
        <table>
            <tr><td>Book Id <label style='color:red'>*</label>
             <td><input type="text" id="txtBookId"></input>
        </table>
        <br><button onclick = "fnSearchBook();">Search Book</button>
             <button onclick = "fnResetSrch();">Reset</button>
```

```
    </div>
    <div th:unless="${userType != 'M'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>

    <div id="divUpdateBook"></div>
    <div id="divStatus" style="color:red"></div>
    <br><hr><br>
    <div>
        <a th:href="@{/book}"><button>Book Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the Book information provided on the page and submits the information
to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnUpdateBook() {
    let apiUrl = server + apiContext + document.getElementById('tdBookId').innerText;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateUpdate()) {
        return;
    }

    // information to be submitted for saving
    let payload = {
        shelfReference: document.getElementById('txtShelfReference').value,
        location: document.getElementById('txtLocation').value,
        edition: document.getElementById('txtEdition').value
    };

    // create json body for submitting the Update Book request
    let options = {method: "PUT", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};
    const response = fetch(apiUrl, options);

    let strStatus = "<br><h3>  &#x25A0  Status</h3>";
    strStatus += "Book '(id= " + document.getElementById('tdBookId').innerText
             + ")' Successfully Updated";
    resultDivStatus.innerHTML= strStatus;

    toggleFields(true);
    fnResetSrch();

    document.getElementById("btnUpdateBook").className = "dbtn";
    document.getElementById("btnUpdateAnotherBook").className = "";
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles PUT requests to update an existing Book.
     * @param bookId the ID of the book to be updated
     * @param book the Book entity with updated information
     * @return the updated Book entity
     */
    @PutMapping("/{bookId}")
    public ResponseEntity<Book> updateBook(@PathVariable Integer bookId,
                                @RequestBody Book book) {
        log.info("Started BookController::updateBook. bookId = " + bookId);
        return ResponseEntity.status(HttpStatus.OK).body(
             bookService.updateBook(bookId, book));
    }
```

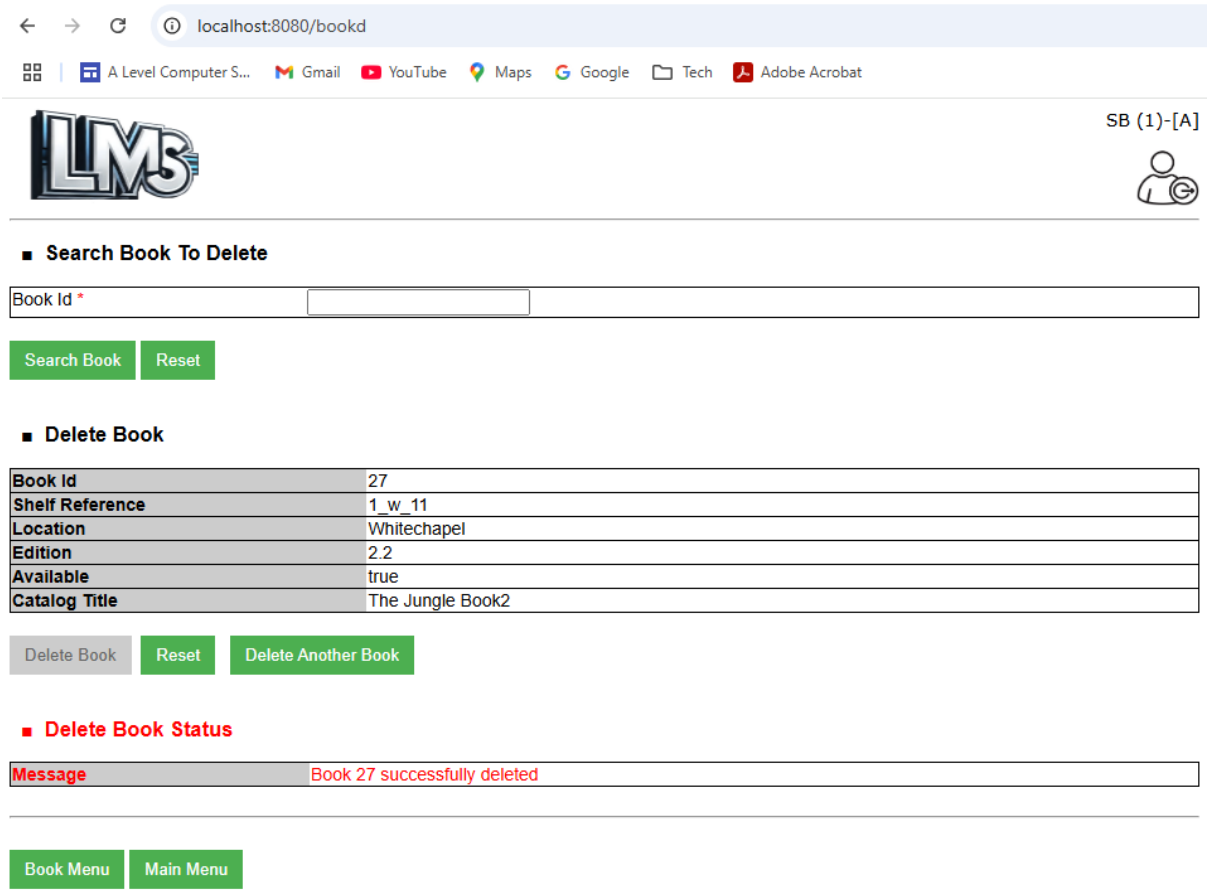
Repository Class [Java Code]

NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save an existing Book.
     * @param bookId the ID of the Book to be updated
     * @param bookFromClient the Book Object with updated values
     * @return the updated Book entity
     */
    public Book updateBook(Integer bookId, Book bookFromClient) {
        log.info("Started BookService::updateBook. bookId = " + bookId);

        // Finds the existing Book by ID.
        Book bookDB = this.getBookById(bookId);

        // Map all non null values
        Book bookUpdated = BookUtils.nonNullMapper(bookFromClient, bookDB);

        // Saves and returns the updated entity.
        log.info("Returning BookService::updateBook");

        // update book set edition=?,location=?,shelf_reference=? where book_id=?
        return bookRepository.save(bookUpdated);
    }
```

Deleted Book After confirmation



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Delete Book</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../book/js/bookd.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div id="divMainWindow">
        <div th:if="${userType != 'M'}">
            <h3>  &#x25A0  Search Book To Delete</h3>
            <table>
                <tr><td>Book Id <label style='color:red'>*</label>
                    <td><input type="text" id="txtBookId"></input>
            </table>
            <br><button onclick = "fnSearchBook();">Search Book</button>
```

```
                    <button onclick = "fnResetSrch();">Reset</button>
        </div>
        <div th:unless="${userType != 'M'}" style="color:red">
            <script>document.write(accessDeniedMsg);</script>
        </div>
        <div id="divDeleteBook"></div>
        <div id="divStatus" style="color:red"></div>
        <br><hr><br>
        <div>
             <a th:href="@{/book}"><button>Book Menu</button></a>
             <a th:href="@{/menu}"><button>Main Menu</button></a>
        </div>
    </div>
    <div id="divModal" style="background-color:#cff5d0;display:none;height:330px;"></div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gets the Book Id on the page and submits the information to the server as an
API request.
It also clears any previous output text from any areas on the page
*/
//async function fnDeleteBook() {
async function fnDoAction() {
    let apiUrl = server + apiContext + glbDelBookId;
    //console.log("apiUrl = " + apiUrl);

    // create json body for submitting the Delete Book request
    let options = {method: "DELETE"};
    const response = await fetch(apiUrl, options);
    if (!response.ok) {
        //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
        displayError(response.status, resultDivStatus, resultDiv);
        return;
    }

    const data = await response.text(); // Text response
    fnDisplayDeleteResponse(data); // Display data on the page
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles DELETE requests to remove a Book by ID.
     * @param bookId the ID of the Book to be deleted
     * @return a success/failure message
     */
    @DeleteMapping("/{bookId}")
    public ResponseEntity<String> deleteBookById(@PathVariable Integer bookId) {
        log.info("Started BookController::deleteBookById. bookId = " + bookId);
        return ResponseEntity.status(HttpStatus.OK).body(
                    bookService.deleteBookById(bookId));
    }
```

Repository Class [Java Code]

NA

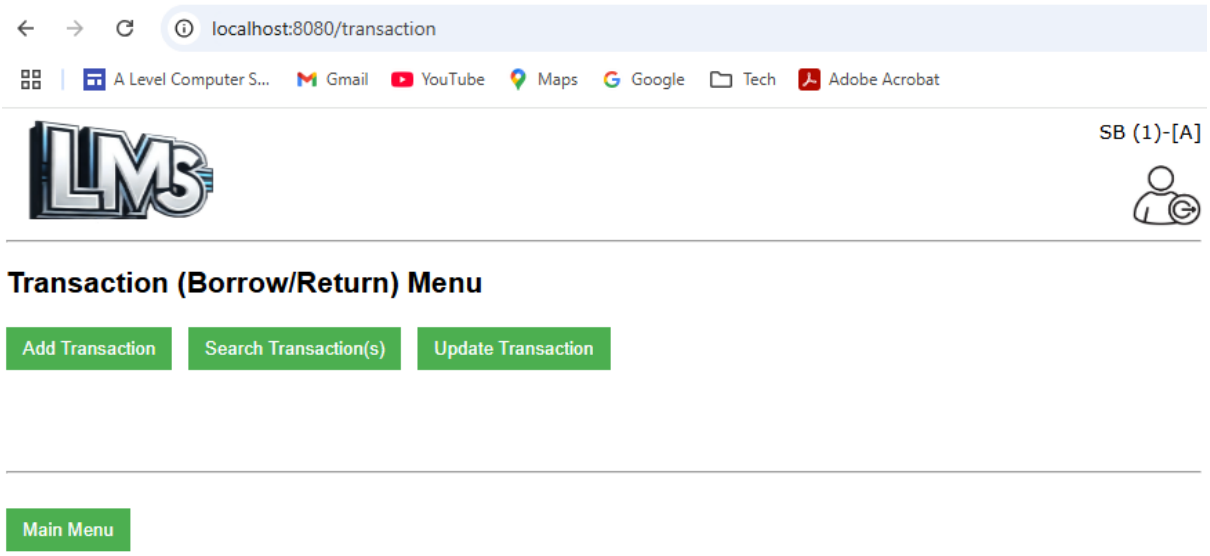
Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to delete a Book by ID.
     * @param bookId the ID of the Book to be deleted
     * @return a success/failure message of deletion
     */
    public String deleteBookById(Integer bookId) {
        log.info("Started BookService::deleteBookById. bookId = " + bookId);

        // Finds the existing Book by ID.
        Book book = this.getBookById(bookId);
        log.info("book = " + book);
        //log.info("book.getAvailable() = " + book.getAvailable());

        // Book cannot be deleted if it is a currently borrowed books and not yet returned
        boolean bookCurrentlyBorrowed = !book.getAvailable();
        if (bookCurrentlyBorrowed) {
            log.info("Book " + bookId + " cannot be deleted. It is currently issued");
            log.info("Returning BookService::deleteBookById");
            return "error:Book " + bookId + " cannot be deleted. It is currently issued";
        }

        // Check if the User has any past transactions
        // SELECT COUNT(*) FROM TRANSACTION WHERE book_id =?
        int bookHistoryEntries =
                    transactionRepository.findCountOfAllTransactionsByBook(bookId);

        // Book cannot be deleted if they have a record of any past transactions because
        // the transaction will first have to be deleted from the system to protect
        // referential integrity of the Database
        boolean bookHasHistoricalTransactions = bookHistoryEntries > 0;
        if (bookHasHistoricalTransactions) {
            log.info("Book " + bookId + " cannot be deleted. Book has " +
                    bookHistoryEntries + " historical transaction(s)");
            String strError = "error:Book " + bookId + " cannot be deleted. Book has " +
                    bookHistoryEntries + " historical transaction(s)";
             strError += "<br>Details: SQL Error: 1451, SQLState: 23000 Cannot delete or "
                    + "update a parent row: a foreign key constraint fails";
            strError += "<br>Please escalate to technical support";

            log.info("Returning BookService::deleteBookById");
            return strError;
        }
        // Get the associated BookInfo entity by its book.
        BookInfo bookInfo = book.getBookInfo();
        log.info("book1 = " + book);
        log.info("bookInfo1 = " + bookInfo);

        // Decrement the count of Total Books in the associated BookInfo entity
        bookInfo.decrementTotalQuantity();
```

```
        // save the updated BookInfo entity
        // update bookinfo set total_quantity=? where bookinfo_id=?
        bookInfoRepository.save(bookInfo);

        // Deletes the book entity by its ID.
        // delete from book where book_id=?
        bookRepository.deleteById(bookId);
        return "success:Book " + bookId + " successfully deleted";
    }
```

CRUD Transaction

Transaction Menu



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>LMS Transaction Menu Page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <h2>Transaction (Borrow/Return) Menu</h2>
    <div th:if="${userType != 'M'}" >
        <a th:href="@{/transactiona}"><button>Add Transaction</button></a>  
        <a th:href="@{/transactionr}"><button>Search Transaction(s)</button></a>  
        <a th:href="@{/transactionu}"><button>Update Transaction</button></a>
    </div>
    <div th:unless="${userType != 'M'}" >
        <a th:href="@{/transactionrm}"><button>Search Self Transaction(s)</button></a>
        <br>
    </div>
    <br><br><br><br><hr><br>
    <div>
        <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code]

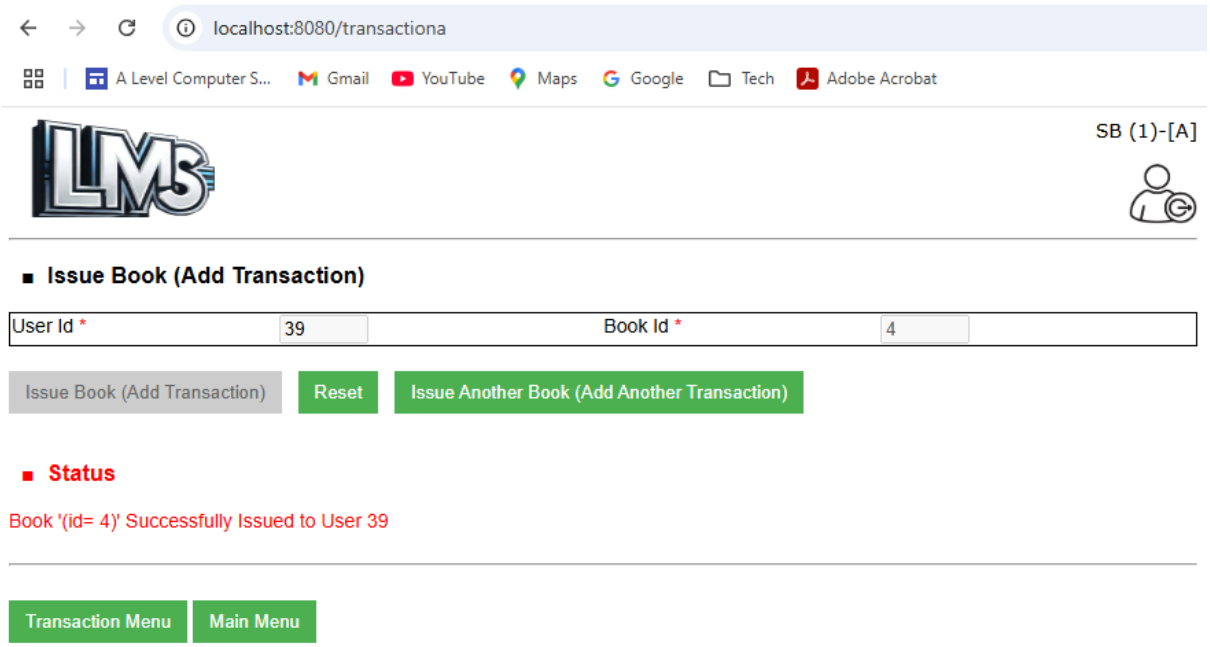
NA

Server Code [Java]

Model/Data Object Class [Java Code] => Git Hub Link

```
public class Transaction {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "transaction_id")
    private Integer transactionId;

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne
    @JoinColumn(name = "book_id")
    private Book book;

    @Column(name = "issue_date")
    private Timestamp issueDate;

    @Column(name = "return_date")
    private Timestamp returnDate;

    @Column(name = "actual_return_date_date")
    private Timestamp actualReturnDate;

    @Column(name = "fine")
    private Double fine = 0.0;

    @Column(name = "returned")
    private Boolean returned = false;

    @Transient
    private LmsFault fault;

    @Transient
    private static final Double FINE_PER_WEEK = 2.0;

}
```

Create Transaction (Issue Book)



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Add Transaction</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../transaction/js/transactiona.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}">
        <h3>  &#x25A0  Issue Book (Add Transaction)</h3>
        <table>
            <tr><td>User Id <label style='color:red'>*</label>
                <td><input type="text" id="txtUserId" maxlength="4" size="4"></input>
            <td><td>Book Id <label style='color:red'>*</label>
                <td><input type="text" id="txtBookId" maxlength="4" size="4"></input></tr>
        </table>
        <br><button id="btnAddTransaction" onclick="fnAddTransaction();">Issue Book (Add
                Transaction)</button>  
        <button onclick="fnReset();">Reset</button>  
        <button id="btnAddAnotherTransaction" onclick="fnAddAnotherTransaction();"
                class='dbtn'>Issue Another Book (Add Another Transaction)</button>
    </div>
    <div th:unless="${userType != 'M'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
```

```
    <div id="divStatus" style="color:red"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/transaction}"><button>Transaction Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the Transaction information provided on the page and submits the
information to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnAddTransaction() {
    resultDivStatus = document.getElementById('divStatus');
    let apiUrl = server + apiContext;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateAdd()) {
        return;
    }
    // information to be submitted for saving
    let payload = {
        user: { userId: document.getElementById('txtUserId').value },
        book: { bookId: document.getElementById('txtBookId').value }
    }

    // create json body for submitting the Add Transaction request
    let options = {method: "POST", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};

    fetch(apiUrl, options).then(response => {if (!response.ok) {
            //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
            displayError(response.status, resultDivStatus, resultDiv);
            return;
        }
        return response.json();
        })
        .then(dataList => {
          if (dataList.fault) {
            let strFault = "<br>  &#x25A0  Error Details";
            strFault += "<table><tr><th width=30%>Http</td><td style='color:red'>"
                   + dataList.fault.http + "</td></tr>";
            strFault += "<tr><th>Code</td><td style='color:red'>" + dataList.fault.code
                   + "</td></tr>";
            strFault += "<tr><th>Message</td><td style='color:red'>"
                   + dataList.fault.message + "</td></tr>";
            strFault += "<tr><th>Path</td><td style='color:red'>" + dataList.fault.path
                   + "</td></tr></table>";

            resultDivStatus.innerHTML = strFault;
        } else { // If server processed the request successfully
```

```
            let strStatus = "<br><h3>  &#x25A0  Status</h3>";
            strStatus += "Book '(id= " + dataList.book.bookId + ")' Successfully Issued to
                    User " + dataList.user.userId;
            resultDivStatus.innerHTML = strStatus;

           // Disable the fields once the request is submitted so no further changes
          // can be done on the same form
           toggleFields(true);

           document.getElementById("btnAddTransaction").className = "dbtn";
           document.getElementById("btnAddAnotherTransaction").className = "";
       }});
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles POST requests to save (issue/borrow a book) a new Transaction.
     * @param transaction the Transaction entity to be saved
      * @return the saved Transaction entity
     */
    @PostMapping
    public ResponseEntity<Transaction> addTransaction(@RequestBody Transaction transaction){
        log.info("Started TransactionController::addTransaction. " +
                "userId = " + transaction.getUser().getUserId() +
                ", bookId = " + transaction.getBook().getBookId());
        return ResponseEntity.status(HttpStatus.CREATED).body(
              transactionService.addTransaction(transaction));
    }
```

Repository Class [Java Code]

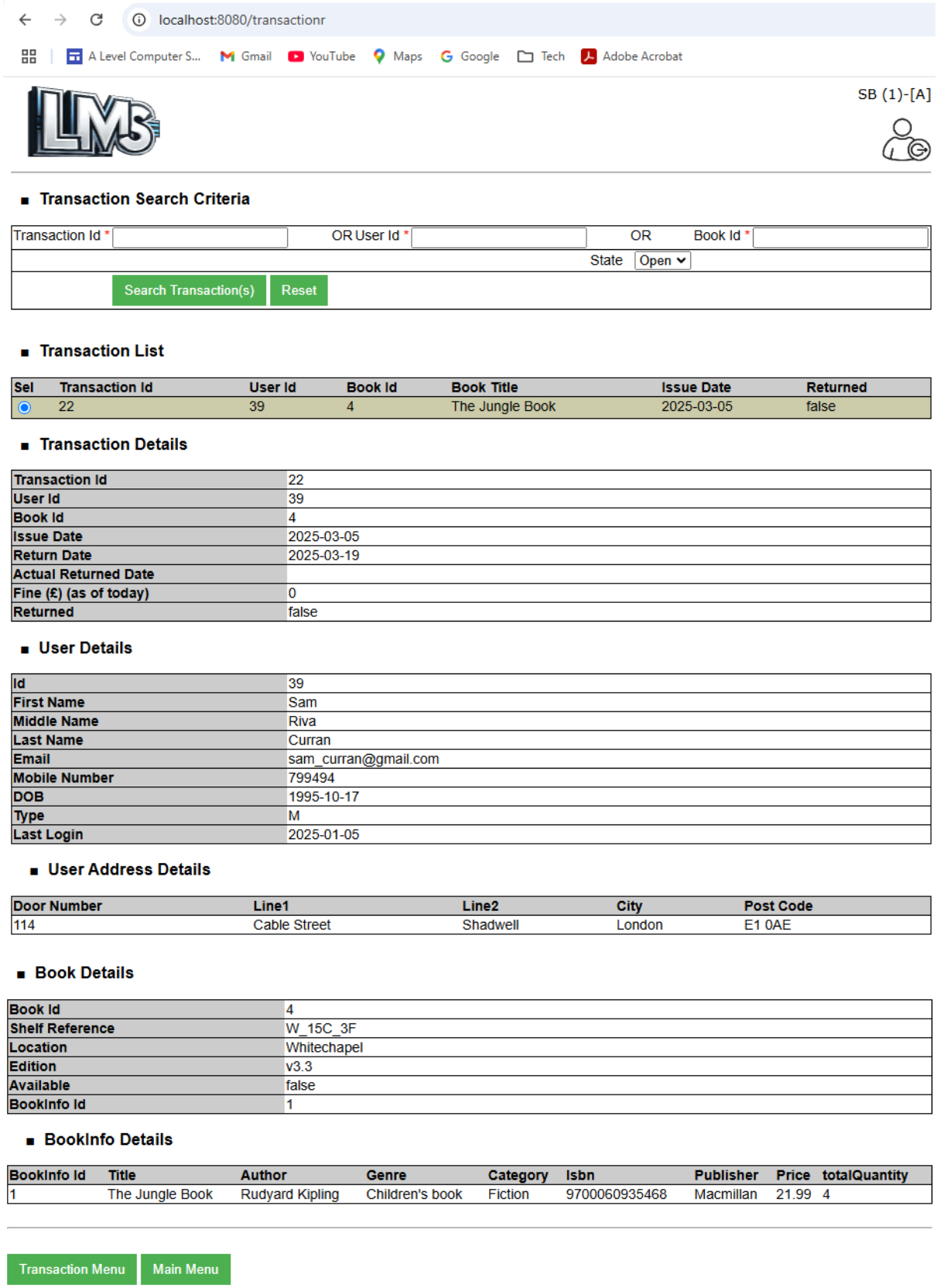
NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save a new Transaction (i.e Issue a Book To a User).
     * @param transaction the Transaction entity to be saved
     * @return the saved Transaction entity
     */
    public Transaction addTransaction(Transaction transaction) {
        log.info("Started TransactionService::addTransaction. userId = "
                + transaction.getUser().getUserId() + ", bookId = "
                + transaction.getBook().getBookId());

        LmsFault lmsFault;
        // Gets the user entity by its ID.
        // select * from user u left join address a on a.address_id=u.address_id
       // where user_id=?
        User userDB = userRepository.findById(transaction.getUser().getUserId())
                    .orElse(null);
```

```
            return Utils.createFaultyTransaction(lmsFault);
        }

        // Gets the book entity by its ID.
        // select * from book b left join bookinfo bi on bi.bookinfo_id=b.bookinfo_id
        // where b.book_id=?
        Book bookDB = bookRepository.findById(transaction.getBook().getBookId())
                    .orElse(null);

        // Get the count of book(s) currently issued by the User to check
        // the borrowing limit
        // SELECT COUNT(*) FROM TRANSACTION WHERE user_id =? and returned=false
        int userBookCount = transactionRepository.findCountOfOpenTransactionsByUser(
                                transaction.getUser().getUserId());

        // User cannot borrow more than 3 books at any point in time
        boolean userAlreadyBorrowedMaxLimit = (userBookCount == MAX_BOOKS_PER_USER);
        if (userAlreadyBorrowedMaxLimit) {
            log.info("User already has " + MAX_BOOKS_PER_USER + " books borrowed");
            lmsFault = new LmsFault("Application Constraint Error", "210",
                    MAX_BOOKS_PER_USER_LIMIT_USED, "/lms/v1/transactions");
            return Utils.createFaultyTransaction(lmsFault);
        }

        // Book to be borrowed cannot be currently borrowed by a User
        boolean bookCurrentlyBorrowed = !bookDB.getAvailable();
        if (bookCurrentlyBorrowed) {
            log.info("Book is already borrowed");
            lmsFault = new LmsFault("Application Constraint Error", "210",
                    BOOKS_ALREADY_BORROWED, "/lms/v1/transactions");
            return Utils.createFaultyTransaction(lmsFault);
        }
        // The user and book entities get associated to the transaction
        transaction.issueBook(transaction.getUser(), transaction.getBook());

        // Issue date is auto set as today's date
        transaction.setIssueDate(Utils.convertNowToSQLTS());

        // Return date is auto set as 2 weeks from issue date
        transaction.setReturnDate(Utils.generateReturnSQLTS(transaction.getIssueDate()));

        // Mark the book as unavailable for borrowing until returned
        bookDB.setAvailable(false);

        // update book set available=?,bookinfo_id=?,edition=?,location=?,
        // shelf_reference=? where book_id=?
        bookRepository.save(bookDB);

        // Associate the updated book back to the Transaction
        transaction.setBook(bookDB);
        log.info("Returning TransactionService::addTransaction");

        // insert into transaction
       (actual_return_date_date,book_id,fine,issue_date,return_date,
        //              returned,user_id) values (?,?,?,?,?,?,?)
        return transactionRepository.save(transaction);
    }
```

Read Transaction



UI Form/Screen [HTML Code] => Git Hub [Link]

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Search Transaction</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../transaction/js/transactionr.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}">
         <h3>  &#x25A0  Transaction Search Criteria</h3>
         <table>
             <tr><td>Transaction Id <label style='color:red'>*</label>
                    <td><input type="text" id="txtTransactionId"></input>
                 <td>OR<td>User Id <label style='color:red'>*</label>
             <td><input type="text" id="txtUserId"></input>
                 <td colspan =2 style="text-align:center;">OR<td>Book Id
                   <label style='color:red'>*</label>
                 <td><input type="text" id="txtBookId"></input>
             <tr><td colspan =5><td>State  
                 <select id='selState'>
                     <option value='O'>Open</option>
                     <option value='A'>All</option>
                 </select></tr>
             <tr><td><td>
             <button onclick = "fnSearchTransaction();">Search Transaction(s)</button>
                 <button onclick = "fnReset();">Reset</button>
         </table>
     </div>
    <div th:unless="${userType != 'M'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
     <div id="divListTransaction"></div>
     <div id="divDtlsTransaction"></div>

     <br><hr><br>
    <div>
        <a th:href="@{/transaction}"><button>Transaction Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the search information provided on the page and submits the
information to the server as an API request.
```

```
It calls the display Transaction function to show the data on the page
*/
async function fnSearchTransaction() {
    resultDivList = document.getElementById('divListTransaction');
    resultDivDtls = document.getElementById('divDtlsTransaction');
    let apiUrl = server + apiContext;
    let payload = "";

    if (document.getElementById('txtTransactionId').value != "") {
        apiUrl += document.getElementById('txtTransactionId').value;
    } else if (document.getElementById('txtUserId').value != "") {
        if (document.getElementById('selState').value == "A")
            apiUrl += 'all/user/' + document.getElementById('txtUserId').value;
        else
            apiUrl += 'available/user/' + document.getElementById('txtUserId').value;
    } else if (document.getElementById('txtBookId').value != "") {
        if (document.getElementById('selState').value == "A")
            apiUrl += 'all/book/' + document.getElementById('txtBookId').value;
        else
            apiUrl += 'available/book/' + document.getElementById('txtBookId').value;
    }
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateSrch()) {
        fnResetSrch();
        return;
    }

    resultDivList.innerHTML = "";

       try {
             const response = await fetch(apiUrl); // Make the API call
             if (!response.ok) {
                 fnReset();
            //throw new Error(`Database Error: ${response.status}`); // Handle HTTP errors
            displayError(response.status, resultDivDtls, resultDivList);
            return;
             }

             const data = await response.json(); // Parse JSON response
        globalData = data;
             //console.log("data = " + data); // csv data items in object
             fnDisplayTransactionList(data); // Display data on the page
       } catch (error) {
             displayError(error, resultDivDtls, resultDivList);
       }
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles GET requests to get/retrieve an existing Transaction
     * @param transactionId the ID of the Transaction to be retrieved
     * @return the requested Transaction entity
     */
    @GetMapping("/{transactionId}")
    public ResponseEntity<Transaction> getTransactionById(
                                @PathVariable Integer transactionId) {
        log.info("Started TransactionController::getTransactionById. transactionId = "
                            + transactionId);
        return ResponseEntity.status(HttpStatus.OK).body(
                            transactionService.getTransactionById(transactionId));
    }

    /**
     * Handles GET requests to get all Transactions (whether returned or not books).
     * @param userId the ID of the User associated to the Transactions to be retrieved
     * @return a list of all Transactions
     */
    @GetMapping("/all/user/{userId}")
    public ResponseEntity<List<Transaction>> getAllTransactionsByUser(
                                        @PathVariable Integer userId) {
        log.info("Started TransactionController::getAllTransactionsByUser");
        return ResponseEntity.status(HttpStatus.OK).body(
                    transactionService.getAllTransactionsByUser(userId));
    }

    /**
     * Handles GET requests to get all Transactions (whether returned or not books).
     * @param bookId the ID of the Book associated to the Transactions to be retrieved
     * @return a list of all Transactions
     */
    @GetMapping("/all/book/{bookId}")
    public ResponseEntity<List<Transaction>> getAllTransactionsByBook(
                                @PathVariable Integer bookId) {
        log.info("Started TransactionController::getAllTransactionsByBook");
        return ResponseEntity.status(HttpStatus.OK).body(
                    transactionService.getAllTransactionsByBook(bookId));
    }

    /**
     * Handles GET requests to get an existing open Transaction (i.e non returned books).
     * @param userId the ID of the User associated to the open Transaction to be retrieved
     * @return a list of all associated open Transactions
     */
    @GetMapping("/available/user/{userId}")
    public ResponseEntity<List<Transaction>> getOpenTransactionByUser(
                        @PathVariable Integer userId) {
        log.info("Started TransactionController::getOpenTransactionByUser");
        return ResponseEntity.status(HttpStatus.OK).body(
                    transactionService.getOpenTransactionByUser(userId));
    }

    /**
```

```
     * Handles GET requests to get an existing open Transaction (i.e non returned books).
     * @param bookId the ID of the Book associated to the open Transaction to be retrieved
     * @return a list of all associated open Transactions
     */
    @GetMapping("/available/book/{bookId}")
    public ResponseEntity<Transaction> getOpenTransactionByBook(
                                @PathVariable Integer bookId) {
        log.info("Started TransactionController::getOpenTransactionByBook");
        return ResponseEntity.status(HttpStatus.OK).body(
                    transactionService.getOpenTransactionByBook(bookId));
    }
```

Repository Class [Java Code] => Git Hub Link

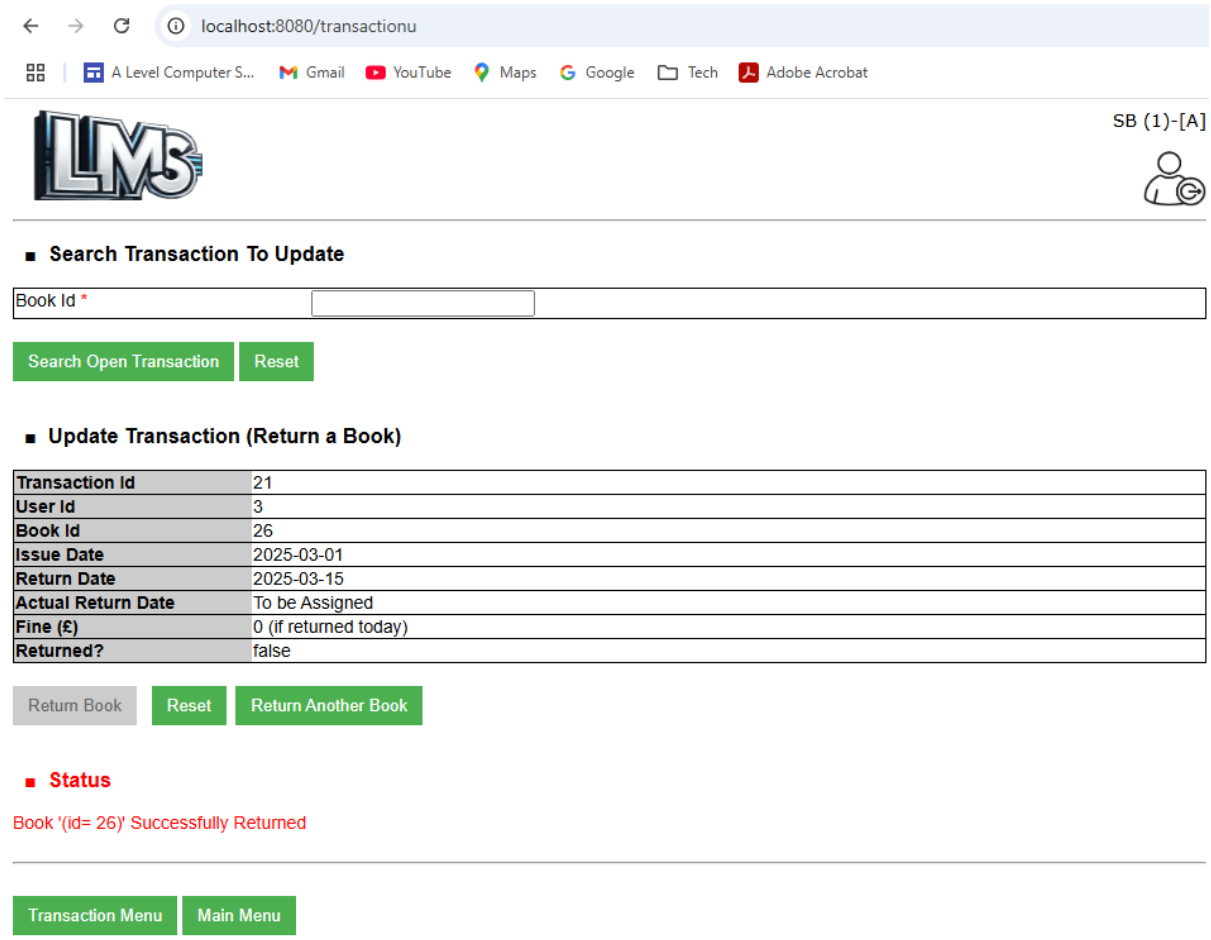
```
    /**
     * Handles the DB call to get/retrieve all open Transactions by its userId
     * @param userId the userId for which all associated open Transactions to be retrieved
     * @return a list of the requested open Transaction entities
     */
    @Query(
            value = "SELECT * FROM TRANSACTION WHERE user_id =?1 and returned=false",
            nativeQuery = true)
    List<Transaction> findOpenTransactionsByUser(Integer userId);

    /**
     * Handles the DB call to get/retrieve a Transaction by its bookId
     * @param bookId the bookId for which the associated Transaction is to be retrieved
     * @return the requested Transaction entity
     */
    @Query(
            value = "SELECT * FROM TRANSACTION WHERE book_id =?1 and returned=false",
            nativeQuery = true)
    Transaction findOpenTransactionByBook(Integer bookId);

    /**
     * Handles the DB call to get/retrieve all Transactions by its userId
     * @param userId the userId for which all associated Transactions to be retrieved
     * @return a list of the requested Transaction entities
     */
    @Query(
            value = "SELECT * FROM TRANSACTION WHERE user_id =?1",
            nativeQuery = true)
    List<Transaction> findAllTransactionsByUser(Integer userId);

    /**
     * Handles the DB call to get/retrieve all Transactions by its bookId
     * @param bookId the bookId for which all associated Transactions to be retrieved
     * @return a list of the requested Transaction entities
     */
    @Query(
            value = "SELECT * FROM TRANSACTION WHERE book_id =?1",
            nativeQuery = true)
    List<Transaction> findAllTransactionsByBook(Integer bookId);
```

Service Class [Java Code] => Git Hub Link

```
/**
 * Handles requests to get/retrieve all existing Transactions of a user
 * @param userId the Id of User for which all Transactions are to be retrieved
 * @return the requested list of Transaction entities
 */
public List<Transaction> getAllTransactionsByUser(Integer userId) {
    log.info("Started TransactionService::getAllTransactionsByUser");

    // Gets the Transaction entity list by its associated User
    // SELECT * FROM TRANSACTION WHERE user_id =?
    List<Transaction> transactionDBList =
         transactionRepository.findAllTransactionsByUser(userId);


    log.info("Returning TransactionService::getAllTransactionsByUser");
    // Prepares every Transaction in the list with its calculated fine
    return prepareTransactionList (transactionDBList);
}

/**
 * Handles requests to get/retrieve all existing Transactions of a Book
 * @param bookId the Id of Book for which all Transactions are to be retrieved
 * @return the requested list of Transaction entities
 */
public List<Transaction> getAllTransactionsByBook(Integer bookId) {
    log.info("Started TransactionService::getAllTransactionsByBook");

    // Gets the Transaction entity list by its associated User
    // SELECT * FROM TRANSACTION WHERE book_id =?
    // select * from user u left join address a on a.address_id=u.address_id
    // where user_id=?
    List<Transaction> transactionDBList =
                transactionRepository.findAllTransactionsByBook(bookId);

    log.info("Returning TransactionService::getAllTransactionsByBook");

    // Prepares every Transaction in the list with its calculated fine
    return prepareTransactionList (transactionDBList);
}

/**
 * Handles requests to get/retrieve existing open Transactions of a User
 * @param userId the Id of User for which open Transactions are to be retrieved
 * @return the requested list of Transaction entities
 */
public List<Transaction> getOpenTransactionByUser(Integer userId) {
    log.info("Started TransactionService::getOpenTransactionByUser");

    // Gets the user entity by its ID.
    // select * from user u left join address a on a.address_id=u.address_id
    // where user_id=?
    User userDB = userRepository.findById(userId).orElse(null);

    // SELECT * FROM TRANSACTION WHERE user_id =? and returned=false
    // select * from book b left join bookinfo bi on bi.bookinfo_id=b.bookinfo_id
    // where b.book_id=?
```

```
        List<Transaction> transactionDBList =
             transactionRepository.findOpenTransactionsByUser(userId);

        log.info("Returning TransactionService::getOpenTransactionByUser");

        // Prepares every Transaction in the list with its calculated fine
        return prepareTransactionList (transactionDBList);
    }

    /**
     * Handles requests to get/retrieve an existing open Transaction of a Book
     * @param bookId the Id of Book for which an open Transaction is to be retrieved
     * @return the requested Transaction entity
     */
    public Transaction getOpenTransactionByBook(Integer bookId) {
        log.info("Started TransactionService::getOpenTransactionByBook");
        // Gets the book entity by its ID.
        // select * from book b left join bookinfo bi on bi.bookinfo_id=b.bookinfo_id
             where b.book_id=?
        Book bookDB = bookRepository.findById(bookId).orElse(null);


        // SELECT * FROM TRANSACTION WHERE book_id =? and returned=false
        // select * from user u left join address a on a.address_id=u.address_id
        // where user_id=?
        Transaction transactionDB =
                    transactionRepository.findOpenTransactionByBook(bookId);

        log.info("Returning TransactionService::getOpenTransactionByBook");
        // Prepares every Transaction in the list with its calculated fine
        return prepareTransaction (transactionDB);
    }

    public Transaction getTransactionById(Integer transactionId) {
        log.info("Started TransactionService::getTransactionById");
        // Gets the Transaction entity by its ID.
        Transaction transactionDB =
             transactionRepository.findById(transactionId).orElse(null);

        // Prepares the Transaction with its calculated fine
        return prepareTransaction (transactionDB);
    }
```

Update Transaction (Return book)



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Update Transaction</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../transaction/js/transactionu.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType != 'M'}">
        <h3>  &#x25A0  Search Transaction To Update</h3>
        <table>
            <tr><td>Book Id <label style='color:red'>*</label>
                    <td><input type="text" id="txtBookId">
        </table>
        <br><button onclick = "fnSearchTransaction();">Search Open Transaction</button>
        <button onclick = "fnResetSrch();">Reset</button>
    </div>
```

```
    <div th:unless="${userType != 'M'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>

    <div id="divUpdateTransaction"></div>
    <div id="divStatus" style="color:red"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/transaction}"><button>Transaction Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the Transaction information provided on the page and submits the
information to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnUpdateTransaction() {
    let apiUrl = server + apiContext + "book/" +
             document.getElementById('tdBookId').innerText;
    //console.log("apiUrl = " + apiUrl);

    // There is nothing to validate because there is no user information
    // but auto calculated values
    /*
    if (!validateUpdate()) {
        return;
    }
    */

    // information to be submitted for saving
    let payload = {
        bookId: document.getElementById('txtBookId').value
    };

    // create json body for submitting the Update Book request
    let options = {method: "PUT", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};
    const response = fetch(apiUrl, options);

    let strStatus = "<br><h3>  &#x25A0  Status</h3>";
    strStatus += "Book '(id= " + document.getElementById('tdBookId').innerText
             + ")' Successfully Returned";
    resultDivStatus.innerHTML= strStatus;

    toggleFields(true);

    fnResetSrch();
    document.getElementById("btnUpdateTransaction").className = "dbtn";
    document.getElementById("btnUpdateAnotherTransaction").className = "";
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles PUT requests to update (return a book) an existing Transaction.
     * @param transactionId the Transaction entity with updated information
     * @return the updated Transaction entity
     */
    @PutMapping("/{transactionId}")
    public ResponseEntity<Transaction> updateTransaction(
                    @PathVariable Integer transactionId) {
        log.info("Started TransactionController::updateTransaction");
        return ResponseEntity.status(HttpStatus.OK).body(
                            transactionService.updateTransaction(transactionId));
    }

    /**
     * Handles PUT requests to update (return a book) an existing Transaction.
     * @param bookId the ID of the Book for which the Transaction has to be updated
     * @return the updated Transaction entity
     */
    @PutMapping("/book/{bookId}")
    public ResponseEntity<Transaction> returnBook(@PathVariable Integer bookId) {
        log.info("Started TransactionController::returnBook. bookId = " + bookId);
        return ResponseEntity.status(HttpStatus.OK).body(
                            transactionService.returnBook(bookId));
    }

    /**
     * Handles PUT requests to update (return a book) an existing Transaction.
     * @param transaction the Transaction entity with updated information
     * @return the updated Transaction entity
     */
    @PutMapping("/book/transaction")
    public ResponseEntity<Transaction> returnBook(@RequestBody Transaction transaction) {
        log.info("Started TransactionController::returnBook transaction");
        return ResponseEntity.status(HttpStatus.OK).body(
                            transactionService.returnBook(transaction));
    }
```

Repository Class [Java Code]

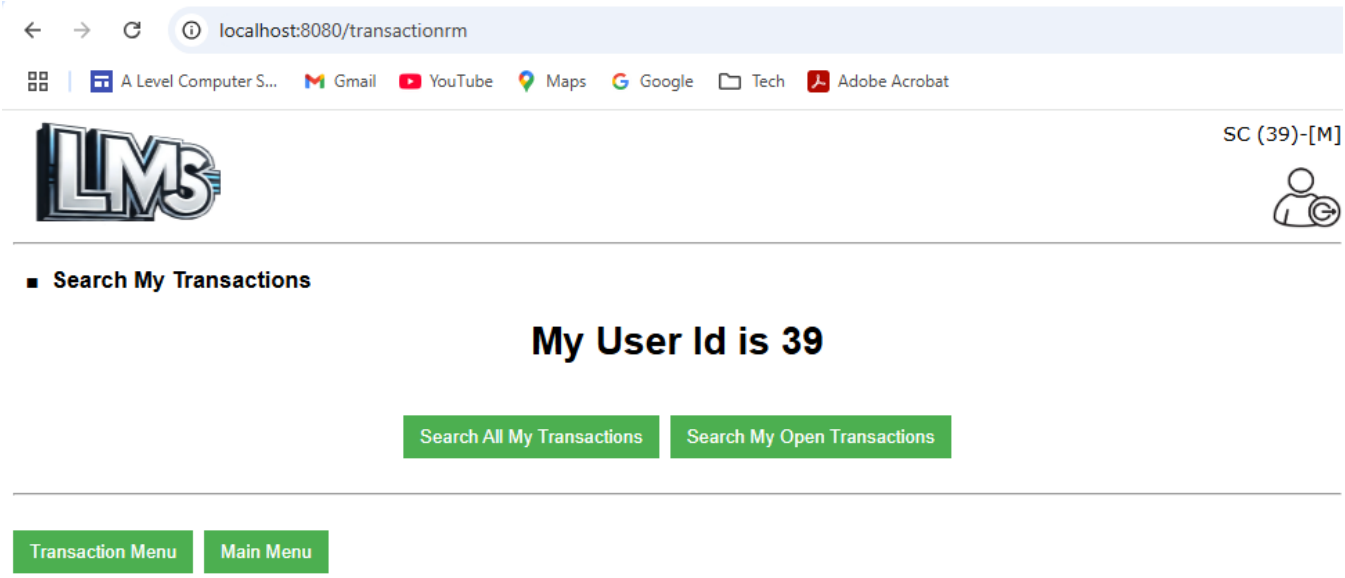
NA

Service Class [Java Code] => Git Hub Link

```
    /** Handles requests to update an existing Transaction (i.e Return a Book From a User).
     * @param transactionId the ID of the Transaction to be updated
     * @return the updated Transaction entity
     */
    public Transaction updateTransaction(Integer transactionId) {
        log.info("Started TransactionService::updateTransaction");

        // Finds the existing Transaction by ID.
        Transaction transactionDB = this.getTransactionById(transactionId);
```

```
        // Process the marking of returning the book
        Transaction validTransaction = returnBook(transactionDB.getBook().getBookId());

        log.info("Returning TransactionService::updateTransaction");
        return validTransaction;
    }

    /**
     * Handles processing of returning a book
     * @param bookId the ID of the Book to be marked as returned
     * @return the updated Transaction entity
     */
    public Transaction returnBook(Integer bookId) {
        log.info("Started TransactionService::returnBook bookId = " + bookId);

        // Finds the existing open Transaction by its Book ID.
        Transaction transactionDB = this.getOpenTransactionByBook(bookId);

        // Set Book to be available
        transactionDB.getBook().setAvailable(true);

        // insert into book (available,bookinfo_id,edition,location,shelf_reference)
        // values (?,?,?,?,?)
        bookRepository.save(transactionDB.getBook());

        // Set Transaction to be returned
        // set today's date to the Actual Returned date
        transactionDB.setActualReturnDate(Utils.convertNowToSQLTS());

        // calculate fine with respect to any late returns
        transactionDB.setFine(transactionDB.calculateFine()); // fine calculation

        // Mark the Transaction as returned and complete
        transactionDB.setReturned(true);

        log.info("Returning TransactionService::returnBook(bookId)");

        // insert into transaction
        // (actual_return_date_date,book_id,fine,issue_date,return_date,
        //              returned,user_id) values (?,?,?,?,?,?,?)
        return transactionRepository.save(transactionDB);
    }


    /**
     * Handles processing of returning a book by its Transaction Object
     * @param transaction the Transaction entity to be marked as returned
     * @return the updated Transaction entity
     */
    public Transaction returnBook(Transaction transaction) {
        log.info("Started TransactionService::returnBook");
        Transaction validTransaction = returnBook(transaction.getBook().getBookId());

        log.info("Returning TransactionService::returnBook(transaction)");
        return validTransaction;
    }
```

Read Member Transaction



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Search Member Transaction</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../transaction/js/transactionrm.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <h3>  &#x25A0  Search My Transactions</h3>
    <div id="divSrchTransaction" style="text-align: center;">
        <h1>My User Id is <span id="txtUserId" th:text="${userId}"></span></h1>
        <br><button id='btnSearchUserA' onclick = "fnSearchTransaction('all');">Search All
            My Transactions</button>
         <button id='btnSearchUserO' onclick = "fnSearchTransaction('open');">Search
            My Open Transactions</button>
    </div>
    <div id="divListTransaction"></div>
    <div id="divDtlsTransaction"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/transaction}"><button>Transaction Menu</button></a> 
        <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

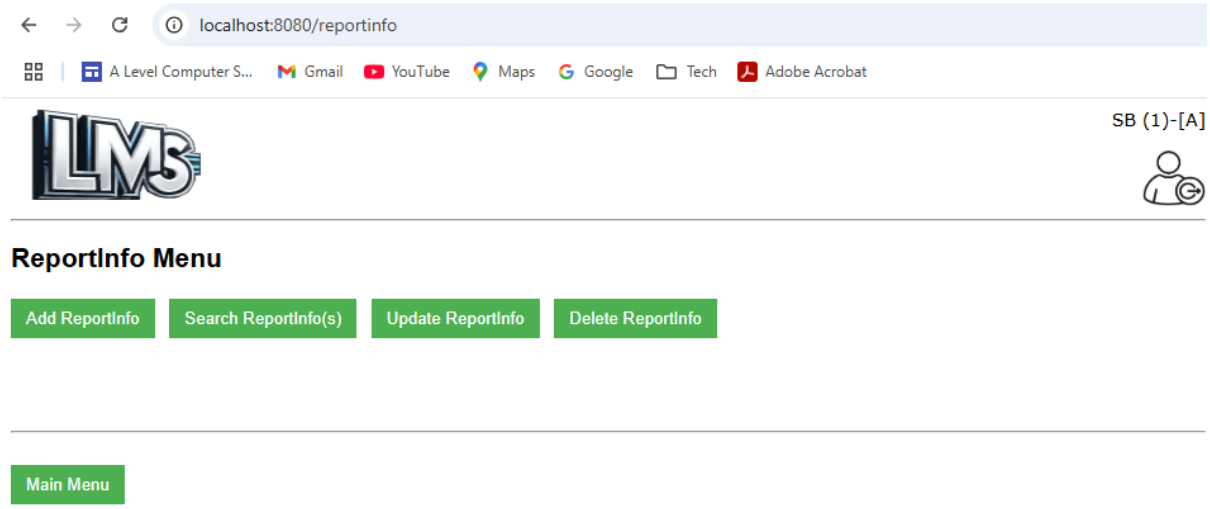
```
/** This function gathers the search information provided on the page and submits the
information to the server as an API request.
It calls the display Transaction function to show the data on the page
*/
async function fnSearchTransaction(mode) {
    resultDivList = document.getElementById('divListTransaction');
    resultDivDtls = document.getElementById('divDtlsTransaction');
    let apiUrl = server + apiContext;

    // check if all transactions need to be searched or only open ones
    let allTransactions = (mode == 'all');
    let openTransactions = (mode == 'open');
    if (allTransactions)
        apiUrl += 'all/user/' + document.getElementById('txtUserId').innerText;
    else if (openTransactions)
        apiUrl += 'available/user/' + document.getElementById('txtUserId').innerText;
    let payload = "";

    //console.log("apiUrl = " + apiUrl);

    resultDivList.innerHTML = "";

      try {
         const response = await fetch(apiUrl); // Make the API call
         if (!response.ok) {
            //throw new Error(`Database Error: ${response.status}`); // Handle HTTP errors
            displayError(response.status, resultDivDtls, resultDivList);
            return;
          }

          const data = await response.json(); // Parse JSON response
          globalData = data;

           fnDisplayTransactionList(data); // Display data on the page
      } catch (error) {
           displayError(error, resultDivDtls, resultDivList);
      }
}
```

Server Code [Java]

Same as Update TransactionService Code details

CRUD ReportInfo

ReportInfo Menu



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>LMS ReportInfo Menu Page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType == 'A'}">
        <h2>ReportInfo Menu</h2>
        <a th:href="@{/reportinfoa}"><button>Add ReportInfo</button></a>  
        <a th:href="@{/reportinfor}"><button>Search ReportInfo(s)</button></a>  
        <a th:href="@{/reportinfou}"><button>Update ReportInfo</button></a>  
        <a th:href="@{/reportinfod}"><button>Delete ReportInfo</button></a>
    </div>
    <div th:unless="${userType == 'A'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
    <br><br><br><br><hr><br>
    <div>
        <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code]

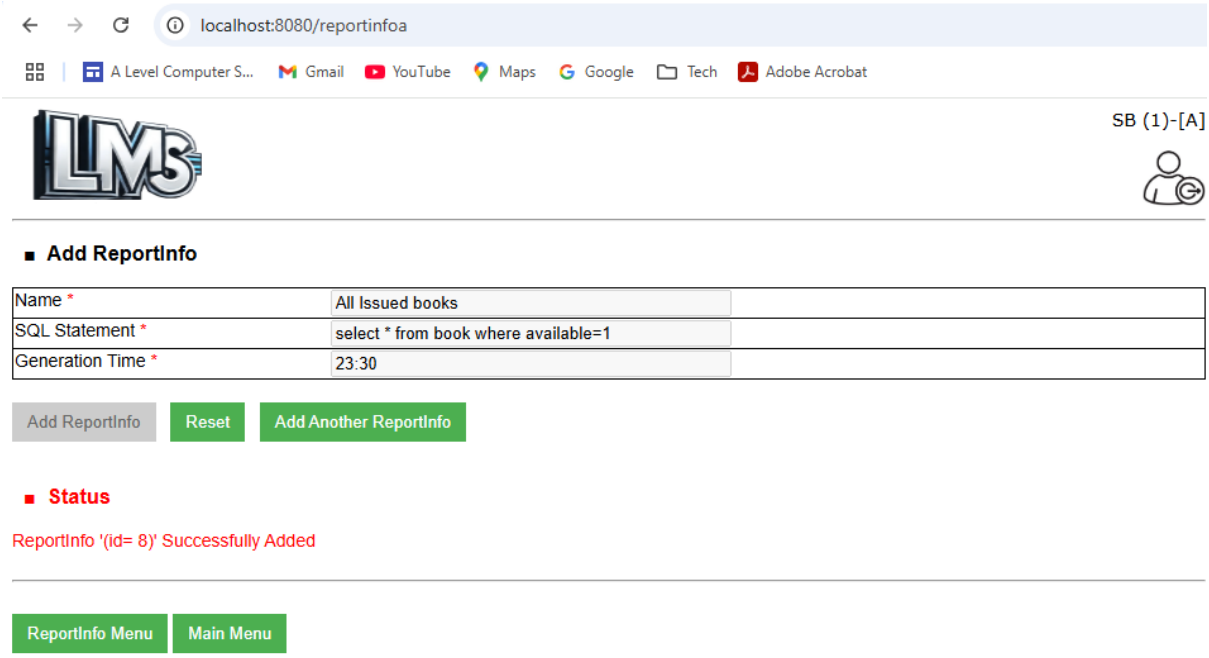
NA

Server Code [Java]

Model/Data Object Class [Java Code] => Git Hub Link

```
public class ReportInfo {
    @Id
    @Column(name = "reportinfo_id")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer reportInfoId;

    @Column(name = "name")
    private String name;

    @Column(name = "sql_statement")
    private String sqlStatement;

    @Column(name = "time_to_generate")
    private String timeToGenerate;

    @Transient
    private LmsFault fault;

}
```

Create ReportInfo



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Add ReportInfo</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../reportinfo/js/reportinfoa.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType == 'A'}">
        <h3>  &#x25A0  Add ReportInfo</h3>
        <table>
            <tr><td>Name <label style='color:red'>*</label>
             <td><input type="text" id="txtName" size="40"></input></tr>
            <tr><td>SQL Statement <label style='color:red'>*</label>
             <td><input type="text" id="txtSql" size="40"></input></tr>
            <tr><td>Generation Time (HH:MM) <label style='color:red'>*</label>
                <td><input type="text" id="txtGenTime" size="40"></input></tr>
        </table>
        <br><button id="btnAddReportInfo" onclick="fnAddReportInfo();">Add
             ReportInfo</button>  
        <button onclick="fnReset();">Reset</button>  
        <button id="btnAddAnotherReportInfo" onclick="fnAddAnotherReportInfo();"
             class='dbtn'>Add Another ReportInfo</button>
    </div>
    <div th:unless="${userType == 'A'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
    <div id="divStatus" style="color:red"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/reportinfo}"><button>ReportInfo Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the Report Info information provided on the page and submits the
information to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnAddReportInfo() {
    resultDivStatus = document.getElementById('divStatus');
```

```
    let apiUrl = server + apiContext;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateAdd()) {
        return;
    }
    // information to be submitted for saving
    let payload = {
        name: document.getElementById('txtName').value,
        sqlStatement: document.getElementById('txtSql').value,
        timeToGenerate: document.getElementById('txtGenTime').value
    }

    // create json body for submitting the Add ReportInfo request
    let options = {method: "POST", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};


    fetch(apiUrl, options).then(response => {if (!response.ok) {
            //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
            displayError(response.status, resultDivStatus, resultDiv);
            return;
        }
        return response.json();
        })
        .then(dataList => {
           if (dataList.fault) { // If server encountered an error
             let strFault = "<br>  &#x25A0  Error Details";
             strFault += "<table><tr><th width=30%>Http</td><td style='color:red'>"
                    + dataList.fault.http + "</td></tr>";
             strFault += "<tr><th>Code</td><td style='color:red'>"
                    + dataList.fault.code + "</td></tr>";
             strFault += "<tr><th>Message</td><td style='color:red'>"
                    + dataList.fault.message + "</td></tr>";
             strFault += "<tr><th>Path</td><td style='color:red'>" + dataList.fault.path
                    + "</td></tr></table>";

             resultDivStatus.innerHTML = strFault;
        } else { // If server processed the request successfully
             let strStatus = "<br><h3>  &#x25A0  Status</h3>";
             strStatus += "ReportInfo '(id= " + dataList.reportInfoId
                    + ")' Successfully Added";
             resultDivStatus.innerHTML = strStatus;

           // Disable the fields once the request is submitted so no further changes
           // can be done on the same form
           toggleFields(true);

           document.getElementById("btnAddReportInfo").className = "dbtn";
           document.getElementById("btnAddAnotherReportInfo").className = "";
      }});
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles POST requests to save a new ReportInfo.
     * @param reportInfo the ReportInfo entity to be saved
     * @return the saved ReportInfo entity
     */
    @PostMapping
    public ResponseEntity<ReportInfo> addReportInfo(@RequestBody ReportInfo reportInfo) {
        log.info("Started ReportInfoController::addReportInfo");
        return ResponseEntity.status(HttpStatus.CREATED).body(
             reportInfoService.addReportInfo(reportInfo));
    }
```

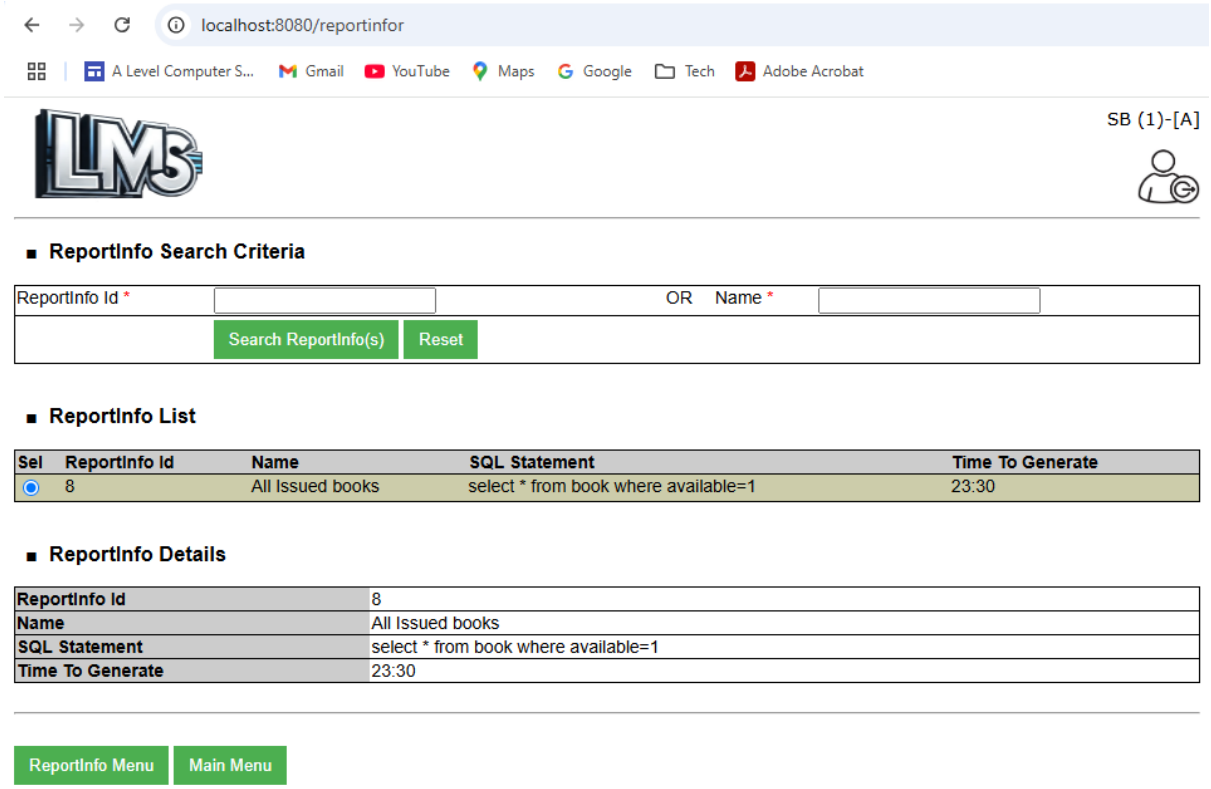
Repository Class [Java Code]

NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save a new ReportInfo.
     * @param reportInfo the ReportInfo entity to be saved
     * @return the saved ReportInfo entity
     */
    public ReportInfo addReportInfo(ReportInfo reportInfo) {
        log.info("Started ReportInfoService::addReportInfo. reportInfo = " + reportInfo);

        // insert into reportinfo (name,sql_statement,time_to_generate) values (?,?,?)
        return reportInfoRepository.save(reportInfo);
    }
```

Read ReportInfo



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Search ReportInfo</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../reportinfo/js/reportinfor.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType == 'A'}">
        <h3>  &#x25A0  ReportInfo Search Criteria</h3>
        <table>
            <tr><td>ReportInfo Id <label style='color:red'>*</label>
                <td><input type="text" id="txtReportInfoId"></input>
                <td colspan=6>OR
                <td>Name <label style='color:red'>*</label>
                <td><input type="text" id="txtReportInfoName"></input>
            <tr><td><td><button onclick = "fnSearchReportInfo();">
                    Search ReportInfo(s)</button>
```

```
            <button onclick = "fnReset();">Reset</button>
        </table>
    </div>
    <div th:unless="${userType == 'A'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
    <div id="divListReportInfo"></div>
    <div id="divDtlsReportInfo"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/reportinfo}"><button>ReportInfo Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the search information provided on the page and submits the
information to the server as an API request.
It calls the display ReportInfo function to show the data on the page
*/
async function fnSearchReportInfo() {
    resultDivList = document.getElementById('divListReportInfo');
    resultDivDtls = document.getElementById('divDtlsReportInfo');

    let apiUrl = server + apiContext;
    if (document.getElementById('txtReportInfoId').value != "") {
        apiUrl += document.getElementById('txtReportInfoId').value;
    } else if (document.getElementById('txtReportInfoName').value != "") {
        apiUrl += "name/" + document.getElementById('txtReportInfoName').value;
    }
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateSrch()) {
        fnResetSrch();
    return;
    }

    resultDivList.innerHTML = "";

      try {
            const response = await fetch(apiUrl); // Make the API call
            if (!response.ok) {
                fnReset();
           //throw new Error(`Database  Error: ${response.status}`); // Handle HTTP errors
           displayError(response.status, resultDivDtls, resultDivList);
           return;
            }
            const data = await response.json(); // Parse JSON response
            globalData = data;
            //console.log("data = " + data); // csv data items in object
```

```
            fnDisplayReportInfoList(data); // Display data on the page
        } catch (error) {
            displayError(error, resultDivDtls, resultDivList);
        }
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles GET requests to get/retrieve an existing ReportInfo.
     * @param reportInfoId the ID of the ReportInfo to be retrieved
     * @return the requested ReportInfo entity
     */
    @GetMapping("/{reportInfoId}")
    public ResponseEntity<ReportInfo> getReportInfoById(
            @PathVariable Integer reportInfoId) {
        log.info("Started ReportInfoController::getBookById");
        return ResponseEntity.status(HttpStatus.OK).body(
            reportInfoService.getReportInfoById(reportInfoId));
    }

    /**
     * Handles GET requests to get/retrieve existing ReportInfos.
     * @param name the name pattern of the ReportInfos to be retrieved
     * @return a list of all requested ReportInfos
     */
    @GetMapping("/name/{name}")
    public ResponseEntity<List<ReportInfo>> getReportInfoByName(@PathVariable String name){
        log.info("Started ReportInfoController::getReportInfoByName. name = " + name);
        return ResponseEntity.status(HttpStatus.OK).body(
                    reportInfoService.getReportInfoByName(name));
    }
```

Repository Class [Java Code] => Git Hub Link

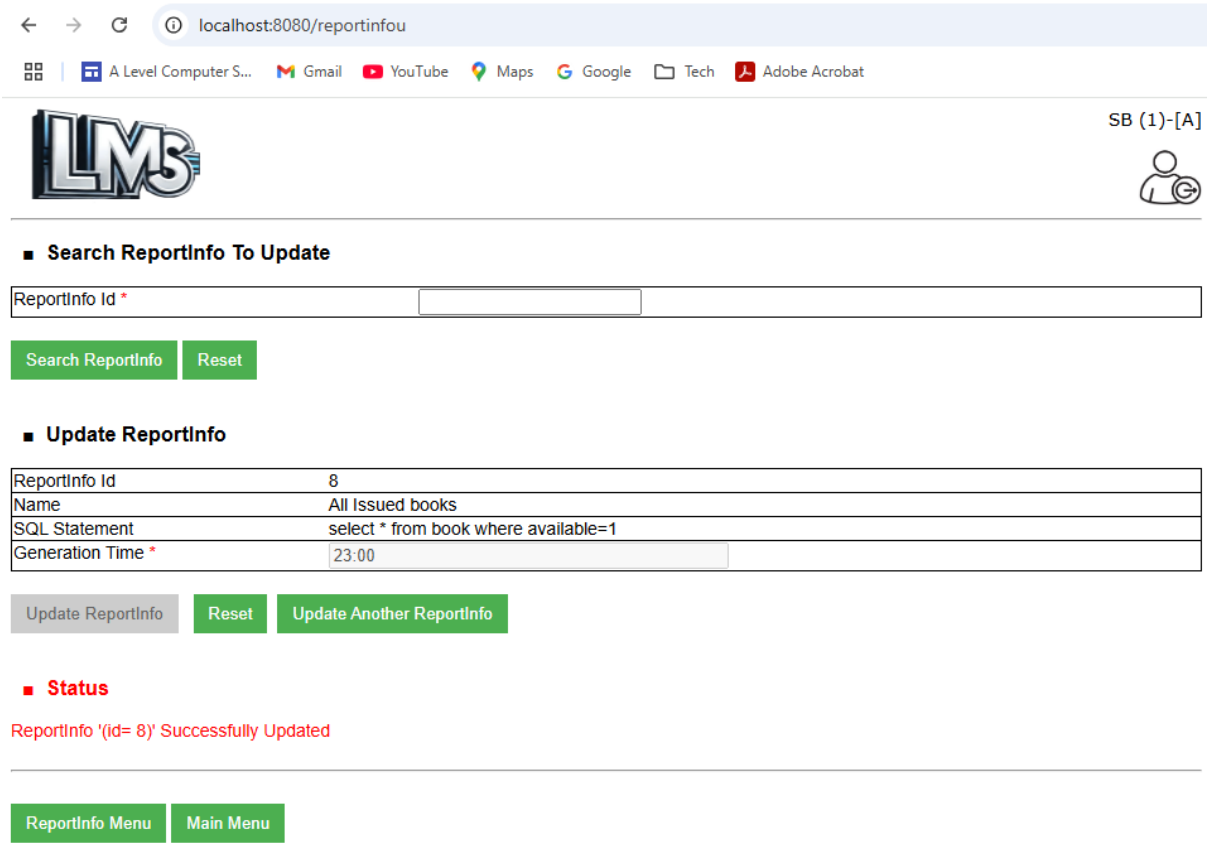
```
    /**
     * Handles the DB call to get/retrieve all ReportInfos by its name pattern
     * @param name the name pattern of all associated ReportInfos to be retrieved
     * @return a list of the requested ReportInfo entities
     */
    @Query(
            value = "SELECT * FROM REPORTINFO WHERE name LIKE %?1%",
            nativeQuery = true)
    List<ReportInfo> findReportInfoByName(String name);
```

Service Class [Java Code] => Git Hub Link

```
    /**
```

```
     * Handles requests to get/retrieve an existing ReportInfo.
     * @param reportInfoId the Id of ReportInfo to be retrieved
     * @return the requested ReportInfo entity
     */
    public ReportInfo getReportInfoById(Integer reportInfoId) {
        log.info("Started ReportInfoService::getReportInfoById");

        // Gets the reportinfo entity by its ID.
        // select * from reportinfo where reportinfo_id=?
        ReportInfo reportInfoDB = reportInfoRepository.findById(reportInfoId).orElse(null);

        log.info("Returning ReportInfoService::getReportInfoById");
        return reportInfoDB;
    }

    /**
     * Handles requests to get/retrieve existing ReportInfos by their title pattern.
     * @param name the title pattern of the ReportInfos to be retrieved
     * @return a list of all requested ReportInfos
     */
    public List<ReportInfo> getReportInfoByName(String name) {
        log.info("Started ReportInfoService::getReportInfoByName. name = " + name);

        // Gets the ReportInfo entity by its name pattern.
        // SELECT * FROM REPORTINFO WHERE name LIKE %?%;
        List<ReportInfo> reportInfoDBList =
                    reportInfoRepository.findReportInfoByName(name);

        log.info("Returning ReportInfoService::getReportInfoByName. name = " + name);
        return reportInfoDBList;
    }
```

Update ReportInfo



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Update ReportInfo</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../reportinfo/js/reportinfou.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType == 'A'}">
        <h3>  &#x25A0  Search ReportInfo To Update</h3>
        <table>
            <tr><td>ReportInfo Id <label style='color:red'>*</label>
             <td><input type="text" id="txtReportInfoId"></input>
        </table><br>
        <button onclick = "fnSearchReportInfo();">Search ReportInfo</button>
        <button onclick = "fnResetSrch();">Reset</button>
    </div>
```

```
    <div th:unless="${userType == 'A'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>

    <div id="divUpdateReportInfo"></div>
    <div id="divStatus" style="color:red"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/reportinfo}"><button>ReportInfo Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the ReportInfo information provided on the page and submits the
information to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnUpdateReportInfo() {
    let apiUrl = server + apiContext + document.getElementById('tdReportInfoId').innerText;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateUpdate()) {
        return;
    }
    // information to be submitted for saving
    let payload = {
        timeToGenerate: document.getElementById('txtGenTime').value
    }

    // create json body for submitting the Update ReportInfo request
    let options = {method: "PUT", headers: {'Content-Type': 'application/json'},
                   body: JSON.stringify(payload)};

    const response = fetch(apiUrl, options);

    let strStatus = "<br><h3>  &#x25A0  Status</h3>";
    strStatus += "ReportInfo '(id= "
      + document.getElementById('tdReportInfoId').innerText + ")' Successfully Updated";
    resultDivStatus.innerHTML= strStatus;

    toggleFields(true);

    fnResetSrch();

    document.getElementById("btnUpdateReportInfo").className = "dbtn";
    document.getElementById("btnUpdateAnotherReportInfo").className = "";
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles PUT requests to update an existing ReportInfo.
     * @param reportInfoId the ID of the ReportInfo to be updated
     * @param reportInfo the ReportInfo entity with updated information
     * @return the updated ReportInfo entity
     */
    @PutMapping("/{reportInfoId}")
    public ResponseEntity<ReportInfo> updateReportInfo(
      @PathVariable Integer reportInfoId, @RequestBody ReportInfo reportInfo) {
        log.info("Started ReportInfoController::updateReportInfo");
        return ResponseEntity.status(HttpStatus.OK).body(
             reportInfoService.updateReportInfo(reportInfoId, reportInfo));
    }
```

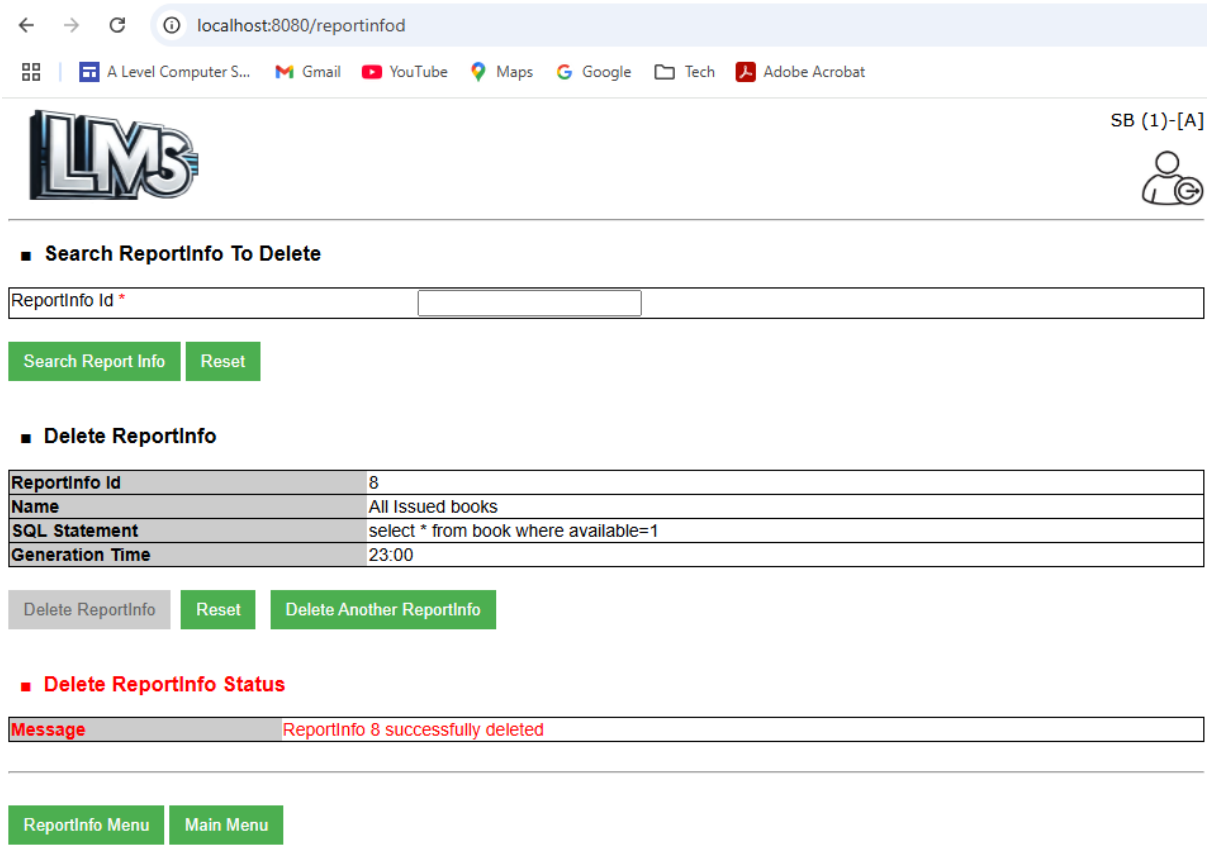
Repository Class [Java Code]

NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save an existing ReportInfo.
     * @param reportInfoId the ID of the ReportInfo to be updated
     * @param reportInfoFromClient the ReportInfo Object with updated values
     * @return the updated ReportInfo entity
     */
    public ReportInfo updateReportInfo(
             Integer reportInfoId, ReportInfo reportInfoFromClient) {
        log.info("Started ReportInfoService::updateReportInfo");

        // Finds the existing ReportInfo by ID.
        ReportInfo reportInfoDB = this.getReportInfoById(reportInfoId);

        // Map all non null values
        ReportInfo reportInfoUpdate =
             ReportInfoUtils.nonNullMapper(reportInfoFromClient, reportInfoDB);

        // Saves and returns the updated entity.
        log.info("Returning ReportInfoService::updateReportInfo");

        // update reportinfo set time_to_generate=? where reportinfo_id=?
        return reportInfoRepository.save(reportInfoUpdate);
    }
```

Deleted ReportInfo After confirmation



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Delete ReportInfo</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../reportinfo/js/reportinfod.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div id="divMainWindow">
        <div th:if="${userType == 'A'}">
            <h3>  &#x25A0  Search ReportInfo To Delete</h3>
            <table>
                <tr><td>ReportInfo Id <label style='color:red'>*</label>
                    <td><input type="text" id="txtReportInfoId"></input>
            </table>
            <br><button onclick = "fnSearchReportInfo();">Search Report Info</button>
            <button onclick = "fnResetSrch();">Reset</button>
```

```
        </div>
        <div th:unless="${userType == 'A'}" style="color:red">
            <script>document.write(accessDeniedMsg);</script>
        </div>

        <div id="divDeleteReportInfo"></div>
        <div id="divStatus" style="color:red"></div>

        <br><hr><br>
        <div>
            <a th:href="@{/reportinfo}"><button>ReportInfo Menu</button></a>
            <a th:href="@{/menu}"><button>Main Menu</button></a>
        </div>
    </div>
    <div id="divModal" style="background-color:#cff5d0;display:none;height:290px;"></div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gets the Report Info ID on the page and submits the information to the server
as an API request.
It also clears any previous output text from any areas on the page
*/
//async function fnDeleteReportInfo() {
async function fnDoAction() {
    let apiUrl = server + apiContext + glbDelReportInfoId;
    //console.log("apiUrl = " + apiUrl);

    // create json body for submitting the Delete ReportInfo request
    let options = {method: "DELETE"};
    const response = await fetch(apiUrl, options);

    if (!response.ok) {
        //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
        displayError(response.status, resultDivStatus, resultDiv);
        return;
    }

    const data = await response.text(); // Text response
    fnDisplayDeleteResponse(data); // Display data on the page
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles DELETE requests to remove a ReportInfo by ID.
     * @param reportInfoId the ID of the ReportInfo to be deleted
     * @return a success/failure message
     */
    @DeleteMapping("/{reportInfoId}")
    public ResponseEntity<String> deleteReportInfoById(@PathVariable Integer reportInfoId){
```

```
        log.info("Started ReportInfoController::deleteReportInfoById. reportInfoId = "
             + reportInfoId);
        return ResponseEntity.status(HttpStatus.OK).body(
             reportInfoService.deleteReportInfoById(reportInfoId));
    }
}
```

Repository Class [Java Code]

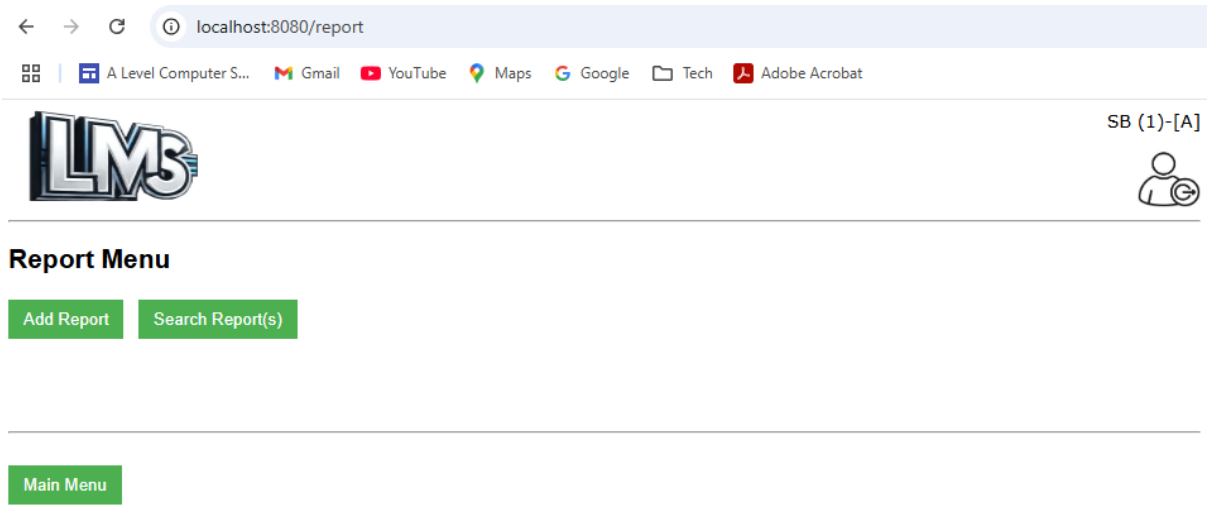
NA

Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to delete a ReportInfo by ID.
     * @param reportInfoId the ID of the ReportInfo to be deleted
     * @return a success/failure message of deletion
     */
    public String deleteReportInfoById(Integer reportInfoId) {
        log.info("Started ReportInfoService::deleteReportInfoById. reportInfoId = "
             + reportInfoId);

        // Finds the existing ReportInfo by ID.
        ReportInfo reportInfo = this.getReportInfoById(reportInfoId);

        // Check if the ReportInfo has any associated reports
        // SELECT COUNT(*) FROM REPORT WHERE reportinfo_id =?
        int reportCounts = eportRepository.findTotalReportCountsByReportInfo(reportInfoId);

        boolean reportInfoHasAssociatedReports = (reportCounts > 0);
        if (reportInfoHasAssociatedReports) {
            log.info("Returning ReportInfoService::deleteReportInfoById");
            return "error:ReportInfo " + reportInfoId + " cannot be deleted. It has "
                    + reportCounts + " associated report(s).";
        }

        // Deletes the ReportInfo entity by its ID.
        // delete from reportinfo where reportinfo_id=?
        reportInfoRepository.deleteById(reportInfoId);
        log.info("Returning ReportInfoService::deleteReportInfoById");
        return "success:ReportInfo " + reportInfoId + " successfully deleted";
    }
```

CR~~UD~~ Report

Report Menu



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>LMS Report Menu Page</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType == 'A'}">
        <h2>Report Menu</h2>
            <a th:href="@{/reporta}"><button>Add Report</button></a>  
            <a th:href="@{/reportr}"><button>Search Report(s)</button></a>
    </div>
    <div th:unless="${userType == 'A'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
    <br><br><br><br><hr><br>
    <div>
        <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code]

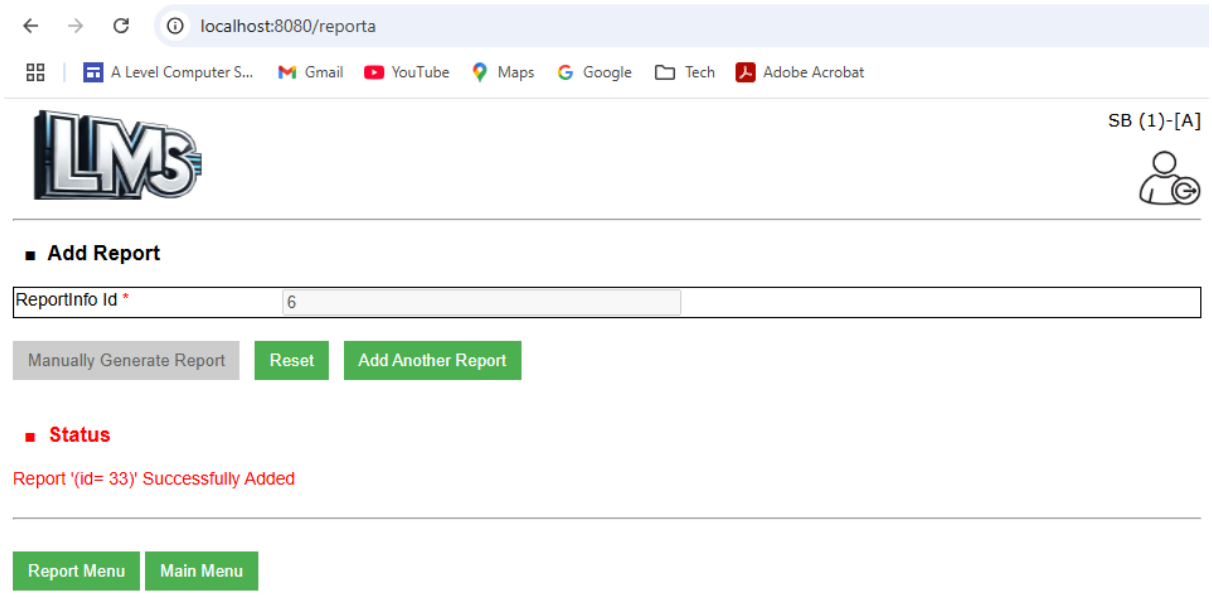
NA

Server Code [Java]

Model/Data Object Class [Java Code] => Git Hub Link

```
public class Report {
    @Id
    @Column(name = "report_id")
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer reportId;

    @Column(name = "generated_date")
    private Timestamp generatedDate;

    @Column(name = "download_link")
    private String downloadLink;

    @ManyToOne
    @JoinColumn(name = "reportinfo_id")
    private ReportInfo reportInfo;

    @Transient
    private String content;

    @Transient
    private LmsFault fault;
}
```

Create Report



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
```

```
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Add Report</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../report/js/reporta.js"></SCRIPT>
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>
<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType == 'A'}">
        <h3>  &#x25A0  Add Report</h3>
        <table>
            <tr><td>ReportInfo Id <label style='color:red'>*</label>
                <td><input type="text" id="txtReportInfoId" size="40"></input></tr>
        </table>
        <br><button id="btnAddReport" onclick="fnAddReport();">
             Manually Generate Report</button>  
        <button onclick="fnReset();">Reset</button>
          <button id="btnAddAnotherReport" onclick="fnAddAnotherReport();"
             class='dbtn'>Add Another Report</button>
    </div>
    <div th:unless="${userType == 'A'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>
    <div id="divStatus" style="color:red"></div>

    <br><hr><br>
    <div>
        <a th:href="@{/report}"><button>Report Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the ReportInfo information provided on the page and submits the
information to the server as an API request.
It also clears any previous output text from any areas on the page
*/
function fnAddReport() {
    resultDivStatus = document.getElementById('divStatus');
    let apiUrl = server + apiContext + document.getElementById('txtReportInfoId').value;
    //console.log("apiUrl = " + apiUrl);

    // validate the information given on the UI
    if (!validateAdd()) {
        return;
    }
    // information to be submitted for saving
    let payload = {
        reportInfo: {
            reportInfoId: document.getElementById('txtReportInfoId').value
```

```
        }
    }

    // create json body for submitting the Add Report request
    let options = {method: "POST"};//, headers: {'Content-Type': 'application/json'}, body:
JSON.stringify(payload)};

    fetch(apiUrl, options).then(response => {if (!response.ok) {
            //throw new Error(`Error: ${response.status}`); // Handle HTTP errors
            displayError(response.status, resultDivStatus, null);
            return;
        }
        return response.json();
        })
        .then(dataList => {
        if (dataList.fault) { // If server encountered an error
            let strFault = "<br><h3>  &#x25A0  Error Details</h3>";
            strFault += "<table><tr><th width=30%>Http</td><td style='color:red'>"
                    + dataList.fault.http + "</td></tr>";
            strFault += "<tr><th>Code</td><td style='color:red'>" + dataList.fault.code
                    + "</td></tr>";
            strFault += "<tr><th>Message</td><td style='color:red'>"
                    + dataList.fault.message + "</td></tr>";
            strFault += "<tr><th>Path</td><td style='color:red'>" + dataList.fault.path
                    + "</td></tr></table>";

            resultDivStatus.innerHTML = strFault;
        } else { // If server processed the request successfully
            let strStatus = "<br><h3>  &#x25A0  Status</h3>";
            strStatus += "Report '(id= " + dataList.reportId + ")' Successfully Added";
            resultDivStatus.innerHTML = strStatus;

          // Disable the fields once the request is submitted so no further
         // changes can be done on the same form
          toggleFields(true);

          document.getElementById("btnAddReport").className = "dbtn";
          document.getElementById("btnAddAnotherReport").className = "";
        }
    });
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     *
     * Handles POST requests to save a new Report.
     * @param reportInfoId the ID of the ReportInfo for which a report is generated
     * @return the saved Report entity
     */
    @PostMapping("/reportinfo/{reportInfoId}")
    public ResponseEntity<Report> addReportForReportInfo(
                    @PathVariable Integer reportInfoId) {
        log.info("Started ReportController::addReport. reportInfoId = " + reportInfoId);
        return ResponseEntity.status(HttpStatus.CREATED).body(
```

```
                reportService.addReportForReportInfo(reportInfoId));
    }
```

Repository Class [Java Code]

NA

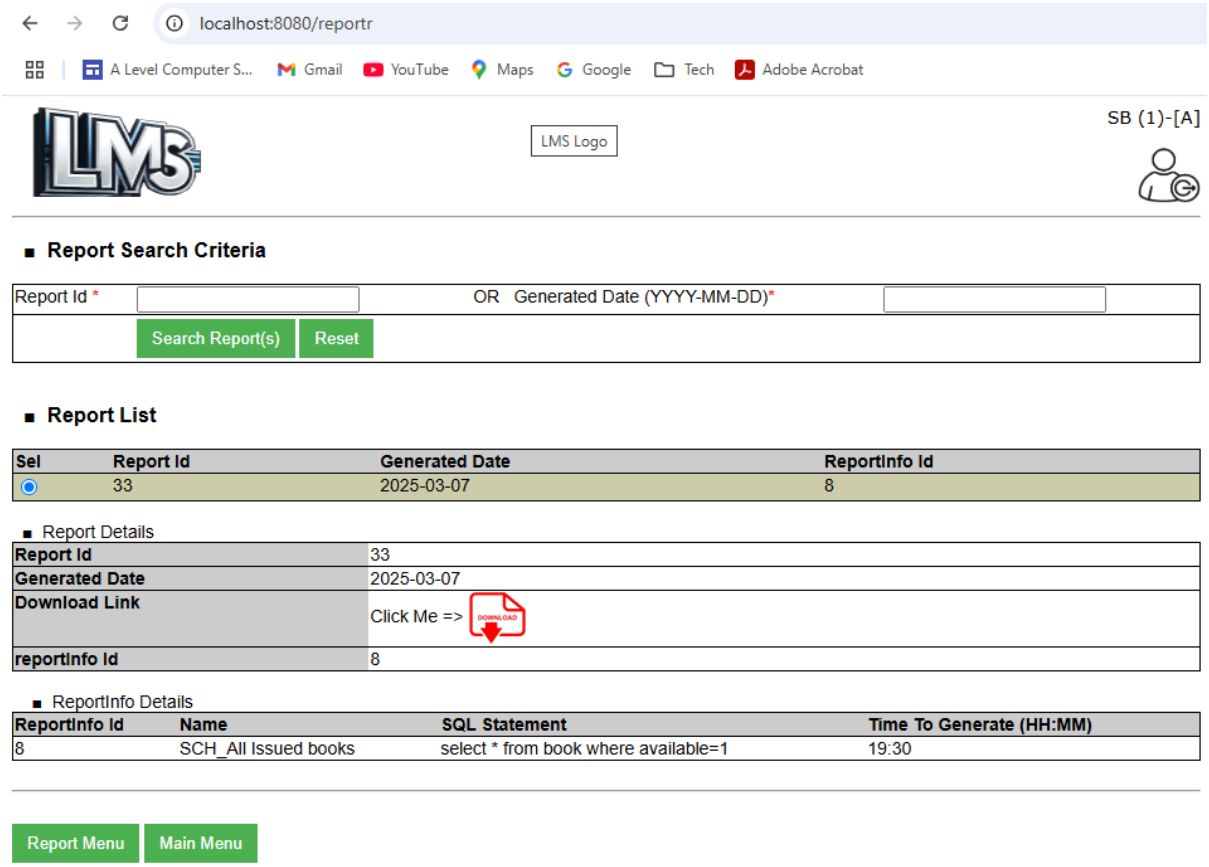
Service Class [Java Code] => Git Hub Link

```
    /**
     * Handles requests to save a new Report for a ReportInfo.
     * @param reportInfoId the Id of the ReportInfo for which a report is to be saved
     * @return the saved Report entity
     */
    public Report addReportForReportInfo(Integer reportInfoId) {
        log.info("Started ReportService::addReportForReportInfo");
        Report report = null;
        String reportContentDB = null;

        // Gets the reportinfo entity by its ID.
        // select * from reportinfo where reportinfo_id=?
        ReportInfo reportInfoDB = reportInfoRepository.findById(reportInfoId).orElse(null);

        // Create Fault Object to give details on UI
        if (reportInfoDB == null) {
            LmsFault lmsFault = new LmsFault("Resource Not Found", "404",
                    "ReportInfo Not Found","/lms/v1/reportinfos/" + reportInfoId);
            return Utils.createFaultyReport(lmsFault);
        }

        // Get the JDBC connection and generate the report
        try {
            reportContentDB = jdbcCall(reportInfoId);

        } catch (Exception ex) {
            log.info("Exception in addReportForReportInfo " + ex.getMessage());
            ex.printStackTrace();
            LmsFault lmsFault = new LmsFault("Resource Not Found", "404",
                    "Report Content Not Found", "/lms/v1/reports");
            return Utils.createFaultyReport(lmsFault);
        }

        // Create the Report Object
        report = new Report();
        report.setContent(reportContentDB);
        report.setReportInfo(reportInfoDB);
        report.setGeneratedDate(Utils.convertNowToSQLTS());// today's date
        log.info("Returning ReportService::addReportForReportInfo");

        // When the report is created, the downlink cannot have the report ID because by
        // this time the DB has not yet assigned any Id to the report.
        // Hence an immediate update also is necessary
        // as part of Report model's @PostPersist
        // insert into report (download_link,generated_date,reportinfo_id) values (?,?,?)
        // update report set download_link=? where report_id=?
```

```
        return reportRepository.save(report);
    }

    /**
     * Handles the connection to the Database, executes the query and gives the resultSet
     * in a report format
     * @param reportInfoId the Id of the ReportInfo for which a report is to be saved
     * @return the report content as a String
     */
    public String jdbcCall(Integer reportInfoId) throws Exception {
        log.info("Starting ReportService::jdbc. reportInfoId = " + reportInfoId);
        String infoQuery = "select * from reportinfo where reportinfo_id ="
                    +  reportInfoId; // query to be run
        Class.forName("com.mysql.cj.jdbc.Driver"); // Driver name

        // Using the datasource configured in the application.properties,
        // establish the DB connection
        Connection con = datasource.getConnection();
        log.info("DB Connection Established successfully");

        // Create a statement to be executed on the DB to get the details of the ReportInfo
        PreparedStatement pst = con.prepareStatement(infoQuery);
        ResultSet resultSet = pst.executeQuery(); // Execute query
        resultSet.next(); // Shift the resultSet pointer to the first element
                          // in the resultSet (Result Table)

        // Get the preset sql statement from the ReportInfo to be executed for Report Add
        String sqlStatement = resultSet.getString("sql_statement"); // sql to be executed
        pst = con.prepareStatement(sqlStatement);

        // Execute the query on the DB
        resultSet = pst.executeQuery(); // Execute the sql on DB

        // Get metadata (i.e Column count and names) of the query being executed
        final ResultSetMetaData metaData = resultSet.getMetaData();
        final int columnCount = metaData.getColumnCount();

        // Formatted to html report but can be a csv file as well if needed
        StringBuilder stringBuilder = new StringBuilder("<table border=1><tr><th>");
        // Write column names as header line
        for (int i = 1; i <= columnCount; i++) {
            stringBuilder.append(metaData.getColumnName(i));
            if (i < columnCount) {
                //sb.append(",");
                stringBuilder.append("<th>");
            }
        }

        // Write data rows
        while (resultSet.next()) {
            stringBuilder.append("<tr><td>");
            for (int i = 1; i <= columnCount; i++) {
                stringBuilder.append(resultSet.getString(i));
                if (i < columnCount) {
                    stringBuilder.append("<td>");
                }
            }
```

```
        }
        //log.info(sb.toString());
        // Close the Statament, ResultSet and Connection to avoid Memory Leakage
        pst.close(); // close statement
        resultSet.close(); // close resultSet
        con.close(); // close connection

        log.info("Connection Closed....");
        log.info("Returning ReportService::jdbc. reportInfoId = " + reportInfoId);
        // return the final html string
        return stringBuilder.toString();
    }
```

Read Report



UI Form/Screen [HTML Code] => Git Hub Link

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Search Report</title>
    <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">
    <SCRIPT src="../report/js/reportr.js"></SCRIPT>
```

```
    <SCRIPT src="../js/main.js"></SCRIPT>
    <link rel="stylesheet" type="text/css" href="../css/main.css">
    <link rel="icon" href="../img/lms.png">
</head>

<body>
    <span style="display:none" id="hdnUserInfo" th:text="${userInfo}"></span>
    <SCRIPT src="../js/header.js"></SCRIPT>
    <div th:if="${userType == 'A'}">
         <h3>  &#x25A0  Report Search Criteria</h3>
             <table>
                 <tr><td>Report Id <label style='color:red'>*</label>
                     <td><input type="text" id="txtReportId"></input><td colspan=6>OR</td>
                     <td>Generated Date (YYYY-MM-DD)<label style='color:red'>*</label>
                     <td><input type="text" id="txtGeneratedDate" maxlength="10"></input>
                 <tr><td><td><td><button onclick = "fnSearchReport();">
                    Search Report(s)</button>
                 <button onclick = "fnReset();">Reset</button>
             </table>
         </div>
    </div>
    <div th:unless="${userType == 'A'}" style="color:red">
        <script>document.write(accessDeniedMsg);</script>
    </div>

    <div id="divListReport"></div>
    <div id="divDtlsReport"></div>
    <br><hr><br>
    <div>
        <a th:href="@{/report}"><button>Report Menu</button></a>
       <a th:href="@{/menu}"><button>Main Menu</button></a>
    </div>
</body>
</html>
```

Client Side scripting [Javascript Code] => Git Hub Link

```
/*
This function gathers the search information provided on the page and submits the
information to the server as an API request.
It calls the display Report function to show the data on the page
*/
async function fnSearchReport() {
    resultDivList = document.getElementById('divListReport');
    resultDivDtls = document.getElementById('divDtlsReport');
    let apiUrl = server + apiContext;

    if (document.getElementById('txtReportId').value != "") {
        apiUrl += document.getElementById('txtReportId').value;
    } else if (document.getElementById('txtGeneratedDate').value != "") {
        apiUrl += 'date/' + document.getElementById('txtGeneratedDate').value;
    }

    //console.log("apiUrl = " + apiUrl);
```

```
    // validate the information given on the UI
    if (!validateSrch()) {
        fnResetSrch();
           return;
      }
    resultDivList.innerHTML = "";

      try {
            const response = await fetch(apiUrl); // Make the API call
            if (!response.ok) {
                fnReset();
           //throw new Error(`Database Error: ${response.status}`); // Handle HTTP errors
           displayError(response.status, resultDivDtls, resultDivList);
           return;
            }

            const data = await response.json(); // Parse JSON response
            globalData = data;
            //console.log("data = " + data); // csv data items in object
            fnDisplayReportList(data); // Display data on the page
      } catch (error) {
            displayError(error, resultDivDtls, resultDivList);
      }
}
```

Server Code [Java]

Controller Class [Java Code] => Git Hub Link

```
    /**
     * Handles GET requests to get/retrieve an existing Report.
     * @param reportId the ID of the Report to be retrieved
     * @return the requested Report entity
     */
    @GetMapping("/{reportId}")
    public ResponseEntity<Report> getReportById(@PathVariable Integer reportId) {
        log.info("Started ReportController::getReportById. reportId = " + reportId);
        return ResponseEntity.status(HttpStatus.OK).body(
                    reportService.getReportById(reportId));
    }

    /**
     * Handles GET requests to get/retrieve existing Reports.
     * @param generatedDate the generatedDate of the Reports to be retrieved
     * @return a list of all requested Reports
     */
    @GetMapping("/date/{generatedDate}")
    public ResponseEntity<List<Report>> getReportsByGeneratedDate(
                            @PathVariable String generatedDate) {
        log.info("Started ReportController::getReportsByGeneratedDate");
        return ResponseEntity.status(HttpStatus.OK).body(
                    reportService.getReportsByGeneratedDate(generatedDate));
    }
```

Repository Class [Java Code] => Git Hub Link

```java
/**
 * Repository interface for Report entity.
 * Provides CRUD operations and custom query methods through JpaRepository.
 * @author Saarah Bedekar
 */
@Repository // Indicates that this interface is a Spring Data repository.
public interface ReportRepository extends JpaRepository<Report, Integer> {

    /**
     * Handles the DB call to get/retrieve all Reports by its date of generation
     * @param generatedDate the date of report creation for all associated Reports to be
retrieved
     * @return a list of the requested Report entities
     */
    @Query(
            value = "SELECT * FROM REPORT WHERE generated_date LIKE %?1% ",
            nativeQuery = true)
    List<Report> findReportsByGeneratedDate(String generatedDate);
}
```

Service Class [Java Code] => Git Hub Link

```java
    /**
     * Handles requests to get/retrieve an existing Report.
     * @param reportId the Id of Report to be retrieved
     * @return the requested Report entity
     */
    public Report getReportById(Integer reportId) {
        log.info("Started ReportService::getReportById. reportId = " + reportId);

        // Gets the reportentity by its ID.
        // select * from report where report_id=?
        Report reportDB = reportRepository.findById(reportId).orElse(null);

        // Create Fault Object to give details on UI
        if (reportDB == null) {
            LmsFault lmsFault = new LmsFault("Resource Not Found", "404",
                    "Report Not Found","/lms/v1/reports/" + reportId);
            return Utils.createFaultyReport(lmsFault);
        }
        log.info("Returning ReportService::getReportById");
        return reportDB;
    }

    /**
     * Handles requests to get/retrieve existing Reports by their dateString pattern
     * @param dateString the dateString pattern (YYYY-MM-DD) of the Reports to be retrieved
     * @return a list of all requested Reports
     */
    public List<Report> getReportsByGeneratedDate(String dateString) {
        log.info("Started ReportService::getReportsByGeneratedDate");

        // Gets the report entity by its dateString pattern.
```

```
        // SELECT * FROM REPORT WHERE generated_date LIKE %?1%
        List<Report> reportDBList =
             reportRepository.findReportsByGeneratedDate(dateString);

        // Create Fault Object to give details on UI
        if (reportDBList == null || reportDBList.isEmpty()) {
            LmsFault lmsFault = new LmsFault("Resource Not Found", "404",
                    "Report Not Found","/lms/v1/reports/date/" + dateString);
            return Utils.createFaultyReportInList(lmsFault);
        }
        log.info("Returning ReportService::getReportsByGeneratedDate");
        return reportDBList;
    }
```

Generic Code

Client Side scripting [Javascript Code] => Git Hub Link

```
// Util method required
const server = "http://localhost:8080/"; //"http://192.168.3.123:8080/";

/**
 * Handles integer check via regular expression.
 * @param value the parameter value
 * @return boolean flag indicating pass or fail check

 Regular expressions used for checks in this js file
 -------------------------
 Part     Meaning
 -------------------------
 ^            Start of the string
 \d*      Zero or more digits (0-9) before the decimal point (optional)
 \d+      At least one digit after the decimal (ensures valid decimal numbers)
 $            End of the string
 */
function isPositiveNumber(value) {
    //
    const pattern = /^\d*$/;
    return pattern.test(value) && parseInt(value) > 0;
}

/**
 * Handles decimal check via regular expression.
 * @param value the parameter value
 * @return boolean flag indicating pass or fail check

 Regular expressions used for checks in this js file
 -------------------------
 Part     Meaning
 -------------------------
 ^            Start of the string
 \d*      Zero or more digits (0-9) before the decimal point (optional)
 \d+      At least one digit after the decimal (ensures valid decimal numbers)
 $            End of the string
 */
function isPositiveDecimal(value) {
    //
    const pattern = /^\d+(\.\d+)?$/;
    return pattern.test(value) && parseDouble(value) > 0.0;
}

/**
 * Handles valid email format check via regular expression.
 * @param value the email parameter value
 * @return boolean flag indicating pass or fail check

 Regular expressions used for checks in this js file
 ---------------------------
 Part             Meaning
```

```
 ----------------------------
 ^                    Start of the string
 [a-zA-Z0-9._%+-]+  Allows letters (a-z, A-Z), digits (0-9), special characters (._%+-), at
least one character required
 @                    Literal "@" symbol (must be present)
 [a-zA-Z0-9.-]+     Allows letters, digits, dots (.), and hyphens (-), at least one
character required
 \.                   Literal "." before the domain extension
 [a-zA-Z]{2,}    Domain extension: Must be at least two letters (e.g. com, org, uk)
 $                      End of the string
 */
function isValidEmailStrict(email) {
    const emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
    return emailPattern.test(email);
}

/**
 * Handles valid UK mobile number format check via regular expression.
 * @param value the UK mobile number parameter value
 * @return boolean flag indicating pass or fail check

 Regular expressions used for checks in this js file
 -------------------------
 Part     Meaning
 -------------------------
 ^            Start of the string
 07        The number must start with "07" (UK mobile numbers)
 \d{9}      Exactly 9 digits (0-9) after 07, ensuring a total of 11 digits
 $            End of the string
 */
function isValidUKMobile(number) {
    const ukMobilePattern = /^07\d{9}$/;  // Starts with 07 and has exactly 11 digits
    return ukMobilePattern.test(number);
}

/**
 * Handles valid Date format check via ISOString.  it does not directly check for leap
years.
 * However, it indirectly enforces valid date constraints because
 * JavaScript's Date object auto-corrects invalid dates
 *
 * @param value the date parameter value
 * @return boolean flag indicating pass or fail check
 */
function isValidDate(dateString) {
    const dateObj = new Date(dateString);
    // Checks for valid timestamp (isNaN(dateObj.getTime()). False means valid)
    // toISOString() returns in YYYY-MM-DDTHH:mm:ss.sssZ format.
    // startsWith(dateString) ensures only the date part matches.
    return !isNaN(dateObj.getTime()) && dateObj.toISOString().startsWith(dateString);
}

/**
 * Handles valid Report generation time format check via regular expression.
 * @param value the date parameter value
 * @return boolean flag indicating pass or fail check
```

```
 Regular expressions used for checks in this js file
 -------------------------
 Part         Meaning
 -------------------------
 ^               Start of the string
 \d{2}              Exactly 2 digits (00-23) for hour (HH) and (00-59) for minutes (MM)
 :              Literal hyphen (:) used for separation
 $               End of the string
 */
function isValidReportTime(timeString) {
    // Check format
    const datePattern = /^\d{2}:\d{2}$/;
    if (!datePattern.test(timeString))
        return false;

    // Parse the date
    const [hours, minutes] = timeString.split(":").map(Number);

    // Check if date is valid
    return (
        (hours >= 0 && hours <= 23) &&
        (minutes >= 0 && minutes <= 59)
    );
}

/**
 * Handles future date check with respect to current system date
 * @param value the date parameter value
 * @return boolean flag indicating future date or not
 */
function isFutureDate(dateString) {
    const [year, month, day] = dateString.split("-").map(num => parseInt(num, 10));

    // Construct Date object (JS months are 0-based)
    const inputDate = new Date(year, month - 1, day);

    // Get today's date without time
    const today = new Date();
    today.setHours(0, 0, 0, 0);

    // Check if input date is in the future
    return inputDate > today;
}

/**
 * Handles displaying the API response error associated to the API call
 * @param error the actual error string
 * @param resultDivStatus the status div place to show the error
 * @param resultDiv the result div to blank any previous contents
 */
function displayError(error, resultDivStatus, resultDiv) {
    //if (error = "400")
      //  error = "400 (Bad Request)";
    console.error('Fetch error:', error); // Log any errors
    let strError = "<br><h3>  &#x25A0  Error Details</h3>";
    strError += "<table><tr><th width=25%>Error fetching data: </td><td style='color:red'>"
+ error + "</td></tr></table>";
```

```
    if (resultDivStatus != null)
        resultDivStatus.innerHTML = strError;
    if (resultDiv != null)
        resultDiv.innerHTML = "";
}

/**
 * Handles the confirmation of delete (and can be extended to add/update) to avoid any
accidental action
 * @param action the action (e.g delete, update, add)
 * @param entity the entity (e.g User, Book etc)
 * @param id the id of the record
 * @param glbTable the table of record details on which the action is to be taken
 */
function fnConfirm(action, entity, id, glbTable) {
    let message = "<br><br><br>";
    message += "<div style='background-color:white'>" + glbTable + "</div>";
    message += "<h2><center>Are you sure you want to " + action + " " + entity
            + " [Id=" + id + "]</center></h2>";
    if (action.toLowerCase() == "delete")
        message += "<h3 style='color:red'><center>Once confirmed, " + entity
            + " [Id=" + id + "]";
        message += " will be deleted permanently. Cancel, if unsure.</center></h3><br>";

    message += "<br>";
    message += "<div style='text-align: center;'><button id='btnAction'
      onclick='fnDoAction();'>Confirm " + action + " " + entity + " </button>  ";
    message += "<button id='btnCancel' onclick='fnHideModal();'>Cancel</button></div>";
    document.getElementById('divModal').innerHTML = message;
    document.getElementById('divModal').style.display= "block";

    document.getElementById('divMainWindow').style.display= "none";
}

/**
 * Handles the cancel functionality in context of confirmations
 * This will hide the modal window and show the main window
 */
function fnHideModal() {
    document.getElementById('divModal').style.display= "none";
    document.getElementById('divMainWindow').style.display= "block";
}

/**
 * Handles the mapping of type short form to its description
 */
function userTypeDescription(type) {
    let description = "";
    if (type == "A") {
        description = "Admin";
    } else if (type == "S") {
        description = "Staff";
    } else if (type == "M") {
        description = "Member";
    }
    return description;
}
```

Header => Git Hub Link

```
var header = "";
header += '  <DIV class="grid-container">';
header += '         <DIV title="LMS Logo" class="grid-child"><img src="../img/lmstr.png"
alt="LMS Logo" width="150" height="75" /></DIV>';
header += '         <div class="grid-child" style="text-align:right"><div
id="spnUserInfo"></div>';
header += '         <a href="/logout"><img src="../img/logout.jpg" width="50" height="60"
/></a></div>';
header += '  </DIV><hr>';

document.write(header);
let result = document.getElementById('hdnUserInfo').innerText;
let glbUserInfo = result.split("-")[0];
let glbUserId = result.split("-")[1];
let glbUserType = result.split("-")[2];
document.getElementById('spnUserInfo').innerText = glbUserInfo + " (" + glbUserId + ")-["+
glbUserType + "]";

// Roles of LMS
let userRole ="";
if (glbUserType == "A") userRole = "Admin"
else if (glbUserType == "S") userRole = "Staff";
else if (glbUserType == "M") userRole = "Member";

// Access denied to pages as per authorisation rules
var accessDeniedMsg = "Access to this page is denied for " + userRole + " Users";
```

Server Code [Java]

Utils Class [Java Code] => Git Hub Link

```
public final class Utils {

    private static final int RETURN_DAYS_LIMIT = 14;
    /**
     * Checks if a String value is null or blank
     * @return the boolean check flag
     */
    public static boolean isStringBlank(String stringInput) {
        return stringInput == null || stringInput.trim().isBlank();
    }

    /**
     * Converts the current date into SQL Timestamp
     * @return the converted SQL timestamp
     */
    public static Timestamp convertNowToSQLTS() {
        log.info("Started Utils::convertNowToSQLTS");
        LocalDateTime now = LocalDateTime.now();
        Timestamp timestamp = Timestamp.valueOf(now);

        return timestamp;
    }
```

```java
/**
 * Calculates future date of exactly 15th day, i.e same day of week but after 2 weeks
 * @return the calculated SQL timestamp to mark the default return date of the Book
 */
public static Timestamp generateReturnSQLTS(Timestamp timestamp) {
    log.info("Started Utils::generateReturnSQLTS. timestamp = {}", timestamp);

    LocalDateTime future;

    //Add 15 days to the timestamp passed as parameter
    future = timestamp.toLocalDateTime().plus(RETURN_DAYS_LIMIT, ChronoUnit.DAYS);

    log.info("Returning Utils::generateReturnSQLTS. timestamp = {}", timestamp);
    return Timestamp.valueOf(future);
}

/**
 * Converts a Timestamp to a dateString (YYYY-MM-DD)
 * @return the converted dateString
 */
public static String convertTsToString(Timestamp timestamp) {
    log.info("Started Utils::convertStringToTs. timestamp = {}", timestamp);
    // Convert Timestamp to LocalDate
    LocalDate localDate = timestamp.toLocalDateTime().toLocalDate();

    log.info("Returning Utils::convertStringToTs. timestamp = {}", timestamp);
    // Format as "YYYY-MM-DD"
    return localDate.format(DateTimeFormatter.ISO_LOCAL_DATE);
}

/**
 * Converts a dateString (YYYY-MM-DD) to a Timestamp
 * @return the converted Timestamp
 */
public static Timestamp convertStringToTs(String dateString) {
    log.info("Started Utils::convertStringToTs. dateString = {}", dateString);
    // Define a DateTimeFormatter for the custom format
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd");

    // Parse the string into a LocalDate
    LocalDate localDate = LocalDate.parse(dateString, formatter);

    // Convert LocalDate to Timestamp (at start of the day)
    return Timestamp.valueOf(localDate.atStartOfDay());
}

/**
 * Handles the mapping between the user session and the GUI's model
 * @param session the user's Http Session
 * @param model the model which lets the thymleaf GUI get model information from server
 * @param returnPath the url path to its respective controller method
 * @return the returnPath as received
 */
public static String handleSession(HttpSession session,
                        Map<String, Object> model, String returnPath) {
    log.info("Starting Utils::authenticate. ", model.values().toString(), returnPath);
```

```
        String  userInfo = (String) session.getAttribute("userInfo");
        String  userId = (String) session.getAttribute("userId");
        String  userType = (String) session.getAttribute("userType");
        if (userInfo == null) {
            //log.info("userInfo is NULL here");
            return "redirect:/";
        }
        model.put("userInfo", userInfo);
        model.put("userId", userId);
        model.put("userType", userType);
        log.info("Returning Utils::authenticate. model = {}", model.values().toString());
        return returnPath;
    }
```

5 Testing

Software testing reduces project risks related to software quality, security and performance. For example, software defects can lead to system failures, slow performance and other significant impacts.

I will come up with test scenarios which can be easily tested out to verify the expected results for those cases. These test scenarios will test successfully that CRUD operations are working as expected for BOOKs, USERs, Transactions, Report Generation modules etc of the **LMS** application.

5.1 Test Overview (Explanation)

Testing has been done using Mockito framework that works well with Java to mock test. In this approach you mock the Class and set expectations on what it should return, and the values associated to the returned Object. Accordingly, you assert that the expected is the same values as when you mock trigger the method on that Class.

In all I have created 130 tests across all Controllers, Models, Repository, Service and Utils .

=> Git Hub Link

Example Test Code [Java]

Controller Test Class [Java Code] => Git Hub Link

```
@Test
@DisplayName("Test 1:Verify User by Id Retrieval Check")
@Order(1)
public void ctrTestGetUserById() throws Exception {

    // arrange
    when(userService.getUserById(1)).thenReturn(user);

    // action
    ResultActions response =
                    mockMvc.perform(get("/lms/v1/users/{userId}", 1));

    // Verify and assert
    response.andExpect(status().isOk()) // Expect HTTP 200
            .andDo(print())
            .andExpect(jsonPath("$.userId").value(1))
            .andExpect(jsonPath("$.firstName").value(user.getFirstName()))
            .andExpect(jsonPath("$.lastName").value(user.getLastName()));
}
```

Repository Test Class [Java Code] => Git Hub Link

```
@Test
@DisplayName("Test 1:Verify User by Id Retrieval Check")
@Order(1)
public void repTestGetUserById() {

    // arrange
    when(userRepository.findById(user.getUserId()))
                       .thenReturn(Optional.of(user));

    // action
    User userDB = userRepository.findById(user.getUserId()).orElse(null);

    // Verify and assert
    Assertions.assertThat(userDB).isNotNull();
    Assertions.assertThat(userDB).isInstanceOf(User.class);
    Assertions.assertThat(userDB).isNotInstanceOf(Book.class);

   Assertions.assertThat(user.getFirstName()).isEqualTo(userDB.getFirstName();
    Assertions.assertThat(user.getLastName()).isEqualTo(userDB.getLastName());
    Assertions.assertThat(userDB.getFault()).isNull();
    verify(userRepository, times(1)).findById(user.getUserId());
}
```

Service Test Class [Java Code] => Git Hub Link

```
@Test
@DisplayName("Test 1:Verify User by Id Retrieval Check")
@Order(1)
public void srvTestGetUserById() {

    // arrange
   when(userRepository.findById(user.getUserId()))
                       .thenReturn(Optional.of(user));

    // action
    User userDB = userService.getUserById(user.getUserId());

    // Verify and assert
    Assertions.assertThat(userDB).isNotNull();
    Assertions.assertThat(userDB).isInstanceOf(User.class);
    Assertions.assertThat(userDB).isNotInstanceOf(Book.class);

   Assertions.assertThat(user.getFirstName()).isEqualTo(userDB.getFirstName();
    Assertions.assertThat(user.getLastName()).isEqualTo(userDB.getLastName());
    Assertions.assertThat(userDB.getFault()).isNull();
    verify(userRepository, times(1)).findById(user.getUserId());

}
```

5.2 Test Plan

Video of Full LMS application in a happy path run through => **Video of Full LMS.mp4**

Below now are short videos of each functional areas of **CRUD** of entities

Test #	Objective #	Purpose of Test	Test Description	Input Test Data	Expected Result	Actual Result	Type of Test OR Data Type	Pass / Fail	Video Timestamp & Comments	
1	LMS_U_01.1	Add User	Unique userId created for the new User	User Data	Incremental userId generated for every new User	This objective has been achieved as expected	Normal	**Pass**	**DB PK auto increment.mp4** Full video	
	LMS_U_01.2		System confirmation of any failures including validation failures	User Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**User Add.mp4** 00:00:00 to 00:02:26	
	LMS_U_01.3		System confirmation of a successful User Add	User Data + User click to Add User	A success message should be shown when the User is Successfully added to the Database	This objective has been achieved as expected	Normal	**Pass**	**User Add.mp4** 00:00:27 to 00:02:40	

			This functionality should not be available to Member user to see details of other details	NA	Menu option missing to Add User	This objective has been achieved as expected	Normal	**Pass**	**User Add.mp4** 00:03:23 to 00:03:43	
2	LMS_U_02.1	Read a user	Search User by userId	userId	Exactly 1 user should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**User Read.mp4** 00:00:41 to 00:01:00	
	LMS_U_02.2		Search User by user's name pattern which will search across the user's First name, Middle Name and Last Name.	A part of user's name pattern from First name, Middle Name and Last Name	Matching user(s) should be searched for by triggering the appropriate query on DB	This objective has been achieved as expected	Normal	**Pass**	**User Read.mp4** 00:01:02 to 00:01:43	
	LMS_U_02.3		List of Users is displayed	A valid name pattern which will search across the user's First name, Middle Name and Last Name	1 or more user(s) should be displayed	This objective has been achieved as expected	Normal	**Pass**	**User Read.mp4** 00:01:07 to 00:01:14	

	LMS_U_02.4		The user should be able to select any user from the list of users to see the full details of that selected user.	User selection of a particular row	Selected user's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**User Read.mp4** 00:01:07 to 00:01:32	
	LMS_U_02.5		When there is only 1 user searched, the system should be able to auto select the user to see the full details of that user.	(The user should not have to select a row when there is only one row.) Row selection done automatically by the system	The only row present is selected and that user's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**User Read.mp4** 00:00:40 to 00:00:48	
	LMS_U_02.6		The selected user's address details should also be displayed	User row selection done automatically by the system	The selected user's address details are displayed along with User details	This objective has been achieved as expected	Normal	**Pass**	**User Read.mp4** 00:00:52, 00:01:19, 00:01:25	
	LMS_U_02.7		System confirmation of any failures including validation failures	User Data	An appropriate error message should be presented	This objective has been achieved as expected	Erroneous	**Pass**	**User Read.mp4** 00:00:08 to 00:00:40	

	LMS_U_02.8		This functionality should be available to Member user	NA	Member user can see only personal details and amend it at the same time	This objective has been achieved as expected	Normal	**Pass**	**User Read.mp4** 00:01:46 to 00:03:42	
3	LMS_U_03.1	Update a user	User should be searched by user Id.	userId	Exactly 1 user should be displayed if present or error displayed	This objective has been achieved as expected		**Pass**	**User Update.mp4** 00:00:00 to 00:00:50	
	LMS_U_03.2		System confirmation of any failures including validation failures	User Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**User Update.mp4** 00:00:08 to 00:00:27	
	LMS_U_03.3		When the search is Successful	User Data + User click to Update User	User details should be retrieved and displayed to the user, giving an option to update only limited fields allowed for	This objective has been achieved as expected	Normal	**Pass**	**User Update.mp4** 00:00:32 to 00:00:40	

					update (email, mobile) via the LMS app.					
	LMS_U_03.4		A success message should be shown when the User is Successfully updated	User Data + User click to Update User	A success message should be shown when the User is Successfully updated to the Database	This objective has been achieved as expected	Normal	**Pass**	**User Update.mp4** 00:00:43 to 00:00:50	
	LMS_U_03.5		This functionality should be available to Member user to update only their persoanl limited information	NA	Menu option missing to Update Any User but available only for self	This objective has been achieved as expected	Normal	**Pass**	**User Update.mp4** 00:01:02 to 00:01:24	
4	LMS_U_04.1	Delete a user	User should first be searched by user Id.	userId	Exactly 1 user should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**User Delete.mp4** 00:00:00 to 00:00:48	
	LMS_U_04.2		System confirmation of any failures	User Data	An appropriate	This objective has been	Erroneous	**Pass**	**User Delete.mp4** 00:00:06 to 00:00:33	

			including validation failures		error message should be presented asking the user to take action.	achieved as expected				
	LMS_U_04.3		When the search is Successful	User Data + User click to Delete User	User details should be retrieved and displayed to the user, who can then confirm the deletion of that user.	This objective has been achieved as expected	Normal	**Pass**	**User Delete.mp4** 00:00:37 to 00:01:04	
	LMS_U_04.4		A User cannot be deleted if the user has one or more issued books and not yet returned. OR if the user has one or more past transactions done as this will require admin to first remove the	User Data + User click to Delete User	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**User Delete.mp4** 00:01:04 to 00:02:04	

			data integrity dependency.							
	LMS_U_04.5		A success message should be shown when the User is Successfully deleted	User Data + User click to Delete User	A success message should be shown when the User is Successfully deleted to the Database	This objective has been achieved as expected	Normal	**Pass**	**User Delete.mp4** 00:02:15 to 00:03:00	
5	LMS_U_05.1	Update Members User's self details	User's self details retrieval	System provided userId	User's self details should be retrieved and displayed to the user, giving an option to update only limited fields allowed for update (password, email, mobile	This objective has been achieved as expected	Normal	**Pass**	**User Update Self.mp4** 00:00:00 to 00:00:10	
	LMS_U_05.2		A success message should be shown when the User is Successfully updated	User Data + User click to Update User	A success message should be shown when the User is Successfully	This objective has been achieved as expected	Normal	**Pass**	**User Update Self.mp4** 00:00:11 to 00:00:26	

					updated to the Database					
	LMS_U_05.3		This functionality should be available to Member user to see only personal details and amend it at the same time	NA	Menu option missing to Delete User	This objective has been achieved as expected	Normal	**Pass**	**User Update Self.mp4** 00:00:00 to 00:00:26	
6	LMS_BI_01.1	Add bookInfo	Unique bookInfoId created for the new Bookinfo	Bookinfo Data	Incremental bookInfoId generated for every new Bookinfo	This objective has been achieved as expected	Normal	**Pass**	**DB PK auto increment.mp4** Full video (Example of User but is applicable to BookInfo as well)	
	LMS_BI_01.2		System confirmation of any failures including validation failures	Bookinfo Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**BookInfo Add.mp4** 00:00:06 to 00:01:52	
	LMS_BI_01.3		System confirmation of a successful Bookinfo Add	Bookinfo Data + User click to Add User	A success message should be shown when	This objective has been	Normal	**Pass**	**BookInfo Add.mp4** 00:01:53 to 00:02:05	

					the BookInfo is Successfully added to the Database	achieved as expected				
	LMS_BI_01.4		This functionality should not be available to Member user	NA	Menu option missing to Add BookInfo	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Add.mp4** 00:02:05 to 00:02:32	
7	LMS_BI_02.1	Read a bookInfo	Search BookInfo by bookInfoId	bookInfoId	Exactly 1 BookInfo should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Read.mp4** 00:00:00 to 00:01:00	
	LMS_BI_02.2		Search BookInfo by bookInfo's title pattern which will search across the bookInfo's title	A part of bookInfo's title pattern	Matching BookInfo(s) should be searched for by triggering the appropriate query on DB	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Read.mp4** 00:01:05 to 00:01:10	
	LMS_BI_02.3		List of BookInfos is displayed	A valid title pattern which will search across the BookInfo's title	1 or more bookInfo(s) should be displayed	This objective has been	Normal	**Pass**	**BookInfo Read.mp4** 00:01:11 to 00:01:16	

						achieved as expected				
	LMS_BI_02.4		The user should be able to select any BookInfo from the list of BookInfos to see the full details of that selected BookInfo.	BookInfo selection of a particular row	Selected bookInfo's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Read.mp4** 00:01:17 to 00:01:26	
	LMS_BI_02.5		When there is only 1 BookInfo searched, the system should be able to auto select the BookInfo to see the full details of that BookInfo.	(The user should not have to select a row when there is only one row.) Row selection done automatically by the system	The only row present is selected and that bookInfo's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Read.mp4** 00:00:34 to 00:00:52	
	LMS_BI_02.6		System confirmation of any failures including validation failures	Bookinfo Data	An appropriate error message should be presented	This objective has been achieved as expected	Erroneous	**Pass**	**BookInfo Read.mp4** 00:00:08 to 00:00:26	

8	LMS_BI_03.1	Update a bookInfo	BookInfo should first be searched by bookInfo Id.	bookInfoId	Exactly 1 BookInfo should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Update.mp4** 00:00:00 to 00:00:36	
	LMS_BI_03.2		System confirmation of any failures including validation failures	Bookinfo Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**BookInfo Update.mp4** 00:00:07 to 00:00:32 and 00:00:52 to 00:02:00	
	LMS_BI_03.3		When the search is Successful	Bookinfo Data + User click to Update Bookinfo	Bookinfo details should be retrieved and displayed to the user, giving an option to update fields.	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Update.mp4** 00:00:36 to 00:00:48	
	LMS_BI_03.4		A success message should be shown when the BookInfo is Successfully updated	BookInfo Data + User click to Update BookInfo	A success message should be shown when the User is Successfully updated to the Database	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Update.mp4** 00:00:36 to 00:02:28	

	LMS_BI_03.5		This functionality should not be available to Member user	NA	Menu option missing to Update BookInfo	This objective has been achieved as expected	Normal		**BookInfo Update.mp4** 00:03:34 to 00:03:50	
9	LMS_BI_04.1	Delete a bookInfo	BookInfo should first be searched by bookInfo Id.	bookInfoId	Exactly 1 BookInfo should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Delete.mp4** 00:00:00 to 00:00:25	
	LMS_BI_04.2		System confirmation of any failures including validation failures	bookinfoId	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**BookInfo Delete.mp4** 00:00:00 to 00:00:15	
	LMS_BI_04.3		When the search is Successful	BookInfo Data + User click to Delete BookInfo	BookInfo details should be retrieved and displayed to the user, who can then confirm the deletion of that BookInfo.	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Delete.mp4** 00:00:15 to 00:00:22	

	LMS_BI_04.4		Book(s) present against BookInfo in the database.	BookInfo Data	A BookInfo cannot be deleted if there are Book(s) against it in the database.	This objective has been achieved as expected	Erroneous	**Pass**	**BookInfo Delete.mp4** 00:00:36 to 00:01:18	
	LMS_BI_04.5		A success message should be shown when the BookInfo is Successfully deleted	BookInfo Data + User click to Delete BookInfo	A success message should be shown when the User is Successfully deleted to the Database	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Delete.mp4** 00:01:23 to 00:02:07	
	LMS_BI_04.6		This functionality should not be available to Member user	NA	Menu option missing to Delete BookInfo	This objective has been achieved as expected	Normal	**Pass**	**BookInfo Delete.mp4** 00:02:40 to 00:02:56	
10	LMS_B_001	Add book (of a BookInfo type)	Unique bookId created for the new Book	Book Data	Incremental userId generated for every new User	This objective has been achieved as expected	Normal	**Pass**	**DB PK auto increment.mp4** Full video (Example of User but is applicable to Book as well)	
			System confirmation of any failures	Book Data	An appropriate	This objective has been	Erroneous	**Pass**	**Book Add.mp4** 00:00:29 to 00:01:20	

			including validation failures		error message should be presented asking the user to take action.	achieved as expected				
			System confirmation of a successful Book Add	Book Data + User click to Add Book	A success message should be shown when the Book is Successfully added to the Database	This objective has been achieved as expected	Normal	**Pass**	**Book Add.mp4** 00:01:21 to 00:01:32	
			Increment copied of BookInfo when a Book is added	Book Data	When a Book is added, the number of books in its associated BookInfo should increment by 1.	This objective has been achieved as expected	Normal	**Pass**	**Book Add.mp4** 00:00:07 to 00:00:25 WITH 00:01:39 to 00:02:01	
			This functionality should not be available to Member user	NA	Menu option missing to Add Book	This objective has been achieved as expected	Normal	**Pass**	**Book Add.mp4** 00:02:39 to 00:01:32	

11	LMS_B_002	Read a book	Search Book by bookId	bookId	Exactly 1 book should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**Book Read.mp4** 00:00:00 to 00:00:50	
			Search Book by associated bookInfoId.	bookInfoId	Matching book(s) should be searched for by triggering the appropriate query on DB	This objective has been achieved as expected	Normal	**Pass**	**Book Read.mp4** 00:00:56 to 00:01:00	
			A list of Books should be displayed for all BookInfos matching the search criteria.	A valid bookInfoId which will search for its associated books	1 or more book(s) should be displayed when present	This objective has been achieved as expected	Normal	**Pass**	**Book Read.mp4** 00:01:02 to 00:01:13	
			The user should be able to select any book from the list of books to see the full details of that selected book.	Book selection of a particular row	Selected book's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**Book Read.mp4** 00:01:02 to 00:01:13	

			When there is only 1 book searched, the system should be able to auto select the book to see the full details of that book.	(The user should not have to select a row when there is only one row.) Row selection done automatically by the system	The only row present is selected and that book's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**Book Read.mp4** 00:00:34 to 00:00:40	
			The selected book's BookInfo details should also be displayed	Book row selection done automatically by the system	The selected book's BookInfo details are displayed along with Book details	This objective has been achieved as expected	Normal	**Pass**	**Book Read.mp4** 00:00:41 to 00:01:52	
			System confirmation of any failures including validation failures	Book Data	An appropriate error message should be presented	This objective has been achieved as expected	Erroneous	**Pass**	**Book Read.mp4** 00:00:06 to 00:00:32	
12	LMS_B_03.1	Update a book	Book should first be searched by bookId	bookId	Exactly 1 book should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**Book Update.mp4** 00:00:00 to 00:00:25	

	LMS_B_03.2		System confirmation of any failures including validation failures	Book Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**Book Update.mp4** 00:00:05 to 00:00:22	
	LMS_B_03.3		When the search is Successful	Book Data + User click to Update Book	Book details should be displayed to the user, giving an option to update only limited fields (Shelf Reference, Location, Edition) allowed for update via the LMS app.	This objective has been achieved as expected	Normal	**Pass**	**Book Update.mp4** 00:00:27 to 00:00:35	
	LMS_B_03.4		A success message should be shown when the Book is Successfully updated	Book Data + User click to Update Book	A success message should be shown when the Book is Successfully updated to the Database	This objective has been achieved as expected	Normal	**Pass**	**Book Update.mp4** 00:00:36 to 00:00:45	

	LMS_B_03.5		This functionality should not be available to Member user	NA	Menu option missing to Update BookInfo	This objective has been achieved as expected	Normal	**Pass**	**Book Update.mp4** 00:01:13 to 00:01:30	
13	LMS_B_04.1	Delete a book	Book should first be searched by bookId	bookId	Exactly 1 book should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**Book Delete.mp4** 00:00:00 to 00:00:30	
	LMS_B_04.2		System confirmation of any failures including validation failures	Book Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**Book Delete.mp4** 00:00:08 to 00:00:32	
	LMS_B_04.3		When the search is Successful	Book Data + User click to Delete Book	Book details should be retrieved and displayed to the user, who can then confirm the deletion of that Book.	This objective has been achieved as expected	Normal	**Pass**	**Book Delete.mp4** 00:00:32 to 00:00:45	

	LMS_B_04.4		Non returned Book	Book Data	A Book cannot be deleted if it is currently borrowed by a user and not yet returned.	This objective has been achieved as expected	Normal	**Pass**	**Book Delete.mp4** 00:00:46 to 00:01:22	
	LMS_B_04.5		Book with historical transactions	Book Data	A Book cannot be deleted if it has historical transactions (to maintain referential integrity).	This objective has been achieved as expected	Normal	**Pass**	**Book Delete.mp4** 00:01:29 to 00:02:45	
	LMS_B_04.6		Decrement copies of BookInfo when a Book is deleted	Book Data	When a Book is deleted, the number of books in its associated BookInfo should decrement by 1.	This objective has been achieved as expected	Normal	**Pass**	**Book Delete.mp4** 00:04:17 to 00:04:35	
	LMS_B_04.7		A success message should be shown when the Book is Successfully deleted	Book Data + User click to Delete Book	A success message should be shown when the Book is Successfully deleted from the Database	This objective has been achieved as expected	Normal	**Pass**	**Book Delete.mp4** 00:03:28 to 00:04:05	

	LMS_B_04.8		This functionality should not be available to Member user	NA	Menu option missing to Delete Book	This objective has been achieved as expected	Normal	**Pass**	**Book Delete.mp4** 00:04:38 to 00:04:56	
14	LMS_T_01.1	Issue book (Add Transaction)	Unique transactionId created for the new Transaction	Transaction Data	Incremental transactionId generated for every new Transaction	This objective has been achieved as expected	Normal	**Pass**	**DB PK auto increment.mp4** Full video (Example of User but is applicable to Transaction as well)	
	LMS_T_01.2		System confirmation of any user and book related failures including validation failures	Transaction Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**Transaction Add.mp4** 00:00:09 to 00:00:40	
	LMS_T_01.3		User already has 3 non-returned, borrowed books	User Data	A transaction cannot be added if the associated user already has 3 non-returned, borrowed books	This objective has been achieved as expected	Normal	**Pass**	**Transaction Add.mp4** 00:00:41 to 00:01:24	

	LMS_T_01.4		Book is already borrowed and not yet marked as returned	Book Data	A transaction cannot be added if the associated book is already borrowed and not yet marked as returned	This objective has been achieved as expected	Normal	**Pass**	**Transaction Add.mp4** 00:01:27 to 00:02:03	
	LMS_T_01.5		Auto assign value of issue date	Transaction Data	Issue date should be auto assigned as of current date.	This objective has been achieved as expected	Normal	**Pass**	**Transaction Add.mp4** 00:02:58 to 00:03:04	
	LMS_T_01.6		Auto assign value of return date	Transaction Data	Return date should be auto calculated as of 2 weeks.	This objective has been achieved as expected	Normal	**Pass**	**Transaction Add.mp4** 00:03:04 to 00:03:13	
	LMS_T_01.7		Book cannot be borrowed after being issued	Transaction Data + Book Data	Book is unavailable for borrowing	This objective has been achieved as expected	Normal	**Pass**	**Transaction Add.mp4** 00:03:14 to 00:03:22	
	LMS_T_01.8		A success message should be	Transaction Data + User click to Add Transaction	A success message should be	This objective has been	Normal	**Pass**	**Transaction Add.mp4**	

			shown when the Transaction is Successfully added (i.e a Book successfully issued to a User)		shown when the Transaction is Successfully added to the Database	achieved as expected			00:02:09 to 00:02:18	
	LMS_T_01.9		This functionality should not be available to Member user	NA	Menu option missing to Add Transaction	This objective has been achieved as expected	Normal	**Pass**	**Transaction Add.mp4** 00:04:01 to 00:04:20	
15	LMS_T_02.1	Return book (Update Transaction)	Transaction should be searched by bookId	bookId	Exactly 1 user should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**Transaction Update.mp4** 00:00:00 to 00:00:23	
	LMS_T_02.2		System confirmation of any failures including validation failures	Transaction Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**Transaction Update.mp4** 00:00:07 to 00:00:18	

	LMS_T_02.3		When the search is Successful	Transaction Data + User click to Update Transaction	Transaction details (i.e Book Issue details) should be displayed to the user, giving an option to update the Transaction (i.e to mark the Book as returned).	This objective has been achieved as expected	Normal	**Pass**	**Transaction Update.mp4** 00:00:23 to 00:00:26	
	LMS_T_02.4		Delayed return of book incurs Fine	Transaction Data	Fine should be auto calculated if the return has gone beyond the Return Date.	This objective has been achieved as expected	Normal	**Pass**	**Transaction Update.mp4** 00:00:28 to 00:00:32	
	LMS_T_02.5		A success message should be shown when the Transaction is Successfully updated	Transaction Data + User click to Update Transaction	A success message should be shown when the Transaction is Successfully updated (i.e The Book is marked as returned).	This objective has been achieved as expected	Normal	**Pass**	**Transaction Update.mp4** 00:00:40 to 00:00:52	

	LMS_T_02.6		Auto assign value of actual return date	Transaction Data	Actual Return date should be auto calculated as of the current date.	This objective has been achieved as expected	Normal	**Pass**	**Transaction Update.mp4** 00:00:56 to 00:01:15	
	LMS_T_02.7		Book can be borrowed after being returned	Transaction Data	Book is available for borrowing	This objective has been achieved as expected	Normal		**Transaction Update.mp4** 00:01:18 to 00:01:25	
16	LMS_T_03.1	Read/View a Transaction (by Id OR by associated User or by associated Book match condition)	Transaction should be searched by transactionId	transactionId	Exactly 1 Transaction should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:02:10 to 00:02:20	
	LMS_T_03.2		Search Transaction by searching userId associated to the Transaction, either for all associated Transactions		Matching Transaction (s) should be searched for by triggering the appropriate query on DB	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:01:30 to 00:01:40	

			or when the Transaction is still open (i.e the issued book is not yet returned).							
	LMS_T_03.3		Search Transaction by searching bookId associated to the Transaction, either for all associated Transactions or when the Transaction is still open (i.e the issued book is not yet returned).		Matching Transaction (s) should be searched for by triggering the appropriate query on DB	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:02:23 to 00:02:40	
	LMS_T_03.4		A list of Transactions should be displayed for all Transactions matching the search criteria.	A valid user or book information which will search across the Transaction's Data	1 or more Transaction(s) should be displayed	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:01:32 to 00:01:42	

	LMS_T_03.5		The user should be able to select any Transaction from the list of Transactions to see the full details of that selected book.	Transaction selection of a particular row	Selected Transaction's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:01:32 to 00:01:42	
	LMS_T_03.6		When there is only 1 Transaction searched, the system should be able to auto select the Transaction to see the full details of that Transaction.	(The user should not have to select a row when there is only one row.) Row selection done automatically by the system	The only row present is selected and that Transaction's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:01:53 to 00:01:59	
	LMS_T_03.7		Auto calculate fine while showing the Trnasaction	Transaction Data	The fine should be auto calculated for each transaction details displayed	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:02:48 to 00:02:53	

					based on the rule that it will be £2 for every 1-week delay. The delay is calculated for days between the current date and the scheduled returnDate of that transaction					
	LMS_T_03.8		Display associated User information	User Data	The selected Transaction's User details will also be displayed.	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:02:59 to 00:03:03	
	LMS_T_03.9		Display User's associated Address information	Address Data	That user's address details will also be displayed.	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:03:03 to 00:03:05	
	LMS_T_03.10		Display associated Book information	Book Data	The selected Transaction's Book details will also be displayed.	This objective has been	Normal	**Pass**	**Transaction Read.mp4** 00:03:05 to 00:03:15	

						achieved as expected				
	LMS_T_03.11		Display Book's associated Bookinfo information	BookInfo Data	That book's BookInfo details should also be displayed.	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:03:15 to 00:03:22	
	LMS_T_03.12		System confirmation of any failures including validation failures	Transaction Data	An appropriate error message should be presented	This objective has been achieved as expected	Erroneous	**Pass**	**Transaction Read.mp4** 00:00:10 to 00:01:30	
	LMS_T_03.13		This functionality should be available to Member user to search only their transaction details	NA	Member user can see only their Transactions details	This objective has been achieved as expected	Normal	**Pass**	**Transaction Read.mp4** 00:04:16 to 00:06:42	
17	LMS_RI_01.1	Add reportInfo	Unique reportInfoId created for the new ReportInfo	ReportInfo Data	Incremental reportInfoId generated for every new ReportInfo	This objective has been achieved as expected	Normal	**Pass**	**DB PK auto increment.mp4** Full video (Example of User but is applicable to ReportInfo as well)	

	LMS_RI_01.2		System confirmation of any failures including validation failures	ReportInfo Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**ReportInfo Add.mp4** 00:00:18 to 00:01:23	
	LMS_RI_01.3		System confirmation of a successful ReportInfo Add	ReportInfo Data + User click to Add ReportInfo	A success message should be shown when the ReportInfo is Successfully added to the Database	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Add.mp4** 00:01:32 to 00:02:05	
	LMS_RI_01.4		This functionality should not be available to Staff and Member user to add ReportInfo details	NA	Menu option missing to Add ReportInfo	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Add.mp4** 00:02:05 to 00:02:45	
18	LMS_RI_02.1	Read a ReportInfo	Search ReportInfo by reportInfoId	reportInfoId	Exactly 1 ReportInfo should be displayed if present or	This objective has been	Normal	**Pass**	**ReportInfo Read.mp4** 00:00:00 to 00:00:31	

					error displayed	achieved as expected				
	LMS_RI_02.2		ReportInfo should be searched by searching the ReportInfo's Name pattern which will search across the Name.	A part of reportInfo's name pattern	Matching ReportInfo(s) should be searched for by triggering the appropriate query on DB	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Read.mp4** 00:00:58 to 00:01:02	
	LMS_RI_02.3		A list of ReportInfos should be displayed for all ReportInfos matching the search criteria.	A valid name pattern which will search across the ReportInfo's name	1 or more ReportInfo(s) should be displayed	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Read.mp4** 00:00:58 to 00:01:20	
	LMS_RI_02.4		The user should be able to select any ReportInfo from the list of ReportInfos to see the full details of that selected ReportInfo.	ReportInfo selection of a particular row	Selected ReportInfo's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Read.mp4** 00:01:07 to 00:01:20	

	LMS_RI_02.5		When there is only ReportInfo searched, the system should be able to auto select the ReportInfo to see the full details of that ReportInfo.	(The user should not have to select a row when there is only one row.) Row selection done automatically by the system	The only row present is selected and that ReportInfo's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Read.mp4** 00:00:39 to 00:00:45	
	LMS_RI_02.6		System confirmation of any failures including validation failures	ReportInfo Data	An appropriate error message should be presented	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Read.mp4** 00:00:04 to 00:00:30	
	LMS_RI_02.7		This functionality should not be available to Staff and Member user	NA	Menu option missing to Read ReportInfo	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Read.mp4** 00:02:00 to 00:02:24	
19	LMS_RI_03.1	Update a ReportInfo	ReportInfo should first be searched by searching the reportInfoId to verify it is	reportInfoId	Exactly 1 ReportInfo should be displayed if present or	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Update.mp4** 00:00:00 to 00:00:31	

			an existing ReportInfo.		error displayed					
	LMS_RI_03.2		System confirmation of any failures including validation failures	ReportInfo Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**ReportInfo Update.mp4** 00:00:00 to 00:00:26 And 00:00:37 to 00:01:07	
	LMS_RI_03.3		When the search is Successful	ReportInfo Data + User click to Update ReportInfo	ReportInfo details should be retrieved and displayed to the user, giving an option to update the field (only the generation time field).	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Update.mp4** 00:00:32 to 00:00:35	
	LMS_RI_03.4		A success message should be shown when the ReportInfo is Successfully updated	ReportInfo Data + User click to Update ReportInfo	A success message should be shown when the ReportInfo is Successfully updated to the Database	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Update.mp4** 00:01:08 to 00:01:45	

	LMS_RI_03.5		This functionality should not be available to Staff and Member user	NA	Menu option missing to Read ReportInfo	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Update.mp4** 00:01:46 to 00:02:13	
20	LMS_RI_04.1	Delete a reportInfo	ReportInfo should first be searched by reportInfoId.	reportInfoId	Exactly 1 ReportInfo should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Delete.mp4** 00:00:00 to 00:00:35	
	LMS_RI_04.2		System confirmation of any failures including validation failures	ReportInfo Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**ReportInfo Delete.mp4** 00:00:05 to 00:00:38	
	LMS_RI_04.3		When the search is Successful	ReportInfo Data + User click to Delete ReportInfo	ReportInfo details should be retrieved and displayed to the user, who can then confirm the deletion of	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Delete.mp4** 00:01:35 to 00:02:22	

					that ReportInfo.					
	LMS_RI_04.4		Reports of this ReportInfo exist	Report Data	A ReportInfo cannot be deleted if there are Report(s) against it in the database.	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Delete.mp4** 00:00:49 to 00:01:32	
	LMS_RI_04.5		This functionality should not be available to Staff and Member user	NA	Menu option missing to Delete ReportInfo	This objective has been achieved as expected	Normal	**Pass**	**ReportInfo Delete.mp4** 00:02:28 to 00:02:50	
21	LMS_R_01.1	Add/Generate report (against a ReportInfo)	Unique reportId created for the new Report	Report Data	Incremental reportId generated for every new User	This objective has been achieved as expected	Normal		**DB PK auto increment.mp4** Full video (Example of User but is applicable to Report as well)	
	LMS_R_01.2		ReportInfo should first be searched by reportInfoId.	reportInfoId		This objective has been achieved as expected	Normal		**Report Add.mp4** 00:00:26 to 00:01:38	
	LMS_R_01.3		System confirmation of any failures	ReportInfo Data	An error message should	This objective has been	Erroneous	**Pass**	**Report Add.mp4** 00:00:07 to 00:00:26	

			including validation failures		be presented asking the user to input mandatory fields.	achieved as expected				
	LMS_R_01.4		A success message should be shown when the Report is Successfully (manually) added	User Data + User click to Add User	A success message should be shown when the User is Successfully added to the Database	This objective has been achieved as expected	Normal	**Pass**	**Report Add.mp4** 00:00:29 to 00:00:35	
	LMS_R_01.5		This functionality should not be available to Staff and Member user	Menu option missing to Add Report	This objective has been achieved as expected	This objective has been achieved as expected	Normal	**Pass**	**Report Add.mp4** 00:01:06 to 00:01:25	
22	LMS_R_02.1	Read a Report	Search Report by reportId	reportId	Exactly 1 user should be displayed if present or error displayed	This objective has been achieved as expected	Normal	**Pass**	**Report Read.mp4** 00:00:00 to 00:01:38	
	LMS_R_02.2		Search Report by Report's Generation Date	Generation Date of Report	Matching Report(s) should be searched for by triggering	This objective has been	Normal	**Pass**	**Report Read.mp4** 00:01:40 to 00:02:08	

					the appropriate query on DB	achieved as expected				
	LMS_R_02.3		List of Reports is displayed	A valid Generation Date which will search across the Report's Generation Date	1 or more Report(s) should be displayed	This objective has been achieved as expected	Normal	**Pass**	**Report Read.mp4** 00:01:55 to 00:02:12	
	LMS_R_02.4		The user should be able to select any Report from the list of Reports to see the full details of that selected Report.	Report selection of a particular row	Selected Report's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**Report Read.mp4** 00:02:13 to 00:02:50	
	LMS_R_02.5		When there is only 1 Report searched, the system should be able to auto select the Report to see the full details of that Report.	(The user should not have to select a row when there is only one row.) Row selection done automatically by the system	The only row present is selected and that Report's details should be displayed	This objective has been achieved as expected	Normal	**Pass**	**Report Read.mp4** 00:01:07 to 00:01:16	

	LMS_R_02.6		The selected Report's Reportinfo details should also be displayed	Report row selection done automatically by the system	The selected Report's ReportInfo details are displayed along with Report details	This objective has been achieved as expected	Normal	**Pass**	**Report Read.mp4** 00:02:13 to 00:02:33	
	LMS_R_02.7		System confirmation of any failures including validation failures	Report Data	An appropriate error message should be presented	This objective has been achieved as expected	Erroneous	**Pass**	**Report Read.mp4** 00:00:22 to 00:01:05	
	LMS_R_02.8		This functionality should not be available to Staff and Member users	NA	Menu option missing to access Report menu	This objective has been achieved as expected	Normal	**Pass**	**Report Read.mp4** 00:03:05 to 00:03:37	
23	LMS_F_01.1	Calculate late fee for overdue books. £2 for every 1-week delay	The user should see fine calculated when Today's (current) Date minus ReturnDate (from the Transaction) Is 9 weeks	Today's (current) Date, ReturnDate (from the Transaction) [9 weeks more than the Return day]	Fine calculated as £18	This objective has been achieved as expected	Normal	**Pass**	**Fine Calculation.mp4** 00:00:00 to 00:01:16	

	LMS_F_01.2		The user should see fine calculated when Today's (current) Date minus ReturnDate (from the Transaction) Is 1 week	Today's (current) Date, ReturnDate (from the Transaction) [1 day more than the Return day]	Fine calculated as £2	This objective has been achieved as expected	Boundary	**Pass**	**Fine Calculation.mp4** 00:01:16 to 00:02:03	
	LMS_F_01.3		The user should see fine calculated when Today's (current) Date minus ReturnDate (from the Transaction) Is 0 week	Today's (current) Date, ReturnDate (from the Transaction) [Both exact same dates]	Fine calculated as £0	This objective has been achieved as expected	Boundary	**Pass**	**Fine Calculation.mp4** 00:02:05 to 00:03:03	
24	LMS_S_01.1	User Authentication	User should attempt to login	Main url on borwser	User should be presented with an option to input user Id and password.	This objective has been achieved as expected	Normal	**Pass**	**User Authentication.mp4** 00:00:00 to 00:03:06	
	LMS_S_01.2		Ensure Password	User password	Password should be encrypted	This objective has been	Normal	**Pass**	**User Authentication.mp4**	

			encryption over network		before transmitting it over the network.	achieved as expected			00:00:22 to 00:01:02	
	LMS_S_01.3		Authenticate User	User Data	User authentication details should be checked against the User's data stored in the DB.	This objective has been achieved as expected	Normal	**Pass**	**User Authentication.mp4** 00:00:25 to 00:00:48	
	LMS_S_01.4		Retrieve User information	User Data	User's Firstname and Surname Initials, User's ID and User's Type should be returned when the User is Successfully searched	This objective has been achieved as expected	Normal	**Pass**	**User Authentication.mp4** 00:01:32 to 00:01:51	
	LMS_S_01.5		System confirmation of any failures including validation failures	User Data	An appropriate error message should be presented asking the user to take action.	This objective has been achieved as expected	Erroneous	**Pass**	**User Authentication.mp4** 00:02:25 to 00:04:40	

25	LMS_S_02.1	User Authorisation	Admin User Access	User Type Data	Admin User will have access to all menu items in the system.	This objective has been achieved as expected	Normal	**Pass**	**User Access - Authorisation.mp4** 00:00:00 to 00:01:20	
	LMS_S_02.2		Staff User Access	User Type Data	Staff User will have access to all menu items in the system except the ReportInfo and Report items.	This objective has been achieved as expected	Normal	**Pass**	**User Access - Authorisation.mp4** 00:01:22 to 00:02:30	
	LMS_S_02.3		Member User Access	User Type Data	Member User will have access to very limited menu items in the system. They can only read their own details, update some of their details (Password, mobile number and email address),	This objective has been achieved as expected	Normal	**Pass**	**User Access - Authorisation.mp4** 00:02:31 to 00:06:57	

					search and read a BookInfo, search and read a Book and search and read their own transaction history.					
26	LMS_H_01.1	Housekeeping logs	Rotating logs	Working of the application	See log files being rotated as per specified configuration	This objective has been achieved as expected	Normal		**Log Housekeeping.mp4** Full video	

5.3 Evidence

Provided as part of videos

6 Evaluation

I have evaluated the LMS application as a whole diving it into 3 parts as below

6.1 Self evaluation

When I look back to all my success criteria in section 2.11.1 (Pg 22 in Analysis section) spread across Login, CRUD Users, CRUD BookInfo, CRUD Book, CRU Transaction (i.e Borrow/Return book), CRUD ReportInfo and CR Report and the Generic functionalities and based on the testing done on my code in section 5.2 (Pg 189 in Testing section), I feel confident that I was able to solve the overall problem that my end users and stakeholders were experiencing in their old manual library system. All the requirements in my success criteria were of High priority, except for the optional Housekeeping of Data. So, there was very little chance I could have taken in terms of missing on any of the core functionalities of the application. Initially I was only planning to provide a json input/output integrity tool (e.g Postman) to connect with the APIs on the server but as I progressed on the application, I realised that if I didn't have UI screens, the end user will not be able to truly appreciate the solution and may feel frustrated working on tools like Postman, that deal with raw json data having to know it's syntax. I went over and above my initial scope of this application development to get the various screens done as well.

I feel the Menu is clearly laid out for ease of Library UI Users to differentiate between the various functionalities and the layout of the page is neatly done. So, first search and then to get a list of all matching records. The user can then select any record and proceed to the required functionality. Here I think instead of just keeping it to read functionality, I could have added the Update and Delete button as well after search so the user would not have to go especially to the Update/Delete menu for doing those functionalities. This way the search functionality would not have repeated in the Update and Delete menus but would only be in one Menu from which there would have been 3 options Read, Update and Delete. To get this done would have taken more time as the screen layout would have to be accordingly set to dynamic, so easier for the end users but more time required for coding from an A level student perspective.

The User search solution allows part of the first or last name of the user to be entered into the search box and if more than one user matches this name pattern then a list of potential users to choose is listed. This makes it very easy to use without the end-user having to know full first/last name. Now, when no User is found, the system is robust in that it says, 'No User found'. However, this could be improved. It would be better if it looked for similar names or misspellings. There is a room for improvement that I could have integrated Fuse.js javascript library to search for not just exact patterns but also for similar patterns as well (e.g if name Rafiq was put for search it could come back with results for Rafik as well). If I had more time I would have attempted to get this done.

I could have given a few more search patterns to search e.g email address, mobile number, user type, DOB etc. I think the fact that I showed name search shows that I know how to write specific SQLs to search within the database and for it to be extended to any number of parameters to be searched.

Similarly for the Book Info search, I had coded for the search on Title. This can be extended to other parameters like Author, Genre Category, ISBN, Publisher etc.

Similarly for the Book search, I had coded for the search on Book Id and BookInfo ID. This can be extended to other parameters like Shelf reference, BookInfo Title etc.

Similarly for the Transaction search, I had coded for the search on Transaction Id, User Id and Book Id. This can be extended to other parameters like Issue Date, Return Date, Username pattern, Book title etc.

Similarly for the Report Info search, I had coded for the search on Name. This can be extended to other parameters like Time to Generate.

Similarly for the Report search, I had coded for the search on generated date. This can be extended to other parameters like ReportInfoId, ReportInfo Name, Time to Generate etc.

Given this is an online system, another improvement could have been that the search box could be further improved to match how search engines like Google® work. For instance, as the name is being entered, a drop-down list of suggested searches could be given. This would make it faster to enter names, titles etc. It would also make it easier to find names, titles etc that are harder to spell.

Another area for improvement would have been to integrate the retrieval of live addresses from a government approved external API provider. I wanted to have a go at this because integrating another API would not have taken up as much time. But all these external address providers work on licence fees to provide the data, which was too expensive for my LMS project. But if there was budget approved and I had more time, I would have put this in scope.

On the technical side, I have used HttpSessions to track logged in users. More effective and latest technology way to do this would have been to integrate JWT (Json Web Token) for authentication. But due to time constraints and not much prior exposure to this technology and very little time to research, I could not put it in my LMS solution. This will be something for me to use my vacation time after A Levels and improve my solution with all the improvements I have listed above.

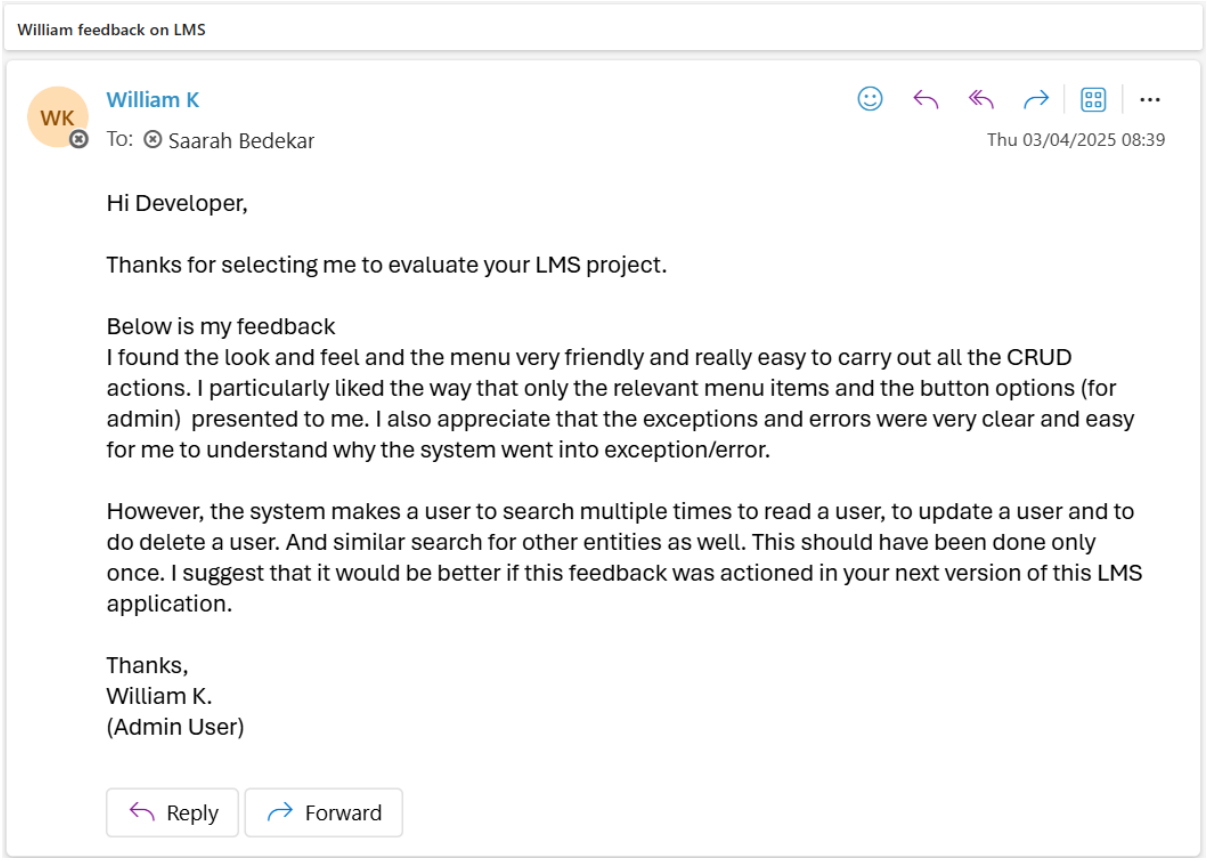
Whilst I have used the latest version of HTML, Maven, SpringBoot and MySQL, I have used a not so recent version of Java i.e Java 17. I could have used a more recent version. But I may not have known all the new syntax in the new versions. So, to keep it safe, I kept it to the industry wide accepted version of Java 17.

I have done a lot of client-side validations, but I could have enhanced a lot more on server-side validations as well, If I had enough time. Most important ones are taken care of already though.

There was a possibility of integrating automatic SMS alerts to users when their due date was approaching. This would have required mobile network SMS expertise which I don't have. Hence this could not have been onboarded.

6.2 Independent feedback

6.2.1 User Feedback Evidence



William feedback on LMS

WK

William K

To: Saarah Bedekar

Thu 03/04/2025 08:39

Hi Developer,

Thanks for selecting me to evaluate your LMS project.

Below is my feedback
I found the look and feel and the menu very friendly and really easy to carry out all the CRUD actions. I particularly liked the way that only the relevant menu items and the button options (for admin) presented to me. I also appreciate that the exceptions and errors were very clear and easy for me to understand why the system went into exception/error.

However, the system makes a user to search multiple times to read a user, to update a user and to do delete a user. And similar search for other entities as well. This should have been done only once. I suggest that it would be better if this feedback was actioned in your next version of this LMS application.

Thanks,
William K.
(Admin User)

Reply Forward

Chris feedback on LMS

CT **Chris T**
To: Saarah Bedekar
Tue 15/04/2025 14:12

Hi Saarah,

I have evaluated your LMS project.

I need to point out the below:
Although I am an admin user, I am not from the DB admin subgroup. Can I request for more pre defined Reports to be setup in the system so that I don't have to know all details of all the tables and the various joins required. These low level details should only be expected from the DB admin subgroup.

Thanks,
Chris T.
(Admin User)

Reply Forward

Mark feedback on LMS

MD **Mark D**
To: Saarah Bedekar
Thu 1/05/2025 15:59

Hi Developer,

I have done a run through your LMS project.

My requested feedback is:
Can I request that the system should scan user ID cards, book scanner stickers instead of manually entering the IDs in the system?
And also if possible, the system should be integrated with contactless Credit/Debit card, Google pay, Apple Pay and similar technologies.

Thanks,
Mark D.
(Staff User)

Reply Forward

6.2.2 User Feedback Summary

Received from End Users and Library owner

A few users with a mix of type Admin, Staff and member users tried my LMS application to perform the CRUD operations on the various entities that they had access to and were authorised to do.

They stated that they found the look and feel and the menu very friendly and easy to carry out all the CRUD actions. They particularly liked the way that only the relevant menu items and the button options (depending on their authorisations) were presented to them. They also appreciated that the exceptions and errors were very clear and easy for users to understand why the system went into exception/error. However, the system makes a user to search multiple times to read a user, to update a user and to do delete a user. This should have been done only once. They suggested that it would be better if this feedback was actioned in the next version of this LMS application.

The admin staff specifically requested that there can be an additional menu to handle the deletes required for entities associated to the linked/join table, so that it becomes a standard set of rules and UI. This will ensure every time those kinds of functionalities are done, it will be done in the same way no matter which user initiates that workflow.

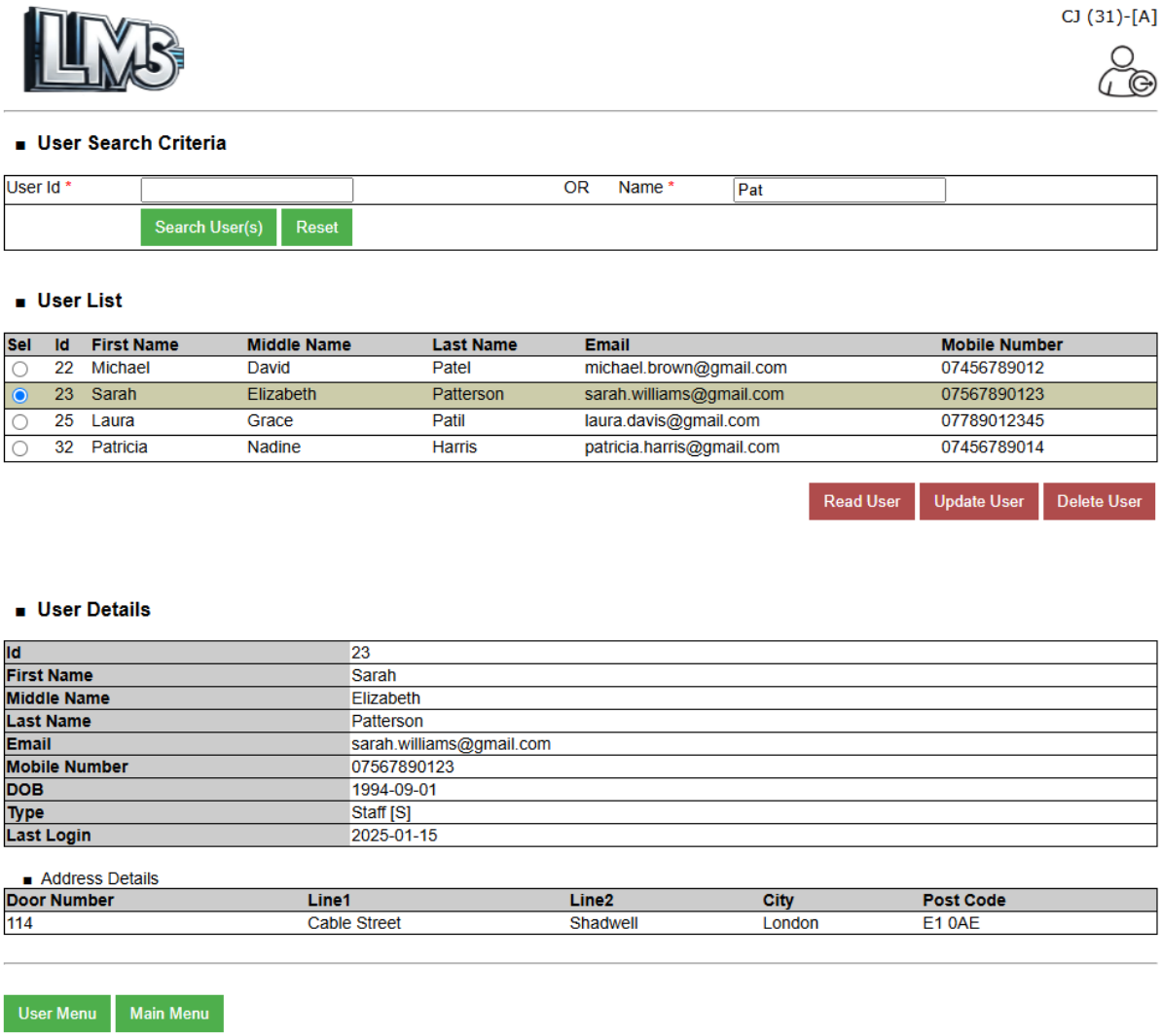
The admin staff requested for more pre-defined Reports to be setup in the system so that they don't have to know all details of all the tables and the various joins required. These low-level details should only be expected from a subset of admins who function specifically as DB admins as well and not expected from all admins.

The Staff members particularly requested that the system should scan user ID cards, book scanner stickers instead of manually entering the IDs in the system.

The Staff members also requested that the system should be integrated with contactless Credit/Debit card, Google pay, Apple Pay and similar technologies

6.3 Evaluation of Independent feedback

I agree with the independent feedback that search functionality should not be repeated for Read, Update and Delete. I, myself had the comment as a part of self-evaluation. This can be integrated in the workflow by ensuring that when the user selects a user from the list of users presented on the screen, there can be options of either reading the details or updating the limited fields of the user or deleting the user as shown in the below screenshot.



I agree with the independent feedback that for MI functionalities, there can be more reports pre-set into the system so the admins can search on the ReportInfo names and accordingly run as and when the reports are to be generated for it.

I agree that it will be a full-fledged payment scanning system integrated for fine collection. But this technology will take time, finance and expertise to be onboarded and so will be too hard to be done at A levels.

7 Appendix

7.1 Hardware support

Our IT systems' hardware is supported by a company known as "Esmail Hardware limited". We have a contract with this company who takes care of our PCs, Screens, Hardware and Networks. They are not responsible for any software development.

Their contact details are

- ✆ 000 9777 8888
- ✉ support@esmailhardware.com

8 References

Section Details	Links
SpringBoot app	https://spring.io/projects/spring-boot https://www.youtube.com/watch?v=a-QBq5uXpLE https://stackabuse.com/how-to-return-http-status-codes-in-a-spring-boot-application/ https://medium.com/javajams/creating-a-rest-api-in-spring-boot-68ce785f652f
Intercalling microservice	https://www.geeksforgeeks.org/spring-boot-microservices-communication-using-resttemplate-with-example/
Log	https://medium.com/@AlexanderObregon/enhancing-logging-with-log-and-slf4j-in-spring-boot-applications-f7e70c6e4cc7
Sec 3.11 Security	• https://nordvpn.com/cybersecurity/glossary/system-security/#:~:text=System%20security%20is%20the%20practice,data%20and%20prevent%20cyber%20threats • https://www.soutron.com/blog/general/top-10-security-issues-library-management-system/ • https://www.qlik.com/us/data-management/data-integrity

Mono vs Micro	https://www.orientsoftware.com/blog/monolithic-architecture-vs-microservices/
JDBC class	https://mkyong.com/spring-boot/how-to-access-values-from-application-properties-in-spring-boot/ https://www.digitalocean.com/community/tutorials/java-datasource-jdbc-datasource-example
Model	https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/Hibernate-JPA-Column-Mapping-Basic-Transient-Annotation-MySQL-Database-Persistence
Test	https://medium.com/@idiotN/junit-with-mockito-example-for-service-class-in-spring-boot-22d2eaaaa4d9
Associ, compo, aggreg	https://www.geeksforgeeks.org/association-composition-aggregation-java/
ERD	https://creately.com/guides/er-diagrams-tutorial/ https://www.geeksforgeeks.org/introduction-of-er-model/
DFD	https://www.geeksforgeeks.org/levels-in-data-flow-diagrams-dfd/
HTML	https://www.w3schools.com/html/
CSS	https://www.w3schools.com/css/
Javascript	https://www.w3schools.com/js/
Maven	https://www.w3schools.blog/maven-tutorial
Github	https://www.youtube.com/watch?v=2e-adccVcEk
Regular Expression	https://www.geeksforgeeks.org/write-regular-expressions/ https://coderpad.io/blog/development/the-complete-guide-to-regular-expressions-regex/